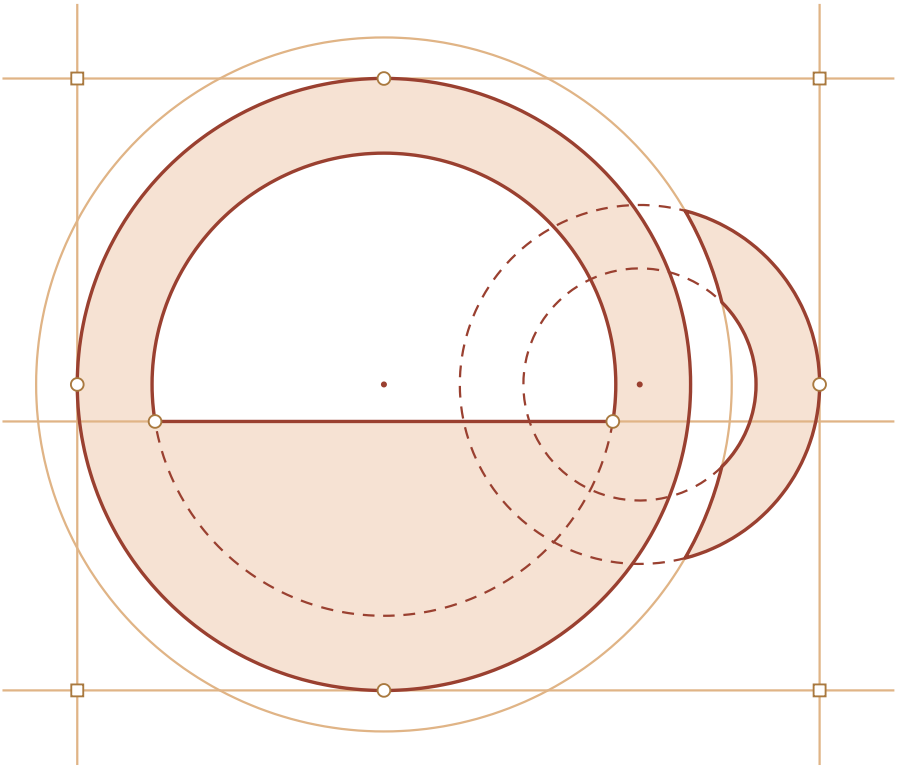
Expresso



Expresso - 익스프레소

채용 공고·커리어 기록 추천 모델 기반
맞춤형 포트폴리오 생성 및 배포 플랫폼 개발

설계서

버전 v1.0

작성일 2026. 10. 13.

소속 가천대학교 컴퓨터공학과

팀장 박민재

팀원 심민규 · 오시건 · 채서현 · 윤규현

DOCUMENT

문서 정보

본 문서는 **Expresso — 채용 공고·커리어 기록 추천 모델 기반 맞춤형 포트폴리오 생성 및 배포 플랫폼 개발**을 위한 요구규격 및 설계서입니다.

버전	1.0
작성일	2026-10-13
상태	■ 완료 □ 진행 중 □ 초안

버전	변경한 사람	변경한 날짜	버전업 변경(또는 추가) 내용
0.1	Expresso 팀	2026-08-14	저장소 형상을 근거로 전 장 초안 작성
0.2	Expresso 팀	2026-09-28	MongoDB 데이터·시스템 구조와 현행 규모·성능 수치 반영
0.3	Expresso 팀	2026-10-05	프로젝트 제안서 기준 AI 모델 · 학습 데이터 · 평가 지표 · 일정 · 조직 정합
0.4	Expresso 팀	2026-10-05	파일럿 학습 결과(4.3 · 9.2)와 ROC 양성 기준 60점 반영, 7.3 API 설계 26개로 확장, 8.2 컬렉션 명세서 82개 자동 생성
0.5	Expresso 팀	2026-10-05	표지 · 문서 정보 · 머리말에 Expresso 브랜딩과 공식 명칭 적용, 개발필요성을 제안서 기획 배경 · 벤치마킹 차별점 기준으로 재작성, 폐기된 근거 검증 컨셉 정리
0.6	Expresso 팀	2026-10-06	구조 그림 9개를 제안서 양식으로 반영(교사 지식 증류 · 두 모델의 단계적 활용 추가), 방문자 용어 통일, 모델 A 판단 기준에서 근무 조건 제외, 웹 화면을 PDF와 같은 구성으로 정리
0.7	Expresso 팀	2026-10-06	서비스 개요 그림과 세 가지 장점, 기존 서비스 지원 범위 벤 다이어그램, 전체 사용자 플로우, 순위 지표(NDCG@k · Recall@k) 설명 추가, 핵심 문장 하이라이트 · 강조, 제공 효과 정리

버전	변경한 사람	변경한 날짜	버전업 변경(또는 추가) 내용
0.8	Expresso 팀	2026-10-06	순위 지표가 평가하는 것을 나타낸 개념 그림(그림 9.1) 추가, 순위 지표 계산 예시를 그림 9.2로 이동
0.9	Expresso 팀	2026-10-13	사용자용 유스케이스 다이어그램(그림 2.3)의 연결선 교차 정리
1.0	Expresso 팀	2026-10-13	설계서 작성 완료

CONTENTS

목차

01. 프로젝트 개요 6

1.1 개발목표와 범위 6

1.2 개발일정/산출물 9

1.3 개발조직/역할 10

02. 요구사항 정의 12

2.1 서비스 구성 및 개요 12

2.2 데이터분석의 요구사항 17

2.3 AI학습서버의 요구사항 19

2.4 AI서비스 유스케이스 모델 21

2.5 요구기능 정의 (웹 사용자용) 22

2.6 요구기능 정의 (웹 관리자용) 30

03. 데이터 분석 및 전처리 33

3.1 데이터 수집출처 및 수집방법 35

3.2 데이터 특징추출 및 특징분포 39

3.3 데이터세트 정의 및 생성 40

04. AI학습모델의 설계 41

4.1 모델 파이프라인의 구성 41

4.2 AI학습 알고리즘의 선정 41

4.3 AI학습 파라메터 설정 44

4.4 AI학습 모델의 생성 45

05. 화면(UI) 설계 48

5.1 사용자 메뉴구성 48

5.2 초기화면(로그인 전화면) 설계 48

5.3 사용자 화면설계(웹) 50

5.4 관리자 메뉴구성 82

5.5 관리자 화면설계(웹) 82

06. 프로세스(기능) 설계 83

6.1 시스템 아키텍쳐 83

6.2 사용자 기능설계 (Sequence Diagram) 84

6.3 관리자 기능설계 (Sequence Diagram) 99

6.4 AI학습서버 기능설계 108

07. API 설계 110

7.1 서버/클라이언트 구조 110
7.2 서비스 REST API 정의 111
7.3 서비스 REST API 설계 121
7.4 AI학습서버 REST API 설계 163

08. 데이터 설계 166

8.1 데이터 모델 166
8.2 논리적 설계(컬렉션 명세서) 166
8.3 로그파일 설계 197
8.4 오류코드 및 오류파일 설계 198

09. 성능시험지표 200

9.1 AI서비스 성능평가지표 200
9.2 AI학습 성능 (ROC) 202

10. 개발환경 구성 210

10.1 개발환경 210
10.2 소스디렉터리 구조 212
10.3 명칭표준안 (변수, 함수, 파일명) 212

01 프로젝트 개요

양식 1장

1.1 개발목표와 범위

Expresso는 사용자가 쌓아 둔 커리어 기록을 지원할 채용 공고에 맞는 웹 포트폴리오로 만들어 배포하는 서비스이다. 아래 그림 1.1은 기록과 공고가 들어가 공고 맞춤 포트폴리오로 나오고, 방문 분석이 다음 포트폴리오의 개선으로 돌아오는 흐름을 한 장으로 나타낸 것이며, 아래 줄은 Expresso가 주는 세 가지 장점이다.



그림 1.1 Expresso 서비스 개요

■ 개발목표

본 설계서는 웹 환경에서 구직자를 위한 **채용 공고·커리어 기록 추천 모델 기반 맞춤형 포트폴리오 생성 및 배포 플랫폼 개발**을 위한 설계서이다. 주로 사용자는 이직과 취업을 준비하는 구직자이며, 데스크톱 웹을 통해서 접속하고 사용하기 편리하도록 고안되는 것을 전제로 한다. 본 서비스를 효과적으로 관리하기 위해 관리기능도 구성하였으며, 사용 편의성과 추천 품질을 높이도록 개발한다.

사용자는 자신의 경력·프로젝트·학력·자격·활동을 **커리어 기록**으로 축적하고, 자신에게 맞는 채용 공고를 탐색하여 지원할 공고를 고른다. 본 서비스는 그 공고의 요구사항을 분석하고, 공고에 담을 커리어 기록을 추천하며, AI 대화로 부족한 경험을 보완한 뒤 레시피 · 디자인 선택 · 다듬기를 거쳐 **공고 맞춤형 포트폴리오**를 생성한다. 생성된 포트폴리오는 공개 주소로 배포되며, 방문자 데이터를 분석하여 다음 판의 개선에 활용하도록 사용자에게 되돌려 준다. 본 서비스는 포트폴리오 생성에 외부의 프론티어 LLM을 이용하며, 채용 공고 추천과 커리어 기록 추천에는 본 프로젝트에서 개발하는 AI학습모델을 사용한다. 두 AI의 구분은 2.1에 기술한다.

아래 그림 1.2는 본 서비스의 전체적인 구성을 나타내며, 구직자 · 관리자 · 방문자 3종류의 사용자가 본 서비스를 사용한다. **점선 안쪽**이 본 프로젝트의 개발 및 설계범위이다.

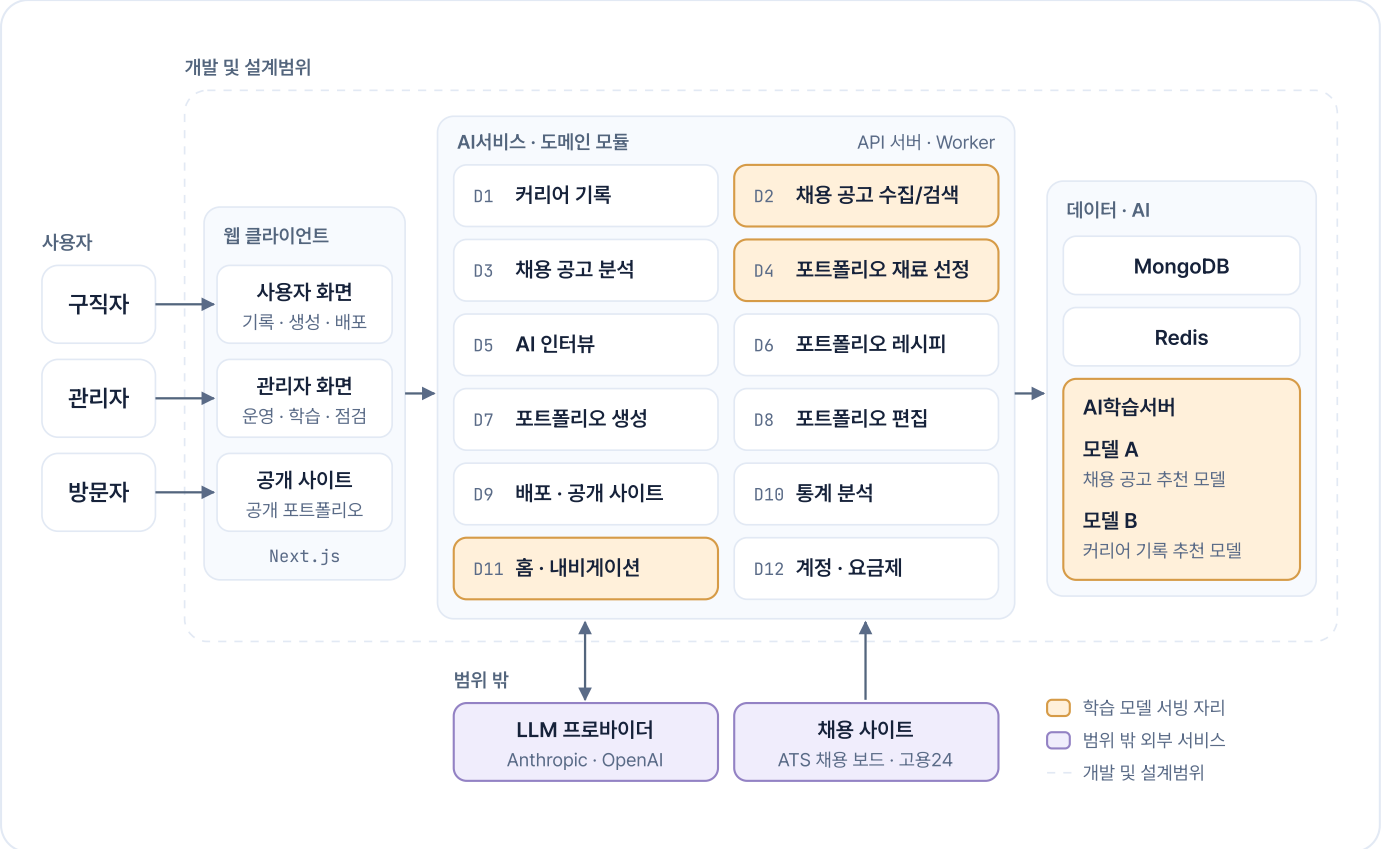


그림 1.2 서비스의 전체 구성과 설계범위

■ 개발필요성

구직자는 지원하는 회사가 바뀔 때마다 흩어진 경험과 성과를 다시 찾고, 공고에 맞는 경험을 골라내고, 포트폴리오를 새로 구성하는 **반복 작업**을 한다. 프로젝트 문서 · 발표 자료 · 활동 기록 · 저장소처럼 자료가 여러 곳에 흩어져 있어 지원을 준비할 때마다 자료 취합부터 다시 해야 하고, 같은 경력이라도 공고가 요구하는 역량에 따라 강조할 경험이 달라지므로 한 벌의 포트폴리오를 그대로 제출하기 어렵다.

기존 서비스는 이 과정의 **일부만 지원**한다. 아래 표는 유사 서비스의 제공 기능과 본 서비스의 차별화 방안이고, 그림 1.3은 각 서비스가 지원하는 영역을 겹쳐 나타낸 것이다.

서비스	제공 기능	결과물	EXPRESSO의 차별화 방안
원티드 에이전트	조건에 맞는 채용 공고 검색, 지원 공고에 맞춘 이력서 문구 수정 제안	추천 공고와 수정된 이력서	프로젝트 과정과 성과를 시각적으로 전달하는 포트폴리오 생성, 사용자가 직접 디자인하는 부담을 줄인 완성된 지면 제공
랠릿	정해진 테마로 경력 · 프로젝트 프로필 작성, 링크 공유와 공고 지원	공유 가능한 프로필 이력서	사용자 선호를 반영한 포트폴리오 디자인 생성, 지원 공고와 강조할 경험에 맞춘 레이아웃과 시각적 구성 조정

서비스	제공 기능	결과물	EXPRESSO의 차별화 방안
Rezi · Teal	공고에 맞춘 이력서 작성 · 수정 (Rezi), 공고 대비 포함 · 누락 키워드 확인(Teal)	공고별 맞춤 이력서	국내 채용 공고 데이터 연동을 통한 공고 추천과 경험 매칭, 국내 직무 · 경력 표현을 반영한 공고 분석과 맞춤 구성

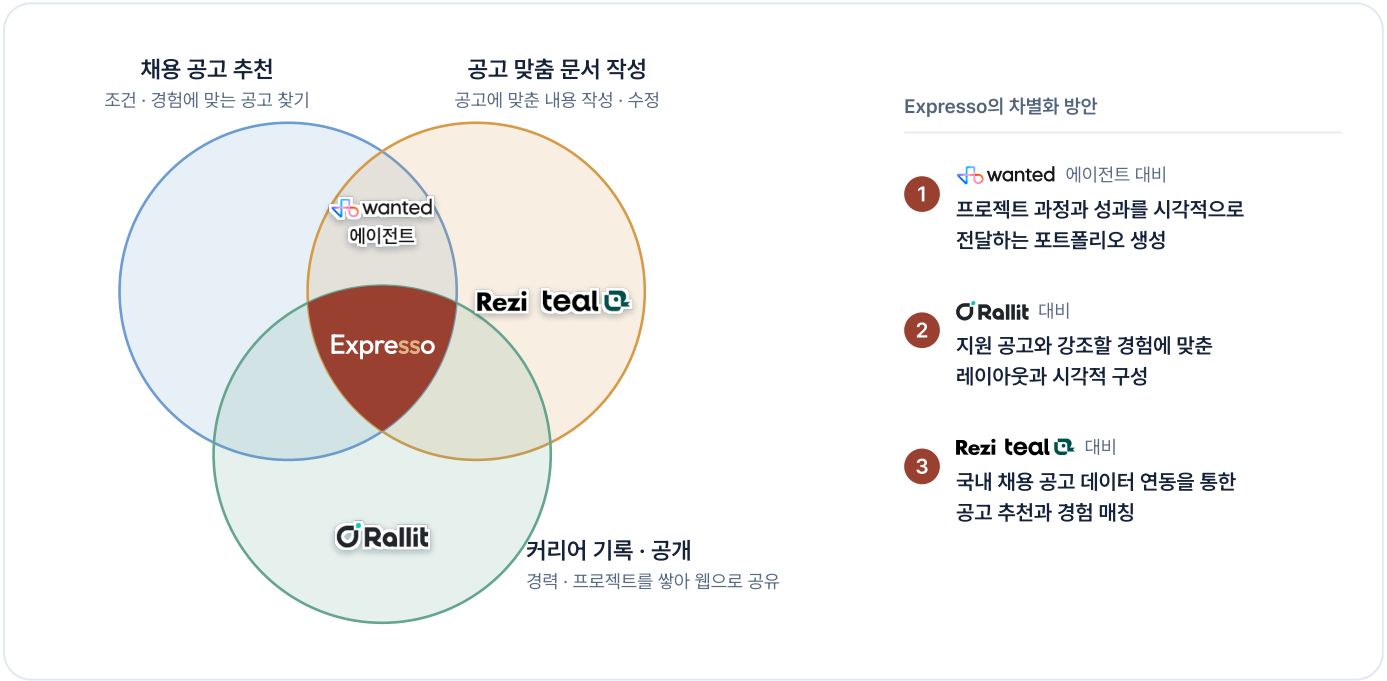


그림 1.3 기존 서비스와 Expresso의 지원 범위

본 서비스는 커리어 기록 축적 · 채용 공고 탐색 · 공고 맞춤형 포트폴리오 제작을 하나의 플랫폼으로 연결하고, 배포 후 방문자 데이터를 분석하여 다음 포트폴리오의 개선에 활용하게 한다. 이 흐름에서 **「이 사용자에게 어떤 공고가 적합한가」**를 판단하는 채용 공고 추천과 **「이 공고에 어떤 경험을 담아야 하는가」**를 판단하는 커리어 기록 추천을 AI학습모델로 구현하는 것이 본 프로젝트의 AI 개발 과제이다.

■ 개발범위

구분	포함	제외
클라이언트	웹 (Next.js)	네이티브 모바일 앱
서버	REST API · 비동기 Worker	—
AI 생성	공고 분석 · 레시피 · 포트폴리오 생성 · 편집 제안 **(외부 LLM API 이용)**	다중 후보 생성 후 선별
AI 학습	채용 공고 추천 모델(모델 A) · 커리어 기록 추천 모델(모델 B)	자체 LLM 사전학습 · 생성 모델 파인튜닝
배포	공개 사이트 배포 · 버전 · 롤백 · 내보내기	커스텀 도메인 자동 발급

구분	포함	제외
분석	방문 지표 수집 · 대시보드 · 해설	외부 광고 플랫폼 연동

1.2 개발일정/산출물

본 프로젝트는 아래 그림 1.4와 같이 **15주** 일정에 따라 진행하며, 각 단계별로 산출물을 개발한다. 주차는 학기 주차이며 제안서 발표가 2주차이다. 서비스 구현은 2026-08-06에 먼저 착수하였고, 설계와 구현을 병행하여 진행한다. AI 모델의 데이터 구축과 학습은 **4주차부터 10주차까지** 서비스 기능 개발과 함께 진행한다.

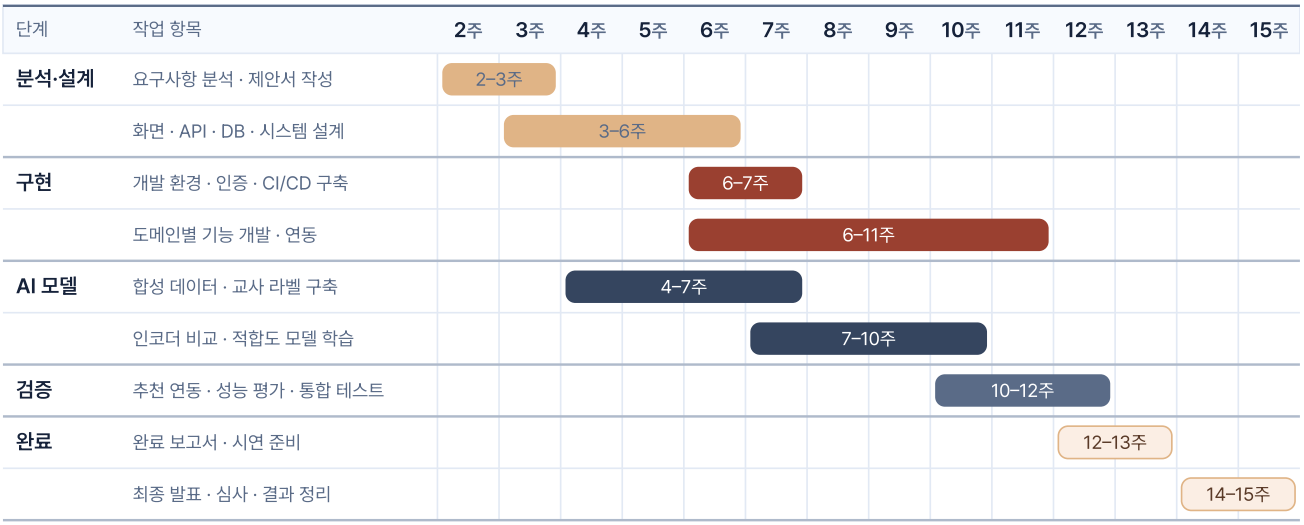


그림 1.4 개발일정 및 산출물

단계	산출물	현재 상태
요구분석	기능 명세서 (12 도메인 · 32 주요기능 · 151 태스크)	작성 완료
설계	요구규격 및 설계서 · 화면 정의서 (29 화면)	본 문서 · 작성 중
구현	서비스 서버 — HTTP 경로 정의 163개 · Worker 작업 유형 11개	구현 중
	웹 클라이언트 — page.tsx 27개	구현 중
	데이터베이스 — MongoDB 컬렉션 접근자 82개	구현 중
	AI학습서버 — 학습 모델 2	미착수

단계	산출물	현재 상태
시험	시험성적서 · 완료보고서 · 매뉴얼	미착수

■ 단계별 제품 목표

개발은 아래 **네 단계의 마일스톤**으로 나누어 진행한다. 각 단계는 사용자가 쓸 수 있는 서비스 하나를 목표로 하며, 앞 단계의 서비스 위에 다음 단계의 기능을 더한다.

단계	제품 목표	구현 범위
M1	**커리어 기록과 공고 탐색** 내 경험을 저장하고 지원할 공고를 찾는 서비스	회원가입 · 로그인 구현 · 커리어 기록 작성 · 수정 · 분류 · 채용 공고 수집 · 검색 · 필터 · 공고 상세 · 지원 요건 제공
M2	**공고 맞춤 포트폴리오 제작** 선택한 공고에 맞는 포트폴리오 제작 서비스	공고 분석 · 관련 기록 선택 · AI 인터뷰로 경험 보완 · 레시피 구성 · 디자인 선택 · 포트폴리오 생성 · 편집
M3	**자체 학습 모델 기반 추천** 내 경험에 맞는 공고와 지원에 쓸 기록 추천 서비스	학습 데이터 · 교사 라벨 구축 · 임베딩 비교 · 적합도 모델 학습 · 공고 · 커리어 기록 추천 연동 · 추천 품질 · 응답 시간 평가
M4	**포트폴리오 배포와 방문 분석** 포트폴리오 공유와 방문자 데이터 분석 서비스	공개 URL 배포 · 수정본 재배포 · 포트폴리오 버전 관리 · 방문 수집 · 분석 대시보드 구현 · 전체 흐름 통합 테스트

M3의 학습 모델 설계는 3장과 4장, 평가 방법은 9장에 기술한다.

1.3 개발조직/역할

본 프로젝트의 개발조직과 역할은 아래 표와 같다. 2장의 기능 표(2.3 · 2.5 · 2.6)에 있는 **개발담당** 칸은 이 표의 이름과 담당 도메인을 따른다.

역할	이름	역할(개발기능)	비고
팀장	박민재	PM · AI · 프론트 — 포트폴리오 생성 · 편집 · 공개 사이트 배포 · 프로젝트 일정 · 업무 조율 · 시스템 구조 · UI/UX 설계	D4 · D6 · D7 · D8 · D9 · D11
팀원	심민규	백엔드 — 채용 공고 수집 · 정규화 · 검색 · 필터 · 공고 탐색 · CI/CD 파이프라인 구축 · 서버 운영 · 모니터링	D2 · 관리자 수집 · 운영 기능

역할	이름	역할(개발기능)	비고
팀원	오시건	백엔드 — 커리어 기록 작성 · 편집 · 분류 · 기록 검색 · 첨부 관리 · 계정 · 인증 기능 개발 · DB 스키마 · 마이그레이션	D1 · 관리자 인증
팀원	채서현	백엔드 · AI — 공고 요건 · 역량 추출 · 인터뷰 질문 · 응답 처리 · 학습 데이터 · 라벨 구축 · 추천 모델 학습 · 평가	D3 · D5 · AI학습서버
팀원	윤규현	데이터 · AI — AI 대화 · 맥락 관리 · 대화 기반 기록 연동 · 사용량 · 요금제 관리 · 방문자 통계 · 분석	D10 · D12

도메인 번호는 2.5의 D1–D12이다. D4 재료 선정과 D11 홈 · 내비게이션은 포트폴리오 생성과 UI/UX 설계를 맡은 팀장이 담당한다.

02 요구사항 정의

양식 2장

2.1 서비스 구성 및 개요

■ 서비스 개요

본 서비스는 구직자가 지원하려는 채용 공고에 맞추어 포트폴리오를 생성하고 공개 주소로 배포하는 웹 서비스이다. 사용자는 경력 · 프로젝트 · 학력 · 자격 · 활동을 구조화된 **커리어 기록**으로 축적하고, 추천된 채용 공고를 탐색하여 지원할 공고를 고른다. 본 서비스는 고른 공고에서 요구 역량을 추출하고, 그 공고에 담을 커리어 기록을 추천하며, 사용자가 고른 기록으로 공고 맞춤형 포트폴리오를 생성한다. 생성된 포트폴리오는 하위 도메인 주소로 배포하며, 방문 지표를 수집하여 사용자에게 제공한다.

본 서비스가 제공하는 기능은 아래 표와 같이 **6종**이다. 각 기능의 상세기능은 2.5에 정의하며, 기능을 수행하는 화면은 5장에 설계한다.

제공 기능	내용	주요 화면
커리어 기록 관리	경력 · 프로젝트 · 학력 · 자격 · 활동 · 스킬을 카테고리별로 축적하고, 기록에서 스킬을 추출한다	내 커리어(05)
채용 공고 수집과 탐색	채용 채널에서 공고를 수집하여 정규화하고, 조건 검색과 추천으로 지원할 공고를 찾는다	공고 탐색(06) · 홈(00)
채용 공고 분석	공고에서 요구 역량을 추출하고, 사용자의 기록으로 충족되는 요건과 부족한 요건을 판정한다	공고 분석(01) · 공고 상세(06b)
포트폴리오 생성	담을 기록을 선정하고, 부족한 내용을 AI 인터뷰로 보완한 뒤, 구성을 확정하여 지면을 생성한다	재료 고르기(01b) · AI 대화(02) · 레시피(02b) · 디자인 선택(03)
편집과 배포	생성 결과를 수정하고, 하위 도메인 주소로 공개하며 버전을 관리한다	다듬기(04) · 배포(08b) · 공개 사이트(08)
방문 분석	공개 사이트의 방문과 체류를 수집하여 지표로 제공하고, 수정 대상을 제안한다	분석(07)
6종의 기능은 12개 도메인 · 32개 주요기능 · 151개 상세기능으로 전개한다.		

■ 이용 절차

사용자가 본 서비스를 이용하는 절차는 아래 그림 2.1 · 표와 같다. 1단계는 최초 1회 수행하며, 2단계부터 10단계까지는 지원하는 공고마다 반복한다. 그림 2.1에서 **주황색 단계**는 본 프로젝트의 학습 모델이, **보라색 단계**는 외부 LLM이 쓰이는 단계이다.

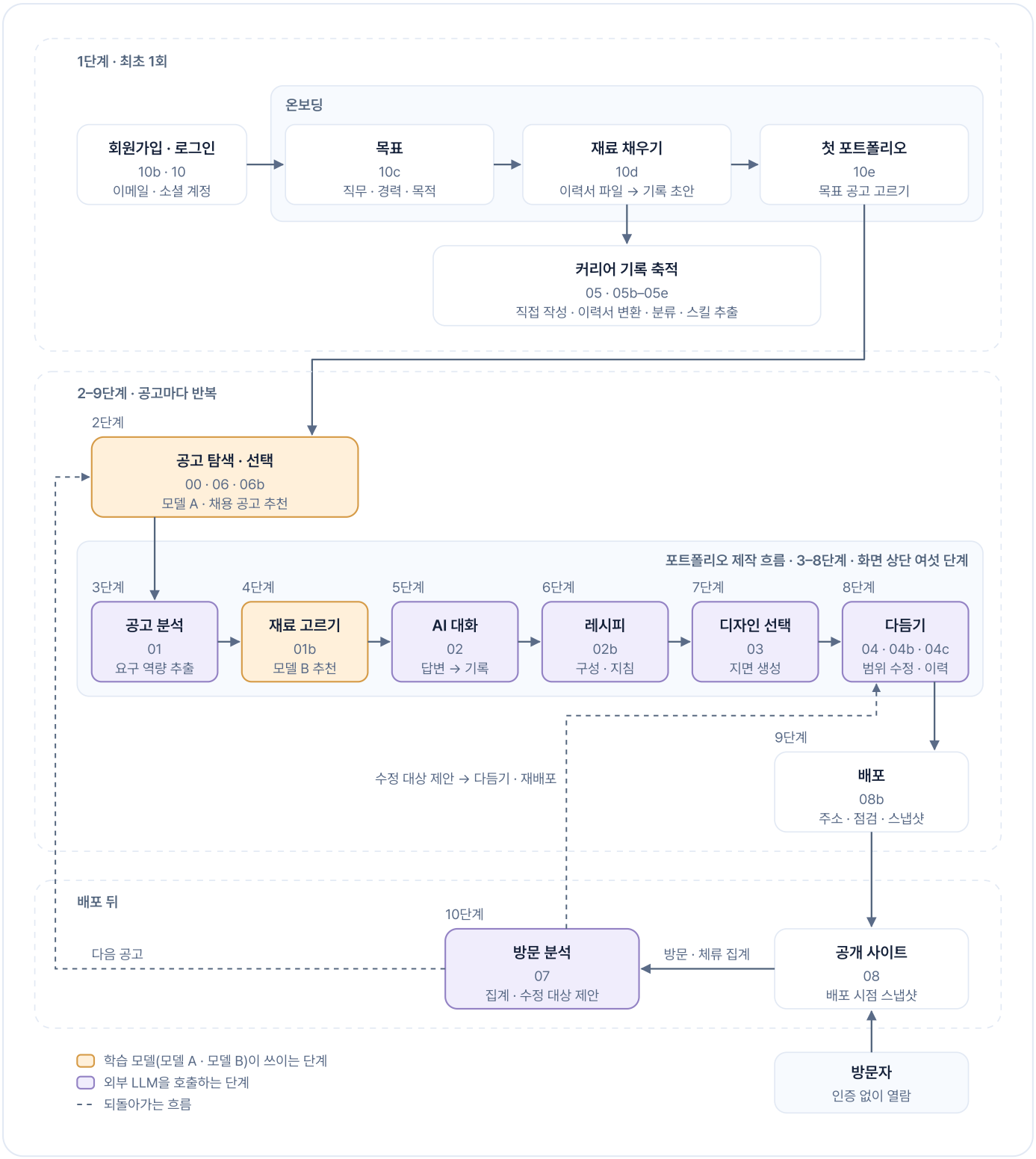


그림 2.1 전체 사용자 플로우

단계	사용자가 하는 일	서비스가 하는 일	화면
1	커리어 기록을 작성한다. 이력서 파일을 올려 기록으로 변환할 수 있다	기록을 카테고리로 분류하고 스킬을 추출한다	05 · 10d

단계	사용자가 하는 일	서비스가 하는 일	화면
2	지원할 채용 공고를 고른다	수집한 공고를 사용자의 기록과 대조하여 순위로 제시한다	00 · 06 · 06b
3	공고 분석을 요청한다	요구 역량을 추출하고 요건별 충족 여부를 판정한다	01
4	포트폴리오에 담을 기록을 확정한다	공고와 관련된 기록을 미리 선택하고 제외 사유를 제시한다	01b
5	AI가 묻는 질문에 답한다	부족한 요건을 질문으로 만들고, 답변을 커리어 기록으로 저장한다	02
6	포트폴리오의 구성을 확인하고 수정한다	지면 순서와 지면별 작성 지침을 근거와 함께 제시한다	02b
7	디자인을 고르고 생성을 실행한다	지침과 고른 디자인에 따라 지면을 생성한다	03
8	생성 결과를 다듬는다	지시한 범위만 수정하고 변경 이력을 남긴다	04
9	주소를 정하고 배포한다	배포 전 점검을 수행하고 그 시점의 문서를 스냅샷으로 고정한다	08b · 08
10	방문 지표를 확인한다	방문과 체류를 집계하고 수정 대상을 제안한다	07

3단계부터 8단계까지가 포트폴리오 제작 흐름이며, 화면 상단에 여섯 단계로 표시한다.

■ 제공 효과

본 서비스가 사용자에게 주는 효과는 그림 1.1 아래의 세 가지 장점과 같다.

단계	효과	제공 방식
지원 준비	커리어 기록 재사용	지원할 공고를 고르면 공고의 요구 역량을 추출하고, 커리어 기록 추천 모델이 공고에 맞는 기록을 골라 준다. 공고마다 자료를 다시 모으지 않고 쌓아 둔 기록에서 포트폴리오를 구성한다
경험 정리	AI 인터뷰로 기록 보완	기록으로 채워지지 않은 공고 요건을 AI가 질문으로 만들고, 답변을 커리어 기록으로 저장한다. 저장한 기록은 다음 지원에서 다시 쓴다
배포 이후	방문 분석 기반 개선	공개 사이트의 방문 · 체류 · 연락처 클릭을 수집하여 섹션별로 보여 주고, 다음 수정의 근거로 제공한다

시스템 구성

본 서비스의 구성은 아래 그림 2.2와 같다. 웹 · API · Worker · MongoDB · Redis로 이루어진 **AI서비스**와, 채용 공고 추천 모델과 커리어 기록 추천 모델을 학습 · 검증 · 서빙하는 **AI학습서버**로 나뉜다.



그림 2.2 서비스 시나리오 구성도

그림 2.2를 보면 사용자는 웹 클라이언트를 통해 AI서비스의 REST API를 호출한다. 공고 분석과 포트폴리오 생성처럼 처리에 시간이 걸리는 요청은 API 서버가 큐에 넣고 즉시 접수 응답을 돌려주며, Worker가 이를 꺼내어 LLM 프로바이더를 호출하고 그 결과를 MongoDB에 기록한다. 사용자는 완료된 결과를 다시 조회하여 확인한다.

AI학습서버는 수집 공고와 합성 프로필을 원천으로 두 추천 모델을 학습하고 서비스에 배포하도록 설계한다. **모델 A**는 공고의 순위를, **모델 B**는 해당 공고에 쓸 커리어 기록의 순위를 산출한다.

구성요소	역할	현재 구현
웹 클라이언트	기록 작성 · 공고 탐색 · 생성 요청 · 편집 · 배포 · 분석	Next.js 16, page.tsx 27개
API 서버	HTTP 경로 정의 163개 · 인증 · 권한 · 검증 · 낙관적 잠금	Fastify 5
Worker	공고 분석 · 생성 · 내보내기 · 집계 · 알림 · 정리	BullMQ, 작업 유형 11개
MongoDB	도메인 문서 · 컬렉션 접근자 82개	MongoDB 8.0
Redis	작업 큐 · 캐시	Redis 8.2

구성요소	역할	현재 구현
LLM 프로바이더	공고 해석 · 레시피 · 문장 생성	Claude Code / Codex / fixture
AI학습서버	모델 A·B 교사 라벨링 · 학습 · 검증 · 배포	미구현

AI서비스와 AI학습서버를 분리하는 이유

축	AI서비스	AI학습서버
호출 시점	사용자 요청마다	배치(주기 또는 수동)
응답 시간 요구	초 단위	분~시간 단위
상태	무상태	데이터세트 · 모델 아티팩트 보유
실패 영향	해당 요청만	다음 배포까지 기존 모델 유지
확장 방식	수평 확장	단일 노드

다섯 축이 모두 다르므로 한 서버에 두면 어느 한쪽의 요구를 만족시킬 수 없다.

사용 AI의 구분

본 서비스는 아래 표와 같이 **2종의 AI**를 사용한다. 하나는 포트폴리오를 생성하는 AI이고, 다른 하나는 채용 공고와 사용자 커리어 기록의 관련성을 판정하는 AI이다. 포트폴리오를 생성하는 AI는 외부의 프론티어 LLM을 API로 이용하고, 관련성을 판정하는 AI는 본 프로젝트에서 직접 개발한다. 그림 2.2에서 포트폴리오 문장 생성은 외부 LLM이 담당한다.

구분	담당 기능	개발 방식	서비스 연동
포트폴리오 생성	공고 분석 · 인터뷰 질문과 답변 정리 · 레시피 구성 · 포트폴리오 지면 생성 · 편집 제안 · 방문 해설	외부 LLM API 이용	Worker가 프로바이더 API를 호출
관련성 판정	공고 추천 순위 산출 · 요건 충족 판정	자체 개발 (채용 공고 추천 모델 · 커리어 기록 추천 모델)	AI학습서버가 배포한 모델을 API 서버가 호출

본 프로젝트에서 개발하는 AI 모델은 관련성 판정 모델이며, 생성 모델의 사전학습과 파인튜닝은 개발범위에서 제외한다.

관련성 판정을 직접 개발하는 근거는 다음과 같다. 프론티어 LLM은 응답에 수 초가 걸리고 호출마다 비용이 과금된다. 공고 목록 화면은 한 번에 **수백 건**을 정렬하므로, 건별로 외부 API를 호출하는 방식으로는 응답 시간과 비용 요구를 만족할 수 없다. 자체 학습 모델은 즉시 응답하고 **호출 비용이 없다**. 프론티어 LLM은 학습 라벨을 만드는 **교사**로만 사용한다(3.1).

■ 개발하는 AI학습모델

본 프로젝트에서 개발하는 AI학습모델은 아래 표와 같이 2종이다. 두 모델은 채용 공고와 커리어 기록의 적합도를 서로 반대 방향으로 판정한다.

모델	판정 방향	판정 내용	입력 데이터	출력	사용 화면
모델 A 채용 공고 추천 모델	기록 → 공고	사용자에게 적합한 공고의 순위	사용자의 기록 전체와 후보 공고 목록	공고별 적합도 점수(0–100)와 추천 순위	홈(00) · 공고 탐색(06) · 공고 AI 검색 결과(00b)
모델 B 커리어 기록 추천 모델	공고 → 기록	공고에 담을 기록과 선택 조합	공고 요건과 사용자의 커리어 기록 목록	기록별 적합도 점수와 추천 선택 조합	재료 고르기(01b)
모델 A는 Bi-Encoder+MLP로 전체 공고를 선별하고 Cross-Encoder로 상위 후보를 재정렬한다. 모델 B는 모델 A의 Cross-Encoder를 재사용한다.					

두 모델의 상세기능은 2.3에, 학습 데이터는 3장에, 모델 구조와 학습 · 배포 절차는 4장에, 성능 지표와 시험 방법은 9장에 각각 기술한다.

2.2 데이터분석의 요구사항

본 프로젝트의 AI 학습은 **사용자 기록과 채용 공고를 잇는 적합도 판정**이다. 모델 A는 사용자의 기록을 보고 그에 맞는 공고를 찾고, 모델 B는 공고를 보고 그 공고에 담을 기록을 찾는다. 두 모델은 같은 교사 판정에서 데이터를 얻으므로 데이터세트는 한 벌이다.

구분	모델 A — 기록에서 공고로	모델 B — 공고에서 기록으로
답하는 질문	이 사용자에게 어떤 공고가 적합한가	이 공고에 어떤 경험을 담아야 하는가
판단 기준	경험 · 역량의 관련성, 역할 · 연차	기록의 관련성, 요건 범위, 경험의 중복
관측 단위	(사용자 프로필 1건, 채용 공고 1건)	(채용 공고 1건, 커리어 기록 1건)
학습 데이터	프로필–공고 10,000쌍과 교사 적합도 점수	별도 학습 없음 — 모델 A의 Cross-Encoder 재사용

구분	모델 A — 기록에서 공고로	모델 B — 공고에서 기록으로
평가 정답	교사 적합도 점수	교사가 요구사항별로 연결한 근거 기록과 충족 수준
쓰이는 자리	공고 목록과 추천 화면의 정렬	재료 고르기의 기록 정렬과 자동 선택

교사는 프로필–공고 쌍을 판정할 때 요구사항마다 근거 기록 ID를 남긴다. 모델 B는 이 기록 단위 판정으로 평가한다.

■ 원천 데이터

학습에 사용할 데이터는 아래 **네 종류의 원천**에서 얻는다.

원천	저장 위치 · 출처	내용	수집 방법
채용 공고	`job_postings`	직무 · 주요 업무 · 필수 자격 · 우대 요건	출처별 어댑터 수집 (3.1)
공고 요구사항	`job_posting_requirements`	공고에서 추출한 요건 문장과 종류 · 원문 위치	공고 분석(LLM) 결과
프로필 골격	한국고용정보원 청년패널조사 YP2021	학력 · 직업 · 경력의 집계 분포	공개 자료 이용
경험 서술	AI Hub 채용면접 인터뷰(71592)	실제 경험의 역할 · 행동 · 성과	데이터셋 신청

공개 데이터에서 다수 사용자의 상세 커리어 기록을 확보하기 어려우므로, 프로필 골격과 경험 서술 두 원천을 근거로 프로필과 기록을 합성한다(3.1).

■ 데이터가 갖추어야 할 특성

- [ ] 합성 기록은 **실제 자료에 근거**하여야 한다. 사건 · 연차 · 수치는 원천 자료의 것을 보존하고, 상황 · 작업 과정 · 회고만 생성한다.
- [ ] 라벨은 **고정된 루브릭**으로 산출한다. 교사는 근거 해석과 판정만 수행하고, 가중 점수 계산과 오류 검사는 코드가 수행한다.
- [ ] **직군이 편중되지 않아야** 한다. 백엔드 공고만으로 학습하면 디자인 직군에서 정확도가 크게 떨어진다.
- [ ] **실제 개인의 식별 정보**는 학습 데이터에 포함하지 않는다. 원천 자료는 집계 분포와 인터뷰 서술만 사용한다.
- [ ] **한국어와 영어**가 함께 나타난다. 인코더가 두 언어를 모두 다루어야 한다.

- [ ] 같은 프로필의 쌍은 학습과 평가에 나뉘어 들어가지 않아야 한다. 분할은 **프로필 단위**로 한다.

2.3 AI학습서버의 요구사항

AI학습서버에서 개발하여야 할 기능을 아래 표와 같이 정의한다. 두 모델은 같은 서버에서 같은 데이터세트로 학습하고 배포하며, 모델 버전은 학습 단위별로 분리한다.

번호	주요모듈	상세기능	개발담당
1	교사 라벨링 모듈	프로필–공고 쌍 구성 기능	채서현
2	교사 라벨링 모듈	LLM 교사 배치 호출 기능	채서현
3	교사 라벨링 모듈	루브릭 · 교사 프롬프트 버전 관리 기능	채서현
4	교사 라벨링 모듈	요구사항별 근거 판정 기록 기능	채서현
5	교사 라벨링 모듈	가중 점수 계산 · 오류 검사 기능	채서현
6	합성 데이터 모듈	프로필 골격 표본 추출 기능	채서현
7	합성 데이터 모듈	조건부 커리어 기록 합성 기능	채서현
8	합성 데이터 모듈	합성 기록 검증 기능	채서현
9	데이터세트 모듈	임베딩 생성 · 캐시 기능	채서현
10	데이터세트 모듈	프로필 단위 분할 기능	채서현
11	데이터세트 모듈	데이터세트 버전 기록 기능	채서현
12	데이터세트 모듈	사람 평가 표본 추출 기능	채서현
13	학습 모듈	Bi-Encoder+MLP 학습 기능	채서현
14	학습 모듈	Cross-Encoder 부분 미세조정 기능	채서현
15	학습 모듈	인코더 비교 실험 기능	채서현
16	학습 모듈	학습 중단 · 재개 기능	채서현

번호	주요모듈	상세기능	개발담당
17	학습 모듈	재학습 기능	채서현
18	검증 모듈	평가 분할 성능 측정 기능	채서현
19	검증 모듈	ROC · AUC 산출 기능	채서현
20	검증 모듈	순위 지표(NDCG@10 · Recall@K) 산출 기능	채서현
21	검증 모듈	점수 지표(MAE · 공고쌍 순서 일치율) 산출 기능	채서현
22	검증 모듈	규칙 baseline 비교 기능	채서현
23	검증 모듈	배포 승인 판정 기능	채서현
24	배포 모듈	모델 아티팩트 배포 기능	채서현
25	배포 모듈	모델별 배포 버전 선택 기능	채서현
26	배포 모듈	이전 버전 롤백 기능	채서현
27	관리 모듈	모델 버전관리 기능	채서현
28	관리 모듈	학습 파라미터 수정 기능	채서현
29	관리 모듈	모델 삭제 기능	채서현
30	인터페이스 모듈	공고 추천 요청 처리 (모델 A)	채서현
31	인터페이스 모듈	기록 추천 요청 처리 (모델 B)	채서현
32	인터페이스 모듈	모델 미배포 시 규칙 기반 폴백 기능	채서현
33	인터페이스 모듈	추천 근거 기록 반환 기능	채서현
34	장애기록 모듈	학습 장애 기록 기능	채서현
35	장애기록 모듈	장애기록 정보 제공 기능	채서현

2.4 AI서비스 유스케이스 모델

본 서비스의 액터는 다섯이다. 서비스를 사용하는 **구직자**와 배포된 포트폴리오를 열람하는 **방문자**가 사람 액터이고, **AI서버 · AI학습서버 · 데이터수집기**가 시스템 액터이다. 아래 그림 2.3은 구직자와 방문자의 유스케이스를 나타낸다.

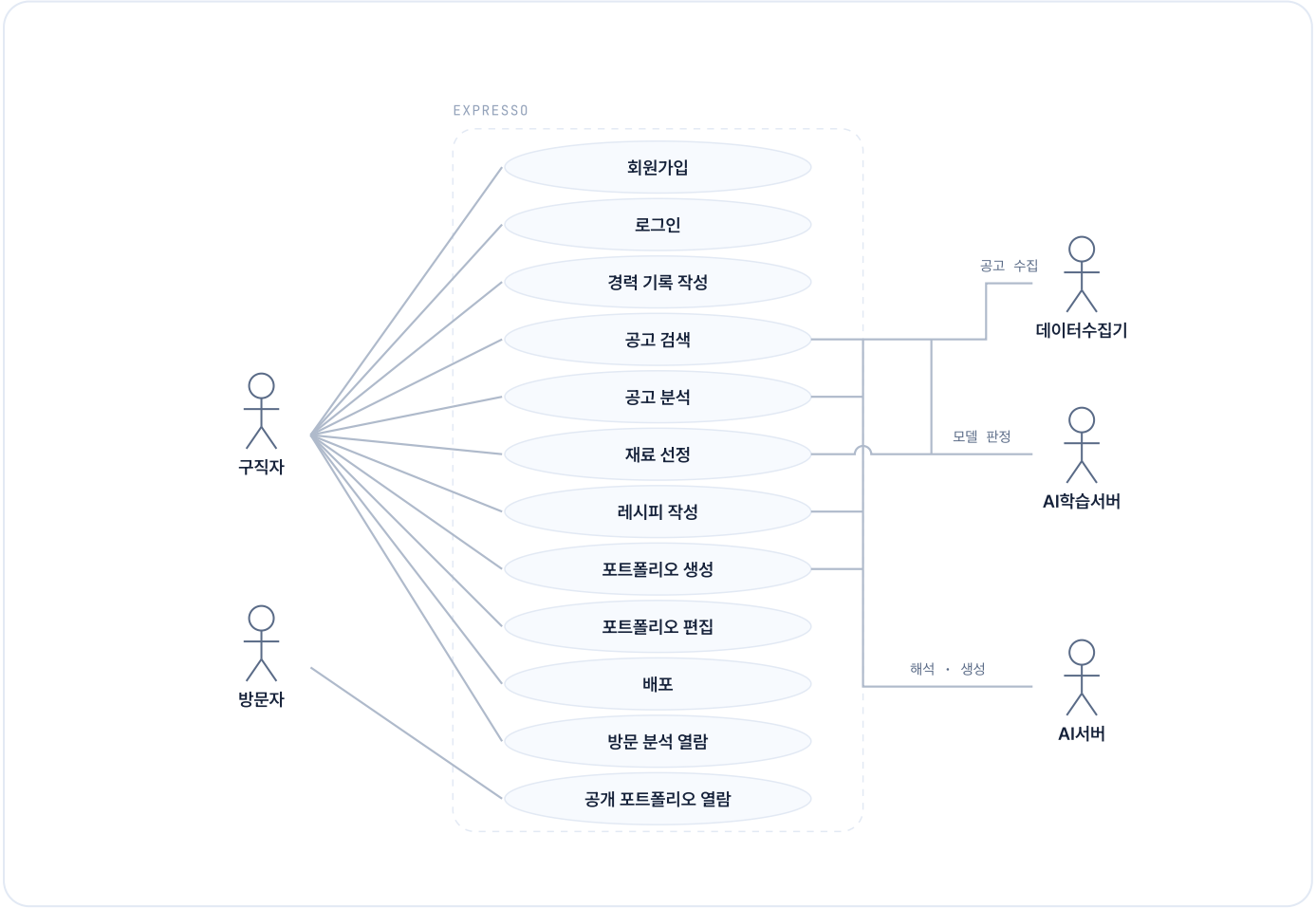


그림 2.3 사용자용 유스케이스 다이어그램

그림 2.3을 보면 구직자는 회원가입과 로그인을 거쳐 경력 기록을 작성하고, 공고를 검색하여 분석을 요청한 뒤 포트폴리오를 생성하고 배포한다. 공고 분석과 포트폴리오 생성은 AI서버가 LLM 프로바이더를 호출하여 처리하며, 공고 추천과 기록 추천은 AI학습서버가 배포할 모델을 사용한다. 데이터수집기는 채용 사이트에서 공고를 수집하여 공고 검색의 대상을 채운다. 방문자는 배포된 포트폴리오를 열람하기만 하며, 그 열람 기록이 분석 지표로 쌓인다.

아래 그림 2.4는 관리자의 유스케이스를 나타낸다.

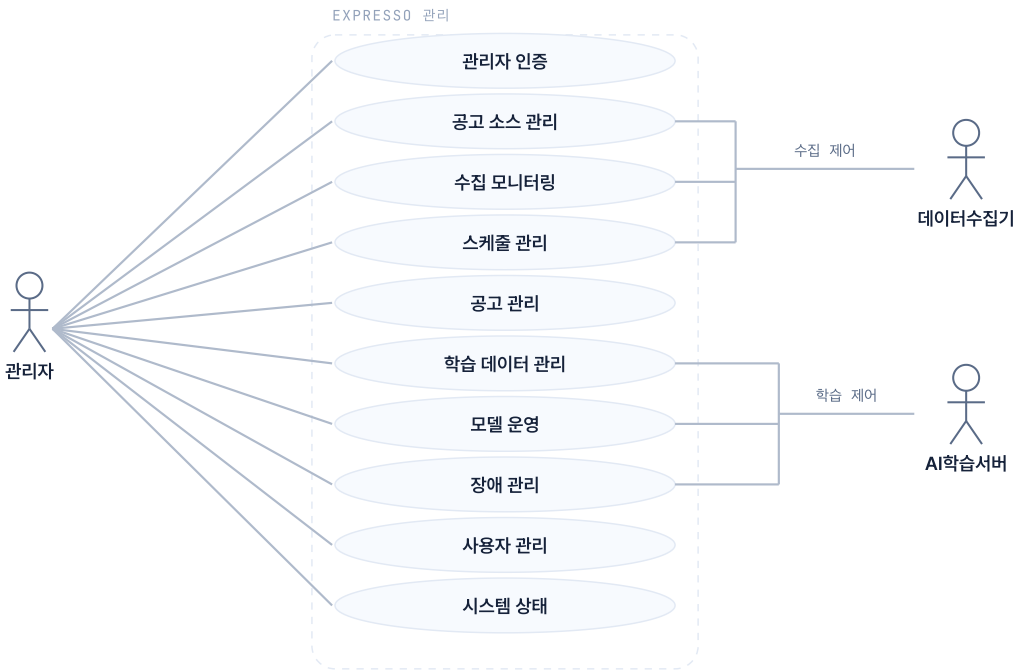


그림 2.4 관리자용 유스케이스 다이어그램

그림 2.4를 보면 관리자는 수집 소스를 등록하고 수집을 실행하며, 수집 실행 이력과 스케줄을 관리한다. 또한 학습 데이터세트를 생성하고 라벨을 검수한 뒤 모델 학습을 실행하며, 성능 지표를 확인하고 모델을 배포하거나 이전 버전으로 되돌린다. 학습 중 발생한 장애는 장애기록으로 조회한다. 관리자의 유스케이스 가운데 수집 계열은 데이터수집기를, 학습 계열은 AI학습서버를 각각 호출한다.

2.5 요구기능 정의 (웹 사용자용)

2.4에서 그린 유스케이스 모델의 주요기능을 좀 더 상세하게 식별하여 아래 표와 같이 정의한다. 기능 명세서에 정의된 12개 도메인 · 32개 주요기능 · **151개 상세기능**이며, 도메인(D) · 주요기능(F) · 상세기능(T) 번호는 이슈 트래커의 에픽 · 스토리 · 태스크와 **1대 1로 대응**한다.

NO	도메인	주요기능명	상세기능명	담당
1	D1 커리어 기록	F1.1 카테고리 관리	`T1.1.1` 기본 카테고리 제공	오시건
2	D1 커리어 기록	F1.1 카테고리 관리	`T1.1.2` 사용자 카테고리 추가	오시건
3	D1 커리어 기록	F1.1 카테고리 관리	`T1.1.3` 카테고리별 속성 스키마	오시건
4	D1 커리어 기록	F1.1 카테고리 관리	`T1.1.4` 트리에서 바로 추가	오시건

NO	도메인	주요기능명	상세기능명	담당
5	D1 커리어 기록	F1.2 기록 생성	T1.2.1 직접 만들기	오시건
6	D1 커리어 기록	F1.2 기록 생성	T1.2.2 AI로 만들기	오시건
7	D1 커리어 기록	F1.2 기록 생성	T1.2.3 대화에서 자동 생성	오시건
8	D1 커리어 기록	F1.2 기록 생성	T1.2.4 외부에서 가져오기	오시건
9	D1 커리어 기록	F1.2 기록 생성	T1.2.5 상태 자동 판정	오시건
10	D1 커리어 기록	F1.3 기록 문서 편집	T1.3.1 문서 패널 열기	오시건
11	D1 커리어 기록	F1.3 기록 문서 편집	T1.3.2 속성 편집	오시건
12	D1 커리어 기록	F1.3 기록 문서 편집	T1.3.3 AI 정리 블록	오시건
13	D1 커리어 기록	F1.3 기록 문서 편집	T1.3.4 문서 단위 AI 요청	오시건
14	D1 커리어 기록	F1.3 기록 문서 편집	T1.3.5 삭제와 영향 범위	오시건
15	D1 커리어 기록	F1.4 뷰 · 필터 · 정렬	T1.4.1 뷰 전환	오시건
16	D1 커리어 기록	F1.4 뷰 · 필터 · 정렬	T1.4.2 뷰 저장	오시건
17	D1 커리어 기록	F1.4 뷰 · 필터 · 정렬	T1.4.3 필터 · 정렬 · 표시 속성	오시건
18	D1 커리어 기록	F1.4 뷰 · 필터 · 정렬	T1.4.4 갤러리 카드 시각화	오시건
19	D1 커리어 기록	F1.4 뷰 · 필터 · 정렬	T1.4.5 목록 집계	오시건
20	D1 커리어 기록	F1.5 기록 간 연결과 사용처 추적	T1.5.1 기록 연결	오시건
21	D1 커리어 기록	F1.5 기록 간 연결과 사용처 추적	T1.5.2 사용처 표시	오시건
22	D1 커리어 기록	F1.5 기록 간 연결과 사용처 추적	T1.5.3 인용 문단 추적	오시건
23	D1 커리어 기록	F1.5 기록 간 연결과 사용처 추적	T1.5.4 중복·병합 제안	오시건

NO	도메인	주요기능명	상세기능명	담당
24	D1 커리어 기록	F1.6 스킬 자동 집계와 수요 갭	T1.6.1 기록에서 스킬 추출	오시건
25	D1 커리어 기록	F1.6 스킬 자동 집계와 수요 갭	T1.6.2 숙련도 산출	오시건
26	D1 커리어 기록	F1.6 스킬 자동 집계와 수요 갭	T1.6.3 근거 열람	오시건
27	D1 커리어 기록	F1.6 스킬 자동 집계와 수요 갭	T1.6.4 공고 수요 비교	오시건
28	D1 커리어 기록	F1.6 스킬 자동 집계와 수요 갭	T1.6.5 숫자 연결 제안	오시건
29	D2 채용 공고 수집/검색	F2.1 공고 수집과 정규화	T2.1.1 정기 수집	심민규
30	D2 채용 공고 수집/검색	F2.1 공고 수집과 정규화	T2.1.2 사용자 직접 등록	심민규
31	D2 채용 공고 수집/검색	F2.1 공고 수집과 정규화	T2.1.3 필드 정규화	심민규
32	D2 채용 공고 수집/검색	F2.1 공고 수집과 정규화	T2.1.4 중복 제거와 만료	심민규
33	D2 채용 공고 수집/검색	F2.1 공고 수집과 정규화	T2.1.5 신규 표시	심민규
34	D2 채용 공고 수집/검색	F2.2 자연어 검색과 조건 해석	T2.2.1 문장 → 조건 변환	심민규
35	D2 채용 공고 수집/검색	F2.2 자연어 검색과 조건 해석	T2.2.2 조건 수정과 재검색	심민규
36	D2 채용 공고 수집/검색	F2.2 자연어 검색과 조건 해석	T2.2.3 결과 0건 대응	심민규
37	D2 채용 공고 수집/검색	F2.2 자연어 검색과 조건 해석	T2.2.4 검색 저장과 알림	심민규
38	D2 채용 공고 수집/검색	F2.2 자연어 검색과 조건 해석	T2.2.5 최근 검색 · 추천 질의	심민규
39	D2 채용 공고 수집/검색	F2.3 일치도 산출과 설명	T2.3.1 점수 계산	심민규
40	D2 채용 공고 수집/검색	F2.3 일치도 산출과 설명	T2.3.3 축별 충족 내역	심민규
41	D2 채용 공고 수집/검색	F2.3 일치도 산출과 설명	T2.3.4 낮은 점수 접기	심민규
42	D2 채용 공고 수집/검색	F2.3 일치도 산출과 설명	T2.3.5 결과 집합 요약	심민규

NO	도메인	주요기능명	상세기능명	담당
43	D2 채용 공고 수집/검색	F2.4 공고 상세 열람과 요건 대조	T2.4.1 원문 렌더링	심민규
44	D2 채용 공고 수집/검색	F2.4 공고 상세 열람과 요건 대조	T2.4.2 요건별 충족 판정	심민규
45	D2 채용 공고 수집/검색	F2.4 공고 상세 열람과 요건 대조	T2.4.3 스택 대조 표시	심민규
46	D2 채용 공고 수집/검색	F2.4 공고 상세 열람과 요건 대조	T2.4.4 보강 경로 제시	심민규
47	D2 채용 공고 수집/검색	F2.4 공고 상세 열람과 요건 대조	T2.4.5 기업 톤 정보	심민규
48	D2 채용 공고 수집/검색	F2.4 공고 상세 열람과 요건 대조	T2.4.6 관심 공고 관리	심민규
49	D3 채용 공고 분석	F3.1 분석 대상 지정	T3.1.1 입력 방식 3종	채서현
50	D3 채용 공고 분석	F3.1 분석 대상 지정	T3.1.2 부가 자료 수집	채서현
51	D3 채용 공고 분석	F3.1 분석 대상 지정	T3.1.3 진행 상태 표시	채서현
52	D3 채용 공고 분석	F3.1 분석 대상 지정	T3.1.4 재분석	채서현
53	D3 채용 공고 분석	F3.2 요구 역량 도출과 근거	T3.2.1 역량 항목 추출	채서현
54	D3 채용 공고 분석	F3.2 요구 역량 도출과 근거	T3.2.2 기록 커버리지 판정	채서현
55	D3 채용 공고 분석	F3.2 요구 역량 도출과 근거	T3.2.3 원문 근거 하이라이트	채서현
56	D3 채용 공고 분석	F3.2 요구 역량 도출과 근거	T3.2.4 반복 표현 추출	채서현
57	D3 채용 공고 분석	F3.2 요구 역량 도출과 근거	T3.2.5 질문 배정 예고	채서현
58	D3 채용 공고 분석	F3.2 요구 역량 도출과 근거	T3.2.6 소스 후보 선별	채서현
59	D4 포트폴리오 재료 선정	F4.1 소스 기록 추천과 선택	T4.1.1 매칭 순 정렬과 자동 선택	박민재
60	D4 포트폴리오 재료 선정	F4.1 소스 기록 추천과 선택	T4.1.2 선택 변경과 즉시 반영	박민재
61	D4 포트폴리오 재료 선정	F4.1 소스 기록 추천과 선택	T4.1.3 제외 사유 제시	박민재

NO	도메인	주요기능명	상세기능명	담당
62	D4 포트폴리오 재료 선정	F4.1 소스 기록 추천과 선택	T4.1.4 현장에서 기록 추가	박민재
63	D4 포트폴리오 재료 선정	F4.2 생성 모드와 분량 설정	T4.2.1 모드 선택	박민재
64	D4 포트폴리오 재료 선정	F4.2 생성 모드와 분량 설정	T4.2.2 분량 프리셋	박민재
65	D4 포트폴리오 재료 선정	F4.2 생성 모드와 분량 설정	T4.2.3 마감 알림 설정	박민재
66	D4 포트폴리오 재료 선정	F4.2 생성 모드와 분량 설정	T4.2.4 사용량 확인	박민재
67	D5 AI 인터뷰	F5.1 질문 생성과 배정	T5.1.1 질문 세트 구성	채서현
68	D5 AI 인터뷰	F5.1 질문 생성과 배정	T5.1.2 질문 근거 표시	채서현
69	D5 AI 인터뷰	F5.1 질문 생성과 배정	T5.1.3 질문 품질 규칙	채서현
70	D5 AI 인터뷰	F5.1 질문 생성과 배정	T5.1.4 질문 교체	채서현
71	D5 AI 인터뷰	F5.1 질문 생성과 배정	T5.1.5 남은 질문 미리보기	채서현
72	D5 AI 인터뷰	F5.2 응답 수집과 기록 생성	T5.2.1 답변 입력 수단	채서현
73	D5 AI 인터뷰	F5.2 응답 수집과 기록 생성	T5.2.2 실시간 기록 구성	채서현
74	D5 AI 인터뷰	F5.2 응답 수집과 기록 생성	T5.2.3 기록 확정과 저장	채서현
75	D5 AI 인터뷰	F5.2 응답 수집과 기록 생성	T5.2.4 추가 질문(꼬리 질문)	채서현
76	D5 AI 인터뷰	F5.2 응답 수집과 기록 생성	T5.2.5 중단 · 재개 · 건너뛰기	채서현
77	D5 AI 인터뷰	F5.2 응답 수집과 기록 생성	T5.2.6 이전 대화 열람	채서현
78	D6 포트폴리오 레시피	F6.1 레시피 생성	T6.1.1 입력 통합	박민재
79	D6 포트폴리오 레시피	F6.1 레시피 생성	T6.1.2 섹션 목록 생성	박민재
80	D6 포트폴리오 레시피	F6.1 레시피 생성	T6.1.3 컨텍스트 문서 작성	박민재

NO	도메인	주요기능명	상세기능명	담당
81	D6 포트폴리오 레시피	F6.1 레시피 생성	T6.1.4 근거 경로 기록	박민재
82	D6 포트폴리오 레시피	F6.1 레시피 생성	T6.1.5 미사용 소스 처리	박민재
83	D6 포트폴리오 레시피	F6.1 레시피 생성	T6.1.6 전체 재생성	박민재
84	D6 포트폴리오 레시피	F6.2 레시피 편집과 공동 작업	T6.2.1 섹션 순서·추가·삭제	박민재
85	D6 포트폴리오 레시피	F6.2 레시피 편집과 공동 작업	T6.2.2 항목 단위 편집	박민재
86	D6 포트폴리오 레시피	F6.2 레시피 편집과 공동 작업	T6.2.3 문장으로 수정 지시	박민재
87	D6 포트폴리오 레시피	F6.2 레시피 편집과 공동 작업	T6.2.4 변경 이력	박민재
88	D6 포트폴리오 레시피	F6.2 레시피 편집과 공동 작업	T6.2.5 완성도 요약	박민재
89	D7 포트폴리오 생성	F7.1 지면 생성	T7.1.1 내 내용으로 후보 비교	박민재
90	D7 포트폴리오 생성	F7.1 지면 생성	T7.1.2 후보 3안과 전체 보기	박민재
91	D7 포트폴리오 생성	F7.1 지면 생성	T7.1.3 기업 톤 반영	박민재
92	D7 포트폴리오 생성	F7.1 지면 생성	T7.1.4 스타일 변형 지시	박민재
93	D7 포트폴리오 생성	F7.1 지면 생성	T7.1.5 선택 요약	박민재
94	D7 포트폴리오 생성	F7.2 문서 생성 실행	T7.2.1 레시피 준수 생성	박민재
95	D7 포트폴리오 생성	F7.2 문서 생성 실행	T7.2.2 진행 표시와 백그라운드	박민재
96	D7 포트폴리오 생성	F7.2 문서 생성 실행	T7.2.3 사용량 차감과 복구	박민재
97	D7 포트폴리오 생성	F7.2 문서 생성 실행	T7.2.4 생성 직후 스냅샷	박민재
98	D8 포트폴리오 편집	F8.1 구조 편집	T8.1.1 섹션 순서·표시	박민재
99	D8 포트폴리오 편집	F8.1 구조 편집	T8.1.2 순서 개선 제안	박민재

NO	도메인	주요기능명	상세기능명	담당
100	D8 포트폴리오 편집	F8.1 구조 편집	T8.1.3 섹션 추가	박민재
101	D8 포트폴리오 편집	F8.1 구조 편집	T8.1.4 뷰포트 전환	박민재
102	D8 포트폴리오 편집	F8.2 대화형 수정	T8.2.1 대상 지정	박민재
103	D8 포트폴리오 편집	F8.2 대화형 수정	T8.2.2 변경 요약 제시	박민재
104	D8 포트폴리오 편집	F8.2 대화형 수정	T8.2.3 기록에서 가져오기	박민재
105	D8 포트폴리오 편집	F8.2 대화형 수정	T8.2.4 인라인 텍스트 편집	박민재
106	D8 포트폴리오 편집	F8.3 속성 조정과 기록 동기화	T8.3.1 내용 속성 편집	박민재
107	D8 포트폴리오 편집	F8.3 속성 조정과 기록 동기화	T8.3.2 표현 속성 편집	박민재
108	D8 포트폴리오 편집	F8.3 속성 조정과 기록 동기화	T8.3.3 원본 기록 동기화	박민재
109	D8 포트폴리오 편집	F8.3 속성 조정과 기록 동기화	T8.3.4 기본값 복원	박민재
110	D8 포트폴리오 편집	F8.4 변경 이력과 잠금	T8.4.1 주체별 이력	박민재
111	D8 포트폴리오 편집	F8.4 변경 이력과 잠금	T8.4.2 개별 되돌리기	박민재
112	D8 포트폴리오 편집	F8.4 변경 이력과 잠금	T8.4.3 변경 위치 비교 보기	박민재
113	D8 포트폴리오 편집	F8.4 변경 이력과 잠금	T8.4.4 복원 지점	박민재
114	D8 포트폴리오 편집	F8.4 변경 이력과 잠금	T8.4.5 잠금 관리	박민재
115	D9 배포 · 공개 사이트	F9.1 배포와 버전	T9.1.1 주소 발급	박민재
116	D9 배포 · 공개 사이트	F9.1 배포와 버전	T9.1.2 배포 전 확인	박민재
117	D9 배포 · 공개 사이트	F9.1 배포와 버전	T9.1.3 버전 관리	박민재
118	D9 배포 · 공개 사이트	F9.1 배포와 버전	T9.1.4 공개 중단·삭제	박민재

NO	도메인	주요기능명	상세기능명	담당
119	D9 배포 · 공개 사이트	F9.1 배포와 버전	T9.1.5 커스텀 도메인	박민재
120	D9 배포 · 공개 사이트	F9.2 공개 사이트 품질과 내보내기	T9.2.1 모바일 대응	박민재
121	D9 배포 · 공개 사이트	F9.2 공개 사이트 품질과 내보내기	T9.2.2 공유 메타 정보	박민재
122	D9 배포 · 공개 사이트	F9.2 공개 사이트 품질과 내보내기	T9.2.3 문서 내보내기	박민재
123	D9 배포 · 공개 사이트	F9.2 공개 사이트 품질과 내보내기	T9.2.4 이력서 파일 첨부	박민재
124	D10 통계 분석	F10.1 방문 데이터 수집	T10.1.1 기본 지표 수집	윤규현
125	D10 통계 분석	F10.1 방문 데이터 수집	T10.1.2 섹션 단위 측정	윤규현
126	D10 통계 분석	F10.1 방문 데이터 수집	T10.1.3 기업 방문 식별	윤규현
127	D10 통계 분석	F10.1 방문 데이터 수집	T10.1.4 표본 부족 처리	윤규현
128	D10 통계 분석	F10.2 대시보드 구성	T10.2.1 기본 대시보드	윤규현
129	D10 통계 분석	F10.2 대시보드 구성	T10.2.2 위젯 추가·배치	윤규현
130	D10 통계 분석	F10.2 대시보드 구성	T10.2.3 위젯 설정	윤규현
131	D10 통계 분석	F10.2 대시보드 구성	T10.2.4 뷰 저장	윤규현
132	D10 통계 분석	F10.2 대시보드 구성	T10.2.5 파생 지표	윤규현
133	D10 통계 분석	F10.2 대시보드 구성	T10.2.6 주석	윤규현
134	D10 통계 분석	F10.3 해석과 개선 제안	T10.3.1 읽기 패턴 해설	윤규현
135	D10 통계 분석	F10.3 해석과 개선 제안	T10.3.2 실행 가능한 제안	윤규현
136	D10 통계 분석	F10.3 해석과 개선 제안	T10.3.3 홈 요약 연동	윤규현
137	D10 통계 분석	F10.3 해석과 개선 제안	T10.3.4 리포트 발송	윤규현

NO	도메인	주요기능명	상세기능명	담당
138	D11 홈 · 내비게이션	F11.1 홈 대시보드	T11.1.1 추천 공고 제시	박민재
139	D11 홈 · 내비게이션	F11.1 홈 대시보드	T11.1.2 진행 중 작업 이어가기	박민재
140	D11 홈 · 내비게이션	F11.1 홈 대시보드	T11.1.3 포트폴리오 현황	박민재
141	D11 홈 · 내비게이션	F11.1 홈 대시보드	T11.1.4 최초 사용자 안내	박민재
142	D11 홈 · 내비게이션	F11.2 전역 내비게이션과 검색	T11.2.1 사이드바 구성	박민재
143	D11 홈 · 내비게이션	F11.2 전역 내비게이션과 검색	T11.2.2 통합 검색	박민재
144	D11 홈 · 내비게이션	F11.2 전역 내비게이션과 검색	T11.2.3 알림	박민재
145	D11 홈 · 내비게이션	F11.2 전역 내비게이션과 검색	T11.2.4 진행 단계 표시	박민재
146	D11 홈 · 내비게이션	F11.2 전역 내비게이션과 검색	T11.2.5 자동 저장 표시	박민재
147	D12 계정 · 요금제	F12.1 사용량과 플랜 게이팅	T12.1.1 사용량 표시	윤규현
148	D12 계정 · 요금제	F12.1 사용량과 플랜 게이팅	T12.1.2 유료 기능 노출	윤규현
149	D12 계정 · 요금제	F12.1 사용량과 플랜 게이팅	T12.1.3 플랜 변경	윤규현
150	D12 계정 · 요금제	F12.1 사용량과 플랜 게이팅	T12.1.4 계정 데이터 관리	윤규현

2.6 요구기능 정의 (웹 관리자용)

2.4의 그림 2.4에서 식별한 관리자의 주요기능을 상세하게 정의하여 아래 표와 같이 작성한다. **주요기능 10개, 상세기능 24개**이다.

NO	주요기능명	상세기능명	현재 상태	개발담당
1	공고 소스 관리	수집 소스 목록 조회	`GET /v1/job-sources`	심민규
2	공고 소스 관리	수집 소스 등록	`POST /v1/job-sources`	심민규
3	공고 소스 관리	전체 소스 수집 실행	`POST /v1/job-sources/runs`	심민규

NO	주요기능명	상세기능명	현재 상태	개발담당
4	공고 소스 관리	개별 소스 수집 실행	`POST /v1/job-sources/:id/runs`	심민규
5	공고 관리	공고 직접 등록	`POST /v1/jobs/submissions`	심민규
6	공고 관리	URL로 공고 가져오기	`POST /v1/jobs/url-imports`	심민규
7	공고 관리	직무 수요 요약 산출	`POST /v1/jobs/demand-summary`	심민규
8	시스템 상태	생존 확인	`GET /health/live`	심민규
9	시스템 상태	준비 확인 (MongoDB · Redis)	`GET /health/ready`	심민규
10	관리자 인증	역할 기반 접근 제어	미구현	오시건
11	관리자 인증	관리자 로그인 · 로그아웃	미구현	오시건
12	수집 모니터링	수집 실행 이력 조회	컬렉션에 기록	심민규
13	수집 모니터링	수집 실패 알림	미구현	심민규
14	스케줄 관리	스케줄 작업 정의 조회 · 수정	컬렉션에 기록	심민규
15	학습 데이터 관리	데이터세트 생성 요청	미구현	채서현
16	학습 데이터 관리	라벨 검수 화면	미구현	채서현
17	모델 운영	학습 실행 요청	미구현	채서현
18	모델 운영	학습 진행 · 결과 조회	미구현	채서현
19	모델 운영	성능 지표(ROC · AUC) 조회	미구현	채서현
20	모델 운영	모델 배포	미구현	채서현
21	모델 운영	모델 버전 조회 · 롤백	미구현	채서현

NO	주요기능명	상세기능명	현재 상태	개발담당
22	장애 관리	학습 장애기록 조회	미구현	심민규
23	사용자 관리	사용량 · 플랜 조회	컬렉션에 기록	윤규현
24	사용자 관리	계정 삭제 요청 처리	컬렉션에 기록, 화면 없음	오시건

03 데이터 분석 및 전처리

학습 대상 · 두 방향

2.1에서 정의한 바와 같이 본 프로젝트에서 개발하는 AI학습모델은 포트폴리오 생성 모델이 아닌 **채용 공고와 커리어 기록의 적합도 판정 모델**이다. 본 서비스의 모든 생성은 사용자 기록과 채용 공고를 연결하는 일에서 시작하며, 본 프로젝트는 그 연결을 두 방향으로 학습하고 사용한다.

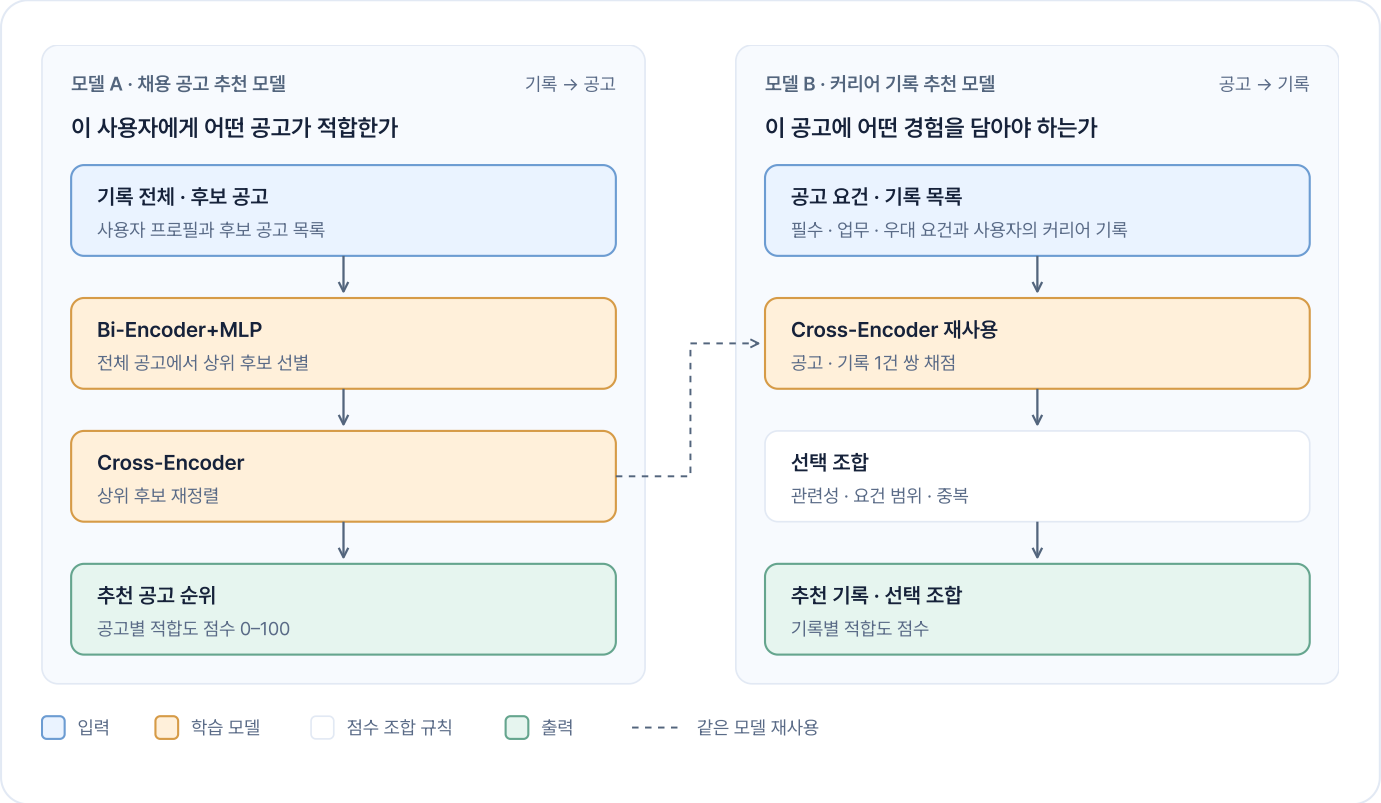


그림 3.1 채용 공고 추천 모델과 커리어 기록 추천 모델의 구성

그림 3.1에서 **채용 공고 추천 모델**(모델 A)은 사용자의 기록 전체를 보고 적합한 공고의 순위를 산출한다. 전체 공고는 Bi-Encoder+MLP로 빠르게 선별하고, 선별된 상위 후보만 Cross-Encoder로 다시 평가한다. **커리어 기록 추천 모델**(모델 B)은 지원할 공고 하나를 보고 포트폴리오에 담을 기록을 고른다. 모델 B는 모델 A의 Cross-Encoder를 재사용하여 공고와 기록 1건의 쌍을 채점하고, 그 점수로 기록의 선택 조합을 정한다.

■ 현행 판정 방식과 그 한계

모델	현행 구현	판정 방법
A	`jobs/match-score.ts` `calculateExplainableMatch()`	`source.includes(normalized(item))`
B	`materials/ranking.ts` `rankMaterials()`	요건과 기록의 토큰 중첩 및 검증 상태로 정렬

두 함수 모두 표기가 다르면 관련성을 낮게 평가한다. 현재 규칙 기반 순위가 학습 모델의 baseline이다.

services/backend/src/modules/materials/ranking.ts

```
const matched = requirementTokens.filter((token) ⇒ normalized.includes(token));
const score = matched.length * 10 + (record.status ⩵ "verified" ? 3 : 0);
return { ...record, score };
```

예를 들어 React 경험을 다른 표현으로 적으면 **겹치는 토큰**이 줄어 순위가 낮아진다. 사전학습 문장 인코더는 표기가 달라도 의미가 가까운 문장을 가까운 벡터로 변환하므로 이 한계를 해소한다. 요건별 `covered/partial/missing` 값은 **별도의 규칙 함수**가 산출한다.

■ 두 단계로 나누는 근거

축	BI-ENCODER+MLP	CROSS-ENCODER
입력	프로필과 공고를 각각 인코딩	공고와 관련 기록을 한 입력으로 함께 인코딩
사전 계산	공고 벡터는 수집 시, 프로필 벡터는 기록 변경 시 계산	요청 시 쌍마다 계산
학습 범위	인코더 동결 · 비교 모듈(MLP)만 학습	마지막 4개 층과 점수 예측 헤드 학습
쓰이는 단계	모델 A 스크리닝 — 전체 공고에서 상위 후보 선별	모델 A 리랭킹 · 모델 B 기록 채점

처리가 빠른 Bi-Encoder를 대규모 스크리닝에, 정확한 Cross-Encoder를 상위 후보에 적용하여 추천 정확도와 처리 비용의 균형을 맞춘다.

■ 두 모델이 제품에서 쓰이는 자리

모델	화면	기능	태스크
A	00	Home · 추천 공고	`T11.1.1` 추천 공고 제시
	06	공고 탐색 · 목록 정렬	`T2.3.1` 점수 계산
	00b	공고 AI 검색 결과	`T2.3.3` 축별 충족 내역
외부 LLM 설계	01	공고 분석 · 요건별 충족	`T3.2.2` 기록 커버리지 판정
B	01b	재료 고르기 · 자동 선택	`T4.1.1` 매칭 순 정렬과 자동 선택
	01b	재료 고르기 · 제외 사유	`T4.1.3` 제외 사유 제시

3.1 데이터 수집출처 및 수집방법

AI학습에는 채용 공고와 사용자 커리어 기록, 그리고 둘의 **적합도 라벨**이 필요하다. 공고와 요건의 수집 경로는 구현돼 있다. 공개 데이터에서는 다수 사용자의 상세 커리어 기록을 충분히 확보하기 어려우므로, 실제 자료의 근거를 결합한 합성 프로필을 구축한다. 아래 표는 원천별 수집 방법과 학습에서 맡는 역할을 정리한 것이다.

원천	수집 방법	학습에서 맡는 역할	상태
채용 공고	`job_sources`에 등록된 소스를 주기 실행. 목록 조회 후 상세 파싱	후보 공고와 요건 원문	구현됨
공고 요구사항	LLM이 공고 본문을 역량 항목 단위로 분해	공고의 필수 · 업무 · 우대 요건	구현됨
청년패널조사 YP2021	한국고용정보원 고용조사분석시스템의 공개 자료	학력 · 직업 · 경력의 집계 분포 — 가상 프로필의 골격	구축 중
AI Hub 채용면접 인터뷰	AI Hub 데이터셋 71592	실제 경험의 역할 · 행동 · 성과 — 커리어 기록 내용의 근거	구축 중
합성 커리어 프로필	위 두 자료를 조건으로 한 LLM 합성	프로필 1,000개	구축 중
적합도 라벨	루브릭 기반 교사 판정	프로필-공고 10,000쌍	구축 중

직군별 표본을 고르게 확보한다. 백엔드 공고만으로 학습하면 디자인 직군에서 정확도가 크게 떨어진다.

■ 채용 공고 수집출처

채용 출처는 `job_sources`에 저장하며 Worker가 **출처별 어댑터**를 실행한다. 다음 상태는 2026-09-28의 `createJobSourceAdapters()`를 기준으로 한다.

PROVIDER	출처	등록 조건
`greenhouse`	Greenhouse 채용 보드	기본 등록
`greeting`	Greeting 채용 보드	기본 등록
`lever`	Lever 채용 보드	기본 등록
`workable`	Workable 채용 보드	기본 등록
`work24web`	고용24 웹	기본 등록

PROVIDER	출처	등록 조건
`work24`	고용24 API	`WORK24_API_KEY`가 있을 때 등록
`ashby`	Ashby 채용 보드	어댑터 구현 전

수집한 공고는 `job_postings`에 정규화하여 저장한다. 중복은 `dedupeHash`의 유일 인덱스로 막는다.

■ 수집 데이터의 필드 형식

공고와 추출 요건의 MongoDB 문서 필드는 다음과 같다. 타입은 `packages/database/src/documents/jobs.ts`를 따른다.

컬렉션	필드	타입	설명
`job_postings`	`_id`	UUID 문자열	공고 식별자
	`companyId`	문자열	회사 참조
	`source`	열거형	`api` · `partner` · `user_input`
	`externalId`	문자열 또는 null	출처의 원본 식별자
	`title`	문자열	공고 제목
	`descriptionRaw`	문자열	요건 추출의 원문
	`dedupeHash`	문자열	중복 제거 해시
`job_posting_requirements`	`jobPostingId`	문자열	공고 참조
	`label`	문자열	요건 설명
	`kind`	열거형	`must` · `nice` · `tone`
	`sourceSpan`	객체	원문 인용과 위치
	`extractorVersion`	정수	추출기 버전

■ 합성 프로필 구축

합성 프로필은 실제 자료 두 가지를 조건으로 생성한다. **청년패널조사**의 집계 분포로 가상 사용자의 학력 · 직업 · 경력 골격을 정하고, **채용면접 인터뷰**의 실제 경험 서술을 근거로 커리어 기록의 내용을 작성한다. 아래 그림 3.2는 그 과정이다.

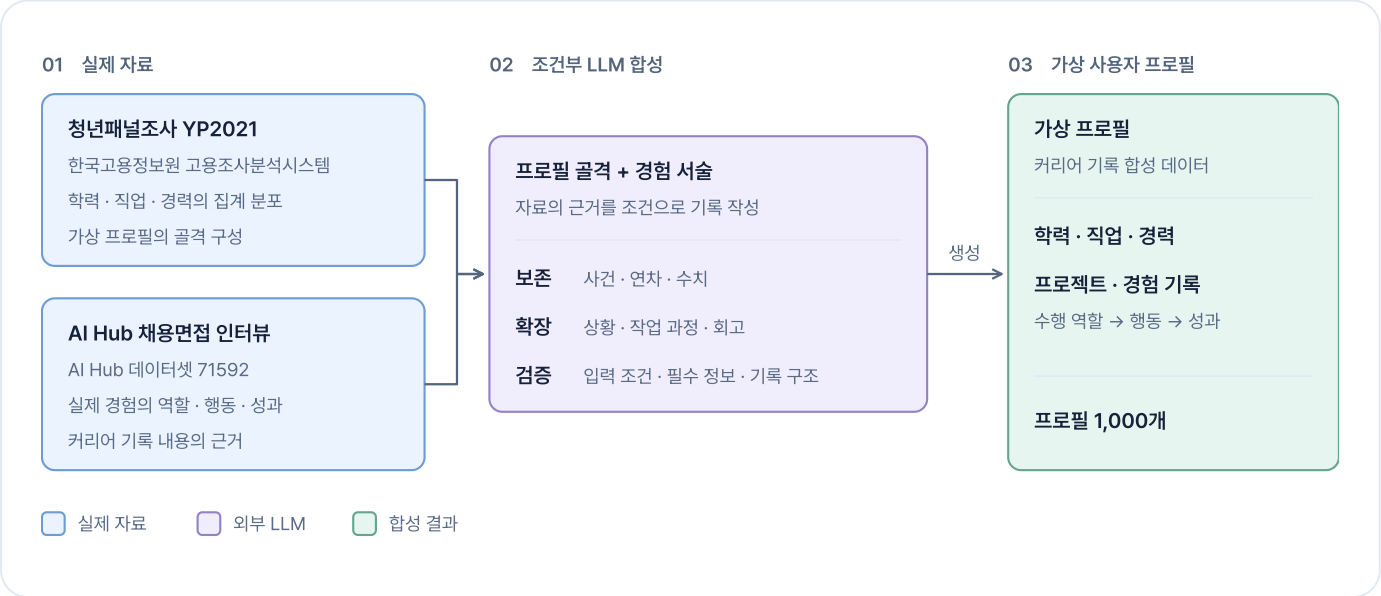


그림 3.2 실제 자료 기반 합성 프로필의 구축 과정

생성 모델은 원천 자료의 사건 · 연차 · 수치를 바꾸지 않고, 상황 · 작업 과정 · 회고만 확장한다. 생성된 기록은 입력 조건을 지켰는지, 필수 정보가 있는지, 기록 구조를 따르는지 검사하고 **통과한 것만** 데이터세트에 넣는다.

■ 라벨 생성 방법

현재 실사용자가 없어 **행동 로그가 0건**이며, 사람이 **1만 쌍**을 직접 라벨링하는 것도 불가능하다. 따라서 프론티어 LLM을 교사로 사용하여 적합도 라벨을 생성하고, 학생 모델이 그 판단을 재현하도록 학습한다. 이를 **지식 증류(distillation)** 구조라고 한다. 교사는 Codex의 GPT-5.6 Luna와 Claude Sonnet이다.

교사가 점수를 직접 매기게 하면 같은 판단에도 점수가 흔들린다. 그래서 교사는 **근거 해석과 판정**만 수행하고, **가중 점수 계산과 오류 검사**는 코드가 수행한다. 교사는 공고에서 핵심 요구사항을 **1–8개** 추리고, 요구사항마다 종류와 충족 수준을 판정한 뒤 근거가 되는 기록의 ID를 연결한다. 아래 그림 3.3은 그 과정이다.

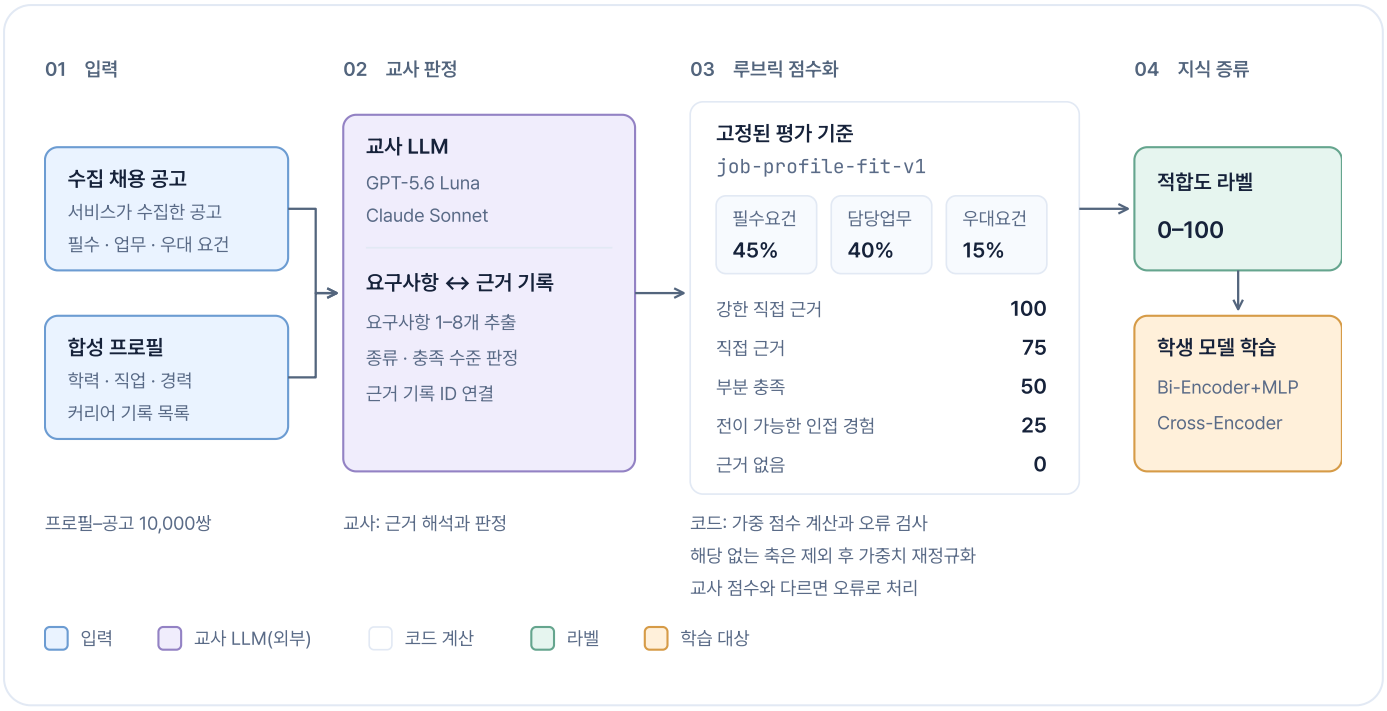


그림 3.3 루브릭 기반 교사 지식 증류의 적합도 라벨 생성 과정

요구사항 종류	가중치	대상
필수요건	45%	명시된 필수 기술 · 경력 · 학력 · 자격 · 언어
담당업무	40%	해당 직무에서 실제로 수행할 핵심 업무
우대요건	15%	공고가 우대 사항임을 명시한 것

근무지 · 고용형태 · 원격 여부 · 급여 같은 근무 조건은 점수에서 제외한다.

충족 수준	점수	기준
강한 직접 근거	100	구체적인 행동 · 범위 · 결과가 있는 직접 근거
직접 근거	75	해당 요구를 수행했다고 볼 수 있는 직접 근거
부분 충족	50	일부만 충족하거나 깊이 · 범위가 부족한 직접 근거
전이 가능한 인접 경험	25	인접 경험은 있으나 해당 요구의 직접 경험은 아님
근거 없음	0	현재 프로필에서 확인할 수 없음. 근거 기록을 연결하지 않는다

근거 없음을 제외한 판정은 근거 기록 ID를 하나 이상 연결하여야 한다.

축 점수는 그 종류에 속한 요구사항 점수의 평균이다. 최종 적합도 라벨(**0–100**)은 요구사항이 있는 축만으로 가중 평균하여 구한다. 해당 요구사항이 없는 축은 제외하고 나머지 축의 가중치를 다시 합이 1이 되도록 맞춘다. 계산 규칙은 루브릭 버전 `job-profile-fit-v1` 로 고정하며, 코드는 교사가 낸 점수와 이 규칙으로 계산한 점수가 다르면 그 라벨을 **오류로 처리**한다.

3.2 데이터 특징추출 및 특징분포

본 프로젝트의 특징은 사람이 설계한 수치 특징이 아니라 **사전학습 문장 인코더가 만든 벡터**이다. 채용 공고와 사용자 프로필은 문서 구조가 서로 다르므로 각각을 텍스트로 구성한 뒤 같은 인코더로 벡터화한다.

입력	텍스트 구성	인코더 입력 형식	출력
채용 공고	직무 · 주요 업무 · 필수 자격 · 우대 요건	`passage:` 접두어, 최대 512토큰	공고 벡터 v · 768차원
사용자 프로필	경력 연차 · 기록 제목 · 기본 프로퍼티 · 본문	`query:` 접두어, 최대 512토큰	프로필 벡터 u · 768차원

기준 인코더는 `multilingual-e5-base` 이다. 두 입력은 가중치를 공유하는 같은 인코더를 지나며, 평균 풀링 후 L2 정규화한다.

인코더는 동결하여 사용하고 학습하지 않는다. 공고 벡터는 수집할 때 **한 번 계산**하고, 프로필 벡터는 기록이 바뀔 때만 다시 계산한다. 적합도 예측에 쓰는 비교 특징은 $[u, v, |u-v|, u \odot v]$ 를 이어 붙인 **3,072차원**이며, 이를 받는 MLP는 4.2에 기술한다.

■ 인코더 비교

인코더는 사전에 하나로 정하지 않고, 같은 데이터와 같은 평가 조건에서 아래 네 모델의 적합도 예측 성능을 비교하여 선정한다.

모델	특성	비교하려는 점
`multilingual-e5-base`	기준 모델	현재 인코더와 동일한 조건의 출발점
`KURE-v1`	한국어 특화	한국어 검색 데이터 추가 학습의 효과
`BGE-M3`	다국어 · 긴 문서	다국어 문맥과 긴 경력 기록의 표현력
`Qwen3-Embedding-0.6B`	지시문 지원	과업 지시문과 가변 벡터 차원의 효과

주요 지표는 NDCG@10이며, MAE · 공고쌍 순서 일치율과 임베딩 생성 시간 · 처리량 · GPU 메모리 · 임베딩 저장 용량을 함께 비교한다.

그림 3.4 · 적합도 라벨 분포와 라벨 구간별 코사인 유사도 분포 (데이터 구축 후 삽입)

그림 3.4 적합도 라벨의 분포와 라벨 구간별 코사인 유사도 분포

그림 3.4에서 확인하여야 할 것은 두 가지이다. 첫째, 적합도 라벨이 **한쪽 구간에 몰리지 않아야** 한다. 둘째, 라벨이 높은 쌍일수록 프로필 벡터와 공고 벡터의 **코사인 유사도가 높아야** 한다. 유사도 분포가 라벨 구간과 무관하면 인코더 입력의 텍스트 구성을 다시 설계한다.

3.3 데이터세트 정의 및 생성

AI학습용으로 최종 결정한 데이터세트는 `fit-v1` 한 벌이다. 합성 프로필마다 **후보 공고 10개**를 연결하고, 교사가 각 공고의 적합도를 평가한다. 두 학생 모델은 같은 데이터세트로 학습하며, 커리어 기록 추천 모델의 평가에는 같은 판정에 남은 요구사항별 근거 기록을 사용한다. 데이터세트는 MongoDB에 적재한 뒤 학습 시점에 **Parquet 파일**로 내보내어 사용한다.

항목	`FIT-V1`
단위	(사용자 프로필, 채용 공고) 쌍
구성	프로필 1,000개 × 공고 10개 = 10,000쌍
입력	프로필 텍스트 · 공고 텍스트 · 공고와 관련된 기록(최대 8개)
라벨	적합도 0–100 (루브릭 `job-profile-fit-v1`)
부가 정보	요구사항별 종류 · 충족 수준 · 근거 기록 ID
분할	학습 8,400 · 검증 900 · 평가 700
분할 단위	프로필 — 한 프로필의 공고 10개는 같은 분할에 들어간다 (프로필 840 · 90 · 70개)

Cross-Encoder 입력의 관련 기록은 공고와 기록의 E5 코사인 유사도 상위 기록으로 고른다.

04 AI학습모델의 설계

이 문서의 심장

4.1 모델 파이프라인의 구성

AI학습을 위한 처리과정은 아래 그림 4.1과 같다. 실제 자료로 합성 프로필을 만들고, 교사 라벨링 · 데이터세트 생성 · 학습 · 검증 · 배포를 거쳐 서빙에 이른다. 두 학습 단위(Bi-Encoder+MLP와 Cross-Encoder)는 **같은 데이터세트**로 학습하고 **모델 버전만 분리**한다.



그림 4.1 학습 파이프라인과 서빙 구성

그림 4.1을 보면 교사 라벨링은 합성 프로필마다 **후보 공고 10개**를 묶어 교사에게 판정을 요청하고, 코드가 루브릭으로 점수를 계산하여 `fit-v1` 데이터세트로 적재한다. 학습은 먼저 네 인코더를 같은 조건에서 비교하여 인코더를 정하고, Bi-Encoder+MLP와 Cross-Encoder를 차례로 학습한다. 평가 분할에서 규칙 baseline과 대조하여 승인된 모델만 AI학습서버로 배포한다. 어느 모델도 배포되어 있지 않으면 서비스는 **현행 규칙 함수**로 동작한다.

4.2 AI학습 알고리즘의 선정

본 프로젝트의 학습 모델은 **사전학습 문장 인코더**를 기반으로 한다. 공고와 기록은 표기가 제각각인 자연어이므로, 표기가 달라도 의미가 가까운 문장을 가까운 벡터로 옮기는 사전학습 인코더가 문자열 비교의 한계를 해소한다. 적합도 예측은 두 구조로 나누어 학습한다.

■ Bi-Encoder+MLP 적합도 모델

프로필 텍스트와 공고 텍스트를 **가중치를 공유하는 같은 인코더**로 각각 벡터화하고, 두 벡터의 비교 특징을 MLP에 넣어 적합도를 예측한다. 인코더는 동결하고 MLP만 학습한다.

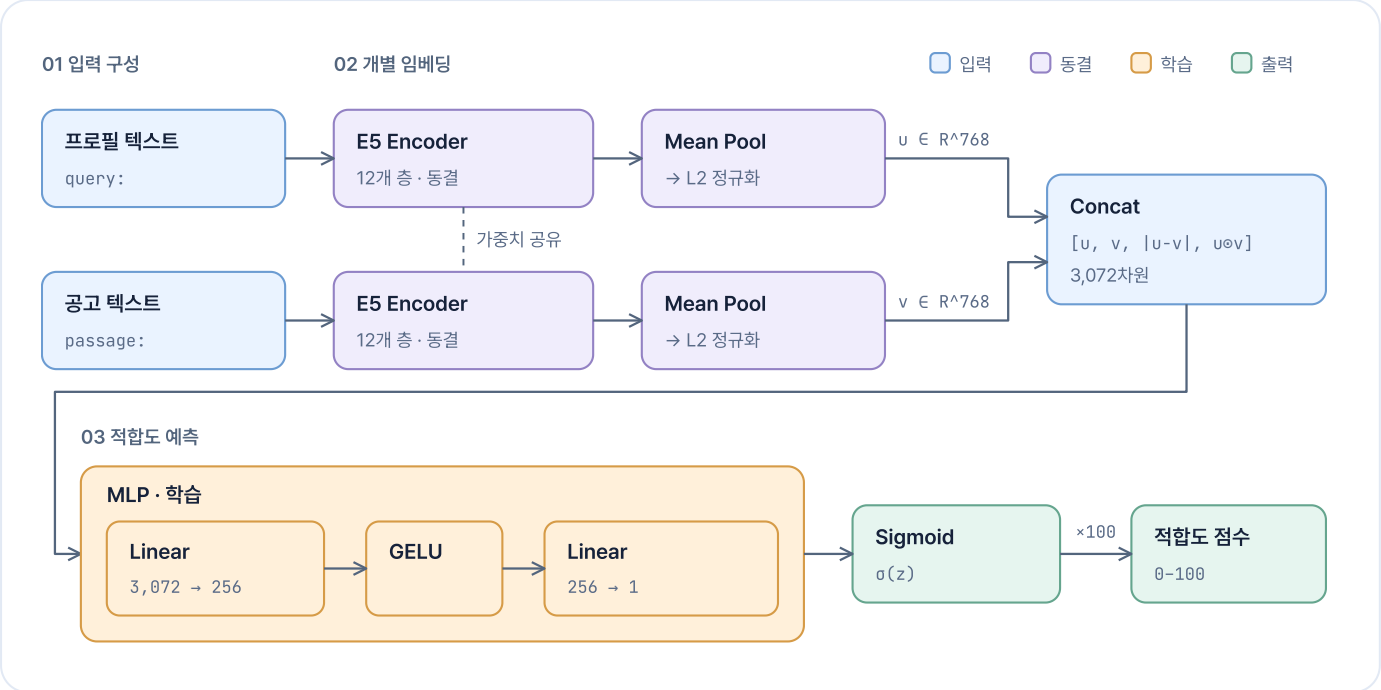


그림 4.2 Bi-Encoder+MLP 적합도 모델의 구조

Cross-Encoder 적합도 모델

공고와 관련 기록을 **한 입력으로 이어 붙여** Transformer가 함께 읽게 한다. 공고와 기록의 토큰이 서로를 참조하므로 Bi-Encoder보다 세밀하게 비교한다. 입력 길이를 맞추기 위해 공고와 기록의 E5 코사인 유사도 상위 기록을 **최대 8개**까지 고른다. 앞쪽 층은 동결하고 마지막 4개 층과 점수 예측 헤드만 학습한다.



그림 4.3 Cross-Encoder 적합도 모델의 구조

■ 두 모델의 단계적 활용

두 모델은 아래 그림 4.4와 같이 단계적으로 활용한다. Bi-Encoder+MLP가 전체 공고에서 상위 후보를 선별하고, Cross-Encoder가 선별된 후보만 다시 평가하여 최종 순위를 정한다.

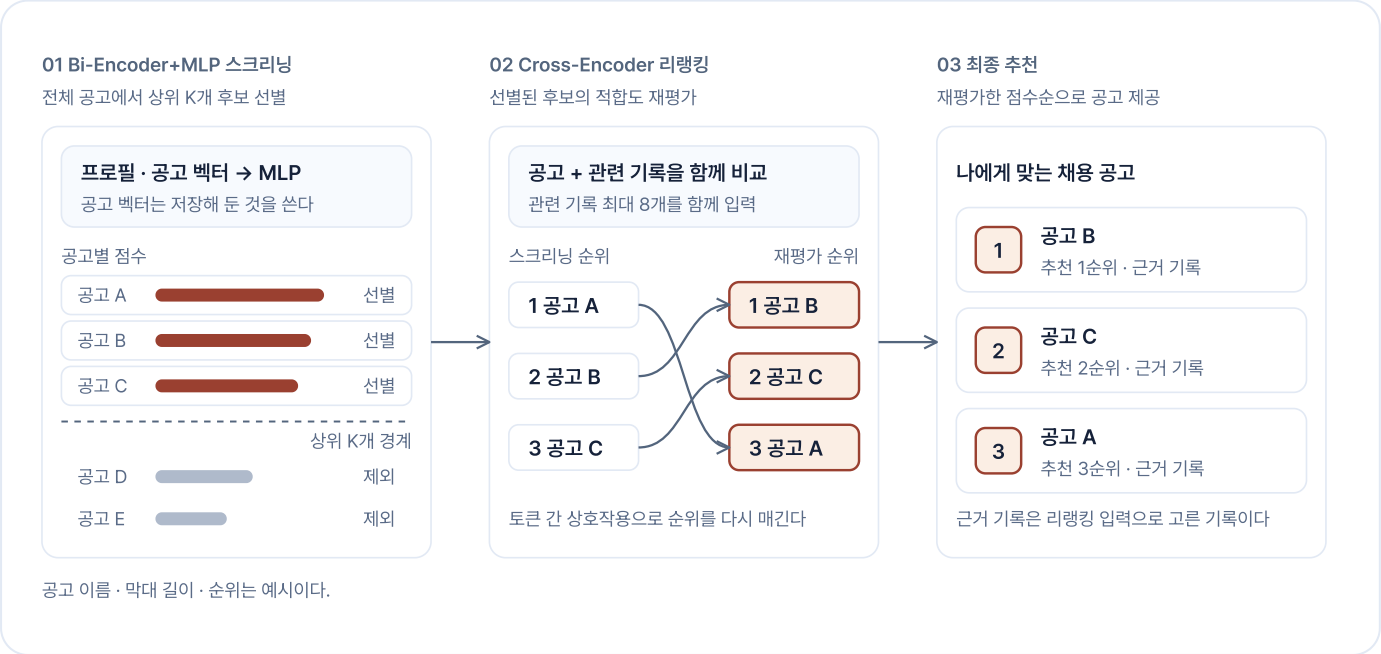


그림 4.4 Bi-Encoder+MLP와 Cross-Encoder의 단계적 활용

단계	사용 모델	처리
01 스크리닝	Bi-Encoder+MLP	프로필 벡터와 저장된 공고 벡터로 전체 공고의 점수를 계산하고 상위 후보를 선별한다
02 리랭킹	Cross-Encoder	선별된 후보마다 공고와 관련 기록을 함께 읽어 적합도를 다시 평가한다
03 최종 추천	—	재평가한 점수순으로 공고를 제공한다
기록 추천	Cross-Encoder 재사용	지원할 공고와 기록 1건의 쌍을 기록마다 채점하고, 선택 조합을 정한다

스크리닝 후보 수는 Recall@K 측정 결과로 정한다(9.2).

커리어 기록 추천의 선택 조합은 **점수 순서만으로 정하지 않는다**. 비슷한 프로젝트가 상위를 모두 차지하면 포트폴리오가 같은 강점을 반복하기 때문이다. 선택 점수는 기록의 관련도에 새로 충족하는 요건과 근거 강도를 더하고 **기록 간 중복**을 뺀 값이다 (출처: `docs/채용-공고-커리어-기록-추천-모델-연구.html`). 각 항의 가중치는 **검증 분할**에서 정한다.

ADR-001 **적합도 모델의 구조** ACCEPTED

배경

공고 목록은 한 번에 수백 건을 정렬하여야 하고, 재료 고르기는 공고 하나에 대해 사용자의 기록을 세밀하게 비교하여야 한다. 학습 데이터는 프로필–공고 **10,000쌍**이다.

선택지

- **규칙 기반** — 현행 방식이다. 표기가 다르면 판정하지 못하므로 baseline으로만 쓴다.
- **Bi-Encoder+MLP 단독** — 공고 벡터를 미리 계산하므로 빠르지만, 공고와 기록의 세부 대응을 보지 못한다.
- **Cross-Encoder 단독** — 토큰 간 상호작용으로 정확하지만, 쌍마다 연산하므로 전체 공고에 적용할 수 없다.
- **Bi-Encoder+MLP 선별 후 Cross-Encoder 재정렬** — 빠른 선별과 정밀한 비교를 단계별로 나눈다.

결정

채용 공고 추천은 Bi-Encoder+MLP로 선별하고 Cross-Encoder로 재정렬한다. 커리어 기록 추천은 같은 Cross-Encoder를 재사용한다. 인코더는 네 후보의 비교 실험으로 정한다.

근거

대규모 스크리닝에는 처리가 빠른 Bi-Encoder를, 상위 후보에는 정확한 Cross-Encoder를 적용하여 추천 정확도와 처리 비용의 균형을 맞춘다. 인코더를 동결하므로 공고 벡터를 수집할 때 한 번 계산하여 재사용할 수 있다.

영향

커리어 기록 추천은 별도 학습 데이터 없이 동작하지만, Cross-Encoder는 공고와 기록 여러 건의 묶음으로 학습하고 기록 추천에서는 기록 1건과 짝지어 쓰므로 입력 분포가 다르다. 이 차이가 성능에 미치는 영향은 교사의 기록 단위 판정으로 따로 평가한다(9.2).

■ 추천 근거의 제시

두 모델의 출력은 사용자에게 그대로 보인다. 화면 06은 「이 공고를 왜 위에 놓았는가」(T2.3.2)를, 화면 01b는 「이 기록을 왜 뺐는가」(T4.1.3)를 사용자에게 설명하여야 한다. 채용 공고 추천은 Cross-Encoder 입력으로 고른 관련 기록을 **근거 기록**으로 함께 반환한다. 커리어 기록 추천은 제외한 기록마다 점수와, 이미 선택된 기록과의 중복 또는 새로 충족하는 요건이 없음을 **사유**로 반환한다.

4.3 AI학습 파라메터 설정

학습 파라메터를 아래 표와 같이 설정한다.

파라미터	BI-ENCODER+MLP	CROSS-ENCODER
기반 모델	`multilingual-e5-base` (인코더 비교 후 확정)	`bge-reranker-v2-m3`
입력 길이	텍스트당 최대 512토큰	공고 + 기록 최대 8개, 최대 1,536토큰
표현 차원	768	1,024
동결 범위	인코더 12개 층 전체	토큰 임베딩과 앞쪽 20개 층
학습 범위	MLP — Linear 3,072 → 256 · GELU · Linear 256 → 1	마지막 4개 층과 점수 예측 헤드
출력	Sigmoid → 적합도 0–100	Sigmoid → 적합도 0–100
손실 함수	점수 MSE + 0.1 × 순서 학습 손실 (같은 프로필 안 정답 차 5점 이상 쌍)	점수 MSE
옵티마이저 · 학습률	AdamW · 0.0005	AdamW · 0.00002
배치 크기	프로필 32명 단위	2쌍 · 기울기 누적 4회
에폭	최대 60 · 검증 MAE 8회 개선 없으면 조기 종료 · 최저 검증 MAE 시점 선택	3 · 최저 검증 MAE 시점 선택
기울기 제한 · 정밀도	—	기울기 노름 1.0 · bfloat16

파일럿 학습(2026-09-04)에서 사용한 값이다. 출처는 `scripts/ml-data/llm_suitability_train.py`와 파일럿 결과 리포트이다.

파일럿에서 Bi-Encoder+MLP는 47 에폭에서 조기 종료되어 39 에폭을, Cross-Encoder는 3 에폭을 선택하였다. 교사 프롬프트와 루브릭 버전은 **데이터세트 메타**에 기록한다.

4.4 AI학습 모델의 생성

학습의 최종 결과로 모델을 생성하는 과정은 아래 **다섯 단계**이다. 각 단계는 앞 단계가 통과하여야만 진행된다.

01

데이터 검사

합성 기록 검증을 통과하지 못한 프로필과, **교사 점수와 루브릭 계산**이 어긋난 라벨을 제외한다.

02 **인코더 비교**

네 인코더로 Bi-Encoder+MLP를 학습하여 **검증 분할**에서 비교하고 인코더를 정한다.

03 **Bi-Encoder+MLP 학습**

정한 인코더로 학습 분할 전체를 학습한다.

04 **Cross-Encoder 학습**

마지막 4개 층과 점수 예측 헤드를 학습 분할로 미세조정한다.

05 **평가 · 승인**

평가 분할에서 **한 번만 평가**하고 규칙 baseline과 대조하여 승인 여부를 판정한다. 평가 분할을 보고 파라메터를 고치면 평가가 사실상 검증이 되어 성능이 부풀려지기 때문이다.

산출물	내용
학습 가중치	MLP 가중치 · Cross-Encoder의 학습한 층과 헤드 가중치
추론 코드	텍스트 구성 · 인코딩 · 점수 계산 코드
`metrics.json`	평가 분할 성능 · ROC 좌표 · baseline 비교
`metadata.json`	데이터세트 버전 · 인코더 이름 · 루브릭 버전 · 교사 모델 · 학습 시각 · 코드 커밋

학습 · 평가를 다시 실행할 수 있는 재현 코드를 함께 남긴다.

모델 버전과 롤백

항목	규칙
버전 형식	`bi-{데이터세트}-{일련번호}` · `cross-{데이터세트}-{일련번호}`
예	`bi-v1-003` · `cross-v1-002`
배포 단위	학습 단위별로 한 번에 하나의 버전만 활성
독립성	두 학습 단위는 따로 배포하고 따로 롤백한다. 커리어 기록 추천은 활성 Cross-Encoder 버전을 따른다

자동 선택과 모델 사용 조건

모델	상황	처리
B	기록 점수가 서로 가까워 순위가 갈리지 않는다	자동 선택을 멈추고 사용자가 고른다.
	기록이 Cross-Encoder 입력 한도(8개)보다 적다	모든 기록을 제시한다.
A	요건을 추출하지 못한 공고	점수를 null로 둔다.
	모델이 배포되어 있지 않다	현행 규칙 함수로 정렬한다.

「가깝다」의 임계값은 검증 분할에서 정한다.

05 화면(UI) 설계

화면 정의서가 기준

5.1 사용자 메뉴구성

■ 사용자

사용자가 접근할 수 있는 메뉴의 구조는 아래 그림 5.1과 같다. 화면 정의서는 `docs/Expresso Screens.dc.html`이며 **화면 정의서가 기준**이다. 본 절은 그 내용을 설계서 양식에 맞추어 옮긴 것이므로, 둘이 어긋나는 경우에는 화면 정의서를 따른다.



그림 5.1 사용자용 메뉴 구조도

그림 5.1을 보면 사용자 메뉴는 여섯 갈래이다. 기록 · 공고 · 생성 · 배포 · 분석 · 설정이며, 각 갈래 아래에 화면번호가 붙은 화면이 놓인다. 화면번호는 화면 정의서를 따르고, 파생 화면(b · c · d · e)은 같은 라우트 안의 패널 또는 탭이며 별도의 주소를 갖지 않는다.

5.2 초기화면(로그인 전화면) 설계

인증 이전에 노출되는 화면은 로그인 · 회원가입 · 비밀번호 재설정의 셋이다. 이 가운데 서비스에 접속하였을 때 최초로 나타나는 것이 로그인 화면이다. 본 절에서는 그 화면의 버튼과 입력란 단위의 동작을 정의한다. 화면 단위의 구성과 제약은 5.3에 있다.

로그인 화면 — 화면번호 10

`expresso.me/login · D12`

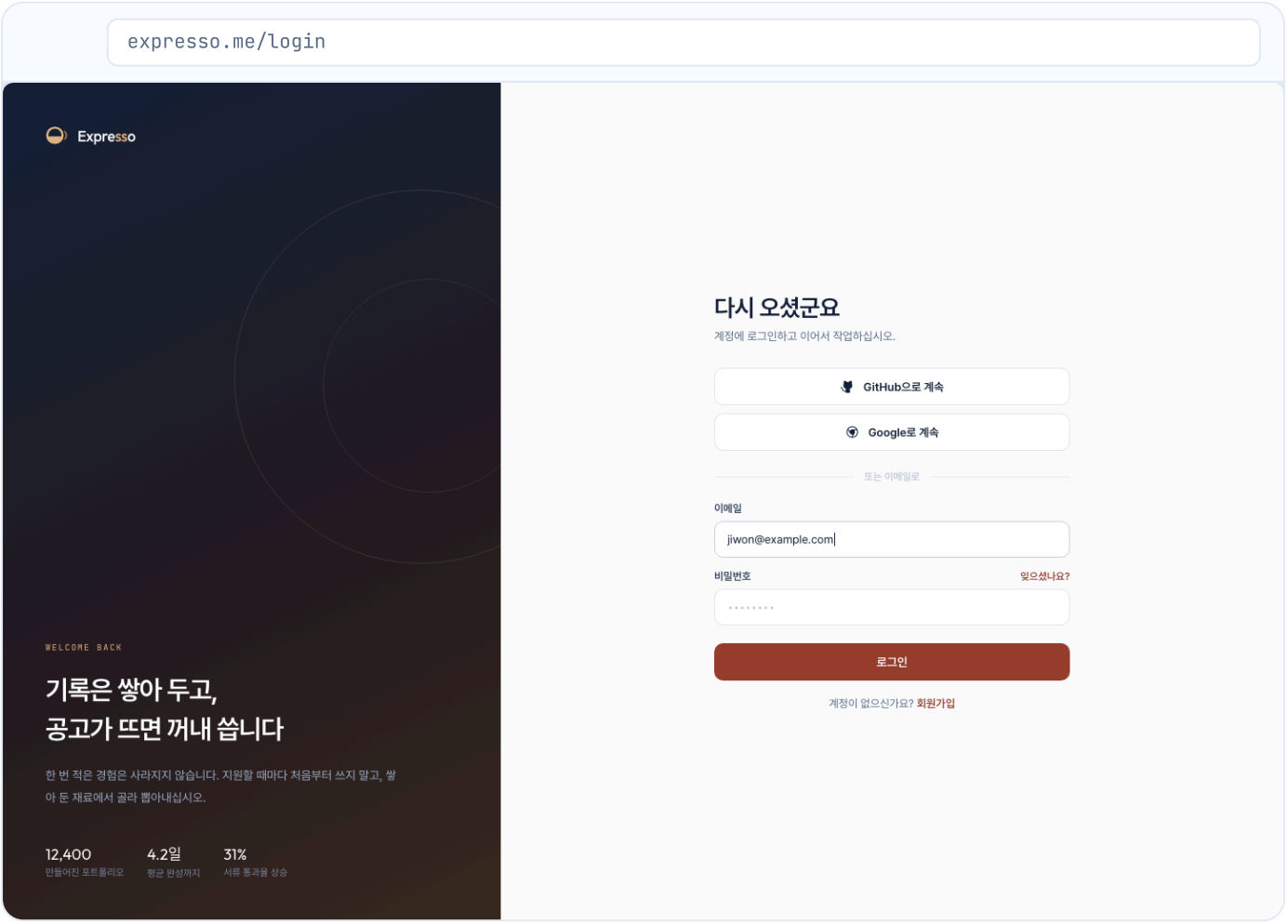


그림 5.2 초기화면(로그인) — 브라우저 프레임 포함 캡처

버튼과 입력란의 동작

요소	동작	이동 · 결과
이메일 입력란	형식을 검사한다. 형식이 어긋나면 입력란 아래에 사유를 적는다	—
비밀번호 입력란	입력값을 가려서 표시한다	—
[로그인]	`POST /v1/auth/login`을 호출한다. 두 입력란이 비어 있으면 누를 수 없다	성공 시 Home(00)
[GitHub으로 계속]	외부 인증을 시작한다	기존 계정은 Home(00) · 처음 쓰는 계정은 온보딩(10c)

요소	동작	이동 · 결과
[Google로 계속]	`POST /v1/auth/google`로 인증 결과를 넘긴다	기존 계정은 Home(00) · 처음 쓰는 계정은 온보딩(10c)
[잊으셨나요?]	등록된 이메일로 재설정 링크를 보낸다	발송 안내 표시
[회원가입]	—	회원가입(10b)

버튼 다섯 개와 입력란 두 개의 동작이다. 화면에 없는 조작은 정의하지 않는다.

오류와 예외 처리

- 인증에 실패하면 입력란 아래에 사유를 표시하되, 이메일과 비밀번호 중 무엇이 틀렸는지는 구분하여 알리지 않는다. 사유를 나누어 알리면 계정의 존재 여부가 드러난다.
- 정해진 횟수 이상 연속으로 실패한 접근은 일정 시간 제한한다.
- 외부 인증 제공자가 응답하지 않으면 소셜 버튼을 비활성 상태로 두고 이메일 방식으로 안내한다.
- 이미 로그인한 상태에서 이 주소로 접근하면 Home(00)으로 자동 이동한다.
- 삭제를 요청한 계정으로 로그인하면 유예 기간과 취소 경로를 먼저 안내한다.

5.3 사용자 화면설계(웹)

5.1에서 식별한 모든 사용자 메뉴의 화면을 설계한다. 아래 표는 화면번호와 라우트의 대응이며, 이어서 화면마다 캡처와 함께 그 화면이 하는 일 · 주요 구성 요소 · 사용자에게 하는 역할 · 기능 요구사항 · 제약조건을 적는다.

화면번호	화면명	라우트	도메인
00	Home	`/(app)/home`	D11
00b	공고 AI 검색 결과	`/(app)/jobs`	D2
00c	통합 검색 · 알림	전 화면 오버레이	D11
01	공고 분석	`/(app)/brew/[brewId]/analyze`	D3
01b	재료 고르기	`/(app)/brew/[brewId]/materials`	D4
02	Barista's Counter	`/(app)/brew/[brewId]/counter`	D5
02b	레시피	`/(app)/brew/[brewId]/outline`	D6

화면번호	화면명	라우트	도메인
03	디자인 선택	`/(app)/brew/[brewId]/design`	D7
04	다듬기 · 문구 고치기	`/(app)/edit/[portfolioId]`	D8
04b	다듬기 · 값 미세 조정	`/(app)/edit/[portfolioId]`	D8
04c	다듬기 · 변경 이력	`/(app)/edit/[portfolioId]`	D8
05	내 커리어	`/(app)/career/[categorySlug]`	D1
05b	내 커리어 · 프로젝트	`/(app)/career/[categorySlug]`	D1
05c	내 커리어 · 학력 · 이력	`/(app)/career/[categorySlug]`	D1
05d	내 커리어 · 스킬 · 도구	`/(app)/career/[categorySlug]`	D1
05e	내 커리어 · 학술 · 집필	`/(app)/career/[categorySlug]`	D1
06	공고 탐색	`/(app)/jobs`	D2
06b	공고 상세	`/(app)/jobs/[jobId]`	D2 · D3
07	분석 · 내가 만든 대시보드	`/(app)/analytics`	D10
07b	지표 추가	`/(app)/analytics`	D10
07c	위젯 설정	`/(app)/analytics`	D10
08	공개 사이트	`/site/[slug]`	D9
08b	배포 · 버전	`/(app)/edit/[portfolioId]/deploy`	D9
09	계정 · 요금제	`/(app)/account`	D12
10	로그인	`/(auth)/login`	D12
10b	회원가입	`/(auth)/signup`	D12

화면번호	화면명	라우트	도메인
10c	온보딩 · 목표	`/(auth)/onboarding/goal`	D12
10d	온보딩 · 재료 채우기	`/(auth)/onboarding/materials`	D12
10e	온보딩 · 첫 포트폴리오	`/(auth)/onboarding/first-portfolio`	D12

화면은 29개이다.

아래 캡처는 화면 정의서(`docs/Expresso Screens.dc.html`)의 **목업을 그대로 찍은 것**이다. 설명은 캡처에 실제로 보이는 요소만을 대상으로 적는다.

Home — 화면번호 00

/(app)/home · D11

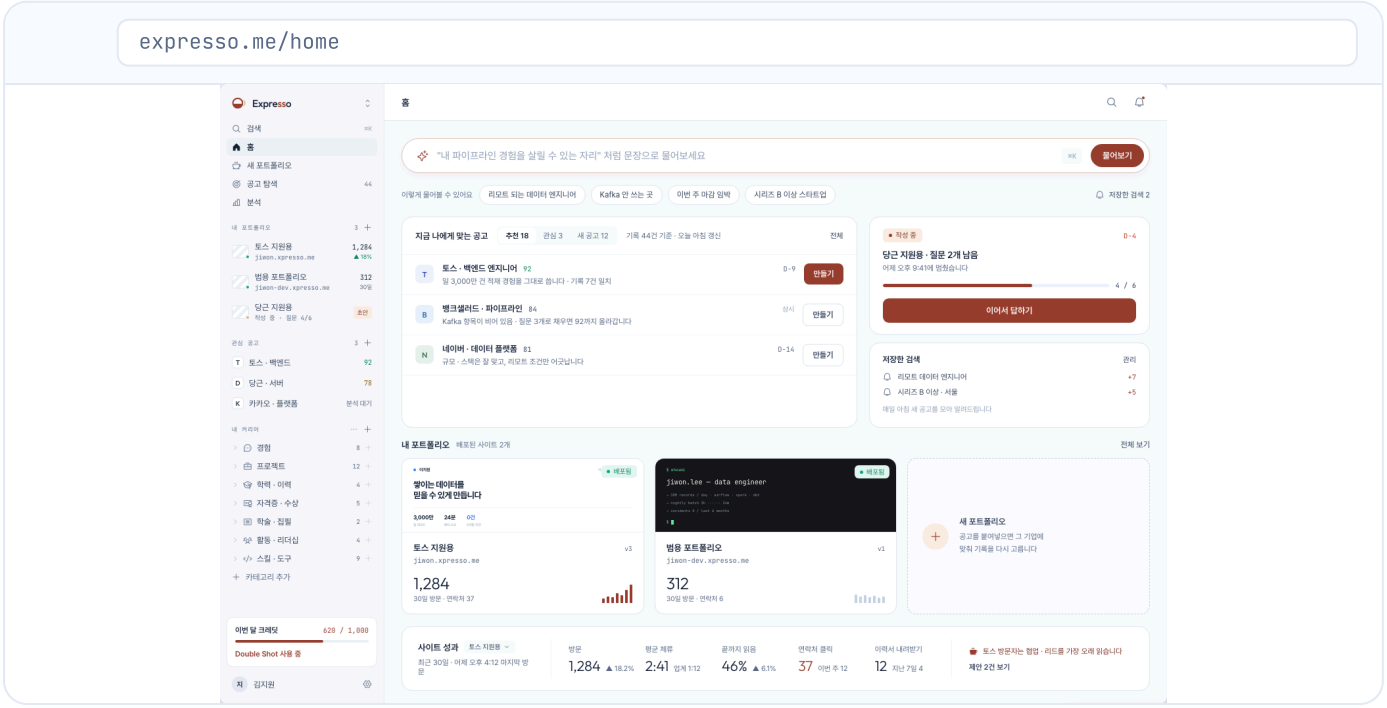


그림 5.3.1 Home (화면번호 00)

본 화면은 로그인 직후 처음 열리는 화면이며, 사용자가 다음에 할 일을 고르는 출발점이다. 제작 중인 포트폴리오 · 관심 공고 · 추천 공고 · 배포한 사이트의 성과를 한 화면에 모아, 사용자가 메뉴를 뒤지지 않고도 지금 손대야 할 것을 판단하게 한다.

주요 구성 요소

- **검색 입력부** — 조건을 문장으로 적어 공고를 찾는다. 아래에 예시 문장 네 개를 두어, 무엇을 어떻게 적어야 하는지 처음 온 사용자도 알 수 있게 한다.
- **추천 공고 목록** — 추천 · 관심 · 새 공고의 세 묶음으로 나누고, 목록 위에 「기록 44건 기준 · 오늘 아침 갱신」과 같이 산출 근거와 시점을 적는다. 공고마다 회사 · 직무 · 마감 잔여일 · 추천 사유 한 문장 · 일치한 기록 수를 표시하고, 「만들기」로 그 공고를 목표로 제작을 시작한다.
- **제작 중 카드** — 중단된 제작이 있으면 남은 질문 수와 마지막으로 멈춘 시각을 적고 「이어서 답하기」를 둔다.
- **사이트 성과 카드** — 배포한 사이트의 최근 30일 방문 · 평균 체류 시간 · 완독률 · 연락처 클릭 · 이력서 내려받기를 전월 대비 변화와 함께 보이고, 해석 한 문장과 개선 제안으로 잇는다.
- **사이드바** — 홈 · 새 포트폴리오 · 공고 탐색 · 분석 · 내 포트폴리오 · 관심 공고 · 내 커리어 카테고리를 세로로 놓고, 맨 아래에 이번 달 남은 생성 횟수와 계정 진입점을 둔다.

사용자에게 하는 역할

홈이 하는 일은 사용자에게 「무엇을 해야 하는지」를 알려 주는 것이다. 서비스의 기능이 많더라도 사용자가 한 번에 마주하는 선택지는 마감이 가까운 공고, 멈춘 제작, 성과가 움직인 사이트의 셋으로 좁혀진다. 또한 추천의 근거와 갱신 시점을 함께 보여 추천 결과를 신뢰할 수 있는지 사용자가 스스로 판단하게 한다.

기능 요구사항

1. 추천 목록은 사용자의 기록 수와 최종 갱신 시각을 함께 제시하며, 추천마다 근거가 된 기록 건수를 표시한다.
2. 제작 중 항목은 마지막 진행 지점을 보존하여, 「이어서 답하기」가 중단된 단계로 정확히 복귀하도록 한다.
3. 성과 지표는 배포된 사이트가 있을 때만 표시하고, 없으면 해당 영역을 아예 두지 않는다.
4. 남은 생성 횟수는 서버의 권한 판정 결과를 그대로 반영하며, 소진 시 제작 진입점 대신 남은 횟수와 초기화 날짜를 표시한다.
5. 검색 입력은 자연어 문장을 그대로 받아 공고 AI 검색 결과 화면으로 전달한다.

제약조건

- 추천은 사용자의 기록이 있을 때만 산출한다. 기록이 없는 신규 계정에는 추천 대신 온보딩으로 유도하는 안내를 둔다.
- 지표는 일 단위로 집계한 값을 사용하므로 당일 데이터는 반영되지 않는다. 이 때문에 화면에 마지막 집계 시점을 명시한다.
- 한 화면에 놓는 카드 수는 고정하지 않고 사용자의 상태에 따라 달라진다. 다만 카드가 없어 빈 영역이 생기는 경우 문구로 채우지 않고 영역 자체를 제거한다.

■ 공고 AI 검색 결과 — 화면번호 00b

`/(app)/jobs · D2`

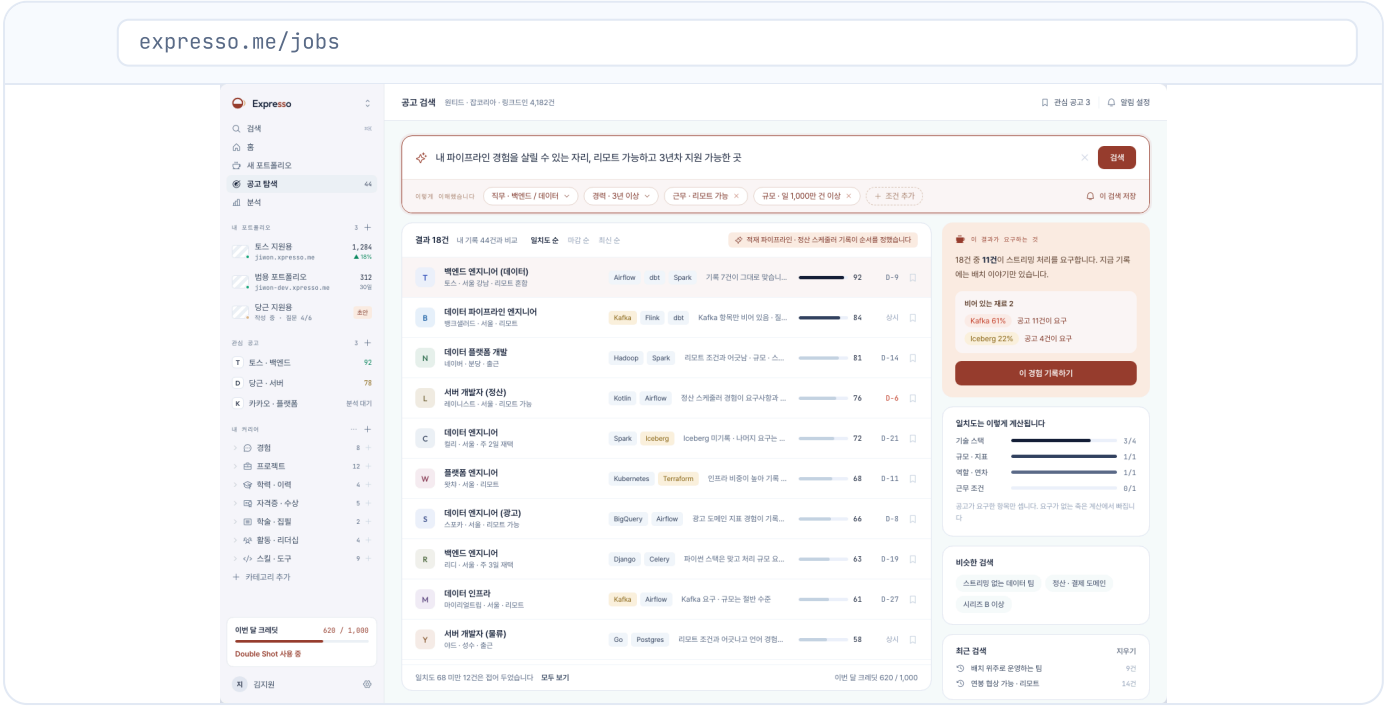


그림 5.3.2 공고 AI 검색 결과 (화면번호 00b)

본 화면은 홈이나 공고 탐색에서 문장으로 검색한 결과를 보여준다. 질의를 조건으로 해석한 결과를 먼저 제시하고, 그 조건으로 찾은 공고를 사용자의 기록과 대조하여 정렬한다. 검색 결과 목록과 그 결과가 사용자에게 요구하는 것의 요약을 좌우로 나누어 배치한다.

주요 구성 요소

- **질의 해석부** — 사용자가 적은 문장과 그것을 해석한 조건(직무 · 경력 · 근무 형태 · 규모)을 칩으로 함께 보인다. 잘못 읽은 조건은 칩을 눌러 수정하고, 「조건 추가」로 조건을 더한다. 「이 검색 저장」으로 같은 조건을 저장한 검색으로 등록한다.
- **결과 목록** — 공고마다 회사 · 근무지 · 근무 형태 · 요구 스택 · 마감 잔여일과 함께, 사용자의 기록과 대조한 결과를 한 문장으로 적는다. 정렬은 일치도 순 · 마감 순 · 최신 순의 세 가지이며 기본값은 일치도 순이다.
- **일치도 산출 안내** — 일치도를 구성하는 네 축(기술 스택 · 규모와 지표 · 역할과 연차 · 근무 조건)을 제시하고, 공고가 요구하지 않은 축은 계산에서 제외한다는 규칙을 함께 적는다.
- **비어 있는 재료** — 결과 전체가 공통으로 요구하는데 사용자의 기록에는 없는 항목을 요구 비율과 공고 건수로 제시하고, 「이 경험 기록하기」로 기록 작성으로 연결한다.
- **검색 이력** — 비슷한 검색 제안과 최근 검색을 오른쪽 아래에 두어 조건을 바꾸어 가며 탐색하게 한다.

사용자에게 하는 역할

검색 결과는 지금 지원할 수 있는 공고와 지원하기 위해 채워야 할 것을 동시에 알려 준다. 결과를 순서대로 나열하는 데 그치지 않고 순서를 정한 근거를 함께 제시하므로, 사용자는 결과를 받아들일지 조건을 고칠지 판단할 수 있다. 또한 여러 공고가 공통으로 요구하는 빈 항목을 묶어 보여, 다음에 쌓을 기록의 우선순위를 정하게 한다.

기능 요구사항

1. 질의 해석 결과는 반드시 화면에 노출하고 사용자가 수정할 수 있어야 한다. 해석이 틀린 상태로 검색이 반복되는 것을 막기 위함이다.
2. 일치도가 임계값에 미달한 결과는 접어 두되 건수를 표시하고, 「모두 보기」로 펼칠 수 있게 한다.
3. 저장한 검색은 주기적으로 재실행하여 새로 올라온 공고를 알림으로 전달한다.
4. 각 공고 행에는 판정 사유를 한 문장으로 반드시 표시한다. 점수만 제시하는 것을 금지한다.
5. 공고를 선택하면 공고 상세 화면으로 이동하고, 검색 조건은 뒤로 이동할 때 유지한다.

제약조건

- 수집 대상은 어댑터가 구현된 채널로 한정한다. 채널마다 제공하는 항목이 달라, 특정 채널에서 근무 형태나 연봉이 비어 있을 수 있다.
- 일치도는 공고가 요구한 축만으로 계산하므로 공고 간 절대 비교에는 사용하지 않는다. 같은 검색 결과 내부의 상대 순서로만 의미를 가진다.
- 자연어 해석은 언어 모델을 호출하므로 응답 지연이 발생할 수 있다. 이 경우 해석 이전에 원문 질의로 수행한 결과를 먼저 표시한다.
- 저장한 검색의 수는 요금제에 따라 제한된다.

■ 통합 검색 · 알림 — 화면번호 00c

전 화면 오버레이 · D11

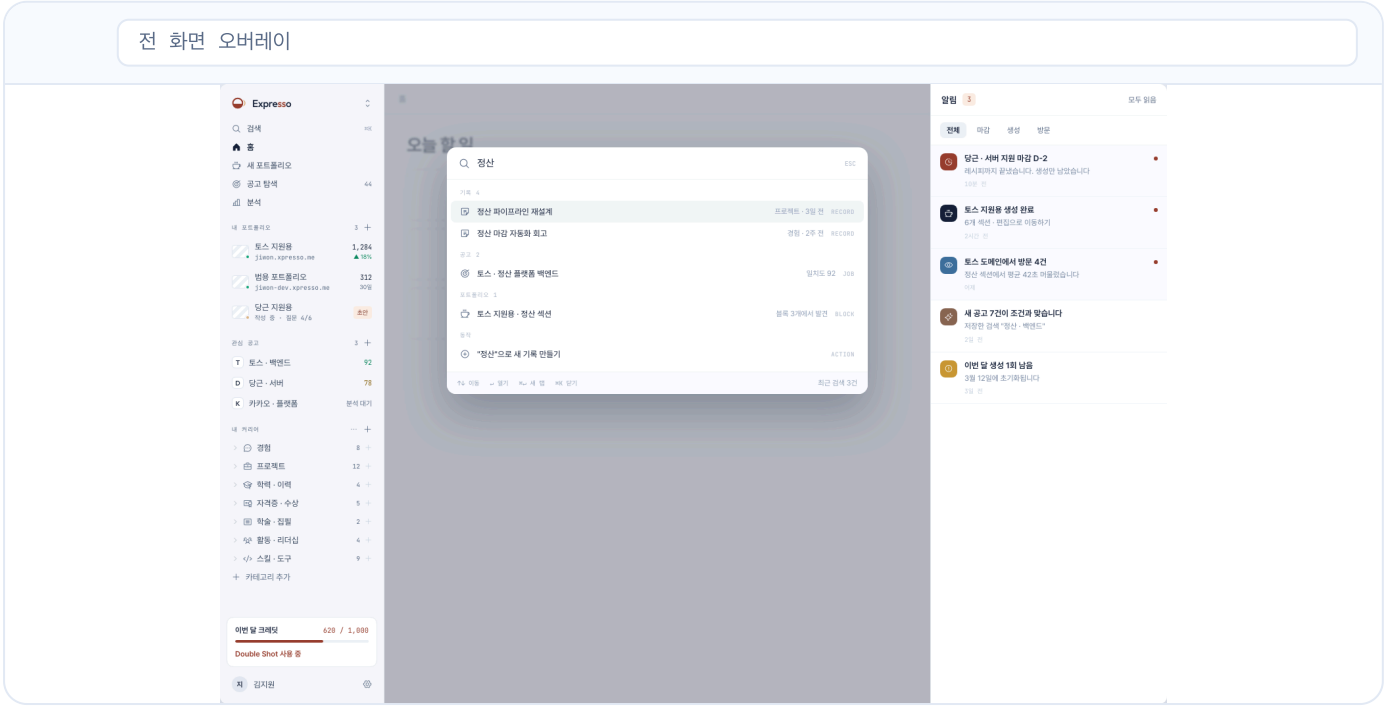


그림 5.3.3 통합 검색 · 알림 (화면번호 00c)

본 화면은 어느 화면에서든 단축키로 열리는 통합 검색과 알림이다. 화면을 이동하지 않고 겹쳐 열리므로 작업 중이던 문맥이 유지된다. 왼쪽은 한 번의 입력으로 여러 종류의 대상을 찾는 검색이고, 오른쪽은 지금 처리할 일을 모은 알림이다.

주요 구성 요소

- **통합 검색** — 입력한 낱말로 기록 · 공고 · 포트폴리오 · 포트폴리오 내부 블록을 함께 찾고, 종류마다 배지를 붙여 구분한다. 결과 그룹마다 건수를 표시한다.
- **동작 결과** — 검색어로 새 기록을 만드는 동작을 결과 목록 안에 함께 둔다. 찾으려던 것이 없을 때 화면을 옮기지 않고 그 자리에서 만들 수 있게 하기 위함이다.
- **키보드 안내** — 이동 · 열기 · 새 탭으로 열기 · 닫기의 단축키를 목록 아래에 상시 표시한다.
- **알림 목록** — 마감 임박 · 생성 완료 · 방문 발생 · 조건에 맞는 새 공고 · 남은 생성 횟수를 시간 역순으로 쌓고, 유형별 필터와 「모두 읽음」을 둔다.
- **알림 본문** — 알림마다 제목과 함께 다음 행동을 지시하는 한 문장을 붙인다. 「생성만 남았습니다」와 같이 무엇이 남았는지 적는다.

사용자에게 하는 역할

통합 검색은 서비스 전체를 관통하는 이동 수단이다. 사용자는 기록이 어느 카테고리에 있는지, 블록이 어느 포트폴리오에 있는지 기억하지 않아도 낱말 하나로 도달할 수 있다. 알림은 사용자가 놓치면 손해가 발생하는 것, 즉 마감과 한도 소진을 우선으로 모아 제시한다.

기능 요구사항

1. 검색은 대상 종류를 가로질러 한 번에 수행하고, 결과는 종류별로 묶어 제시한다.
2. 검색과 이동은 키보드만으로 완결되어야 한다. 마우스 조작을 필수로 요구하지 않는다.
3. 최근 검색을 보관하여 입력 이전에 제시하고, 사용자가 이력 전체를 지울 수 있게 한다.
4. 알림은 유형으로 필터링할 수 있어야 하며, 읽음 처리는 개별과 일괄을 모두 지원한다.
5. 알림을 선택하면 해당 대상 화면의 정확한 위치로 이동한다.

제약조건

- 검색 대상은 요청한 사용자가 소유한 자료로 한정한다. 다른 사용자의 기록과 포트폴리오는 검색 결과에 포함되지 않는다.
- 블록 검색은 편집 중인 최신 내용을 대상으로 한다. 배포 시점의 스냅샷은 검색 대상에서 제외한다.
- 알림은 서비스 내부 알림과 발송 알림을 같은 원본에서 만들지만, 발송 여부는 알림 설정에 따른다.
- 오버레이 상태에서는 배경 화면의 조작을 차단한다. 편집 중 데이터가 두 경로로 동시에 수정되는 것을 막기 위함이다.

공고 분석 — 화면번호 01

`/(app)/brew/[brewId]/analyze · D3`

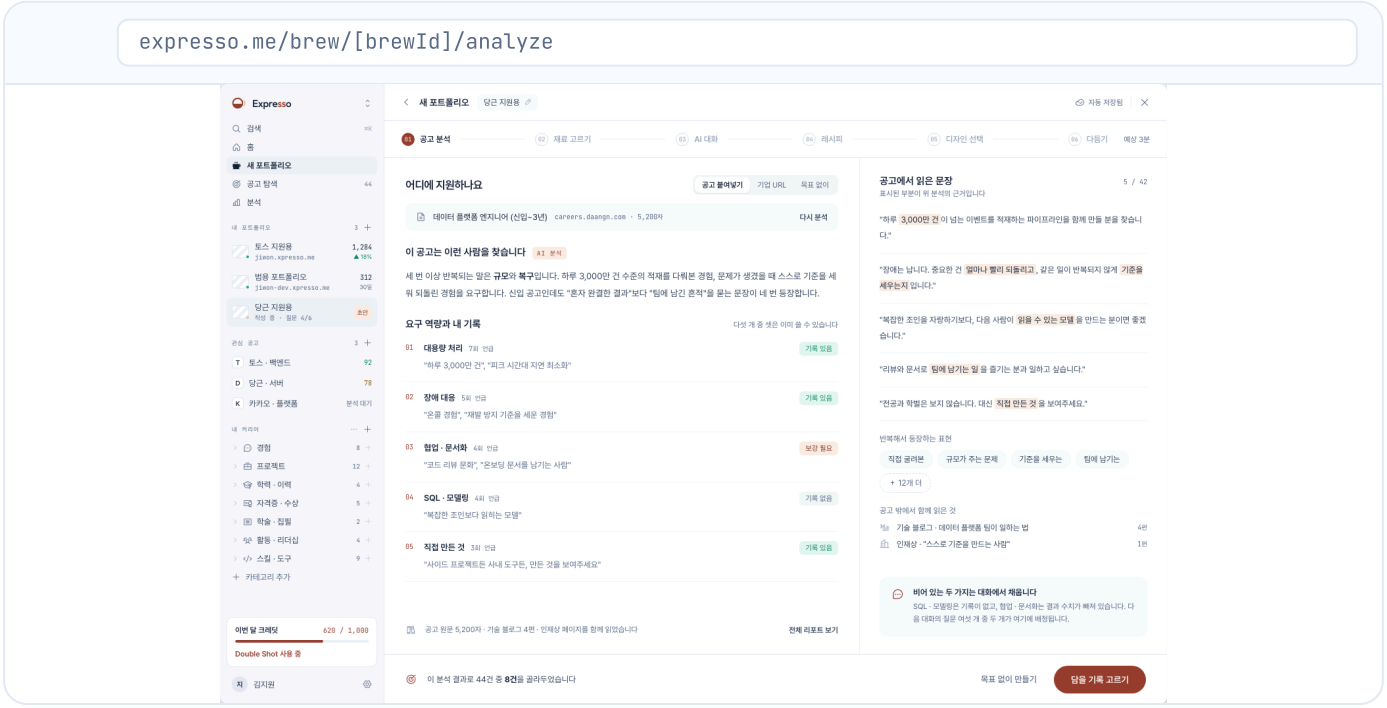


그림 5.3.4 공고 분석 (화면번호 01)

본 화면은 포트폴리오 제작의 첫 단계이며, 지원할 공고를 읽어 요구 사항을 정리한다. 제작 흐름 전체를 여섯 단계로 상단에 고정 표시하고 현재 단계를 강조한다. 왼쪽은 분석 결과, 오른쪽은 그 결과의 근거가 된 공고 원문이다.

주요 구성 요소

- **단계 표시부** — 공고 분석 · 재료 고르기 · AI 대화 · 레시피 · 디자인 선택 · 다듬기의 여섯 단계와 남은 예상 소요 시간, 자동 저장 상태를 함께 표시한다.
- **공고 입력부** — 입력 방법을 세 가지로 제시한다. 공고 전문을 붙여넣는 방법, 기업 채용 페이지 주소를 넣는 방법, 목표 공고 없이 범용으로 만드는 방법이다. 입력이 끝나면 원본의 출처와 분량을 표시한다.
- **요구 역량 분석** — 공고에서 추출한 역량마다 언급 횟수와 사용자 기록의 충족 상태를 「기록 있음 · 보강 필요 · 기록 없음」의 세 값으로 표시하고, 그 판정의 근거가 된 공고 문장을 인용한다.
- **원문 대조부** — 공고 원문을 그대로 두고 분석에 사용된 문장을 강조 표시한다. 반복해서 등장하는 표현과 공고 밖에서 함께 읽은 자료(기술 블로그 · 인재상 페이지)의 목록을 함께 제시한다.
- **다음 단계 진입부** — 분석 결과로 미리 선별한 기록 건수를 알리고 「담을 기록 고르기」로 다음 단계로 이동한다. 「다시 분석」은 같은 공고를 다시 읽는다.

사용자에게 하는 역할

공고 분석은 사용자가 무엇을 써야 하는지를 공고에서 직접 확인하게 한다. 분석 결과를 결론만 제시하면 사용자는 그것이 맞는지 알 수 없으므로, 판정마다 원문 문장을 함께 두어 검증 가능하게 한다. 또한 기록이 없는 역량을 미리 드러내어 이후 대화 단계에서 무엇을 물을지 예고하는 역할도 한다.

기능 요구사항

1. 공고 입력은 붙여넣기 · URL · 미지정의 세 경로를 모두 지원하며, 어느 경로로 들어와도 이후 단계는 동일하다.
2. 분석 결과의 각 항목은 근거 문장을 반드시 포함한다. 근거를 제시할 수 없는 항목은 결과에 포함하지 않는다.
3. 분석에 사용한 자료의 출처와 분량을 화면에 명시한다.
4. 분석은 재실행할 수 있어야 하며, 재실행 결과가 이전 결과를 대체하더라도 진행 중인 제작은 중단되지 않는다.
5. 분석이 끝나면 사용자 기록 중 해당 공고와 관련된 것을 선별하여 다음 단계의 초기 선택으로 전달한다.

제약조건

- 기업 채용 페이지 주소로 입력하는 경우, 페이지 구조에 따라 본문 추출이 실패할 수 있다. 이때는 붙여넣기로 전환하도록 안내한다.
- 분석은 언어 모델 호출을 포함하므로 즉시 완료되지 않는다. 진행 중에는 단계 표시부에 상태를 표시하고 사용자가 화면을 떠나도 작업은 계속된다.
- 공고 밖 자료 수집은 기업의 공개 페이지로 한정한다. 로그인이 필요한 자료는 수집하지 않는다.
- 분석 결과는 사용자의 기록 상태를 기준으로 산출하므로, 기록이 추가되면 같은 공고의 판정이 달라질 수 있다.

재료 고르기 — 화면번호 01b

`/(app)/brew/[brewId]/materials · D4`

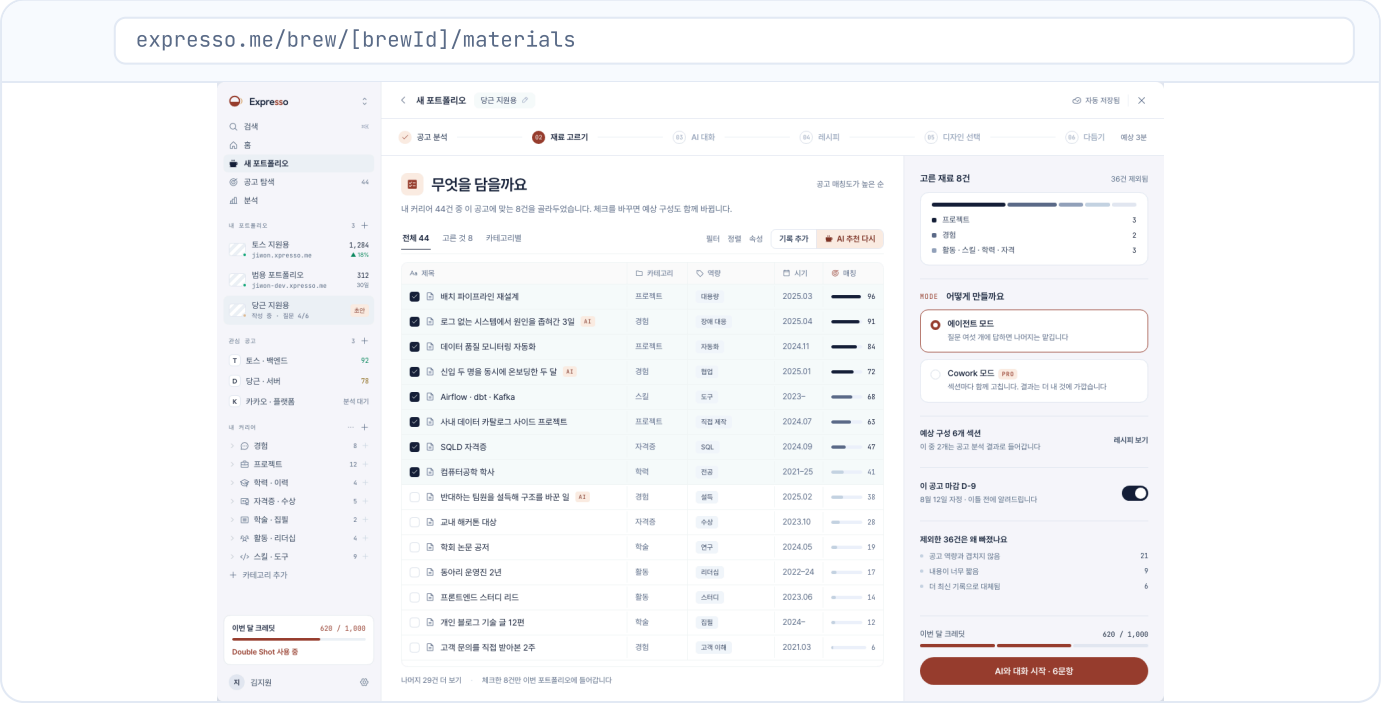


그림 5.3.5 재료 고르기 (화면번호 01b)

본 화면은 공고 분석 다음 단계이며, 포트폴리오에 담을 기록을 고르고 제작 방식을 정한다. 왼쪽에 사용자의 기록 전체를 표로 놓고, 오른쪽에 선택 결과와 제작 방식 · 마감을 모은다. 공고 분석 결과로 선별된 기록은 미리 선택된 상태로 제시된다.

주요 구성 요소

- **기록 표** — 제목 · 카테고리 · 역량 · 시기 · 공고 일치도를 열로 갖는 표이다. 기본 정렬은 일치도가 높은 순이며, 카테고리별 묶음 보기 · 필터 · 정렬 · 표시 속성 선택을 지원한다.
- **선택 요약** — 고른 기록 수와 제외된 기록 수를 함께 표시하고, 제외된 기록의 사유를 「공고 역량과 겹치지 않음 · 내용이 너무 짧음 · 더 최신 기록으로 대체됨」과 같이 유형으로 제시한다.
- **제작 방식 선택** — 질문에 답하면 나머지를 맡기는 방식과, 섹션마다 함께 고치는 방식의 두 가지를 제시한다. 후자는 상위 요금제에서만 선택할 수 있으며 등급 표시를 함께 붙인다.
- **기록 보강** — 「기록 추가」로 이 화면을 벗어나지 않고 기록을 새로 만들고, 「AI 추천 다시」로 선별을 다시 받는다.
- **마감 안내** — 목표 공고의 마감 일시와 사전 알림 시점을 표시하여, 제작 속도를 사용자가 조절하게 한다.

사용자에게 하는 역할

포트폴리오에 담을 내용을 사용자가 최종 결정하는 자리가 이 단계이다. 추천은 초기값일 뿐이며, 제외 사유를 함께 제시하여 사용자가 그 판단을 뒤집을 수 있게 한다. 또한 예상 섹션 수를 미리 알려 결과물의 규모를 가늠하게 한다.

기능 요구사항

1. 선택은 사용자가 언제든 변경할 수 있어야 하며, 추천에 의한 초기 선택과 사용자의 수정을 구분하여 표시한다.
2. 제외된 기록은 감추지 않고 사유와 함께 열람할 수 있게 한다.
3. 제작 방식 중 상위 요금제 전용 항목은 선택 이전에 필요한 등급을 표시하고, 선택 시 요금제 안내로 연결한다.
4. 선택한 기록의 수와 종류로부터 예상 섹션 구성을 산출하여 표시한다.
5. 이 화면의 선택 결과는 다음 단계인 대화의 질문 구성에 사용된다.

제약조건

- 선택할 수 있는 기록은 본인이 소유한 기록으로 한정한다.
- 제작 방식은 이 단계에서 확정하며, 대화가 시작된 뒤에는 같은 제작 안에서 변경할 수 없다.
- 포트폴리오 하나에 담을 수 있는 기록 수의 상한은 템플릿의 섹션 구조에 따라 달라진다.
- 기록이 일정 수 미만이면 선별의 의미가 없으므로, 이 경우 온보딩의 기록 채우기 단계로 안내한다.

Barista's Counter — 화면번호 02

`/(app)/brew/[brewId]/counter · D5`

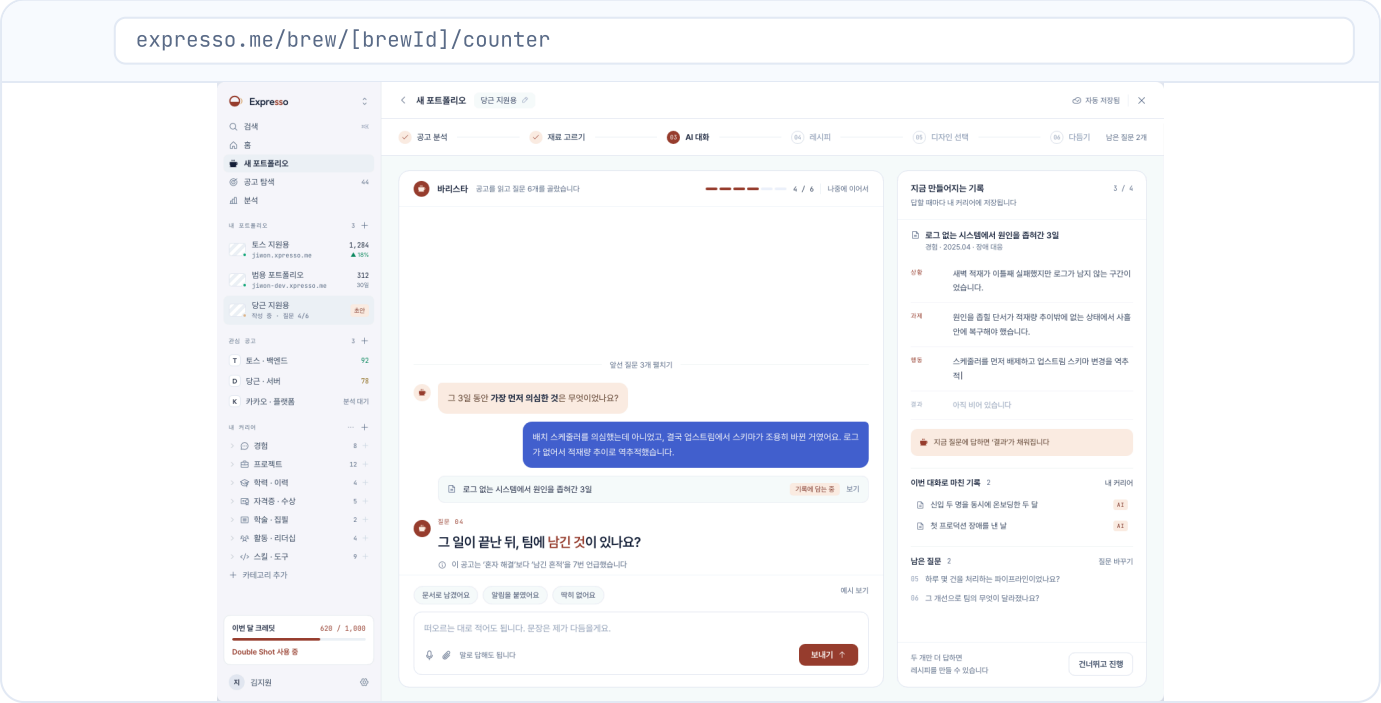


그림 5.3.6 Barista's Counter (화면번호 02)

본 화면에서는 AI가 묻고 사용자가 답한다. 공고 분석에서 비어 있던 항목을 질문으로 바꾸어 묻고, 사용자의 답을 그 자리에서 커리어 기록으로 구조화한다. 왼쪽은 대화, 오른쪽은 그 대화로 만들어지는 중인 기록이다.

주요 구성 요소

- **진행 표시** — 준비된 질문 수와 남은 질문 수를 상단에 표시하고, 이미 답한 질문은 접어 둔다. 「나중에 이어서」로 대화를 중단하고 나중에 같은 지점에서 재개한다.
- **질문부** — 질문마다 그 질문을 고른 근거를 함께 적는다. 공고에서 반복된 표현과 그 횟수를 제시하여, 사용자가 질문의 의도를 이해한 상태로 답하게 한다.
- **답변 입력부** — 자유 입력을 기본으로 하고, 짧은 선택지와 예시 보기를 함께 제공한다. 음성으로도 답할 수 있다.
- **기록 미리보기** — 답변이 상황 · 과제 · 행동 · 결과의 네 항목으로 채워지는 과정을 실시간으로 보인다. 아직 비어 있는 항목은 비어 있다고 명시하고, 지금 질문이 그 항목을 채운다는 사실을 함께 적는다.
- **질문 관리** — 남은 질문을 미리 보이고 「질문 바꾸기」로 교체하며, 「건너뛰고 진행」으로 남은 질문 없이 다음 단계로 넘어간다.

사용자에게 하는 역할

대화는 사용자가 글쓰기의 부담 없이 자신의 경험을 정리하게 한다. 사용자는 문장을 완성하지 않아도 되고 떠오르는 대로 답하면 되며, 문장 정리는 서비스가 담당한다. 또한 그 답변이 커리어 기록으로 저장된다는 것을 오른쪽 영역에서 상시 확인하게 한다.

기능 요구사항

1. 질문은 공고 분석 결과와 선택된 기록을 근거로 생성하며, 질문마다 선정 근거를 표시한다.
2. 답변은 입력하는 즉시 기록으로 저장한다. 대화를 끝까지 진행하지 않아도 답한 부분까지는 남아야 한다.
3. 기록 미리보기는 채워진 항목과 비어 있는 항목을 구분하여 표시한다.
4. 사용자는 질문을 교체하거나 남은 질문을 건너뛸 수 있으며, 건너뛴 경우에도 다음 단계로 진행할 수 있다.
5. 중단 시점을 저장하여 홈의 제작 중 카드와 이 화면의 재개 지점이 일치하게 한다.

제약조건

- 답변으로 만든 기록은 사용자의 확인 없이 확정하지 않는다. 확정 이전 상태를 화면에 명시한다.
- 질문 수는 제작 방식과 선택한 기록 수에 따라 달라지며, 사용자가 임의로 늘릴 수 없다.
- 음성 입력은 브라우저의 음성 인식 기능에 의존하므로 지원하지 않는 환경에서는 노출하지 않는다.
- 대화 응답은 언어 모델 호출을 포함하므로 지연이 발생할 수 있다. 지연 중에도 이미 입력한 답변은 유실되지 않아야 한다.

■ 레시피 — 화면번호 02b

`/(app)/brew/[brewId]/outline · D6`

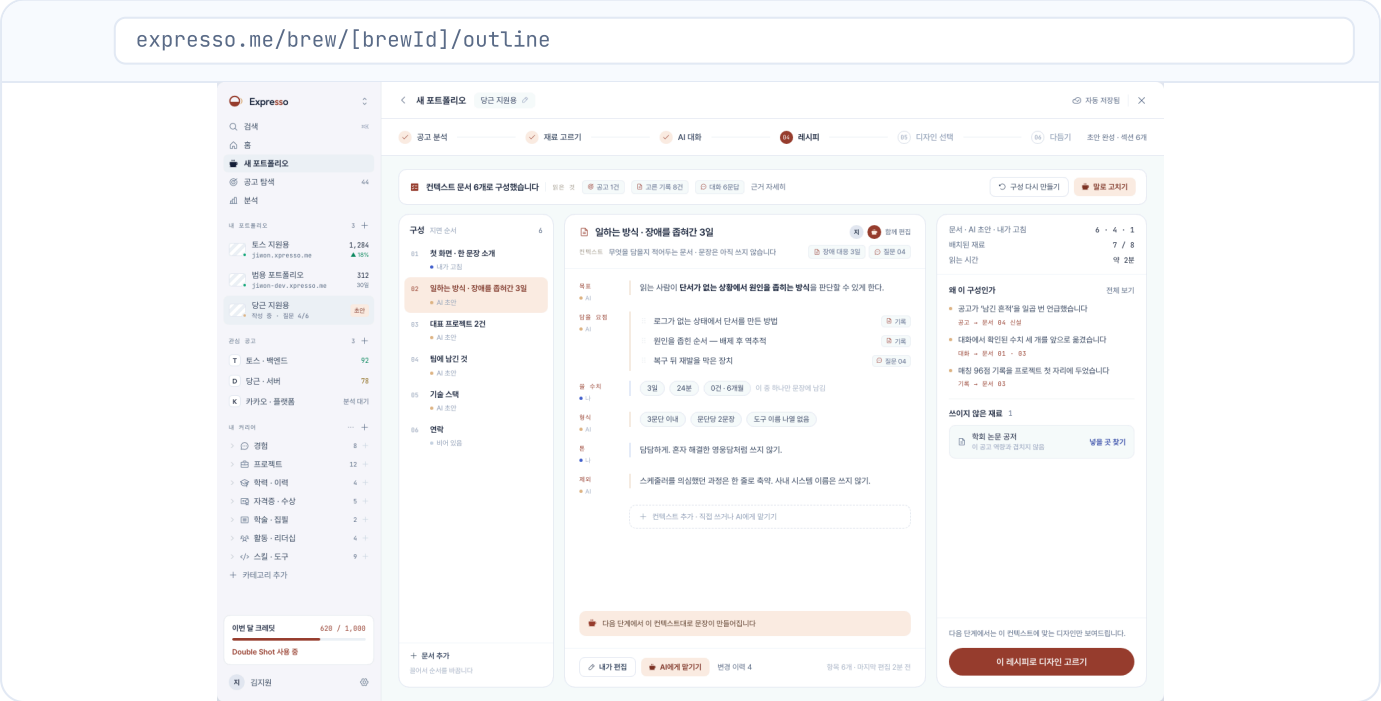


그림 5.3.7 레시피 (화면번호 02b)

본 화면은 대화 다음 단계이며, 공고와 기록과 대화를 근거로 구성한 포트폴리오의 구조를 확인하고 수정한다. 이 단계에서는 문장을 쓰지 않는다. 어떤 지면을 어떤 순서로 두고 각 지면에 무엇을 담을지만 정한다. 왼쪽은 지면 목록, 가운데는 선택한 지면의 작성 지침, 오른쪽은 이 구성을 만든 근거이다.

주요 구성 요소

- **지면 목록** — 섹션을 순서대로 나열하고 끌어서 순서를 바꾼다. 항목마다 출처를 「AI 초안 · 내가 고침」으로 구분하여 표시하고, 아직 내용이 없는 항목은 「비어 있음」으로 명시한다. 「문서 추가」로 지면을 늘린다.
- **작성 지침** — 선택한 지면에 대하여 목표 · 담을 요점 · 사용할 수치 · 형식(문단 수와 길이) · 어조 · 제외할 내용을 항목으로 적는다. 요점마다 근거가 된 기록을 함께 표시한다.
- **구성 근거** — 각 지면이 왜 그 자리에 있는지를 「공고 → 문서 04 신설」과 같이 원인과 결과의 쌍으로 제시한다.
- **쓰이지 않은 재료** — 선택했으나 배치되지 않은 기록을 사유와 함께 두고, 「넣을 곳 찾기」로 배치를 다시 시도한다.
- **요약 지표** — 배치된 기록 수, 예상 읽는 시간, 마지막 편집 시각, 변경 건수를 함께 표시한다.

사용자에게 하는 역할

레시피 단계는 문장을 생성하기 전에 작성 방향을 확정하는 단계이다. 완성된 글을 놓고 고치면 사용자가 무엇을 바꾸어야 할지 판단하기 어렵고 수정 범위도 커진다. 지침 단계에서 미리 고치게 하여, 이후 생성된 문장이 사용자의 의도와 크게 어긋나지 않게 한다.

기능 요구사항

1. 레시피는 지면 단위로 구성하며, 각 지면은 목표 · 요점 · 수치 · 형식 · 어조 · 제외 사항을 갖는다.
2. 각 지면의 출처를 초안과 사용자 수정으로 구분하여 표시한다.
3. 지면 순서는 끌어서 변경할 수 있고, 변경 결과는 즉시 저장한다.
4. 구성의 근거를 사용자가 열람할 수 있어야 한다. 근거를 제시하지 못하는 구성 변경은 수행하지 않는다.
5. 레시피는 버전과 완성도를 함께 관리하여, 다시 생성하여도 이전 버전을 잃지 않는다.

제약조건

- 이 단계에서는 본문 문장을 생성하지 않는다. 문장 생성은 다음 단계 이후에 수행한다.
- 지면 수를 늘리면 다음 단계의 생성 시간이 늘어난다. 이 때문에 예상 읽는 시간을 화면에 함께 표시한다.
- 레시피를 다시 만들면 사용자가 직접 고친 지침도 초기화된다. 이 때문에 재생성 이전에 확인을 받는다.
- 함께 편집 기능은 상위 요금제에서만 사용할 수 있다.

디자인 선택 — 화면번호 03

`/(app)/brew/[brewId]/design · D7`

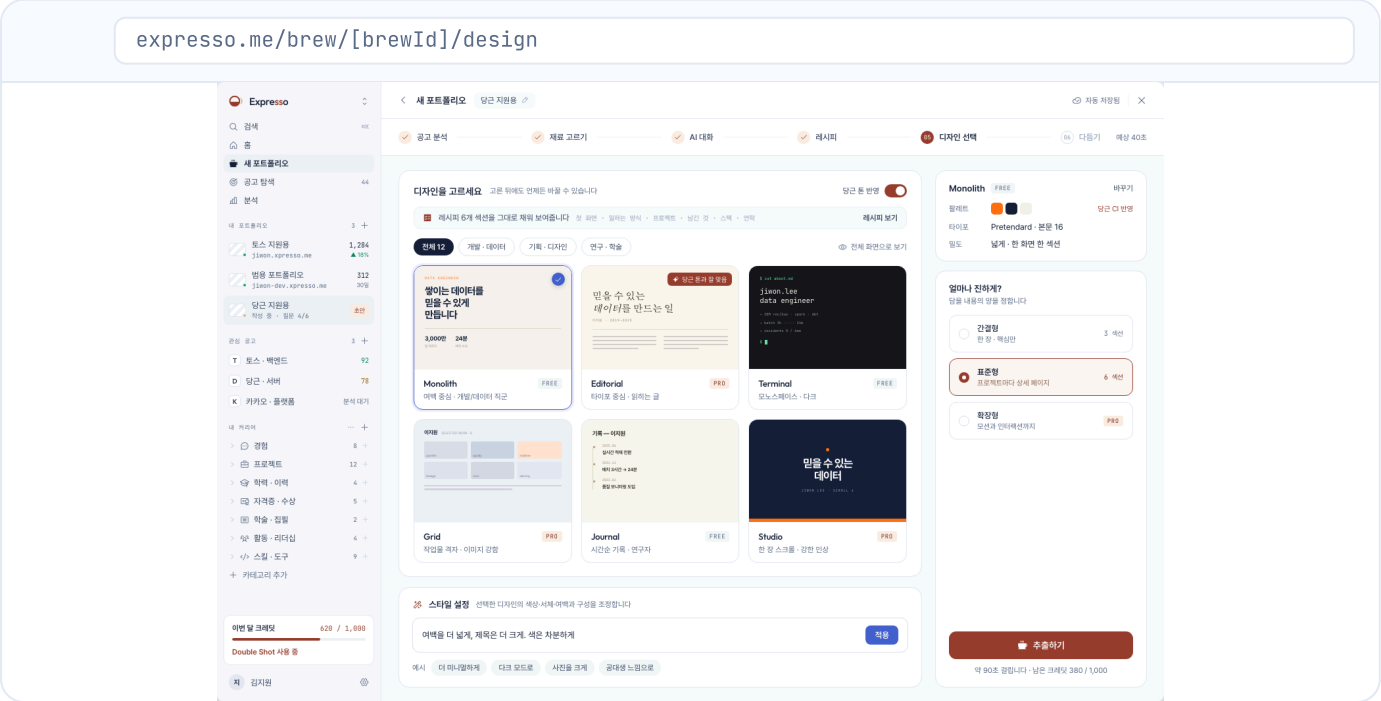


그림 5.3.8 디자인 선택 (화면번호 03)

본 화면은 레시피에 맞는 디자인을 고르는 단계이다. 레시피의 섹션을 실제 내용으로 채운 견본을 제시하여, 사용자가 자신의 포트폴리오가 어떻게 보일지 확인한 뒤에 고르게 한다.

주요 구성 요소

- **견본 목록** — 템플릿마다 사용자의 내용이 채워진 축소 견본을 놓고, 이름 · 성격 요약 · 필요 요금제를 함께 표시한다. 직군별 묶음으로 나누어 볼 수 있고, 「전체 화면으로 보기」로 실제 크기로 확인한다.
- **추천 표시** — 목표 공고의 인상과 잘 맞는 템플릿에 사유를 붙여 표시한다.
- **문장으로 변형** — 고른 템플릿을 문장 하나로 비트는 입력부이다. 예시 문장을 함께 제시하고, 적용 결과를 팔레트 · 타이포그래피 · 밀도의 세 항목으로 요약하여 보인다.
- **분량 선택** — 담을 내용의 양을 세 등급으로 제시한다. 핵심만 담는 한 장 구성, 프로젝트마다 상세 지면을 두는 구성, 모션까지 포함하는 구성이다. 등급마다 예상 섹션 수를 표시한다.
- **생성 진입부** — 예상 소요 시간과 이번 달 남은 생성 횟수를 함께 표시하고 생성을 시작한다.

사용자에게 하는 역할

디자인 선택은 결정의 부담을 낮추는 데 목적이 있다. 사용자는 색과 글꼴을 직접 정하지 않고, 완성된 견본에서 고른 뒤 문장으로 조정한다. 또한 고른 뒤에도 언제든 바꿀 수 있음을 화면에 명시하여, 이 선택이 되돌릴 수 없는 결정이 아님을 알린다.

기능 요구사항

1. 견본은 사용자의 실제 내용으로 채워 제시하며, 빈 서식은 사용하지 않는다.
2. 상위 요금제 전용 템플릿은 목록에서 감추지 않고 필요 등급을 표시한다.
3. 문장으로 지시한 변형은 적용 결과를 항목별 요약으로 확인할 수 있어야 한다.
4. 생성을 시작하기 전에 예상 소요 시간과 남은 생성 횟수를 표시한다.
5. 생성은 사용자가 화면을 떠나도 계속 진행되며, 완료 시 알림으로 통지한다.

제약조건

- 템플릿은 레시피의 섹션 구조를 수용할 수 있는 것만 제시한다. 섹션 수가 맞지 않는 템플릿은 목록에서 제외한다.
- 생성은 이번 달 생성 한도를 소비한다. 한도가 남지 않으면 생성 진입점 대신 남은 횟수와 초기화 날짜를 표시한다.
- 문장 변형은 템플릿이 노출한 조정 범위 안에서만 반영된다. 구조 자체를 바꾸는 지시는 반영하지 않고 그 사실을 알린다.
- 생성 도중에는 같은 제작에 대한 중복 생성을 허용하지 않는다.

■ 다듬기 · 문구 고치기 — 화면번호 04

`/(app)/edit/[portfolioId] · D8`

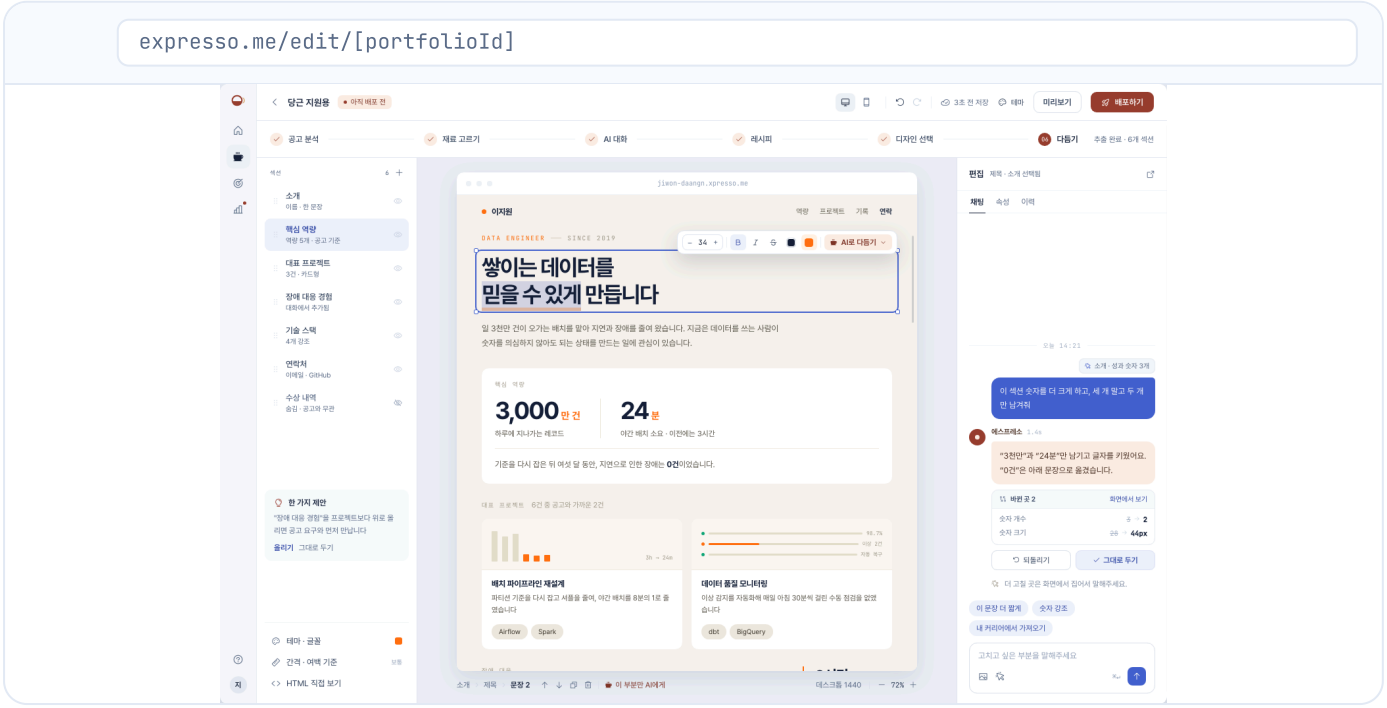


그림 5.3.9 다듬기 · 문구 고치기 (화면번호 04)

본 화면은 생성이 끝난 포트폴리오를 고치는 단계이며, 제작 흐름의 마지막이다. 왼쪽은 섹션 목록, 가운데는 실제 결과물, 오른쪽은 편집 수단이다. 고치는 방법은 두 가지이며, 화면에서 부분을 집어 말로 지시하는 것을 기본으로 하고 글자를 직접 고치는 것을 함께 지원한다.

주요 구성 요소

- **섹션 목록** — 섹션마다 구성 요약을 붙여 나열하고, 감춘 섹션은 감춘 사유와 함께 표시한다. 순서를 바꾸거나 표시 여부를 전환한다.
- **구성 제안** — 공고 요구와 대조하여 순서를 바꾸는 제안을 한 번에 하나만 제시하고, 「올리기」와 「그대로 두기」로 즉시 처리한다.
- **결과물 미리보기** — 배포될 화면을 그대로 보이며, 요소를 선택하면 선택 상태를 표시한다. 미리보기 폭을 전환하여 다른 화면 크기에서의 모습을 확인한다.
- **대화 편집부** — 선택한 요소를 대상으로 고칠 내용을 문장으로 지시한다. 처리 결과는 바뀐 항목 수와 항목별 변경 내용으로 요약되고, 「화면에서 보기」로 해당 위치로 이동하며 「되돌리기」로 그 지시만 취소한다.
- **상단 상태부** — 배포 여부와 마지막 저장 시각을 표시하고, 테마 · 미리보기 · 배포하기로 이동한다.

사용자에게 하는 역할

다듬기는 사용자가 생성 결과를 자신의 의도에 맞게 수정하는 단계이다. 생성된 문장이 어색한 부분을 사용자가 직접 다시 쓰지 않아도 되도록, 고칠 곳을 집어 말로 지시하는 방식을 기본으로 둔다. 또한 지시의 결과를 항목 단위로 되돌릴 수 있게 하여, 수정 결과가 이전보다 나빠지더라도 복구할 수 있게 한다.

기능 요구사항

1. 편집은 요소를 선택한 상태에서 수행하며, 지시의 적용 범위를 선택한 요소로 한정한다.
2. 모든 편집은 자동 저장하고 마지막 저장 시각을 표시한다.
3. 대화 지시의 결과는 변경 항목과 변경 전후를 함께 제시하고, 지시 단위로 되돌릴 수 있어야 한다.
4. 섹션의 표시 여부와 순서는 사용자가 직접 변경할 수 있다.
5. 편집 중 다른 경로에서 같은 포트폴리오가 수정된 경우, 나중의 저장을 거부하고 사용자에게 알린다.

제약조건

- 편집은 배포된 문서를 즉시 바꾸지 않는다. 공개된 내용은 새로 배포할 때에만 갱신된다.
- 대화 편집은 언어 모델 호출을 포함하므로 지시마다 지연이 발생한다. 처리 중에는 같은 요소에 대한 중복 지시를 받지 않는다.
- 구조를 바꾸는 지시는 템플릿이 허용하는 범위로 제한한다.
- 제안은 한 번에 하나만 제시한다. 여러 제안을 동시에 띄우면 사용자가 무엇을 먼저 처리해야 하는지 알 수 없기 때문이다.

■ 다듬기 · 값 미세 조정 — 화면번호 04b

`/(app)/edit/[portfolioId] · D8`

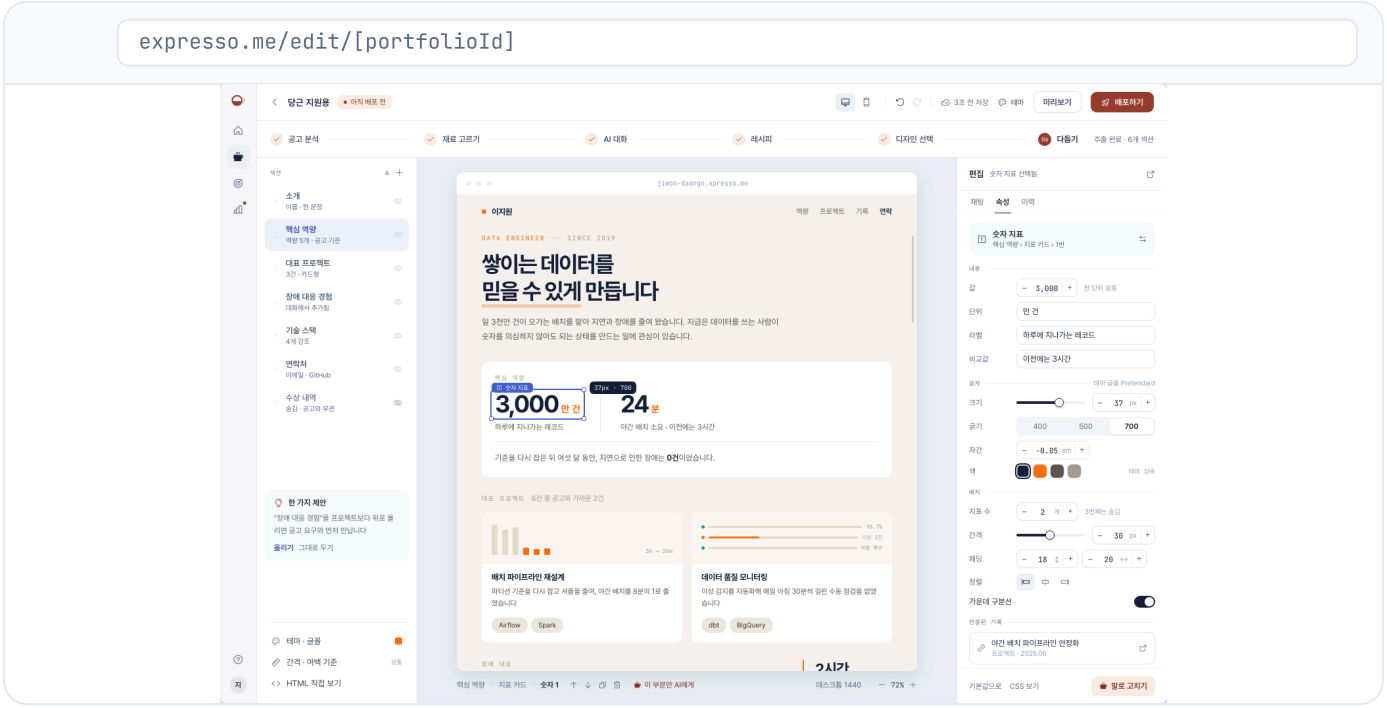


그림 5.3.10 다듬기 · 값 미세 조정 (화면번호 04b)

본 화면은 다듬기 화면의 속성 편집 상태이다. 말로 지시하는 방식으로는 원하는 값에 정확히 도달하지 못하는 경우가 있으므로, 선택한 요소의 내용과 글자와 배치를 값으로 직접 조정하는 수단을 함께 둔다.

주요 구성 요소

- **선택 경로 표시** — 지금 편집하는 요소의 위치를 상위 섹션부터 순서대로 표시하여, 어느 요소를 고치는 중인지 분명히 한다.
- **내용 항목** — 값 · 단위 · 라벨 · 비교값과 천 단위 구분 표시 여부를 조정한다.
- **글자 항목** — 글꼴 · 크기 · 굵기 · 자간 · 색을 조정한다. 값을 지정하지 않은 항목은 테마에서 상속받는다는 것을 명시한다.
- **배치 항목** — 표시 개수 · 간격 · 여백 · 정렬을 조정한다. 개수를 줄이면 감춰지는 항목을 함께 알린다.
- **기록 동기화 표시** — 이 값과 연결된 커리어 기록을 표시하고, 기록이 바뀌면 값도 바뀐다는 규칙을 적는다. 직접 고친 값은 기록과 달라질 수 있음을 알리고 「기본값으로」로 되돌린다.

사용자에게 하는 역할

속성 편집은 결과물의 세부까지 사용자가 통제할 수 있게 한다. 다만 조정 항목을 무제한으로 열지 않고 테마가 정한 범위 안에서만 노출하여, 사용자가 값을 만지다가 디자인이 무너지는 일을 방지한다.

기능 요구사항

1. 속성 편집은 선택한 요소에만 적용하고, 편집 대상의 경로를 화면에 표시한다.
2. 지정하지 않은 속성은 테마 값을 상속하고, 상속 중임을 사용자가 구분할 수 있어야 한다.
3. 직접 고친 값과 기록에서 온 값을 구분하여 표시하고, 기본값으로 되돌리는 수단을 제공한다.
4. 값 변경은 미리보기에 즉시 반영한다.
5. 속성으로 고친 결과도 대화 편집과 동일하게 변경 이력에 기록한다.

제약조건

- 조정 가능한 속성의 범위는 템플릿이 정의한 것으로 한정한다. 임의의 스타일 규칙을 사용자가 추가하지는 못한다.
- 직접 수정한 값은 기록과의 동기화가 해제된다. 이후 기록이 바뀌어도 자동으로 갱신되지 않으며, 이 상태를 화면에 명시한다.
- 값 조정은 화면 크기에 따라 결과가 달라질 수 있으므로, 미리보기 폭을 바꾸어 확인하도록 안내한다.
- 속성 편집은 데스크톱 환경을 기준으로 설계한다. 좁은 화면에서는 대화 편집을 기본으로 제시한다.

■ 다듬기 · 변경 이력 — 화면번호 04c

`/(app)/edit/[portfolioId] · D8`

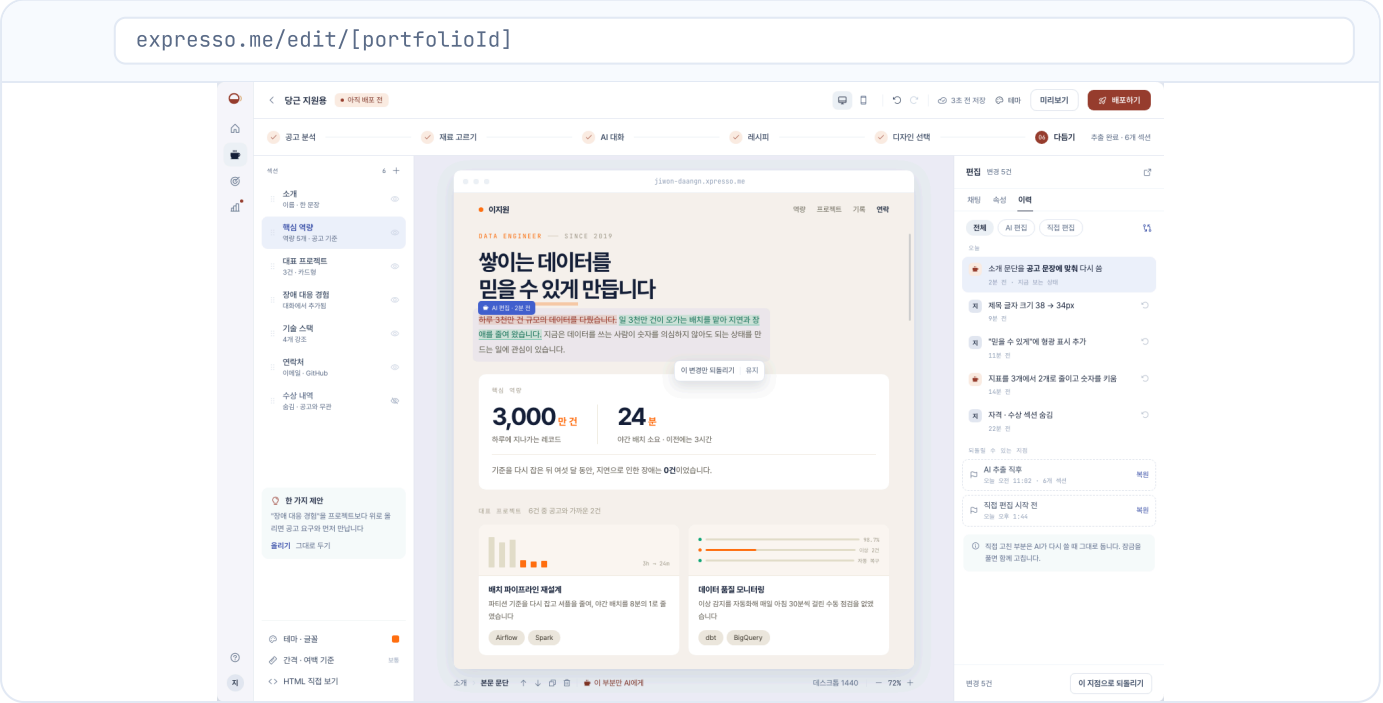


그림 5.3.11 다듬기 · 변경 이력 (화면번호 04c)

본 화면은 다듬기 화면의 변경 이력 상태이다. AI가 고친 것과 사용자가 직접 고친 것을 구분하여 시간 순으로 쌓고, 변경 하나만 되돌리는 것과 특정 시점 전체로 되돌리는 것을 모두 지원한다.

주요 구성 요소

- **이력 목록** — 변경마다 무엇이 어떻게 바뀌었는지와 경과 시간을 적고, 현재 보고 있는 상태에 표시를 둔다. 전체 · AI 편집 · 직접 편집으로 필터링한다.
- **변경 대조** — 문장이 바뀐 경우 이전 문장과 이후 문장을 나란히 놓고, 「이 변경만 되돌리기」와 「유지」로 처리한다.
- **복원 지점** — 생성 직후, 직접 편집 시작 전과 같이 의미 있는 시점을 따로 표시하여 그 지점으로 되돌린다.
- **보호 규칙 안내** — 사용자가 직접 고친 부분은 이후 AI가 다시 쓸 때 그대로 둔다는 규칙과, 잠금을 풀면 함께 고쳐진다는 사실을 함께 적는다.

사용자에게 하는 역할

변경 이력은 사용자가 마음 놓고 고치게 하는 장치이다. 무엇이 바뀌었는지 되짚을 수 있고 언제든 되돌릴 수 있어야, 사용자는 AI에게 편집을 맡기는 위험을 감수할 수 있다. 또한 직접 고친 부분을 보호하여 사용자의 수정이 다음 AI 편집에 지워지지 않게 한다.

기능 요구사항

1. 모든 편집은 주체(AI · 사용자)와 시각과 변경 내용을 함께 기록한다.
2. 되돌리기는 변경 단위와 시점 단위를 모두 지원한다.
3. 사용자가 직접 고친 요소는 잠긴 상태로 표시하고, AI 편집에서 제외한다. 잠금은 사용자가 해제할 수 있다.
4. 현재 보고 있는 상태가 이력의 어느 지점인지 표시한다.
5. 복원 지점은 생성 직후와 직접 편집 시작 시점을 자동으로 남긴다.

제약조건

- 이력은 편집 중인 포트폴리오를 대상으로 하며, 이미 배포된 버전은 배포 화면의 버전 관리로 되돌린다.
- 되돌리기 이후의 변경은 새 이력으로 쌓이고, 되돌리기 자체도 이력에 남는다.
- 이력의 보관 기간과 건수에는 상한을 둔다. 상한을 넘은 이력은 복원 지점만 남기고 정리한다.
- 여러 사람이 함께 편집하는 경우 주체는 사용자 단위로 구분하여 기록한다.

■ 내 커리어 — 화면번호 05

`/(app)/career/[categorySlug] · D1`

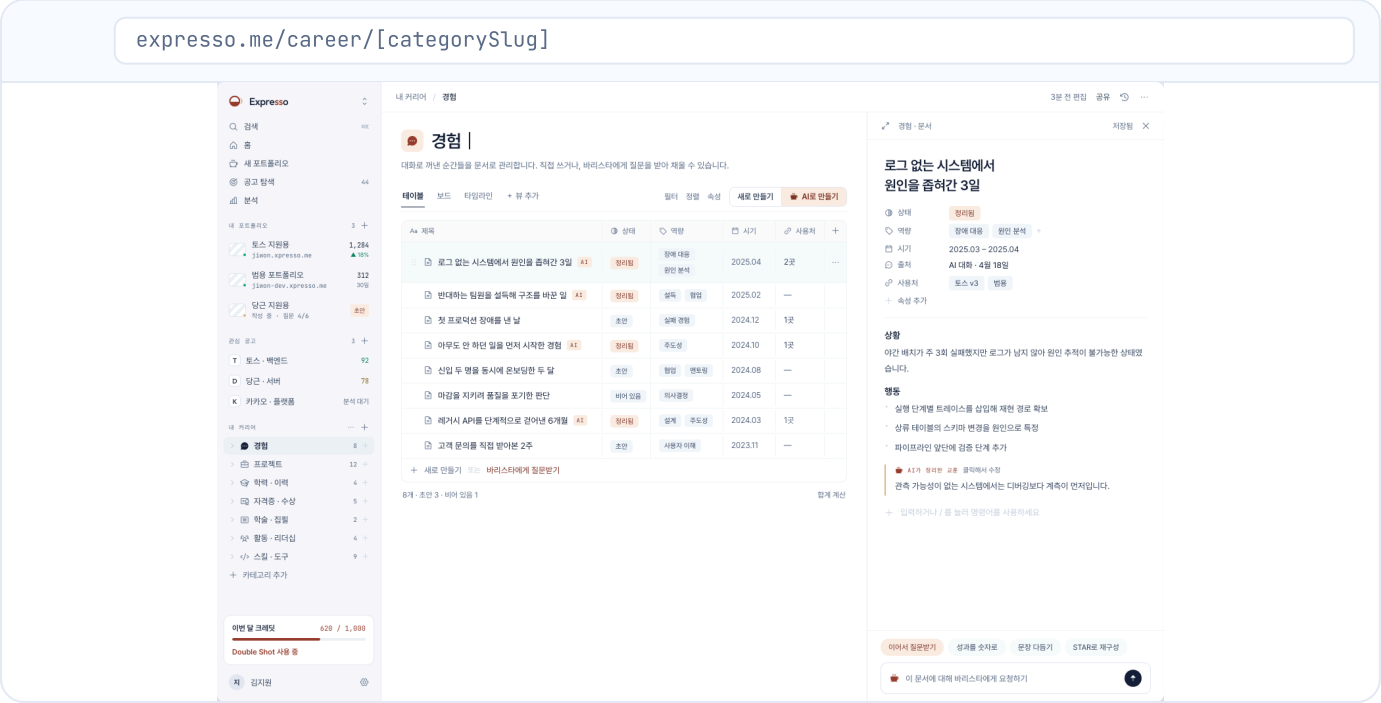


그림 5.3.12 내 커리어 (화면번호 05)

본 화면은 사용자의 커리어 기록을 관리하며, 경험 카테고리를 예로 든 상태이다. 기록은 표 · 보드 · 타임라인의 여러 뷰로 볼 수 있고, 항목을 선택하면 오른쪽에 문서로 열린다. 기록은 직접 쓰거나 대화로 채우며, 두 방법의 결과가 같은 형식으로 저장된다.

주요 구성 요소

- **뷰 전환부** — 같은 기록을 표 · 보드 · 타임라인으로 바꾸어 본다. 뷰마다 필터 · 정렬 · 표시 속성을 따로 저장하고 「뷰 추가」로 뷰를 늘린다.
- **기록 목록** — 제목 · 상태 · 역량 · 시기 · 사용처를 열로 갖는다. 대화로 만든 기록에는 표시를 붙이고, 내용이 없는 기록은 「비어 있음」으로, 정리가 끝난 기록은 「정리됨」으로 상태를 구분한다. 사용처 열은 그 기록이 어느 포트폴리오에 쓰였는지 건수로 알린다.
- **기록 문서** — 선택한 기록을 문서로 열어 제목 · 기간 · 출처 · 사용처와 속성을 보이고, 본문은 상황 · 행동과 같은 구조로 나누어 적는다. 대화로 만든 기록에는 출처와 그 대화의 날짜를 남긴다.
- **AI 정리 결과** — 본문에서 뽑은 교훈이나 요점을 별도로 두고, 눌러서 수정할 수 있게 한다.
- **요청 도구** — 「성과를 숫자로」 · 「문장 다듬기」 · 「STAR로 재구성」과 같이 이 문서에 대하여 자주 하는 요청을 버튼으로 제시하고, 「이어서 질문받기」로 대화로 되돌아간다.

사용자에게 하는 역할

커리어 기록은 포트폴리오에 사용할 재료를 축적하는 영역이다. 포트폴리오는 지원할 때마다 새로 생성하지만 기록은 한 번 축적하면 계속 사용하므로, 사용자가 기록을 지속적으로 관리하게 하는 것이 이 화면의 목적이다. 또한 어느 기록이 실제로 포트폴리오에 쓰였는지 표시하여, 다음에 무엇을 보강할지 판단하게 한다.

기능 요구사항

1. 기록은 카테고리에 속하고, 카테고리마다 속성 구성과 기본 뷰가 다르다.
2. 뷰의 필터 · 정렬 · 표시 속성은 사용자별로 저장하여 다시 방문할 때 유지한다.
3. 기록의 출처(직접 작성 · 대화 · 문서 추출)를 구분하여 표시한다.
4. 기록마다 어느 포트폴리오에 사용되었는지 건수와 대상을 표시한다.
5. 기록은 삭제하여도 즉시 지우지 않고 삭제 표시로 남겨, 사용 중인 포트폴리오가 깨지지 않게 한다.

제약조건

- 기록의 속성 구성은 카테고리의 속성 정의를 따른다. 특정 기록에만 임의의 속성을 추가하지는 못한다.
- 이미 배포된 포트폴리오는 배포 시점의 스냅샷을 사용하므로, 기록을 고쳐도 공개된 내용은 바뀌지 않는다.
- 대화로 만든 기록도 사용자가 수정할 수 있으나, 수정 이후에는 대화의 원문과 달라질 수 있다.
- 기록의 열람과 편집은 소유자로 한정한다. 공유는 읽기 전용 링크로만 제공한다.

■ 내 커리어 · 프로젝트 — 화면번호 05b

`/(app)/career/[categorySlug] · D1`

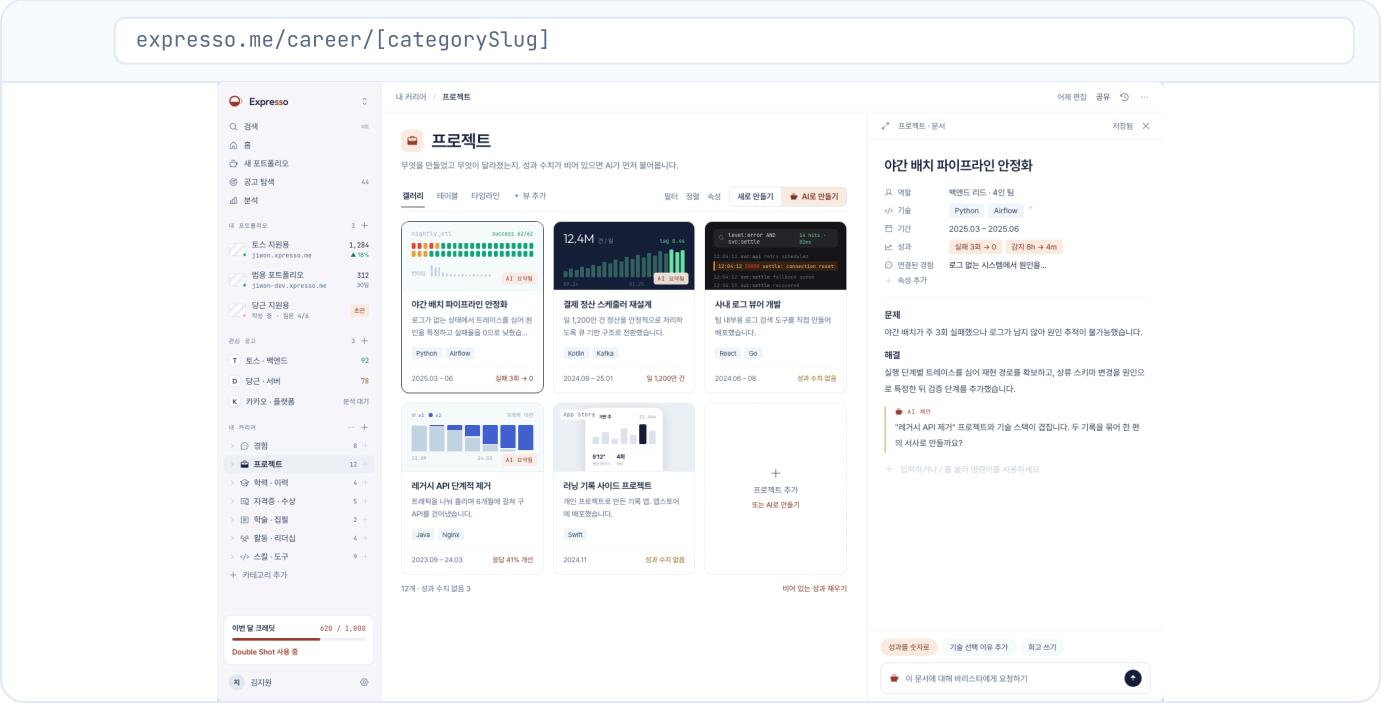


그림 5.3.13 내 커리어 · 프로젝트 (화면번호 05b)

본 화면은 커리어 기록 중 프로젝트 카테고리이다. 카테고리마다 속성과 뷰가 다르다는 원칙을 보이는 예로, 프로젝트는 갤러리 뷰를 기본으로 하고 결과물의 화면이나 지표를 카드의 앞면에 둔다.

주요 구성 요소

- **갤러리 뷰** — 프로젝트마다 대표 화면과 한 문장 요약 · 사용 기술 · 기간 · 핵심 성과를 카드로 보인다. 성과 수치가 없는 프로젝트에는 「성과 수치 없음」을 표시한다.
- **보강 안내** — 목록 아래에 전체 건수와 성과 수치가 없는 건수를 함께 적고, 「비어 있는 성과 채우기」로 해당 항목만 모아 처리한다.
- **프로젝트 문서** — 역할 · 사용 기술 · 기간 · 성과 · 연결된 경험을 속성으로 갖고, 본문은 문제와 해결로 나누어 적는다.
- **기록 간 연결** — 프로젝트에서 파생된 경험 기록을 연결로 표시하여, 같은 사건을 두 카테고리에 중복 입력하지 않게 한다.
- **요청 도구** — 「성과를 숫자로」 · 「기술 선택 이유 추가」 · 「회고 쓰기」와 같이 프로젝트에 특화된 요청을 제시한다.

사용자에게 하는 역할

프로젝트 카테고리는 결과물을 보여줄 수 있는 형태로 정리하게 한다. 프로젝트는 결과물이 있으므로 글보다 화면과 수치가 먼저 읽히며, 이 화면은 그 순서대로 정보를 배치한다. 또한 성과 수치가 빠진 프로젝트를 목록에서 식별하게 하여, 포트폴리오에 쓰기 전에 채우도록 유도한다.

기능 요구사항

1. 카테고리마다 기본 뷰와 속성 구성을 다르게 정의할 수 있어야 한다.
2. 프로젝트는 대표 이미지를 가질 수 있고, 이미지가 없으면 지표나 텍스트로 대체 표시한다.
3. 성과 수치의 유무를 목록에서 식별할 수 있게 표시하고, 없는 항목만 모아 보강할 수 있게 한다.
4. 프로젝트와 경험 기록을 서로 연결할 수 있어야 하며, 연결된 기록은 양쪽에서 함께 보인다.
5. 업로드한 이미지는 원본과 표시용 크기를 함께 보관한다.

제약조건

- 업로드 파일의 형식과 용량에는 상한을 둔다. 상한을 넘는 파일은 업로드 단계에서 거부한다.
- 대표 이미지는 포트폴리오 템플릿에 따라 잘리는 비율이 달라지므로, 목록에서는 동일 비율로 맞추어 표시한다.
- 기록 간 연결은 같은 사용자의 기록 사이에서만 만든다.
- 성과 수치는 사용자가 입력한 값이며 서비스가 검증하지 않는다. 이 때문에 출처가 된 기록을 함께 연결하도록 유도한다.

■ 내 커리어 · 학력 · 이력 — 화면번호 05c

`/(app)/career/[categorySlug] · D1`

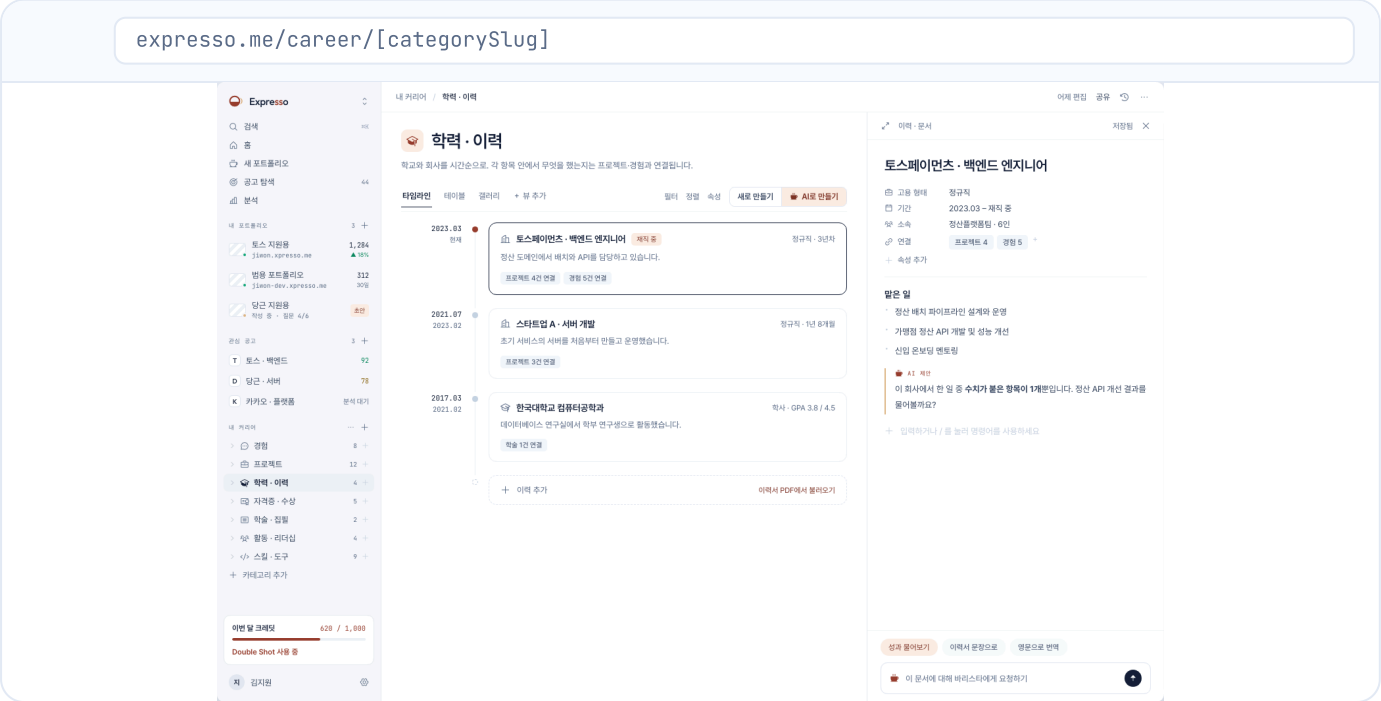


그림 5.3.14 내 커리어 · 학력 · 이력 (화면번호 05c)

본 화면은 커리어 기록 중 학력과 이력 카테고리이다. 시간 순서가 중요한 카테고리이므로 타임라인 뷰를 기본으로 하고, 현재 재직 중인 항목을 맨 위에 둔다. 각 이력은 그 기간에 수행한 프로젝트와 경험 기록을 연결로 거느린다.

주요 구성 요소

- **타임라인 뷰** — 재직과 재학 이력을 최근 순으로 세로로 놓고, 진행 중인 항목에 상태 표시를 붙인다. 항목마다 고용 형태 · 기간 · 한 문장 요약과 연결된 기록 건수를 적는다.
- **이력 문서** — 고용 형태 · 기간 · 소속 조직과 규모를 속성으로 갖고, 본문에 담당한 일을 목록으로 적는다.
- **연결 표시** — 이 이력에 연결된 프로젝트와 경험의 건수를 표시하고, 눌러서 해당 기록으로 이동한다.
- **이력서 불러오기** — 이력서 파일에서 이력 항목을 추출하여 목록으로 만든다.
- **AI 제안** — 수치가 붙은 항목이 부족한 이력을 지적하고, 그 항목을 채우는 질문을 제안한다.

사용자에게 하는 역할

학력과 이력은 사용자의 경력을 시간 축으로 정리하여, 포트폴리오와 이력서가 공통으로 사용하는 기본 구조를 구성한다. 개별 기록은 사건 단위이지만 채용 담당자는 시간 순의 경력을 먼저 확인하므로, 두 표현을 모두 갖추게 한다. 또한 이력마다 연결된 기록 수를 보여, 근거가 비어 있는 기간을 사용자가 발견하게 한다.

기능 요구사항

1. 이력은 시작과 종료 시점을 갖고, 종료가 없는 항목은 진행 중으로 표시한다.
2. 타임라인은 기간이 겹치는 이력을 함께 표시할 수 있어야 한다.
3. 이력과 프로젝트 · 경험 기록을 연결하고, 연결 건수를 이력 항목에 표시한다.
4. 이력서 파일에서 추출한 항목은 저장 전에 사용자가 확인하고 수정할 수 있어야 한다.
5. 이력의 순서는 시간을 기준으로 자동 정렬하며 사용자가 임의로 바꾸지 않는다.

제약조건

- 이력서 추출은 파일의 구조에 따라 정확도가 달라진다. 이 때문에 추출 결과를 그대로 저장하지 않고 확인 단계를 반드시 둔다.
- 기간이 겹치거나 순서가 맞지 않는 입력은 저장은 허용하되 화면에서 표시하여 사용자가 확인하게 한다.
- 학력과 재직 이력은 사용자가 입력한 내용이며 서비스가 사실 여부를 검증하지 않는다.
- 타임라인 뷰는 항목 수가 많아지면 가독성이 떨어지므로, 일정 수를 넘으면 표 뷰를 함께 권한다.

■ 내 커리어 · 스킬 · 도구 — 화면번호 05d

`/(app)/career/[categorySlug] · D1`

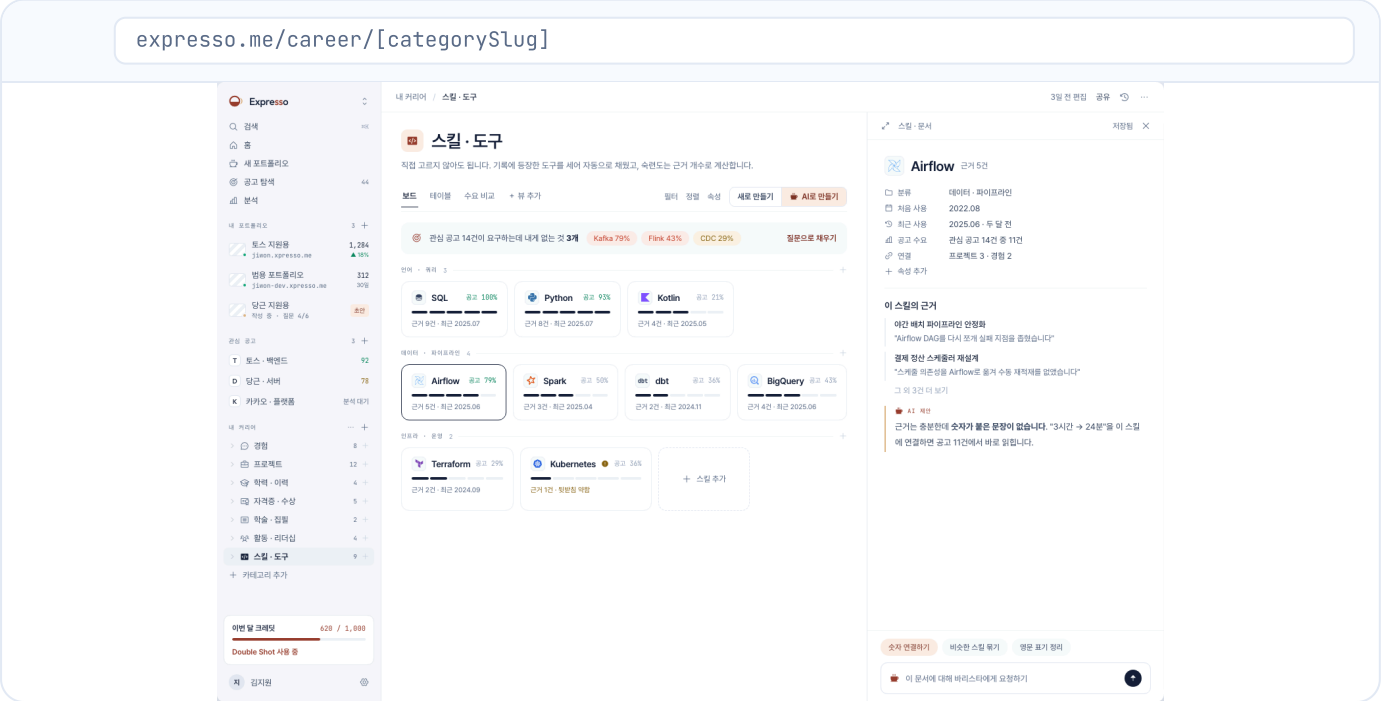


그림 5.3.15 내 커리어 · 스킬 · 도구 (화면번호 05d)

본 화면은 커리어 기록 중 스킬과 도구 카테고리이다. 스킬은 기록에서 추출하며 사용자가 직접 나열하지 않는다. 근거가 있는 스킬만 포트폴리오에 사용한다. 화면은 스킬을 분류별로 묶고, 각 스킬의 공고 수요와 근거 건수를 함께 제시한다.

주요 구성 요소

- **수요 대비 표시** — 관심 공고가 요구하는데 사용자에게 없는 스킬을 상단에 모아 요구 비율과 함께 제시하고, 「질문으로 채우기」로 대화로 연결한다.
- **스킬 목록** — 언어 · 데이터 · 인프라와 같은 분류별로 묶고, 스킬마다 공고 수요 비율 · 근거 건수 · 최근 사용 시점을 적는다. 근거가 부족한 스킬은 「뒷받침 약함」으로 표시한다.
- **스킬 문서** — 분류 · 처음 사용 시점 · 최근 사용 시점 · 공고 수요 · 연결된 기록을 속성으로 갖는다.
- **근거 목록** — 이 스킬을 뒷받침하는 기록과 그 기록에서 해당 스킬이 언급된 문장을 인용으로 함께 보인다.
- **AI 제안** — 근거는 있으나 수치가 없는 스킬을 지적하고, 연결할 수치를 제안한다.

사용자에게 하는 역할

스킬 카테고리가 관리하는 것은 목록의 신뢰도이다. 스킬을 자유롭게 적을 수 있게 하면 근거 없는 나열이 되므로, 기록에서 추출하고 근거를 함께 보관한다. 또한 공고 수요와 대조하여, 사용자가 다음에 쌓거나 문서화할 스킬의 우선순위를 정하게 한다.

기능 요구사항

1. 스킬은 기록에서 추출하며, 추출한 근거의 건수와 원문 인용을 함께 보관한다.
2. 근거가 없는 스킬은 포트폴리오 생성에 사용하지 않는다.
3. 각 스킬의 공고 수요는 사용자의 관심 공고를 대상으로 산출하고, 산출 대상 건수를 함께 표시한다.
4. 최근 사용 시점을 관리하여 오래 쓰지 않은 스킬을 구분할 수 있게 한다.
5. 사용자가 스킬을 직접 추가할 수 있으나, 근거가 없는 상태임을 표시한다.

제약조건

- 공고 수요는 사용자가 관심으로 등록한 공고에 한정하여 계산한다. 전체 채용 시장의 통계가 아니다.
- 스킬 추출은 기록의 문장을 근거로 하므로, 기록에 적히지 않은 역량은 목록에 나타나지 않는다.
- 표기가 다른 같은 스킬이 별개로 잡힐 수 있다. 이 경우 사용자가 병합하도록 수단을 제공한다.
- 숙련도는 서비스가 임의로 등급을 매기지 않는다. 근거 건수와 최근 사용 시점으로만 표현한다.

■ 내 커리어 · 학술 · 집필 — 화면번호 05e

`/(app)/career/[categorySlug] · D1`

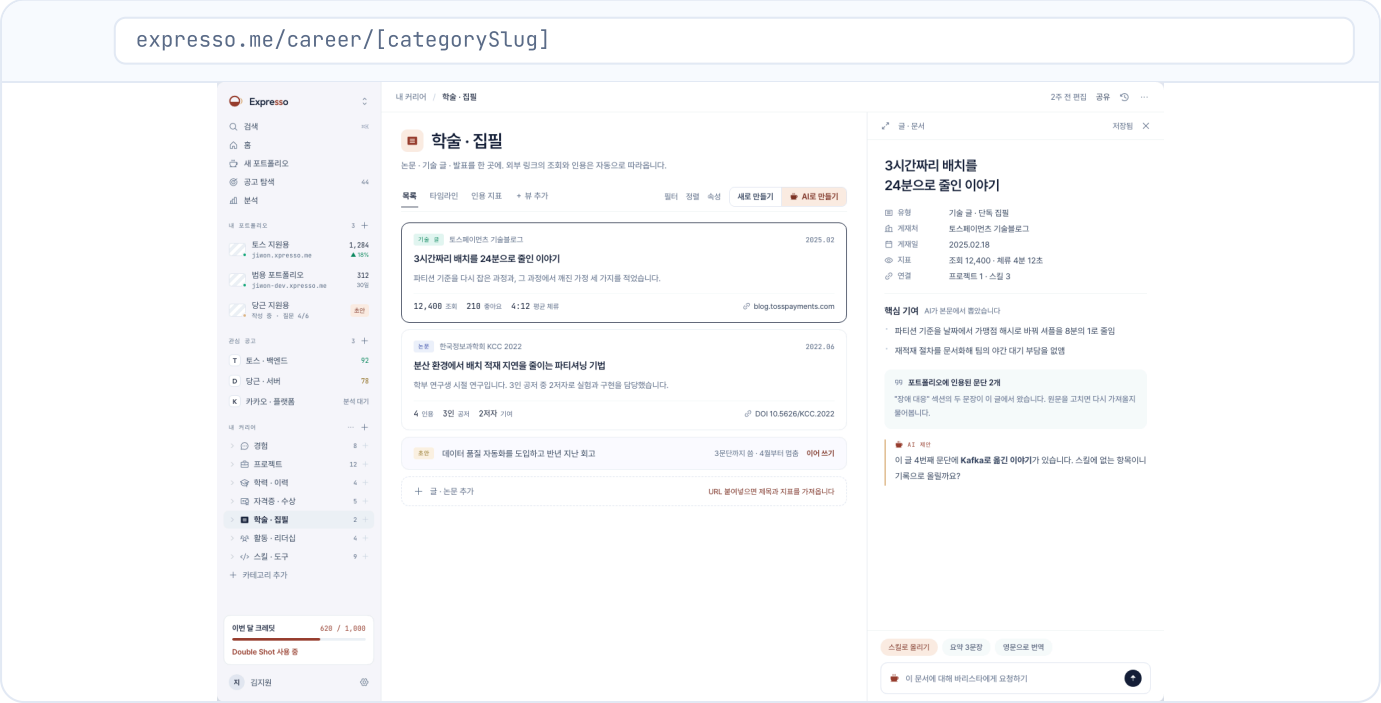


그림 5.3.16 내 커리어 · 학술 · 집필 (화면번호 05e)

본 화면은 커리어 기록 중 학술과 집필 카테고리이다. 글과 논문은 외부에 공개된 결과물이므로 조회 수 · 인용 수와 같은 외부 지표를 함께 관리하고, 그 내용이 포트폴리오의 어느 문단으로 인용되었는지도 함께 표시한다.

주요 구성 요소

- **목록 뷰** — 기술 글과 논문을 유형 배지와 함께 나열하고, 항목마다 게재처 · 제목 · 요약과 외부 지표를 적는다. 작성이 중단된 글은 진행 상태와 함께 「이어 쓰기」를 둔다.
- **지표 표시** — 글은 조회 · 좋아요 · 평균 체류 시간을, 논문은 인용 수와 공저 정보와 기여 역할을 제시한다.
- **주소로 등록** — 글의 주소를 붙여넣으면 제목과 지표를 가져와 항목을 만든다.
- **기록 문서** — 유형 · 게재처 · 게재일 · 지표 · 연결된 기록을 속성으로 갖고, 본문에서 뽑은 핵심 기여를 목록으로 둔다.
- **인용 표시** — 이 글의 어느 문단이 포트폴리오에 인용되었는지 건수로 표시한다.
- **AI 제안** — 본문에 있으나 스킬로 등록되지 않은 항목을 찾아 기록으로 올리도록 제안한다.

사용자에게 하는 역할

학술과 집필 카테고리는 사용자가 이미 외부에 써 놓은 글을 커리어 자산으로 회수하게 한다. 글과 논문은 그 자체로 근거가 되므로, 본문에서 핵심 기여를 뽑아 두면 포트폴리오 작성에 바로 쓸 수 있다. 또한 인용된 문단을 표시하여, 어떤 글이 실제로 성과에 기여했는지 확인하게 한다.

기능 요구사항

1. 글과 논문은 유형을 구분하여 저장하고, 유형에 따라 표시하는 지표 항목을 달리한다.
2. 외부 주소로 등록한 항목은 제목과 지표를 자동으로 수집하되, 사용자가 수정할 수 있어야 한다.
3. 본문에서 추출한 핵심 기여는 사용자가 수정 및 삭제할 수 있다.
4. 포트폴리오에 인용된 문단의 건수를 항목에 표시한다.
5. 작성 중인 글은 진행 상태를 보관하여 같은 지점에서 이어 쓸 수 있게 한다.

제약조건

- 외부 지표는 수집 시점의 값이므로 실시간 값과 다를 수 있다. 이 때문에 수집 시점을 함께 보관한다.
- 지표 수집은 공개된 페이지에 한정한다. 로그인이 필요한 매체의 지표는 사용자가 직접 입력한다.
- 논문의 인용 수는 외부 색인 서비스의 값에 의존하므로 매체에 따라 제공되지 않을 수 있다.
- 본문 전체를 서비스에 저장하지 않고 요약과 인용 문단만 보관한다. 저작권이 매체에 있는 경우가 있기 때문이다.

공고 탐색 — 화면번호 06

`/(app)/jobs · D2`

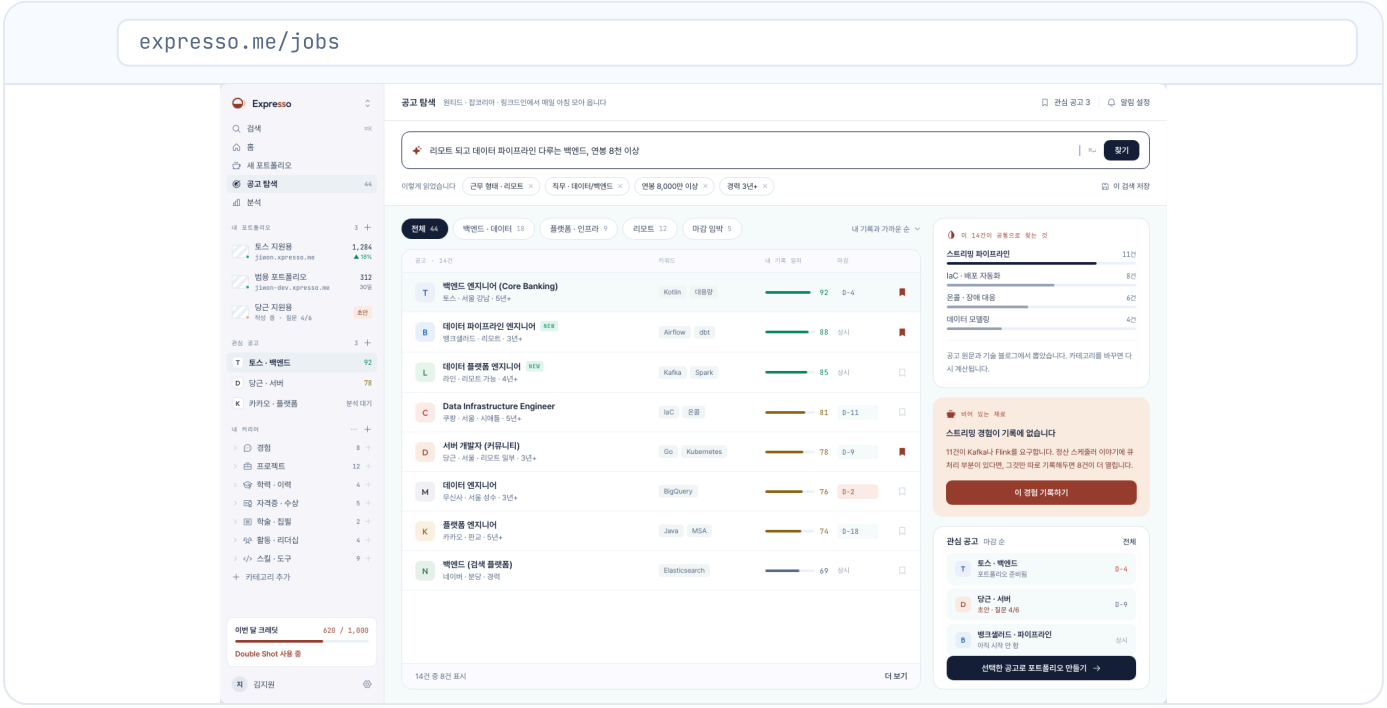


그림 5.3.17 공고 탐색 (화면번호 06)

본 화면은 수집한 공고를 탐색한다. 카테고리로 묶어 보는 방식과 조건을 문장으로 적어 찾는 방식을 함께 제공하고, 목록 오른쪽에 그 결과 전체가 공통으로 요구하는 것을 요약하여 개별 공고와 시장 경향을 함께 읽게 한다.

주요 구성 요소

- **검색 입력부** — 조건을 문장으로 받아 해석 결과를 칩으로 제시한다. 「이 검색 저장」으로 조건을 등록하면 새 공고가 올라올 때 알린다.
- **카테고리 탭** — 전체 · 직무별 · 근무 형태별 · 마감 임박 · 기록과 가까운 순의 묶음으로 목록을 바꾼다.
- **공고 목록** — 회사 · 근무지 · 요구 경력 · 주요 키워드 · 기록 일치 여부 · 마감 잔여일을 열로 갖는다. 최근 등록된 공고에는 표시를 붙이고, 표시 건수와 전체 건수를 함께 적는다.
- **공통 요구 요약** — 현재 목록의 공고들이 공통으로 요구하는 항목을 건수와 함께 나열하고, 산출에 사용한 자료와 카테고리를 바꾸면 다시 계산된다는 사실을 명시한다.
- **제작 진행 표시** — 관심 공고마다 포트폴리오 준비 상태를 「준비됨 · 초안 · 아직 시작 안 함」으로 표시하고, 「선택한 공고로 포트폴리오 만들기」로 제작을 시작한다.

사용자에게 하는 역할

공고 탐색은 지원할 곳을 고르는 자리이면서, 시장이 요구하는 것을 파악하는 자리이기도 하다. 공고를 한 건씩 읽는 것만으로는 무엇을 준비해야 하는지 알기 어려우므로, 목록 전체의 공통 요구를 함께 제시하여 준비의 방향을 잡게 한다.

기능 요구사항

1. 공고는 정기적으로 수집하고, 마지막 수집 시점을 화면에 표시한다.
2. 카테고리를 바꾸면 공통 요구 요약을 다시 산출한다.
3. 각 공고에 사용자의 기록 일치 여부를 표시하고, 마감이 임박한 공고를 구분한다.
4. 관심 공고로 등록한 공고는 제작 진행 상태를 함께 표시한다.
5. 저장한 검색은 조건과 알림 여부를 함께 보관한다.

제약조건

- 수집 대상은 어댑터가 구현된 채널로 한정한다. 채널이 제공하지 않는 항목은 목록에서 비어 있는 상태로 표시한다.
- 공고의 원문 권리는 게시처에 있으므로 원문 전체를 재배포하지 않고, 열람과 지원은 원 게시처로 연결한다.
- 마감이 지난 공고는 목록에서 제외하되 관심으로 등록한 공고는 지난 사실을 표시하여 남긴다.
- 공통 요구 요약은 현재 목록을 대상으로 하므로, 필터를 바꾸면 값이 달라진다.

■ 공고 상세 — 화면번호 06b

`/(app)/jobs/[jobId] · D2 · D3`

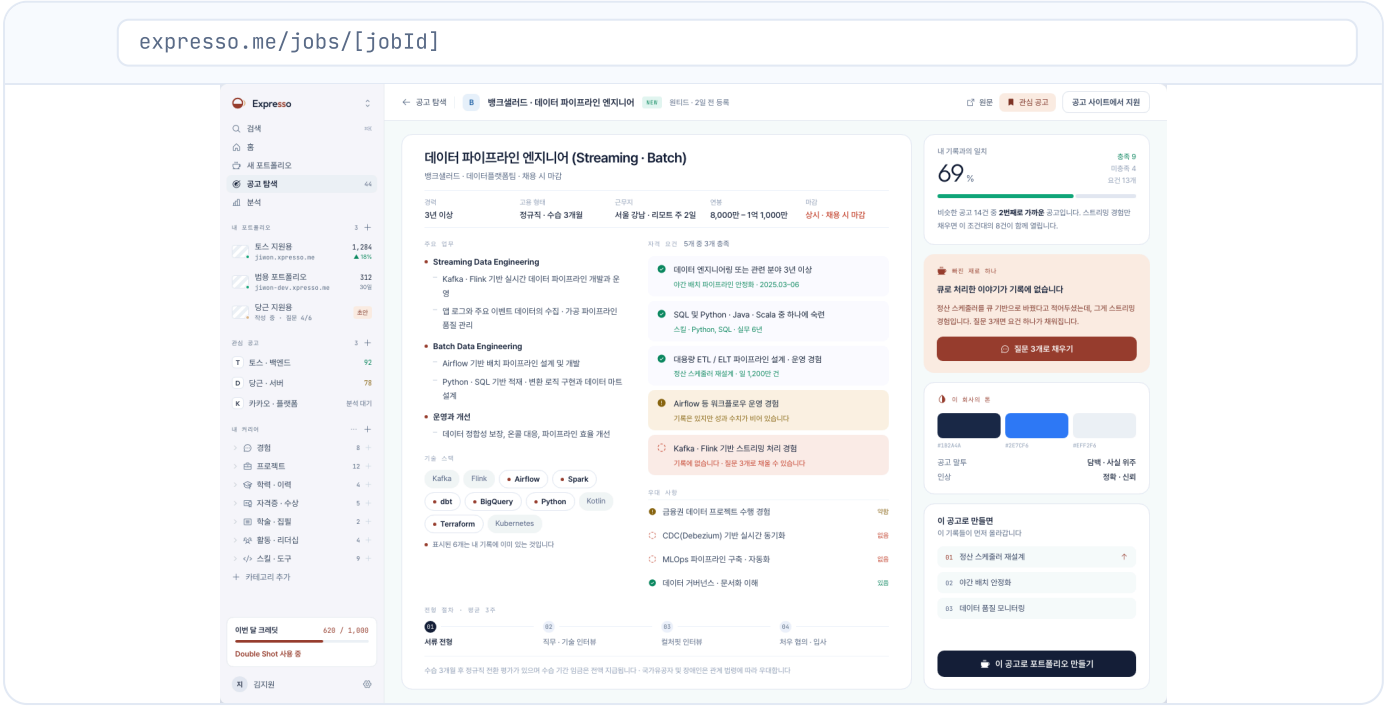


그림 5.3.18 공고 상세 (화면번호 06b)

본 화면은 공고 한 건의 상세 화면이다. 왼쪽에 공고 원문을 구조대로 두고, 오른쪽에 사용자의 기록과 대조한 결과를 둔다. 원문을 읽는 동시에 자신의 기록이 어느 요건에 닿는지 확인하게 하는 것이 이 화면의 구성 원리이다.

주요 구성 요소

- **공고 요약부** — 회사 · 팀 · 경력 · 고용 형태 · 근무지 · 연봉 · 마감을 표로 정리하고, 원문 보기와 지원 페이지로 이동하는 경로를 함께 둔다.
- **원문 본문** — 주요 업무 · 기술 스택 · 자격 요건 · 우대 사항 · 전형 절차를 게시된 구조 그대로 표시한다. 사용자의 기록에 이미 있는 기술에는 표시를 붙인다.
- **요건 대조** — 자격 요건과 우대 사항의 항목마다 충족 여부를 판정하고, 충족한 항목에는 근거가 된 기록을, 부족한 항목에는 보강 방법을 함께 적는다.
- **일치 요약** — 전체 요건 수와 충족 · 미충족 건수를 제시하고, 비슷한 공고 가운데 이 공고의 상대 위치를 알린다.
- **회사 톤 분석** — 공고의 색과 말투에서 읽은 인상을 제시한다. 이후 디자인 선택 단계의 추천 근거로 사용된다.
- **제작 진입부** — 이 공고로 만들면 먼저 배치될 기록을 미리 보이고 제작을 시작한다.

사용자에게 하는 역할

공고 상세가 하는 일은 사용자가 지원 여부를 판단하게 하는 것이다. 요건 충족 여부를 근거와 함께 제시하므로, 사용자는 지금 지원할지 기록을 먼저 채울지 결정할 수 있다. 또한 부족한 요건을 채우는 경로를 화면 안에 두어, 판단이 곧 다음 행동으로 이어지게 한다.

기능 요구사항

1. 공고 원문의 구조를 보존하여 표시하고, 원 게시처로 이동하는 경로를 제공한다.
2. 요건별 판정에는 근거가 된 기록 또는 부족한 이유를 반드시 함께 표시한다.
3. 충족 · 미충족 건수를 요약하여 제시한다.
4. 부족한 요건은 이를 채우는 대화로 연결한다.
5. 이 공고로 제작을 시작하면 분석 결과를 다음 단계로 전달한다.

제약조건

- 요건 판정은 사용자의 기록에 적힌 내용만을 근거로 한다. 기록하지 않은 경험은 미충족으로 판정된다.
- 연봉과 마감 등의 항목은 게시처가 제공하지 않으면 표시하지 않는다. 추정값을 만들지 않는다.
- 판정 결과는 서비스의 해석이며 채용 결과를 보장하지 않는다. 이 사실을 화면에 명시한다.
- 회사 톤 분석은 공개된 공고와 채용 페이지에 한정하여 수행한다.

■ 분석 · 내가 만든 대시보드 — 화면번호 07

`/(app)/analytics · D10`

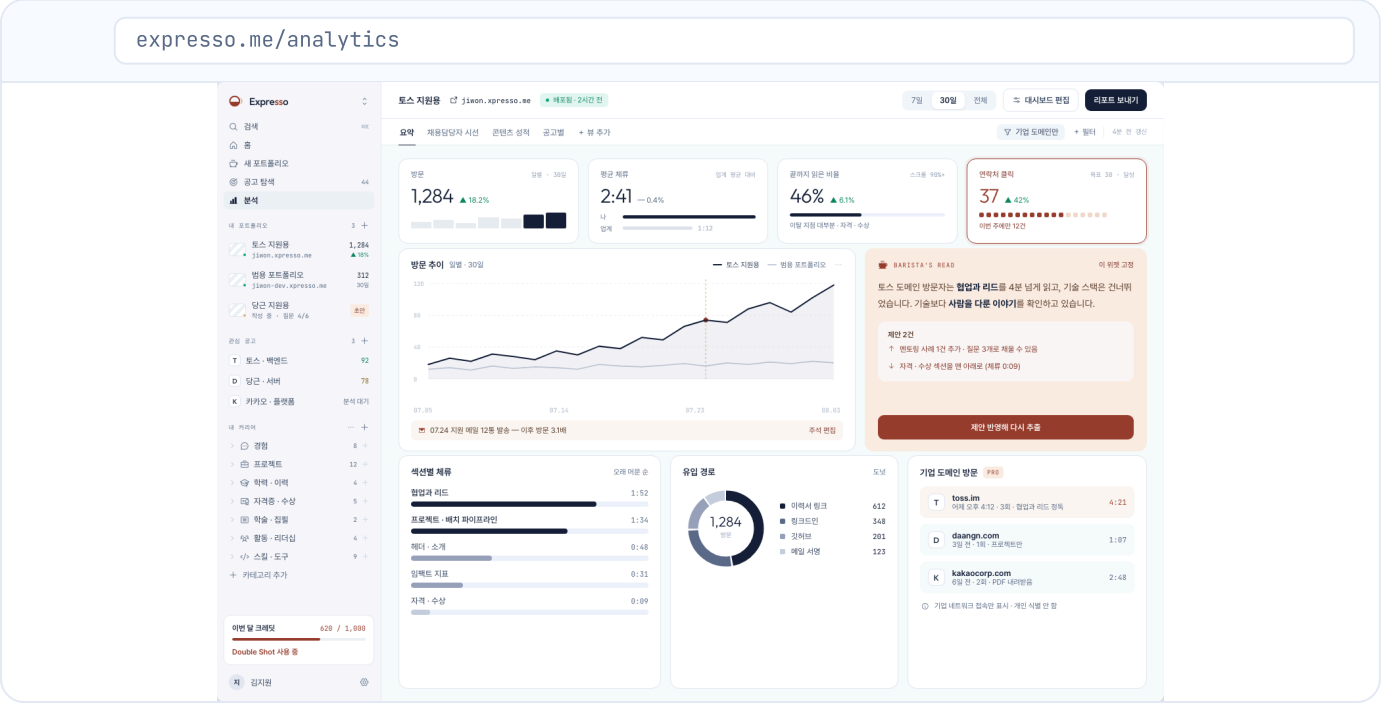


그림 5.3.19 분석 · 내가 만든 대시보드 (화면번호 07)

본 화면은 배포한 포트폴리오의 방문 데이터를 보여준다. 고정된 보고서를 제시하지 않고 사용자가 지표를 골라 구성한 대시보드를 보여준다. 위쪽에 요약 지표를, 아래에 추이와 구성 지표를 두며, 해석은 별도의 위젯으로 분리한다.

주요 구성 요소

- **뷰 탭** — 요약 · 채용담당자 시선 · 콘텐츠 성적 · 공고별과 같이 목적이 다른 대시보드를 탭으로 나누고, 「뷰 추가」로 새 대시보드를 만든다. 뷰마다 기간과 필터를 따로 둔다.
- **요약 지표** — 방문 · 평균 체류 · 완독률 · 연락처 클릭을 전 기간 대비 변화와 함께 제시한다. 비교 대상이 있는 지표는 업계 평균을 함께 표시한다.
- **추이 그래프** — 일별 방문 추이를 그리고, 지원 메일 발송과 같은 사건을 주석으로 표시하여 변화의 원인을 함께 읽게 한다.
- **해석 위젯** — 데이터에서 읽은 내용을 문장으로 제시하고, 그에 따른 개선 제안을 근거와 함께 둔다. 제안을 받아들이면 포트폴리오 재생성으로 연결한다.
- **구성 지표** — 섹션별 체류 시간, 유입 경로, 기업 도메인 방문 목록을 제시한다. 기업 도메인 항목은 상위 요금제에서만 열린다.

사용자에게 하는 역할

분석 화면은 사용자가 포트폴리오를 수정할 근거를 확인하는 화면이다. 방문 수만 제시하면 수정 대상을 판단할 수 없으므로, 섹션별 열람 정도와 이탈 지점을 함께 제시한다. 해석과 제안을 데이터 옆에 두어, 분석이 다시 제작으로 이어지게 한다.

기능 요구사항

1. 대시보드는 사용자가 구성하며, 뷰 단위로 지표 구성과 기간과 필터를 저장한다.
2. 모든 지표는 비교 기준(전 기간 · 업계 평균 등)을 함께 제시한다.
3. 추이 그래프에는 사용자가 주석을 추가할 수 있어야 한다.
4. 해석 위젯의 제안은 근거가 된 지표를 함께 표시하고, 반영 경로를 제공한다.
5. 상위 요금제 전용 지표는 목록에서 감추지 않고 필요 등급을 표시한다.

제약조건

- 방문 데이터는 일 단위로 집계하므로 당일 값은 확정되지 않는다. 마지막 집계 시각을 화면에 표시한다.
- 기업 도메인 방문은 접속한 네트워크의 조직 정보만 표시하고 개인은 식별하지 않는다. 이 원칙을 화면에 명시한다.
- 업계 평균은 직군 단위의 익명 집계값이며, 표본이 부족한 직군에는 제시하지 않는다.
- 배포되지 않은 포트폴리오에는 지표가 없으므로 이 화면을 제공하지 않는다.

지표 추가 — 화면번호 07b

`/(app)/analytics · D10`

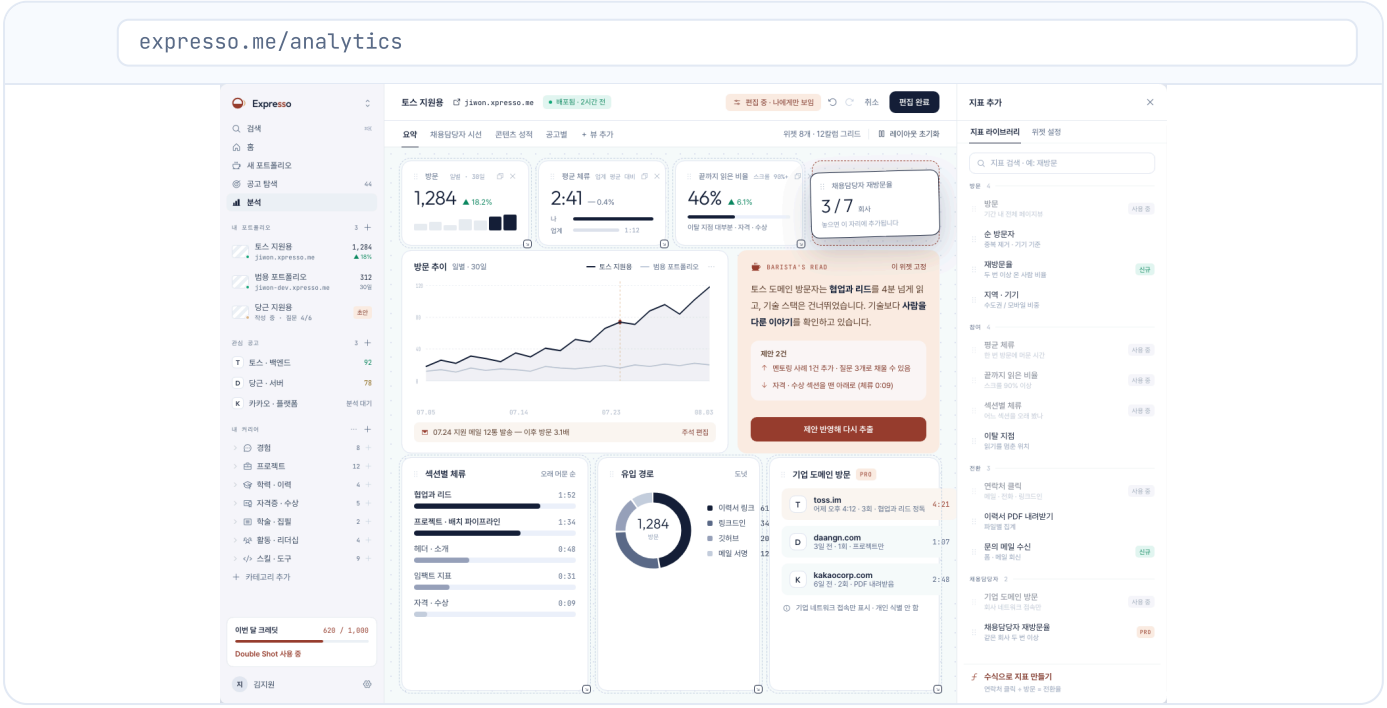


그림 5.3.20 지표 추가 (화면번호 07b)

본 화면은 분석 대시보드의 편집 상태이며, 지표를 추가할 때 나타난다. 오른쪽 지표 라이브러리에서 원하는 지표를 끌어 격자 위에 놓는 방식으로 구성하고, 편집 중임을 상단에 표시하여 지금 보이는 화면이 다른 사람에게는 보이지 않음을 알린다.

주요 구성 요소

- **편집 상태 표시** — 편집 중이며 본인에게만 보인다는 사실과 함께 취소 · 완료를 둔다. 위젯 수와 격자 규격, 「레이아웃 초기화」를 함께 표시한다.
- **격자 배치** — 대시보드를 열 단위의 격자로 다루고, 끌어 놓을 위치를 미리 표시하며 놓을 자리의 크기를 함께 알린다.
- **지표 라이브러리** — 지표를 목적별로 묶어 제시한다. 방문 규모 · 참여 · 전환 · 채용담당자로 나누고, 지표마다 정의를 한 줄로 적으며 이미 사용 중인 지표에는 표시를 붙인다.
- **지표 검색** — 이름으로 지표를 찾는다.
- **수식 지표** — 기존 지표를 나누거나 더하여 새 지표를 만든다. 정의한 수식을 함께 표시한다.

사용자에게 하는 역할

지표 추가는 사용자가 자신의 관심에 맞는 지표만 남기게 한다. 제공하는 지표를 모두 나열하면 화면의 가독성이 떨어지므로, 라이브러리에 두고 필요한 것만 꺼내 쓰게 한다. 지표마다 정의를 함께 적어, 이름만으로 오해하고 고르는 일을 막는다.

기능 요구사항

1. 편집 중인 변경은 완료를 누를 때까지 적용하지 않으며, 취소하면 이전 구성으로 되돌린다.
2. 위젯은 격자 단위로 배치하고 크기를 조절할 수 있어야 한다.
3. 지표 라이브러리의 각 지표는 정의를 함께 제시한다.
4. 이미 사용 중인 지표는 라이브러리에서 구분하여 표시한다.
5. 사용자가 기존 지표를 조합하여 새 지표를 정의할 수 있어야 한다.

제약조건

- 지표는 수집하고 있는 원본 데이터로 산출할 수 있는 것만 제공한다. 정의만 만들 수는 없다.
- 대시보드 하나에 놓을 수 있는 위젯 수에는 상한을 둔다. 상한을 넘으면 새 뷰로 나누도록 안내한다.
- 수식 지표는 분모가 0이 되는 기간을 표시할 수 없으므로, 해당 기간은 값 없음으로 처리한다.
- 편집은 데스크톱 환경을 기준으로 설계한다. 좁은 화면에서는 열람만 제공한다.

위젯 설정 — 화면번호 07c

`/(app)/analytics · D10`

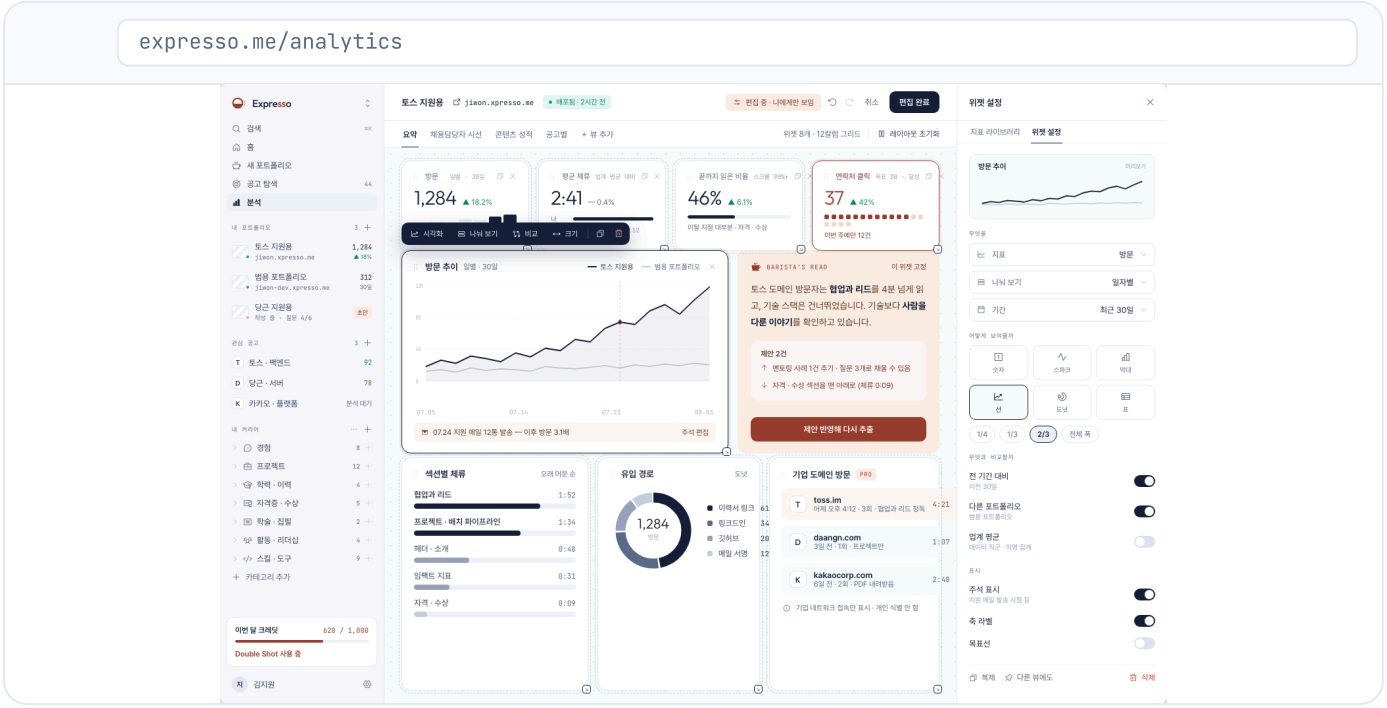


그림 5.3.21 위젯 설정 (화면번호 07c)

본 화면은 대시보드 위젯 하나의 설정 화면이다. 설정은 무엇을 보일지, 어떻게 보일지, 무엇과 비교할지의 세 질문으로 구성하고, 설정을 바꿀 때마다 결과를 미리보기로 확인하게 한다.

주요 구성 요소

- **설정 탭** — 시각화 · 나눠 보기 · 비교 · 크기의 네 묶음으로 항목을 나눈다.
- **대상 지정** — 표시할 지표와 집계 단위와 기간을 정한다.
- **표현 방식** — 숫자 · 스파크라인 · 막대 · 선 · 표 가운데 하나를 고르고, 위젯의 폭을 정한다.
- **비교 대상** — 전 기간 · 이전 동일 기간 · 다른 포트폴리오 · 업계 평균 가운데 선택한다. 업계 평균은 집계 범위를 함께 적는다.
- **표시 항목** — 주석 · 축 라벨 · 목표선의 표시 여부를 전환한다.
- **위젯 관리** — 위젯을 다른 뷰로 복제하거나 삭제한다.

사용자에게 하는 역할

위젯 설정은 같은 데이터를 사용자의 목적에 맞는 형태로 바꾸게 한다. 숫자 하나로 보고 싶은 지표와 추이로 보아야 하는 지표가 다르므로, 표현 방식을 지표와 분리하여 고르게 한다. 비교 대상을 명시적으로 고르게 하여, 변화의 크기를 해석할 기준을 사용자가 정하게 한다.

기능 요구사항

1. 지표와 표현 방식과 비교 대상을 각각 독립적으로 선택할 수 있어야 한다.
2. 설정 변경은 미리보기에 즉시 반영한다.
3. 비교 대상을 선택한 경우 위젯에 그 기준을 함께 표시한다.
4. 위젯은 다른 뷰로 복제할 수 있어야 한다.
5. 표현 방식은 선택한 지표가 지원하는 것만 활성화한다.

제약조건

- 지표의 성질에 맞지 않는 표현 방식은 선택할 수 없다. 비율 지표를 누적 막대로 표시하는 것과 같은 경우이다.
- 다른 포트폴리오와의 비교는 본인이 소유한 포트폴리오 사이에서만 가능하다.
- 업계 평균은 표본이 확보된 직군에만 제공하며, 표본 수를 함께 표시한다.
- 목표선은 사용자가 지정한 값이며 서비스가 권장값을 제시하지는 않는다.

공개 사이트 — 화면번호 08

`/site/[slug] · D9`

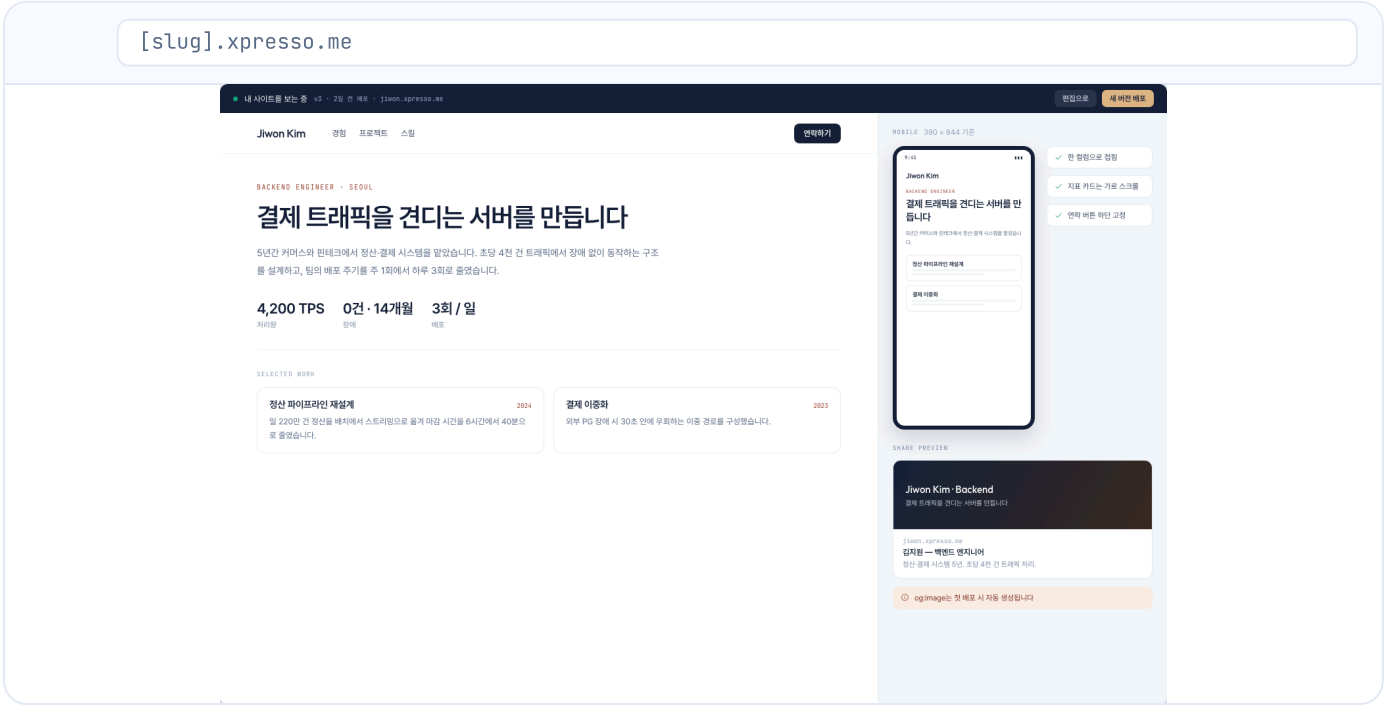


그림 5.3.22 공개 사이트 (화면번호 08)

본 화면은 배포된 포트폴리오를 방문자가 열었을 때의 지면이다. 로그인 없이 주소만으로 열리며, 배포 시점의 스냅샷을 보여주므로 이후의 편집이 공개된 내용을 바꾸지 않는다. 같은 배포에 데스크톱 지면 · 모바일 지면 · 공유 카드가 함께 묶인다.

주요 구성 요소

- **머리부** — 이름 · 직무 · 근무지와 한 문장 소개를 두고, 핵심 지표를 숫자로 함께 제시한다. 섹션 이동을 위한 내부 링크를 상단에 고정한다.
- **본문 섹션** — 레시피에서 정한 순서대로 경험 · 프로젝트 · 스킬 · 연락처를 배치한다. 프로젝트는 연도와 성과 문장을 함께 적는다.
- **모바일 지면** — 좁은 화면에서는 한 열로 접히고, 지표 카드는 가로로 넘기며, 연락 버튼은 화면 아래에 고정한다.
- **공유 카드** — 다른 서비스에 주소를 붙였을 때 표시되는 제목 · 요약 · 이미지이다. 첫 배포 시 자동으로 생성한다.
- **소유자 표시줄** — 소유자가 자신의 사이트를 열면 배포 버전과 배포 시점을 알리고, 편집 화면과 새 버전 배포로 이동하는 경로를 함께 표시한다. 방문자에게는 표시하지 않는다.

사용자에게 하는 역할

공개 사이트는 사용자가 생성한 결과물을 방문자가 열람하는 화면이다. 채용 담당자는 짧은 시간 안에 판단하므로, 이름과 직무와 핵심 지표를 첫 화면에서 모두 확인할 수 있게 배치한다. 또한 이 화면에서 발생한 방문과 체류가 분석 화면의 자료가 되어, 결과물을 다시 고치는 근거가 된다.

기능 요구사항

1. 공개 사이트는 인증 없이 접근할 수 있어야 한다.
2. 표시하는 내용은 배포 시점의 스냅샷으로 고정하고, 편집 중인 내용을 노출하지 않는다.
3. 좁은 화면에서도 모든 내용에 접근할 수 있어야 하며, 연락 수단은 화면 어디에서나 도달 가능해야 한다.
4. 공유 카드는 첫 배포 시 자동 생성하고, 이후 배포에서 제목과 요약을 갱신한다.
5. 방문과 체류와 연락처 클릭을 집계하여 분석 화면에 제공한다.
6. 소유자로 인증된 접근에는 배포 버전과 편집 경로를 함께 표시한다.

제약조건

- 연락처의 공개 범위는 소유자가 배포 설정에서 정한 값을 따른다. 전체 공개가 기본값이 아니다.
- 검색 엔진 노출 여부도 배포 설정을 따르며, 노출을 끈 경우 색인을 거부하는 표시를 함께 내려보낸다.
- 방문 집계는 개인을 식별하지 않는다. 접속한 조직의 도메인 수준까지만 기록한다.
- 공개를 중단하면 주소로 접근할 수 없게 되지만 문서와 기록은 삭제되지 않는다.

배포 · 버전 — 화면번호 08b

`/(app)/edit/[portfolioId]/deploy · D9`

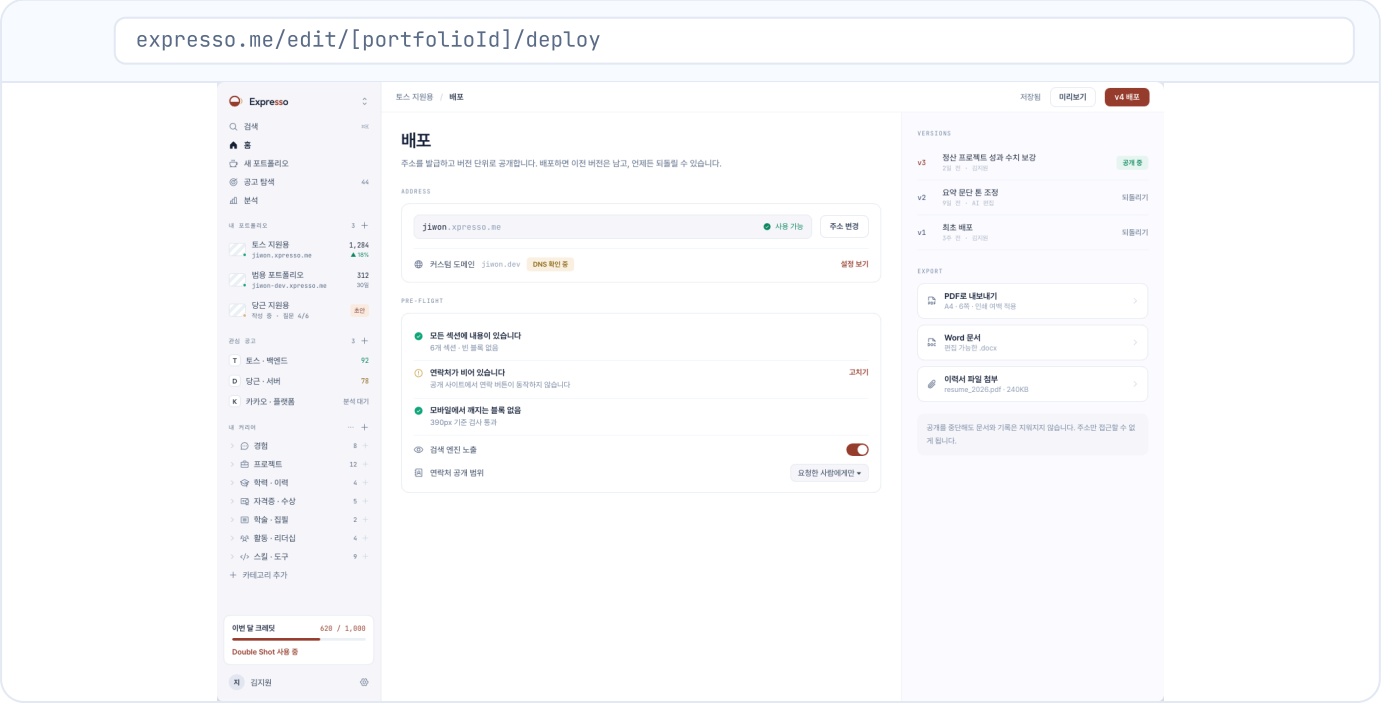


그림 5.3.23 배포 · 버전 (화면번호 08b)

본 화면은 포트폴리오를 공개하고 관리한다. 주소 발급 · 배포 전 점검 · 공개 범위 설정 · 버전 관리 · 파일 내보내기를 한 화면에 모았다. 공개와 관련된 결정을 여러 화면에 흩어 놓지 않기 위한 구성이다.

주요 구성 요소

- **주소 설정** — 하위 도메인을 입력하면 사용 가능 여부를 즉시 확인한다. 사용자 도메인을 연결하는 경우 이름 서버 확인 상태를 함께 표시한다.
- **배포 전 점검** — 빈 섹션 · 비어 있는 연락처 · 좁은 화면에서 깨지는 블록을 검사하여 통과와 경고를 항목별로 제시하고, 「고치기」로 해당 위치로 이동한다.
- **공개 범위 설정** — 검색 엔진 노출 여부와 연락처 공개 범위를 정한다.
- **버전 목록** — 배포마다 버전 번호 · 변경 요약 · 배포 시각 · 주체를 기록하고, 현재 공개 중인 버전을 표시한다. 「되돌리기」로 이전 버전을 다시 공개한다.
- **내보내기** — 인쇄 여백을 적용한 문서 파일과 편집 가능한 문서 파일로 내보내고, 이력서 파일을 함께 첨부한다.

사용자에게 하는 역할

배포 화면은 사용자가 공개의 결과를 예측한 상태로 배포하게 한다. 점검 결과를 배포 이전에 제시하므로, 연락처가 비어 있거나 좁은 화면에서 깨진 상태로 공개되는 일을 줄인다. 또한 버전을 남겨 두어 배포 이후에 문제를 발견하여도 즉시 이전 상태로 되돌릴 수 있게 한다.

기능 요구사항

1. 주소의 사용 가능 여부는 입력 중에 확인하여 표시한다.
2. 배포 전 점검은 항목별 결과를 제시하고, 문제 항목마다 수정 위치로 이동하는 경로를 제공한다.
3. 배포는 버전으로 관리하며, 버전마다 변경 요약과 주체와 시각을 기록한다.
4. 이전 버전으로 되돌릴 수 있어야 하고, 되돌린 결과도 새 배포로 기록한다.
5. 공개 중단은 주소 접근만 차단하고 문서와 기록은 보존한다.
6. 내보낸 파일은 발급 이력을 남기고, 필요 시 접근을 무효화할 수 있어야 한다.

제약조건

- 사용자 도메인 연결은 이름 서버 설정이 완료될 때까지 적용되지 않는다. 확인 중 상태를 화면에 표시한다.
- 배포 전 점검의 경고는 배포를 차단하지 않는다. 사용자가 확인한 뒤 진행할 수 있게 한다.
- 사용자 도메인 연결과 게시자 표시 제거는 상위 요금제에서만 사용할 수 있다.
- 배포는 스냅샷을 생성하므로 즉시 완료되지 않을 수 있다. 진행 상태를 표시하고 완료 시 알린다.
- 삭제 요청 중인 계정의 포트폴리오는 새로 배포할 수 없다.

■ 계정 · 요금제 — 화면번호 09

/(app)/account · D12

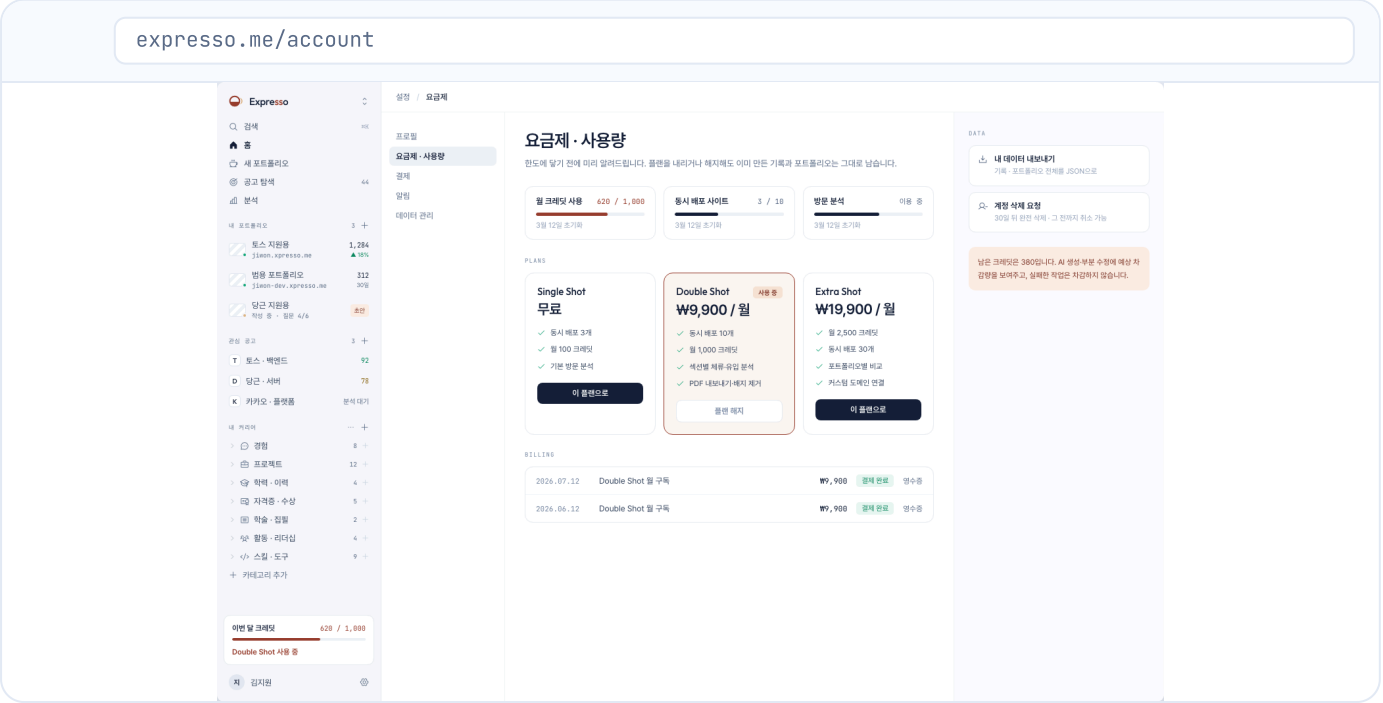


그림 5.3.24 계정 · 요금제 (화면번호 09)

본 화면은 계정 정보와 구독 상태를 한자리에서 확인하고 변경한다. 사이드바 아래의 계정 항목에서 진입하며, 왼쪽 탭으로 프로필 · 요금제 · 사용량 · 결제 · 알림 · 데이터 관리의 영역을 이동한다. 서비스의 기능 대부분이 요금제에 따라 열리고 닫히므로, 사용자가 지금 무엇을 얼마나 더 쓸 수 있는지 확인하는 지점이 이 화면이다.

주요 구성 요소

- **사용량 카드** — 포트폴리오 생성 · AI 인터뷰 · 배포 사이트의 세 지표를 사용량과 한도의 쌍으로 제시하고, 그 아래 다음 초기화 날짜를 함께 적는다.
- **요금제 비교** — 세 등급을 나란히 놓고 등급별로 포트폴리오 보유 수 · 월 생성 횟수 · 템플릿 범위 · 사용자 도메인 · 방문 분석 범위를 대조한다. 현재 등급에는 사용 중 표시가 붙고 나머지 등급에는 전환 경로가 놓인다.
- **결제 내역** — 결제일 · 상품명 · 금액 · 상태를 시간 역순으로 나열하고 건마다 영수증을 내려받는다.
- **데이터 관리** — 계정의 기록과 포트폴리오 전체를 문서 파일 하나로 내보내는 기능과 계정 삭제를 요청하는 기능을 둔다.

사용자에게 하는 역할

계정 화면은 사용자가 자신의 이용 한계를 미리 파악하도록 돕는다. 생성이 막힌 뒤에 이유를 찾게 하지 않고 남은 횟수와 초기화 시점을 상시 노출하여, 사용자가 언제 등급을 올릴지 스스로 판단하게 한다. 또한 데이터 내보내기와 삭제 요청을 같은 화면에 두어, 서비스를 떠나는 결정 역시 사용자의 통제 아래 있음을 보인다.

기능 요구사항

1. 사용량은 요청 시점의 집계 값을 조회하여 표시하며, 무제한 등급은 숫자 대신 무제한으로 표기한다.
2. 등급 변경은 다음 결제일부터 적용하고, 변경 예정 상태를 요금제 영역에 명시한다.
3. 한도를 소진한 경우 생성 진입점을 비활성화하는 대신 남은 횟수와 초기화 날짜를 그 자리에 표시한다.
4. 데이터 내보내기는 커리어 기록 · 공고 분석 · 레시피 · 포트폴리오 · 배포 · 방문 지표를 스키마 버전이 붙은 하나의 파일로 생성한다.
5. 계정 삭제는 즉시 삭제가 아닌 요청으로 처리하며, 요청 시점과 완전 삭제 예정 시점을 함께 제시한다.

제약조건

- 요금제별 기능 개방 여부는 화면이 판단하지 않는다. 서버가 내려준 권한 판정 결과를 그대로 반영하며, 권한 미달과 한도 소진을 서로 다른 문구로 구분하여 안내한다.
- 생성 한도의 집계 주기는 한국 표준시 월 단위로 고정한다. 사용자의 지역 설정이 달라도 초기화 시점은 동일하다.
- 계정 삭제 요청 후 30일 동안은 유예 기간이며 요청을 취소하여 계정을 복구할 수 있다. 다만 유예 기간에도 공개 중이던 포트폴리오는 비공개로 전환되고 내려받기 링크는 즉시 무효화된다.
- 삭제 요청이 진행 중인 계정은 신규 생성 권한을 받지 못한다.
- 결제 수단 정보는 이 화면에서 다루지 않으며 결제 대행사의 결제창으로 위임한다.

■ 로그인 — 화면번호 10

`/(auth)/login · D12`

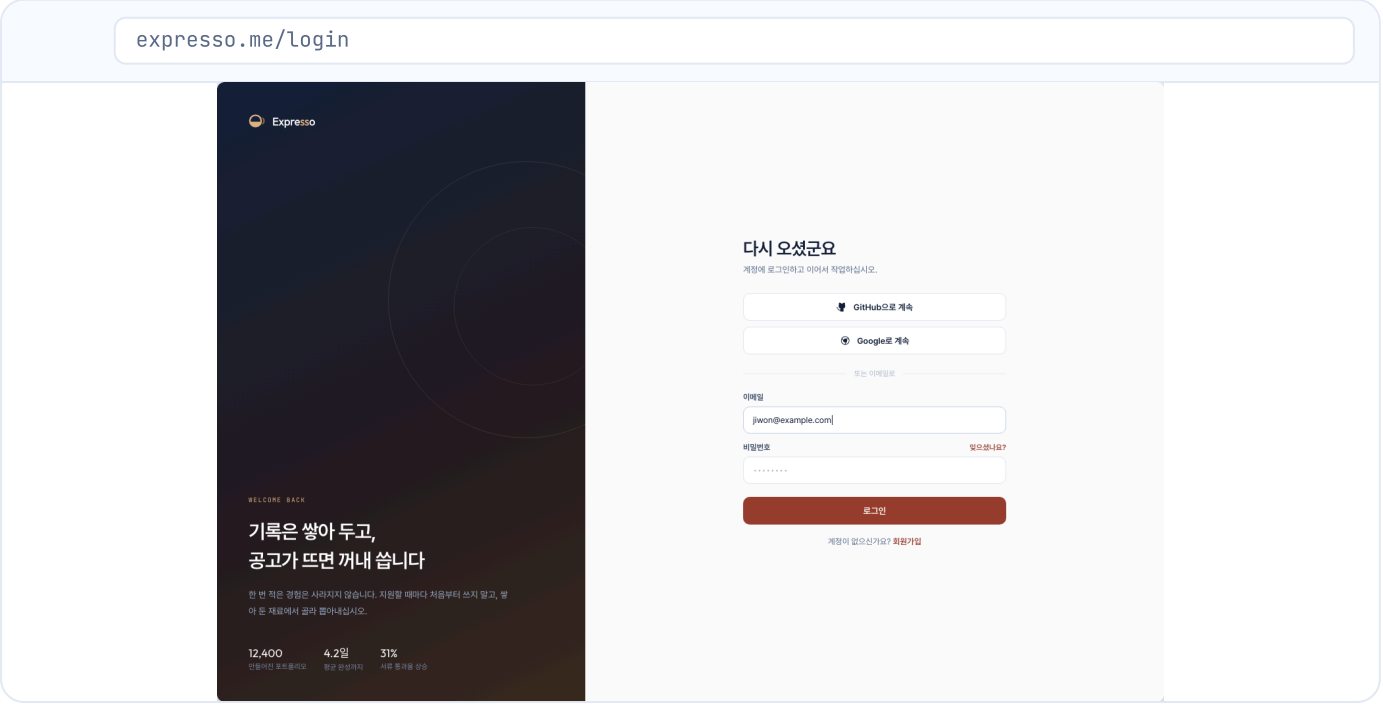


그림 5.3.25 로그인 (화면번호 10)

본 화면은 서비스에 접속하였을 때 최초로 나타나는 로그인 화면이다. 왼쪽에 서비스의 성과 지표를 두고 오른쪽에 로그인 수단을 두어, 처음 온 사용자에게 서비스의 성격을 알리면서 기존 사용자의 진입을 방해하지 않게 한다.

주요 구성 요소

- **소개 영역** — 서비스의 한 문장 설명과 함께 누적 생성 수 · 평균 완성 소요 · 서류 통과율 변화의 세 지표를 제시한다.
- **소셜 로그인** — 개발자 사용자가 이미 보유한 계정을 우선으로 배치한다.
- **이메일 로그인** — 소셜 수단을 쓸 수 없는 사용자를 위한 대안으로 아래에 둔다. 비밀번호 재설정 경로를 함께 제공한다.
- **회원가입 연결** — 계정이 없는 사용자를 회원가입 화면으로 보낸다.

사용자에게 하는 역할

로그인 화면이 하는 일은 진입 장벽을 낮추는 것이다. 이메일과 비밀번호를 새로 만들지 않고도 시작할 수 있게 소셜 수단을 위에 두고, 그 수단을 쓰지 않는 사용자를 위해 이메일 방식을 함께 남긴다.

기능 요구사항

1. 소셜 로그인과 이메일 로그인을 모두 제공하고, 같은 사용자가 두 수단을 함께 쓸 수 있게 계정을 연결한다.
2. 인증 실패 사유는 계정 존재 여부를 노출하지 않는 범위에서 안내한다.
3. 비밀번호 재설정은 등록된 이메일로만 진행한다.
4. 인증 성공 후에는 온보딩 완료 여부에 따라 홈 또는 온보딩으로 분기한다.
5. 일정 횟수 이상 실패한 접근은 제한한다.

제약조건

- 소셜 로그인은 외부 인증 제공자에 의존하므로, 제공자의 장애 시 이메일 방식으로 안내한다.
- 삭제 요청 중인 계정으로 로그인하면 유예 기간과 취소 경로를 먼저 안내한다.
- 화면에 제시하는 지표는 서비스 전체의 집계값이며 개별 사용자의 결과를 보장하지 않는다.
- 버튼별 동작과 유효성 검사 규칙은 5.2에 별도로 정의한다.

회원가입 — 화면번호 10b

/(auth)/signup · D12

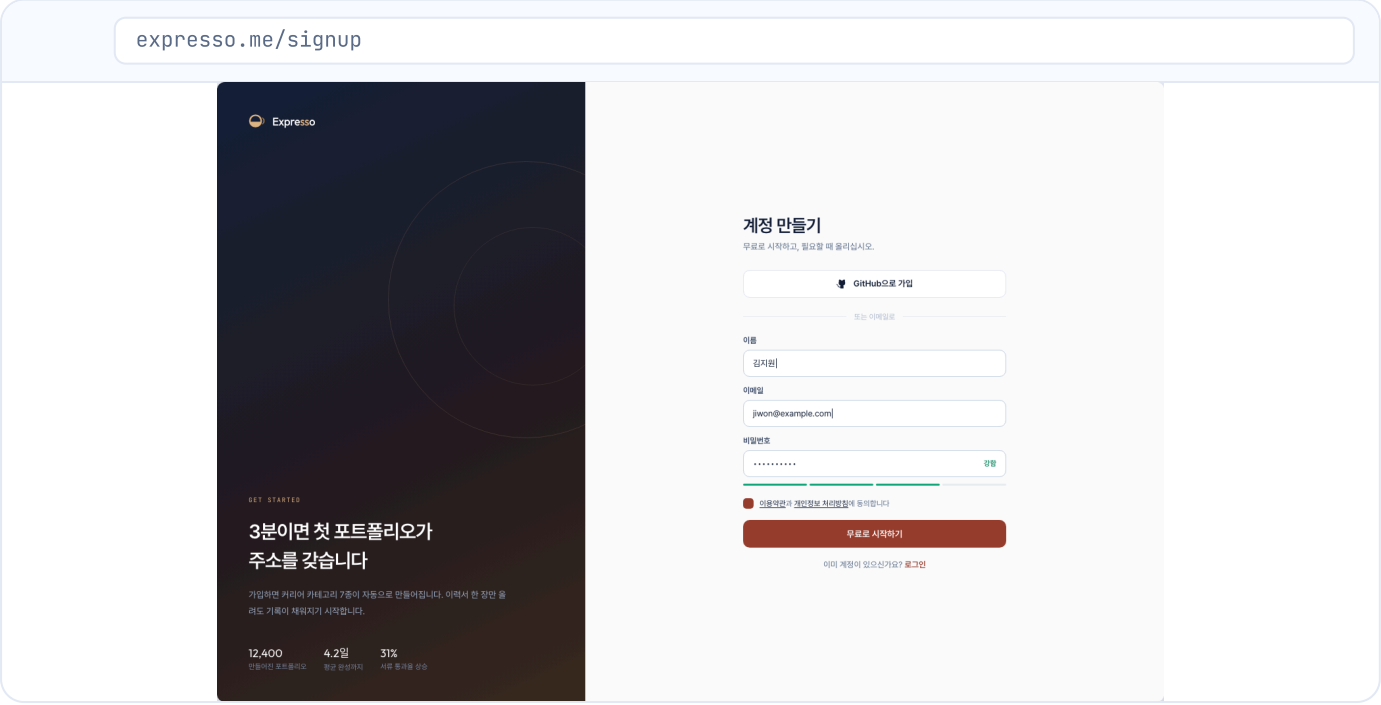


그림 5.3.26 회원가입 (화면번호 10b)

본 화면은 이메일로 계정을 만드는 회원가입 화면이다. 입력 항목을 이메일과 비밀번호로 최소화하고, 약관 동의를 같은 화면에서 받는다. 가입이 완료되면 커리어 기록의 기본 카테고리가 함께 만들어진다.

주요 구성 요소

- **입력부** — 이메일과 비밀번호를 받는다. 비밀번호는 입력 중에 강도를 표시한다.
- **약관 동의** — 이용약관과 개인정보 처리방침에 대한 동의를 받고, 각 문서로 이동하는 경로를 함께 둔다.
- **가입 실행** — 요금제를 고르는 단계 없이 무료로 시작한다.
- **로그인 연결** — 이미 계정이 있는 사용자를 로그인 화면으로 보낸다.

사용자에게 하는 역할

회원가입은 가입 단계에서 사용자가 이탈하지 않게 하는 데 목적이 있다. 직무나 경력 같은 정보는 이 화면에서 묻지 않고 온보딩으로 넘기며, 계정을 만드는 데 필요한 것만 받는다. 또한 가입과 동시에 기록을 담을 카테고리를 만들어, 사용자가 빈 화면을 마주하지 않게 한다.

기능 요구사항

1. 이메일의 중복 여부와 형식을 입력 중에 검사한다.
2. 비밀번호는 정해진 규칙을 만족해야 하며, 충족 여부를 입력 중에 표시한다.
3. 약관 동의는 필수이며, 동의 시점과 동의한 문서의 버전을 기록한다.
4. 가입이 완료되면 기본 카테고리를 함께 생성한다.
5. 가입 직후에는 온보딩 화면으로 이동한다.

제약조건

- 가입 시 요금제를 선택하지 않는다. 모든 계정은 무료 등급으로 시작한다.
- 동의 기록은 법적 요구에 따라 보관하며, 계정 삭제 시에도 보관 의무가 있는 항목은 남긴다.
- 같은 이메일로 소셜 계정이 이미 있는 경우 새 계정을 만들지 않고 기존 계정에 연결하도록 안내한다.
- 비밀번호는 복원 불가능한 형태로만 저장하며, 서비스가 원문을 보관하지 않는다.

■ 온보딩 · 목표 — 화면번호 10c

`/(auth)/onboarding/goal · D12`

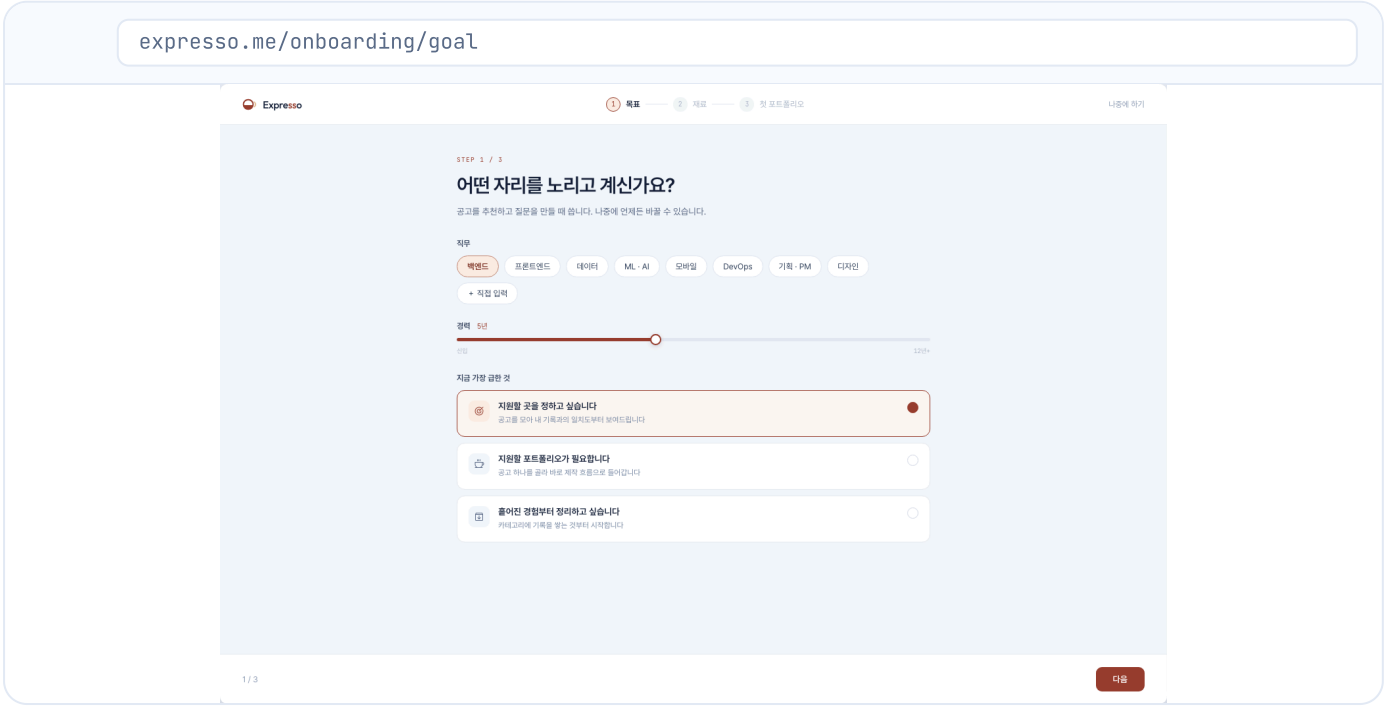


그림 5.3.27 온보딩 · 목표 (화면번호 10c)

본 화면은 온보딩의 첫 단계이며, 추천과 질문의 기준이 되는 최소 정보를 받는다. 직무와 경력, 그리고 지금 가장 급한 일을 묻는다. 세 단계 중 첫 단계임을 상단에 표시하고, 건너뛰는 경로를 함께 둔다.

주요 구성 요소

- **단계 표시** — 목표 · 재료 · 첫 포트폴리오의 세 단계와 현재 위치를 표시하고, 「나중에 하기」로 온보딩을 미룬다.
- **직무 선택** — 대표 직무를 선택지로 제시하고, 목록에 없으면 직접 입력한다.
- **경력 입력** — 신입부터 상한까지의 범위에서 값을 지정한다.
- **목적 선택** — 지원할 곳을 정하는 것, 지원용 포트폴리오를 만드는 것, 흩어진 경험을 정리하는 것의 세 목적을 제시하고, 각 선택이 어떤 화면으로 이어지는지 함께 적는다.

사용자에게 하는 역할

온보딩의 첫 단계는 이후 모든 추천의 기준을 만든다. 직무와 경력이 없으면 공고 추천과 대화 질문을 만들 수 없으므로, 최소한의 정보만 이 단계에서 받는다. 또한 사용자가 지금 무엇을 급하게 여기는지 물어, 온보딩 이후의 첫 화면을 그 목적에 맞추어 정한다.

기능 요구사항

1. 묻는 항목은 추천과 질문 생성에 실제로 사용되는 것으로 한정한다.
2. 선택한 값은 이후 설정 화면에서 변경할 수 있으며, 이 사실을 화면에 명시한다.
3. 목적 선택마다 이어지는 화면을 미리 설명한다.
4. 온보딩은 건너뛸 수 있어야 하고, 건너뛴 경우에도 서비스를 사용할 수 있어야 한다.
5. 입력 값은 단계를 이동할 때마다 저장하여 중단 후 재개할 수 있게 한다.

제약조건

- 직무 선택지는 목록으로 제한하지 않고 직접 입력을 허용한다. 다만 목록에 없는 직무는 추천 품질이 낮아질 수 있다.
- 온보딩을 건너뛰면 추천 기능은 기록이 쌓인 뒤에야 동작한다.
- 경력 값은 사용자가 신고한 값이며 서비스가 검증하지 않는다.
- 이 단계에서 개인 식별 정보를 추가로 수집하지 않는다.

■ 온보딩 · 재료 채우기 — 화면번호 10d

`/(auth)/onboarding/materials · D12`

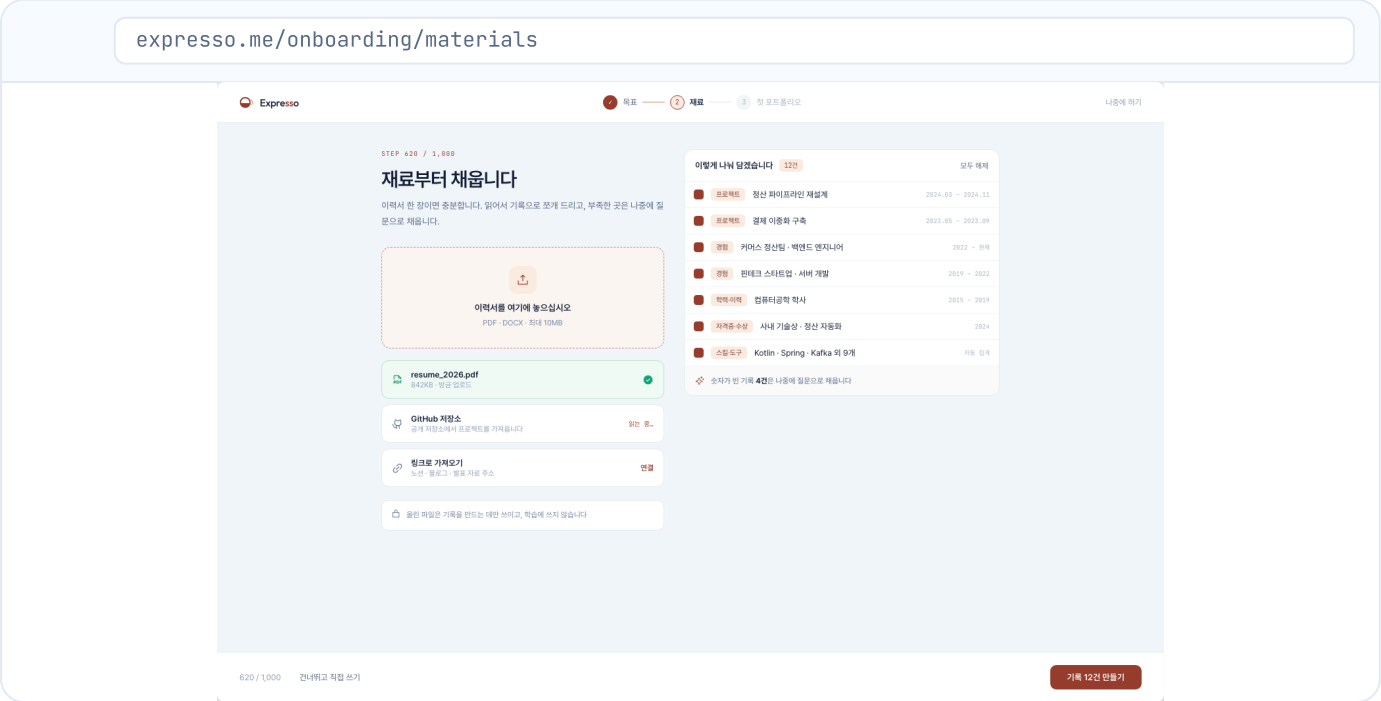


그림 5.3.28 온보딩 · 재료 채우기 (화면번호 10d)

본 화면은 온보딩의 두 번째 단계이며, 기존 자료에서 커리어 기록을 만든다. 이력서 파일과 공개 저장소와 외부 문서 주소를 입력으로 받아 기록의 초안을 만들고, 사용자가 확인한 뒤에 저장한다.

주요 구성 요소

- **파일 업로드** — 이력서 파일을 놓아 올린다. 허용 형식과 용량 상한을 함께 표시하고, 업로드된 파일의 크기와 시각을 알린다.
- **외부 연결** — 공개 저장소에서 프로젝트를 가져오고, 외부 문서의 주소로 자료를 연결한다. 진행 상태를 항목마다 표시한다.
- **사용 범위 안내** — 올린 파일은 기록을 만드는 데만 쓰이고 학습에 사용하지 않는다는 사실을 업로드 영역 아래에 명시한다.
- **추출 결과 확인** — 추출한 항목을 카테고리별로 묶어 제목과 기간과 함께 나열하고, 항목마다 저장 여부를 선택한다. 전체 해제와 전체 선택을 함께 둔다.
- **보강 예고** — 수치가 비어 있는 기록의 건수를 알리고, 이후 질문으로 채운다는 사실을 적는다.
- **대안 경로** — 자료가 없는 사용자를 위해 직접 작성으로 넘어가는 경로를 둔다.

사용자에게 하는 역할

재료 채우기는 사용자가 빈 화면에서 시작하지 않게 한다. 커리어 기록을 처음부터 입력하는 것은 부담이 크므로, 이미 가지고 있는 이력서를 재료로 삼아 초안을 만든다. 동시에 저장 이전에 확인 단계를 두어, 잘못 추출된 항목이 그대로 기록이 되는 것을 막는다.

기능 요구사항

1. 추출 결과는 저장 이전에 사용자가 확인하고 선택할 수 있어야 한다.
2. 허용하는 파일 형식과 용량 상한을 업로드 이전에 표시한다.
3. 업로드한 파일의 사용 범위를 화면에 명시한다.
4. 외부 자료 수집은 진행 상태를 항목별로 표시한다.
5. 자료가 없는 사용자를 위해 직접 작성으로 넘어가는 경로를 제공한다.
6. 추출된 기록에는 출처를 기록하여, 이후 어디에서 온 기록인지 확인할 수 있게 한다.

제약조건

- 추출 정확도는 원본 문서의 구조에 따라 달라진다. 이 때문에 결과를 그대로 저장하지 않고 확인 단계를 반드시 둔다.
- 가져오는 저장소는 공개 저장소로 한정한다. 비공개 저장소의 접근 권한은 요구하지 않는다.
- 업로드한 원본 파일은 기록 생성 목적으로만 사용하고 모델 학습에 사용하지 않는다.
- 파일 형식과 용량 상한을 넘는 자료는 업로드 단계에서 거부한다.

■ 온보딩 · 첫 포트폴리오 — 화면번호 10e

/(auth)/onboarding/first-portfolio · D12

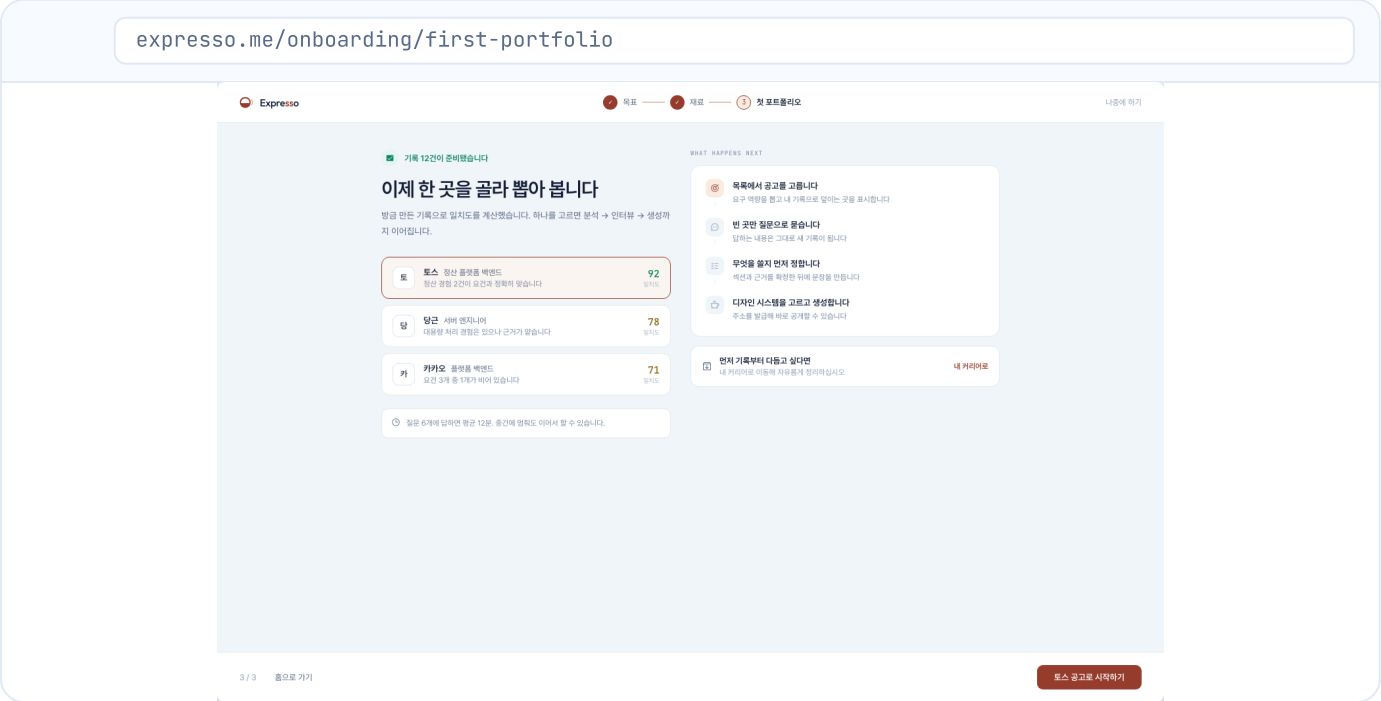


그림 5.3.29 온보딩 · 첫 포트폴리오 (화면번호 10e)

본 화면은 온보딩의 마지막 단계이며, 만들어진 기록으로 첫 포트폴리오를 시작한다. 온보딩을 완료 안내로 끝내지 않고, 목표 공고를 고르는 자리로 잇는다. 이후 진행될 절차를 미리 요약하여 사용자가 무엇을 하게 되는지 알고 시작하게 한다.

주요 구성 요소

- **준비 상태 안내** — 만들어진 기록 건수를 알리고 다음 행동을 한 문장으로 제시한다.
- **추천 공고 목록** — 기록과 목표를 근거로 공고 세 건을 제시하고, 공고마다 일치도와 판정 사유를 함께 적는다. 근거가 얕은 공고에는 그 사실을 적는다.
- **소요 안내** — 질문 수와 평균 소요 시간을 제시하고, 중간에 멈춘 뒤 이어서 할 수 있음을 명시한다.
- **절차 예고** — 공고 선택 · 요구 역량 대조 · 질문으로 빈 곳 채우기 · 구성 확정 · 생성과 공개의 순서를 요약한다.
- **대안 경로** — 기록을 먼저 다듬고 싶은 사용자를 위해 커리어 화면으로 이동하는 경로와 홈으로 가는 경로를 둔다.

사용자에게 하는 역할

온보딩의 마지막 단계는 온보딩과 제작을 잇는다. 기록을 만들어 두고 끝내면 사용자는 그 기록으로 무엇을 할 수 있는지 알지 못하므로, 바로 첫 결과물을 만드는 경로를 제시한다. 동시에 강제하지 않고 다른 경로도 함께 남긴다.

기능 요구사항

1. 추천 공고는 온보딩에서 받은 목표와 만들어진 기록을 근거로 산출하고, 사유를 함께 표시한다.
2. 소요 시간과 질문 수를 미리 제시한다.
3. 이후 절차를 요약하여 제시한다.
4. 제작을 시작하지 않고 다른 화면으로 이동하는 경로를 함께 제공한다.
5. 온보딩 완료 상태를 기록하여 다음 접속 시 다시 표시하지 않는다.

제약조건

- 추천할 공고가 부족하면 목록 대신 공고 탐색으로 안내한다. 억지로 채우지 않는다.
- 이 단계에서 제작을 시작하면 이번 달 생성 한도를 소비한다.
- 제시하는 평균 소요 시간은 통계값이며 사용자마다 다를 수 있다.
- 기록이 일정 수 미만이면 추천의 근거가 부족하므로, 커리어 화면으로 먼저 안내한다.

5.4 관리자 메뉴구성

■ 관리자

관리자가 접근할 수 있는 메뉴의 구조는 아래 그림 5.4과 같다.

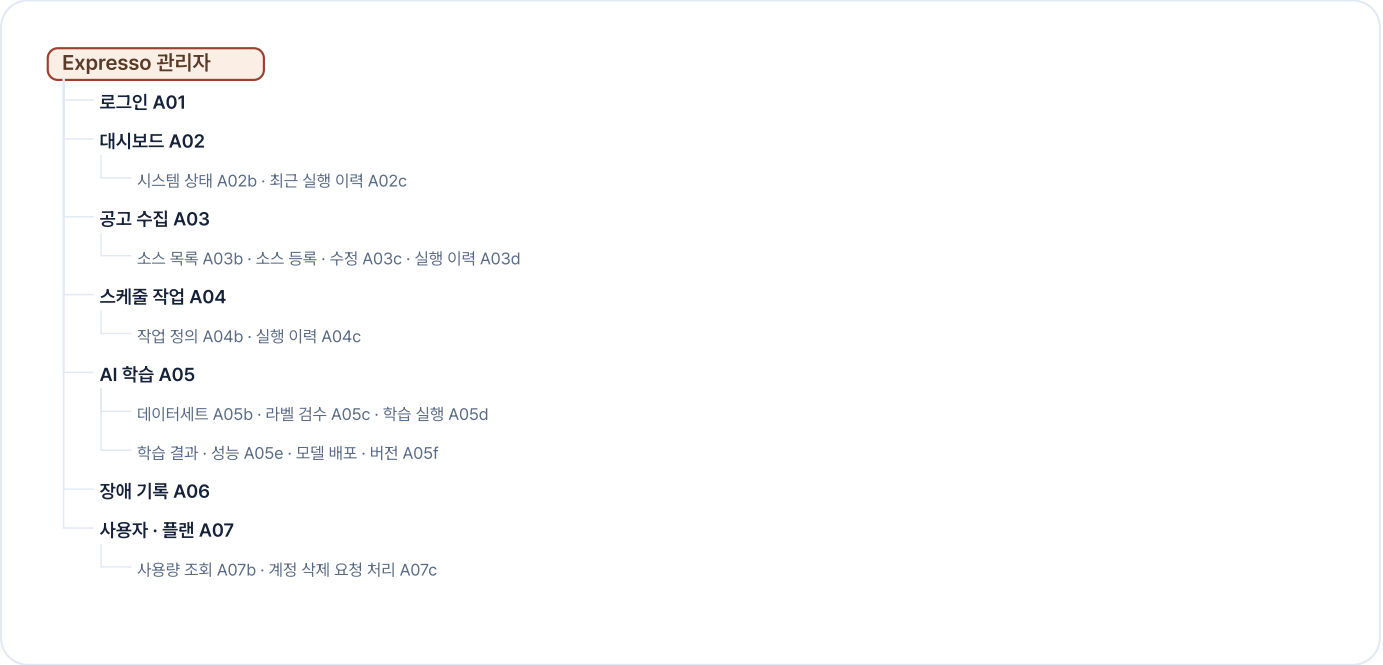


그림 5.4 관리자용 메뉴 구조도

관리자 메뉴는 일곱 갈래이다. 공고 소스 · 공고 · 수집 모니터링 · 스케줄 · AI 학습 · 장애 기록 · 사용자와 플랜이며, 화면번호는 A로 시작한다. 이 가운데 AI 학습(A05)이 관리자 화면을 만드는 가장 큰 이유이다. 학습 데이터세트 생성 · 라벨 검수 · 학습 실행 · 성능 확인 · 배포와 롤백을 화면 없이 수행할 수 없기 때문이다.

5.5 관리자 화면설계(웹)

5.4에서 식별한 관리자 메뉴의 모든 화면을 설계한다. 관리자 화면은 웹 전용이므로 웹 규격을 따르며, 화면 1개당 1쪽으로 작성한다. 화면은 21개이다.

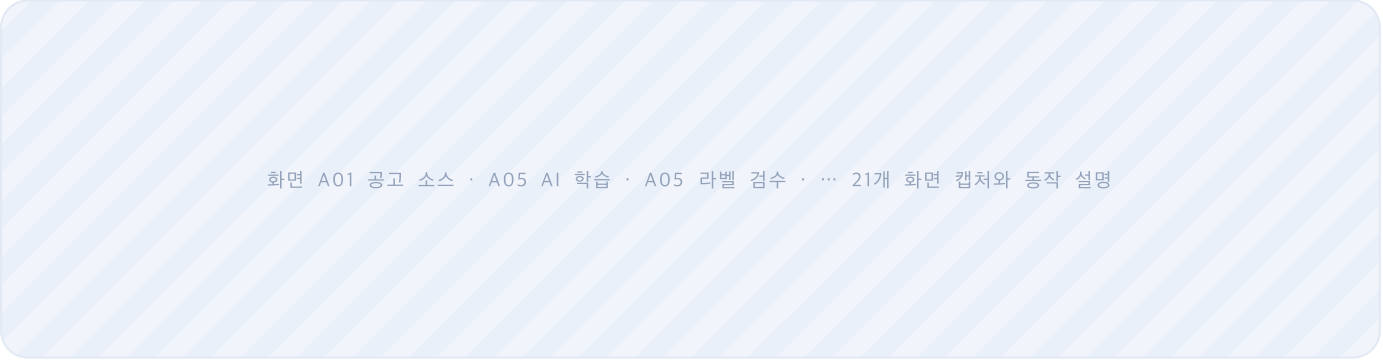


그림 5.5 관리자 화면설계 — 21개 화면 (작성 예정)

06 프로세스(기능) 설계

양식 6장

6.1 시스템 아키텍쳐

본 서비스의 시스템 아키텍쳐는 아래 그림 6.1과 같다.

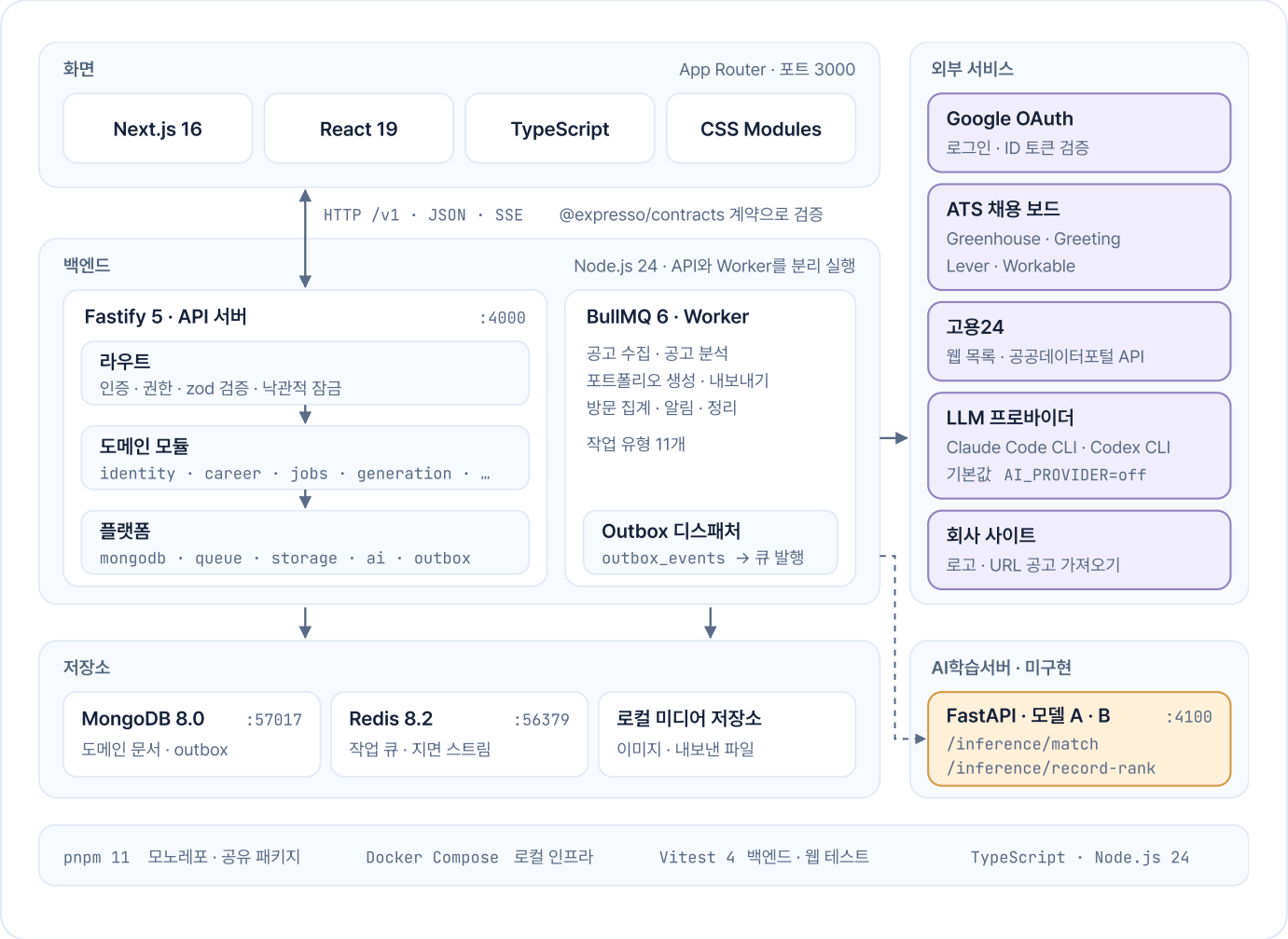


그림 6.1 시스템 아키텍쳐

계층	책임	하지 않는 것
라우트	요청 파싱 · 인증 확인 · 스키마 검증 · 응답 직렬화	비즈니스 규칙
도메인 모듈	비즈니스 규칙 · 트랜잭션 경계	HTTP 세부사항
플랫폼	DB · 큐 · 저장소 · LLM · 로깅	도메인 지식
Worker	오래 걸리는 작업 · 재시도	사용자 응답

그림 6.1을 보면 계층은 **위에서 아래로 의존**한다. 웹 클라이언트는 API 서버의 REST만 호출하고, API 서버는 도메인 모듈을 부르며, 도메인 모듈은 플랫폼 계층을 통해 데이터베이스와 큐에 접근한다. 화살표의 방향이 곧 의존의 방향이며, 플랫폼 계층은 도메인을 알지 못하고 도메인 모듈은 HTTP를 알지 못한다. 이 규칙 덕분에 도메인 로직을 HTTP 없이 시험할 수 있다.

■ 동기와 비동기의 경계

사용자 요청 중 초 단위로 끝나지 않는 것은 큐로 넘깁니다.

작업	처리	이유
공고 분석	비동기	LLM 호출 수십 초
포트폴리오 생성	비동기	LLM 호출 수 분
문서 내보내기	비동기	렌더링 · 변환
방문 지표 집계	비동기	배치
기록 CRUD · 공고 검색	동기	DB만 씁니다

DB 커밋과 큐 발행이 따로 성공하거나 실패하면 상태가 갈라진다. 이를 막기 위해 **Outbox 패턴**을 사용한다. 이벤트를 `outbox_events`에 도메인 변경과 같은 트랜잭션으로 기록하고, 별도의 **폴러**가 그것을 읽어 큐로 보낸다. 트랜잭션이 실패하면 이벤트도 남지 않으므로 두 상태가 어긋나지 않는다.

6.2 사용자 기능설계 (Sequence Diagram)

2.5에서 정의한 **151개**의 사용자 기능을 시퀀스 다이어그램으로 설계한다. 다이어그램은 반드시 서버와 클라이언트가 주고받는 흐름을 보여야 한다. 다만 151개 가운데 상당수는 **주고받는 흐름이 서로 같다.** 예를 들어 기록 조회 · 공고 조회 · 알림 조회는 경로와 컬렉션만 다를 뿐 한 번의 왕복으로 끝나는 같은 흐름이다.

따라서 흐름이 서로 다른 17개를 대표로 설계하고, 나머지 기능은 아래 유형에 귀속시킨다. 유형이 같은 기능은 경로와 대상 컬렉션만 바뀌며, 그 값은 2.5의 기능 표와 7.2의 API 목록에 있다.

■ 상호작용 유형

유형	흐름의 특징	대표 그림
동기 조회	웹이 API를 한 번 호출하고 그 응답으로 화면을 그린다. 왕복이 한 번이다.	그림 6.2.17
동기 저장	스키마를 검증한 뒤 데이터베이스에 기록한다. `Idempotency-Key`로 중복 저장을 막는다.	그림 6.2.2 · 6.2.13

유형	흐름의 특징	대표 그림
낙관적 잠금 저장	`If-Match`로 버전을 확인하고, 어긋나면 409를 반환한다.	그림 6.2.3
비동기 작업	202를 먼저 돌려주고 outbox와 Worker가 이어받는다. 사용자는 기다리지 않는다.	그림 6.2.7 · 6.2.10 · 6.2.11
AI 동기 호출	API 서버가 AI서버를 직접 호출하고 응답을 기다린다. 초 단위로 끝나는 작업만 이 방식을 쓴다.	그림 6.2.5 · 6.2.9 · 6.2.12 · 6.2.16
학습 모델 서빙	배포된 모델로 판정하고, 모델이 없으면 규칙 함수로 되돌아간다.	그림 6.2.6 · 6.2.8 · 6.2.17
파일 내보내기	Worker가 파일을 만들어 로컬 미디어 저장소에 두고, 만료가 있는 서명 URL로만 내려받는다.	그림 6.2.14
공개 사이트	인증이 없는 경로이다. 열람과 지표 수집을 분리한다.	그림 6.2.15

여덟 유형이 151개 기능의 흐름을 모두 포함한다. 유형이 같으면 다이어그램의 모양이 같고 경로만 달라진다.

도메인별 귀속

도메인	기능 수	나타나는 유형	대표 그림
D1 커리어 기록	28	동기 조회 · 동기 저장 · 낙관적 잠금 저장	그림 6.2.2 · 6.2.3 · 6.2.4
D2 채용 공고 수집/검색	21	동기 조회 · AI 동기 호출 · 학습 모델 서빙	그림 6.2.5 · 6.2.6
D3 채용 공고 분석	10	비동기 작업	그림 6.2.7
D4 포트폴리오 재료 선정	8	동기 조회 · 학습 모델 서빙	그림 6.2.8
D5 AI 인터뷰	11	AI 동기 호출 · 동기 저장	그림 6.2.9
D6 포트폴리오 레시피	11	비동기 작업 · 낙관적 잠금 저장	그림 6.2.10 · 6.2.3
D7 포트폴리오 생성	9	비동기 작업	그림 6.2.11
D8 포트폴리오 편집	17	AI 동기 호출 · 낙관적 잠금 저장	그림 6.2.12 · 6.2.3

도메인	기능 수	나타나는 유형	대표 그림
D9 배포 · 공개 사이트	9	동기 저장 · 파일 내보내기	그림 6.2.13 · 6.2.14
D10 통계 분석	14	공개 사이트 · 동기 조회 · AI 동기 호출	그림 6.2.15 · 6.2.16
D11 홈 · 내비게이션	9	동기 조회 · 학습 모델 서빙	그림 6.2.17
D12 계정 · 요금제	4	동기 조회 · 동기 저장	그림 6.2.1
합계	**151**	8개 유형	그림 6.2.1 – 6.2.17

1) 로그인 기능

D12 계정 · 유형 **동기 저장** · `POST /v1/auth/login`

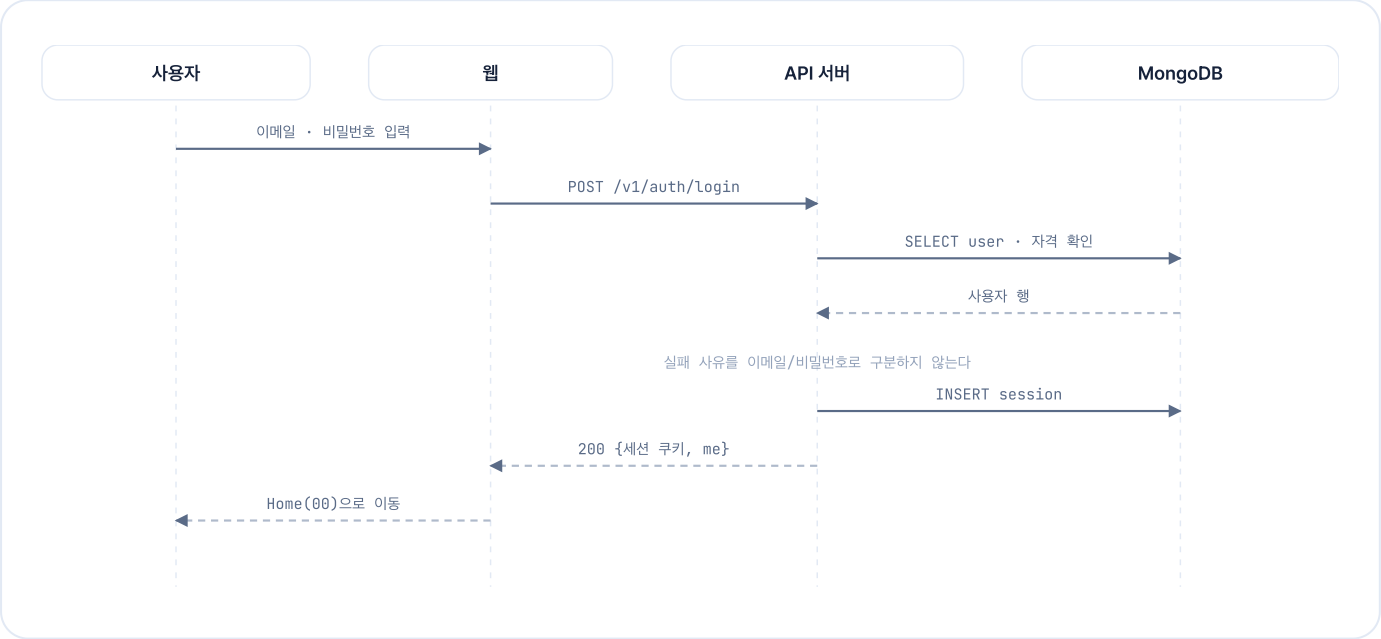


그림 6.2.1 로그인 기능의 시퀀스 다이어그램

사용자가 이메일과 비밀번호를 입력하면 웹이 `POST /v1/auth/login`을 호출한다. API 서버는 계정 행을 읽어 **자격을 확인**하고, 통과하면 세션을 만들어 쿠키와 함께 **200**으로 답한다. 웹은 응답을 받은 뒤 홈으로 이동한다.

실패 응답은 이메일과 비밀번호 중 무엇이 틀렸는지 구분하지 않는다. 사유를 나누어 알리면 계정의 존재 여부가 드러나고, 그것만으로도 공격자에게 쓸 정보가 된다. 세션은 서버가 발급한 쿠키로 유지하므로 클라이언트가 **토큰을 따로 보관하지 않는다**.

2) 기록 직접 만들기 기능

D1 · T1.2.1 · 유형 **동기 저장** · `POST /v1/career/records`

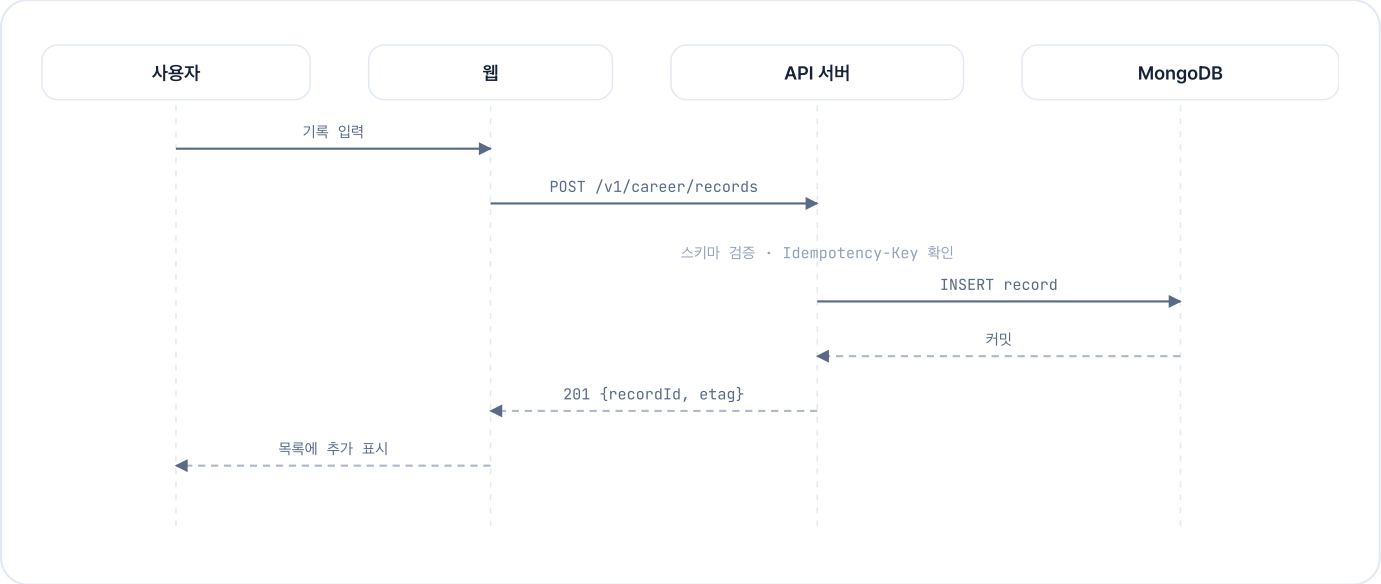


그림 6.2.2 기록 직접 만들기 기능의 시퀀스 다이어그램

웹이 기록을 `POST /v1/career/records`로 보내면 API 서버가 **스키마를 검증**하고 `Idempotency-Key`를 확인한 뒤 `record` 행을 넣는다. 커밋한 다음 **201**과 함께 식별자와 `etag`를 돌려주고, 웹은 목록에 새 항목을 표시한다.

같은 요청이 두 번 도착하여도 기록이 두 벌 생기지 않아야 한다. 응답이 늦으면 사용자가 다시 누르는 일이 흔하기 때문이다. 서버는 키를 기억하여 두 번째 요청에는 첫 번째의 결과를 그대로 돌려준다. 응답에 담긴 `etag`는 이어지는 수정에서 **버전 확인**에 쓰인다.

3) 기록 속성 편집 기능

D1 · T1.3.2 · 유형 **낙관적 잠금** · `PATCH /v1/career/records/:recordId`

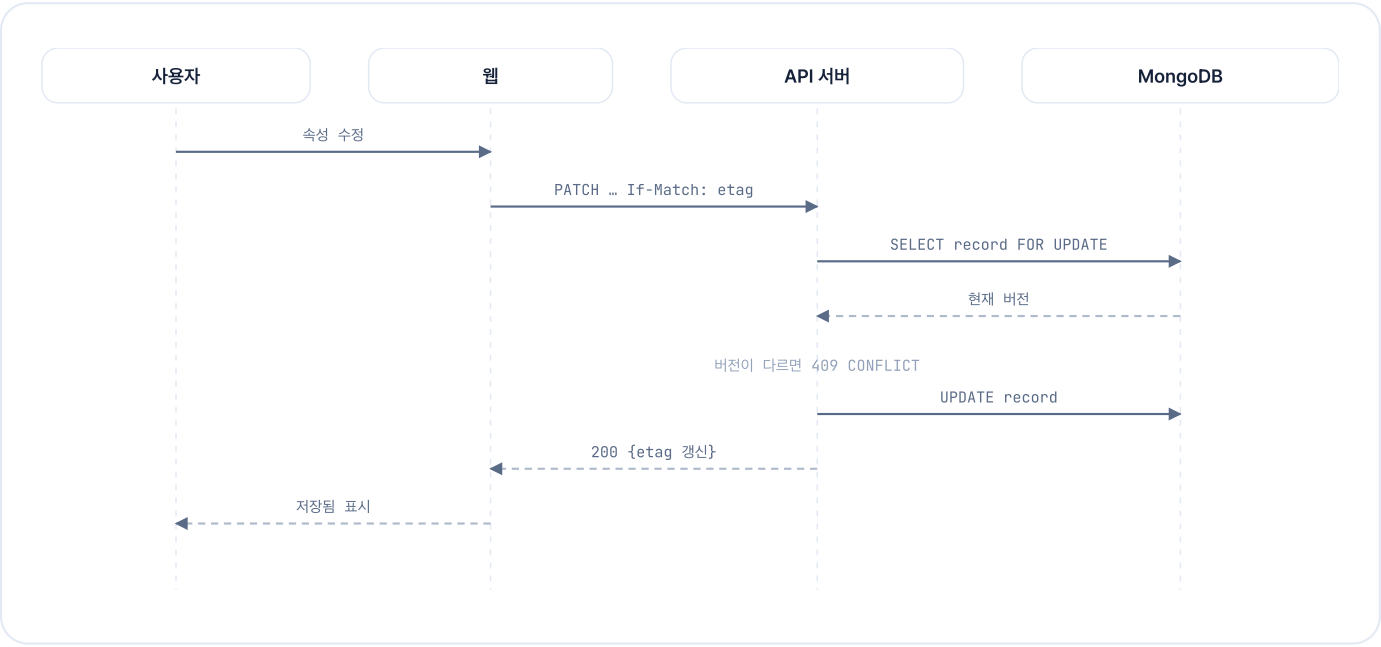


그림 6.2.3 기록 속성 편집 기능의 시퀀스 다이어그램

수정 요청은 `If-Match` 헤더에 직전에 받은 `etag`를 담는다. 서버는 대상 행을 **잠금 조회**하여 현재 버전과 비교하고, 같으면 수정한 뒤 갱신된 `etag`를 돌려준다. 다르면 **409**를 반환하고 아무것도 바꾸지 않는다.

같은 기록을 여러 탭에서 동시에 여는 경우가 발생한다. 버전을 확인하지 않으면 나중에 저장한 요청이 앞선 수정을 **통지 없이 대체**한다. 본 설계는 이를 허용하지 않고 409로 반환하며, 최신 상태를 다시 조회하도록 안내한다.

4) 스킬 자동 집계 기능

D1 · T1.6.1 · T1.6.2 · 유형 **동기 저장** · `POST /v1/career/skills/recompute`

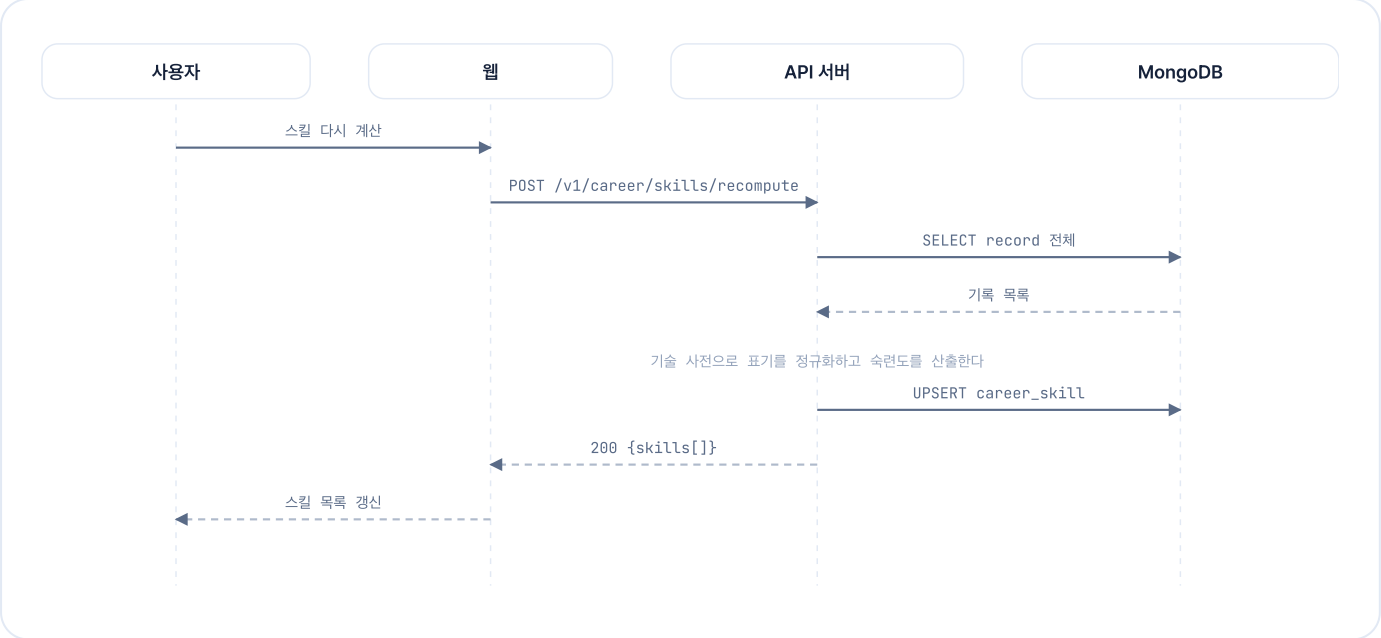


그림 6.2.4 스킬 자동 집계 기능의 시퀀스 다이어그램

웹이 다시 계산을 요청하면 서버는 사용자의 기록 전체를 읽어 기술 표기를 **사전으로 정규화**하고 **숙련도**를 산출한 뒤 `career_skill`을 갱신하고, 그 목록을 돌려준다.

스킬을 사용자가 직접 입력하게 하면 근거 없는 나열이 된다. 그래서 기록에서 뽑되, 표기가 제각각이므로 **정규화 단계**를 둔다. 어떤 기록이 근거였는지는 `GET /v1/career/skills/:skillId/evidence`로 되짚을 수 있고, 되짚지 못하는 스킬은 포트폴리오 생성에 쓰지 않는다.

5) 자연어 검색 조건 변환 기능

D2 · T2.2.1 · 유형 **AI 동기 호출** · `POST /v1/jobs/search/interpret`

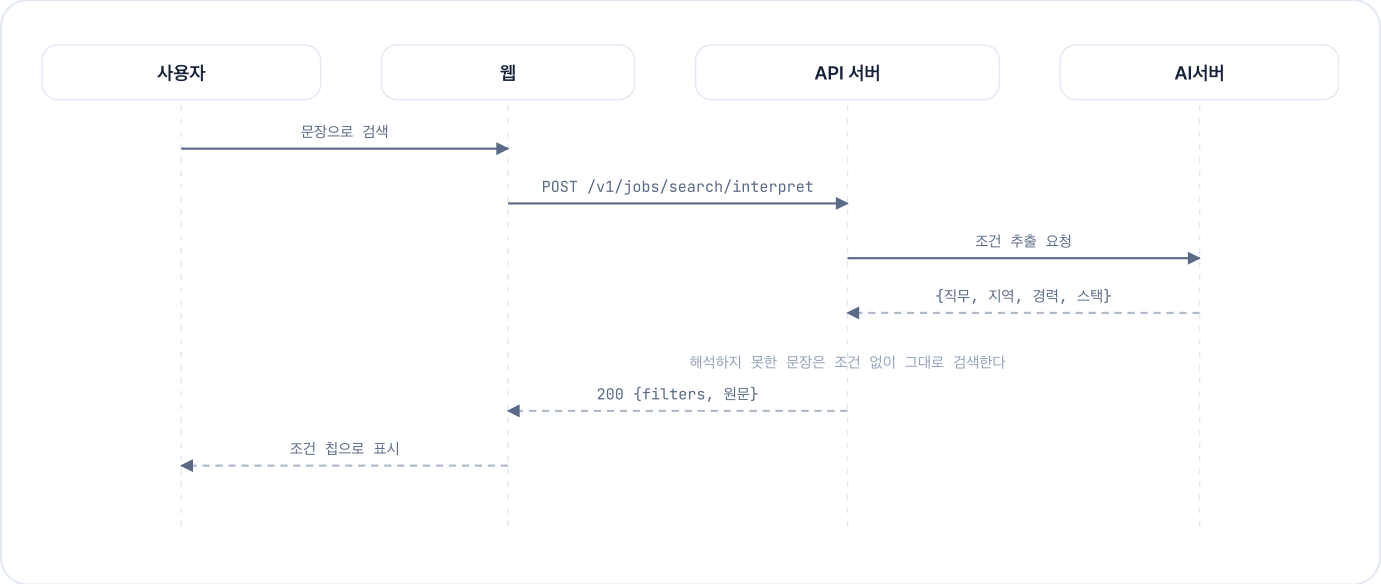


그림 6.2.5 자연어 검색 조건 변환 기능의 시퀀스 다이어그램

사용자가 문장으로 검색하면 API 서버가 AI서버에 **조건 추출**을 요청한다. 직무 · 지역 · 경력 · 스택으로 나뉜 결과를 **원문과 함께** 돌려주고, 웹은 그 조건을 칩으로 표시한다.

해석 결과는 **감추지 않고 표시**한다. 잘못 해석한 경우 사용자가 그 자리에서 수정하여야 한다. 해석하지 못한 문장은 조건 없이 원문으로 검색한다. 조건을 지어내면 결과가 조용히 좁아지고, 사용자는 왜 좁아졌는지 알 수 없다.

6) 공고 일치도 산출 기능

D2 · T2.3.1 · 유형 **학습 모델 서빙** · `POST /v1/jobs/postings/:id/match`

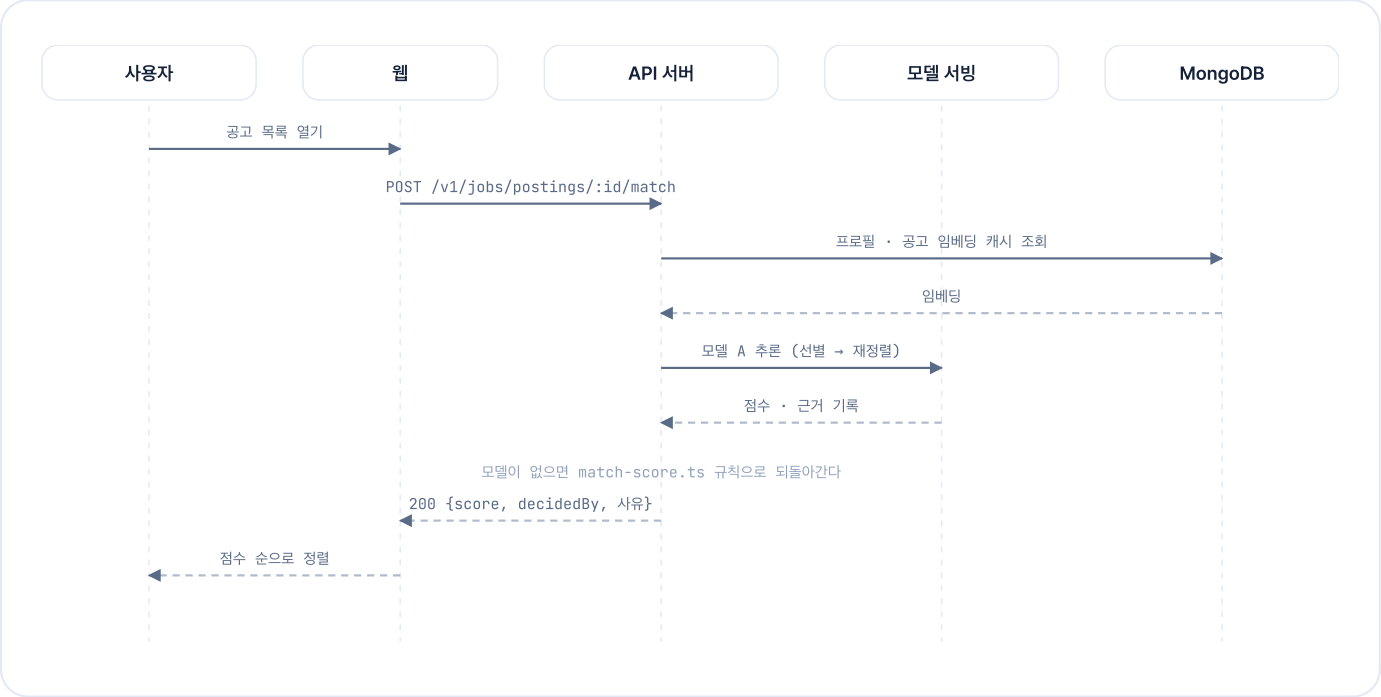


그림 6.2.6 공고 일치도 산출 기능의 시퀀스 다이어그램

AI학습서버가 구현되면 목록 요청을 받은 API 서버는 프로필과 공고의 **임베딩 캐시**를 모델 서빙에 넘기고, **모델 A**의 점수와 근거 기록을 받는다. 서버는 점수와 판정 주체와 사유를 함께 돌려주고 웹은 점수 순으로 정렬한다.

본 기능은 모델 A를 사용하는 경로이다. 공고 **200건**을 목록 한 화면 안에서 세워야 하므로 임베딩은 **미리 계산**해 두고 추론만 요청한다. 모델이 배포되어 있지 않거나 응답이 없으면 `match-score.ts` 의 규칙 판정으로 되돌아가며, 응답의 `decidedBy` 로 어느 쪽이 판정하였는지 밝힌다.

7) 공고 분석 기능

D3 · T3.1.1 · 유형 **비동기 작업** · `POST /v1/jobs/postings/:id/analyses`

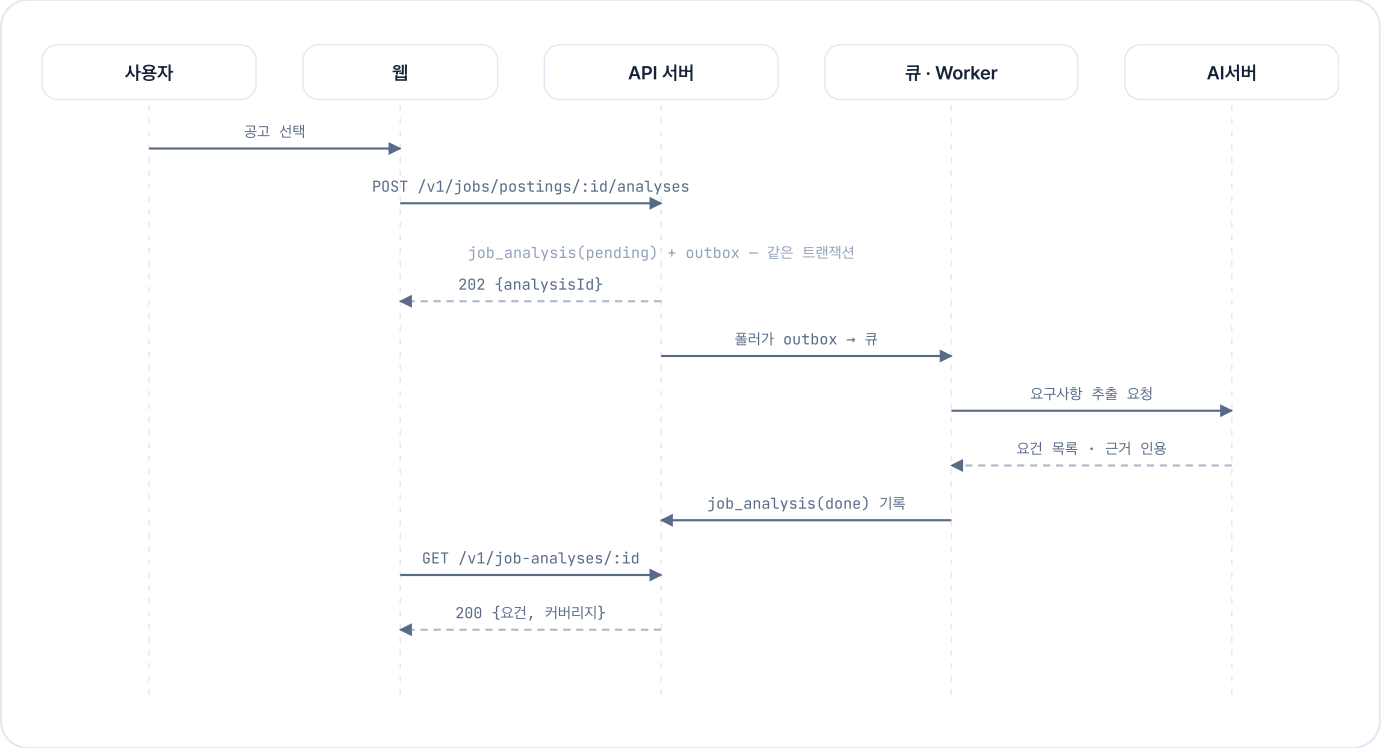


그림 6.2.7 공고 분석 기능의 시퀀스 다이어그램

분석 요청을 받은 API 서버는 `job_analysis` 행과 outbox 이벤트를 같은 트랜잭션에 기록하고 202로 접수만 알린다. 폴러가 outbox를 큐로 옮기고 Worker가 AI서버에 **요구사항 추출**을 요청하여 결과를 기록하며, 웹은 조회 경로로 완료된 분석을 읽는다.

분석에는 수십 초가 걸리므로 사용자를 기다리게 하지 않는다. 상태 행과 이벤트를 **한 트랜잭션**에 넣는 이유는 둘 중 하나만 남는 상태를 막기 위함이다. 트랜잭션이 실패하면 이벤트도 남지 않으므로, **「분석 중」** 상태로 멈춘 채 처리되지 않는 행이 남지 않는다.

8) 포트폴리오 재료 선정 기능

D4 · T4.1.1 · 유형 **학습 모델 서빙** · `GET · PUT /v1/brews/:id/materials`

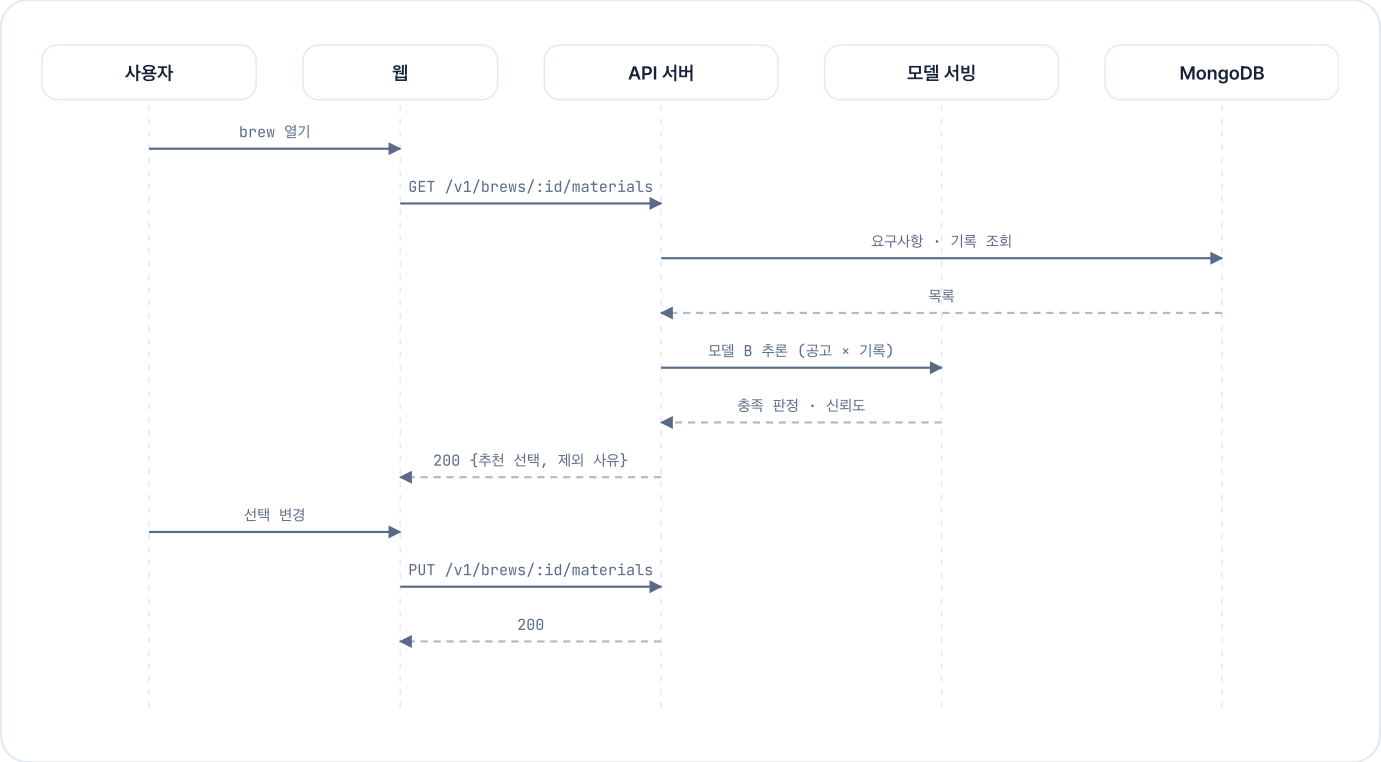


그림 6.2.8 포트폴리오 재료 선정 기능의 시퀀스 다이어그램

brew를 열면 서버가 공고의 요구사항과 사용자의 기록을 읽어 모델 서빙에 **요구사항 × 기록 쌍**을 넘긴다. **모델 B**가 기록마다 적합도와 순위를 산출하고, 서버는 상위 기록을 추천한다. 사용자가 선택을 바꾸면 저장 요청으로 반영한다.

추천은 **초기값**이고 결정은 사용자가 한다. 제외된 기록에 사유를 붙이는 것은 그 판단을 뒤집을 수 있게 하기 위함이다. 신뢰도가 낮은 쌍은 충족으로 확정하지 않고 확인이 필요한 상태로 남긴다. 판정을 단정하면 사용자가 검토할 자리가 없어진다.

9) 인터뷰 답변과 기록 확정 기능

D5 · T5.2.3 · 유형 **AI 동기 호출** · `PUT /v1/interview-sessions/:id/answers/:questionId`

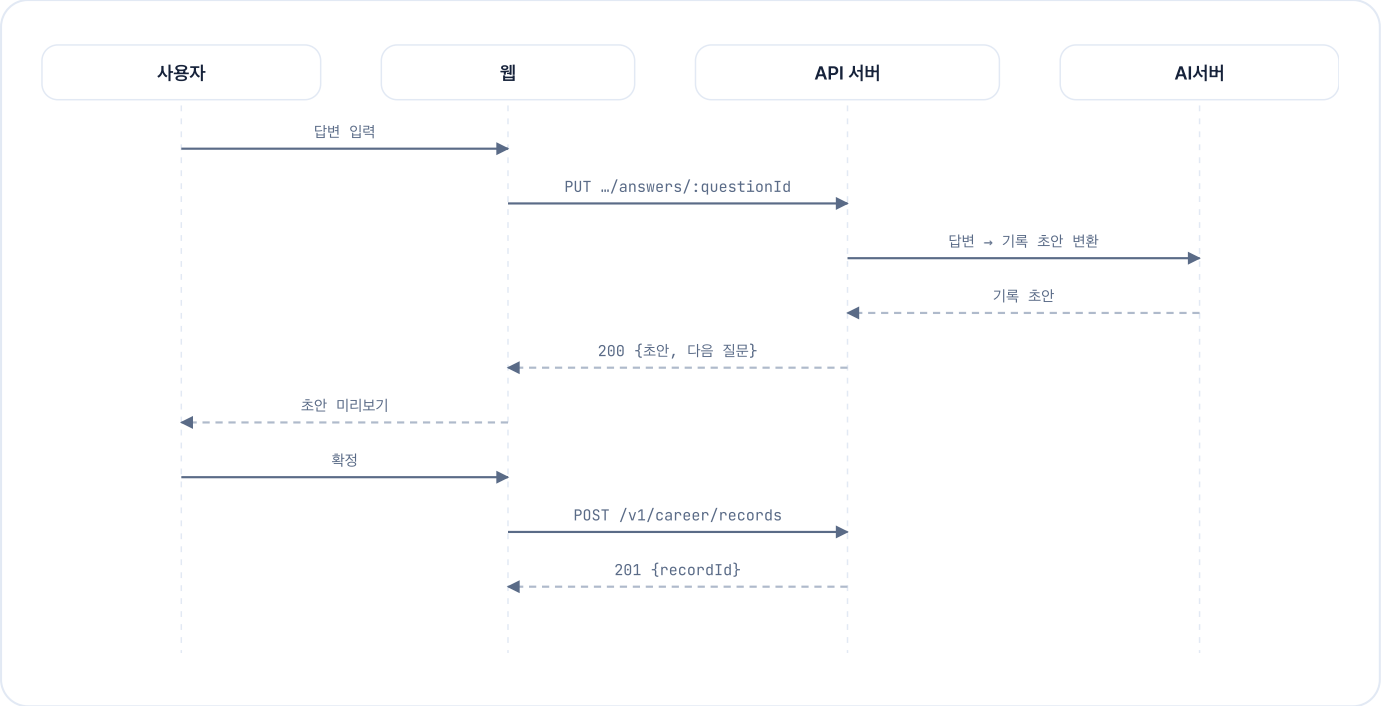


그림 6.2.9 인터뷰 답변과 기록 확정 기능의 시퀀스 다이어그램

답변이 들어오면 API 서버가 AI서버에 답변을 **기록 초안**으로 바꾸도록 요청하고, 초안과 다음 질문을 함께 돌려준다. 사용자가 초안을 확인하고 **확정**하면 기록 생성 요청으로 저장한다.

인터뷰의 목적은 커리어 기록의 생성이다. 다만 초안은 곧바로 저장하지 않는다. 사용자가 말한 내용과 모델이 정리한 내용이 다를 수 있다. **확정 이전 상태**를 화면에 표시하여 사용자가 무엇을 저장하는지 알게 한다.

10) 레시피 생성 기능

D6 · T6.1.1 · 유형 **비동기 작업** · `POST /v1/brews/:id/recipes`

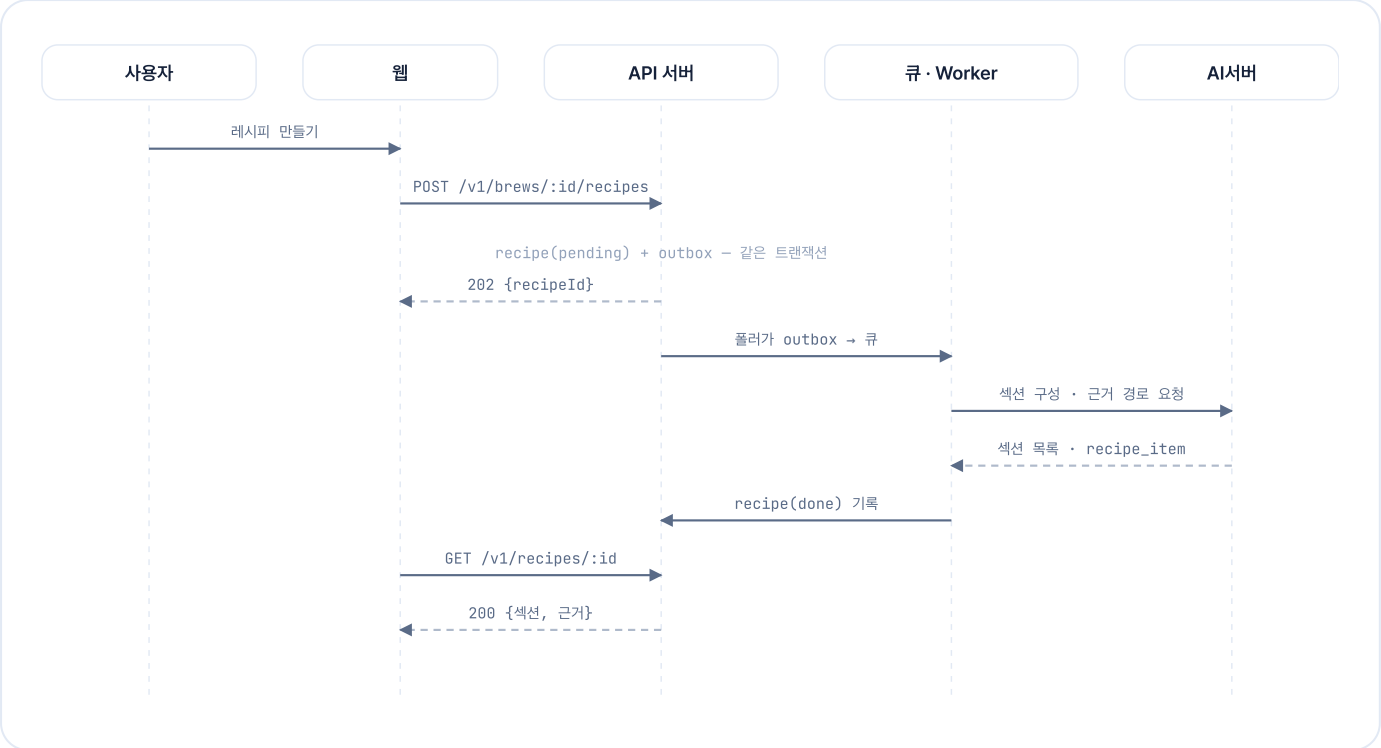


그림 6.2.10 레시피 생성 기능의 시퀀스 다이어그램

요청을 받으면 `recipe` 행과 outbox 이벤트를 **같은 트랜잭션**에 넣고 **202**로 답한다. Worker가 AI서버에 섹션 구성을 요청하여 `recipe_item`으로 저장하며, 항목마다 근거로 삼을 기록을 함께 적는다.

레시피는 생성할 문서의 **구성 정의**이다. 항목마다 근거 경로가 붙으며, 근거를 지정할 수 없는 항목은 레시피에 남기지 않는다.

11) 포트폴리오 생성 실행 기능

D7 · T7.2.1 · 유형 **비동기 작업** · `POST /v1/generation-jobs`

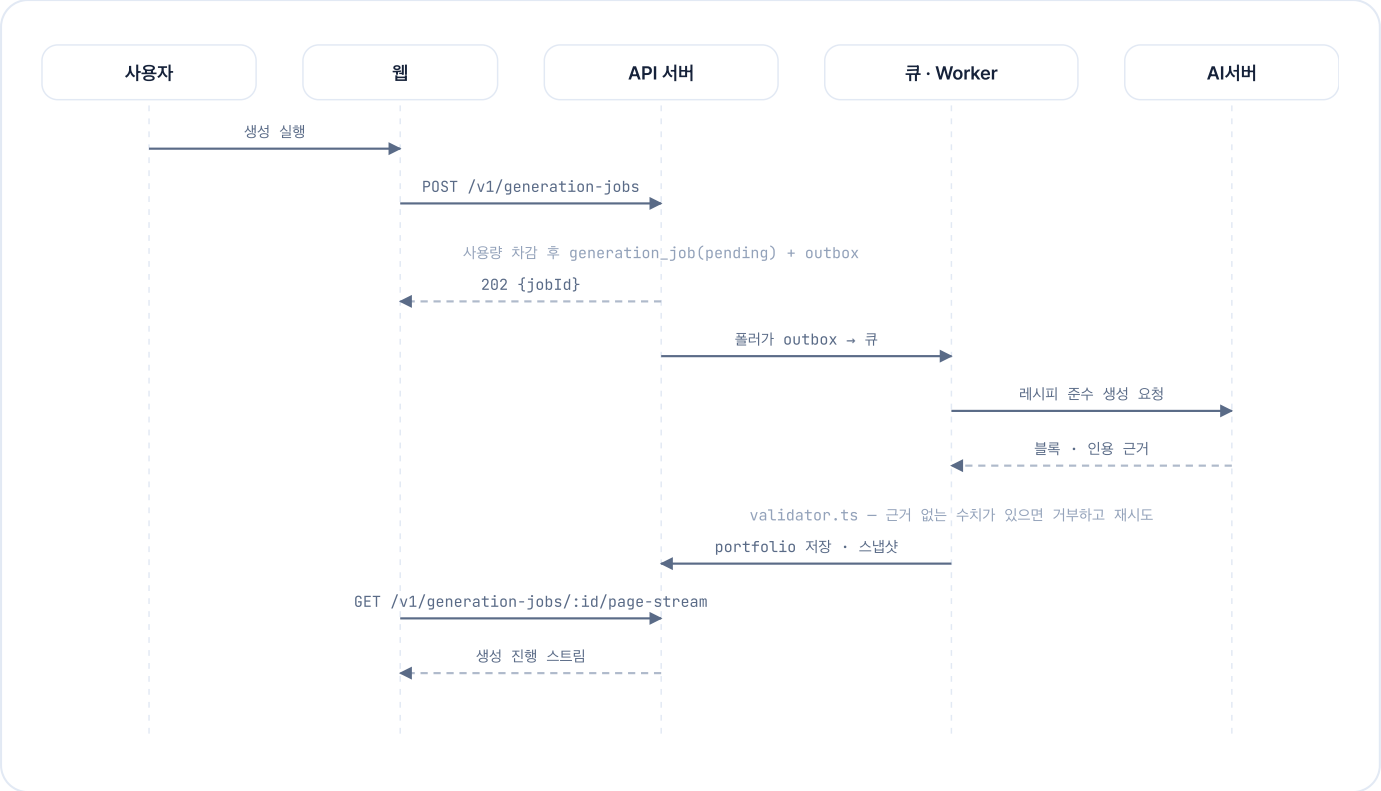


그림 6.2.11 포트폴리오 생성 실행 기능의 시퀀스 다이어그램

생성 요청을 받으면 **사용량을 먼저 차감**한 뒤 `generation_job`과 outbox를 같은 트랜잭션에 기록하고 202로 답한다. Worker는 레시피를 지키도록 지시하여 블록과 인용 근거를 받고, `validator.ts`가 블록에 적힌 수치가 인용한 근거 안에 있는지 검사한다. **통과한 결과만 저장**하며 진행 상황은 스트림으로 내려보낸다.

검증에 걸린 결과는 저장하지 않고 다시 생성한다. 사용량을 **작업 생성 이전**에 차감하는 것은 같은 요청이 여러 번 접수되어 한도를 넘기는 것을 막기 위함이다.

12) 대화형 수정 기능

D8 · T8.2.1 · T8.2.2 · 유형 **AI 동기 호출** · `edit-preview → apply`

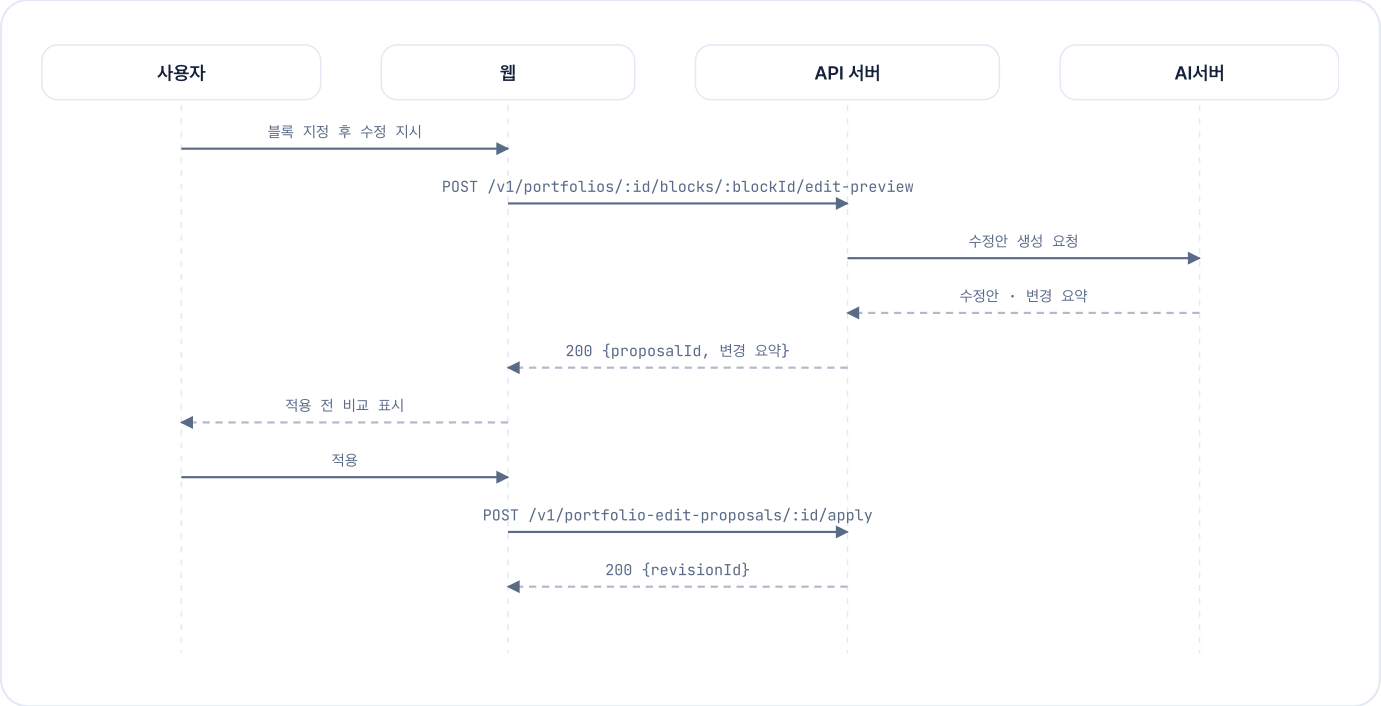


그림 6.2.12 대화형 수정 기능의 시퀀스 다이어그램

사용자가 블록을 지정하고 수정을 지시하면 API 서버가 AI서버에 수정안을 요청하여 변경 요약과 함께 **제안 식별자**를 돌려준다. 웹이 적용 전 비교를 보이고, 사용자가 적용을 누르면 별도의 경로로 제안을 확정하여 **개정**을 남긴다.

제안과 적용은 두 단계로 분리한다. 사용자가 변경 내용을 먼저 확인한 뒤 결정하여야 한다. 적용된 변경은 개정으로 쌓이므로 되돌릴 수 있고, 사용자가 직접 고친 블록은 **이후 AI 수정에서 제외**한다. 사람이 고친 문장이 다음 요청에 지워지면 편집을 맡길 수 없다.

13) 배포와 주소 발급 기능

D9 · T9.1.1 · 유형 **동기 저장** · `POST /v1/portfolios/:id/deployments`

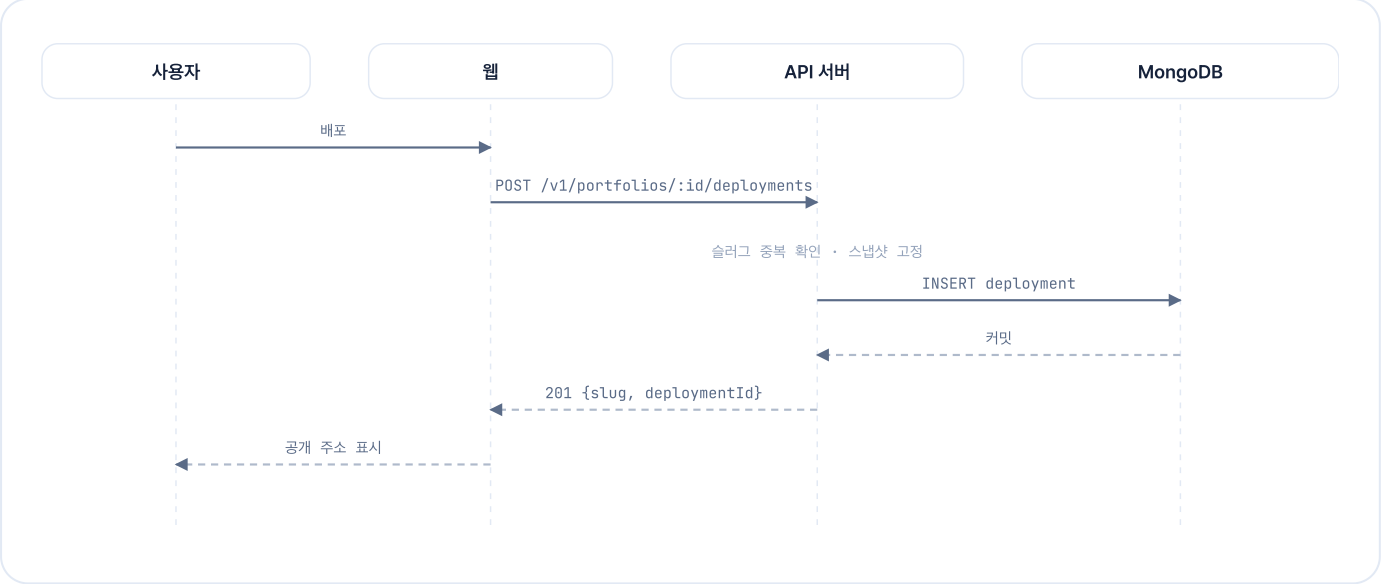


그림 6.2.13 배포와 주소 발급 기능의 시퀀스 다이어그램

배포 요청을 받으면 서버가 **슬러그의 중복**을 확인하고 그 시점의 문서를 **스냅샷으로 고정**한 뒤 `deployment` 행을 넣는다. 커밋 후 슬러그와 배포 식별자를 돌려주고 웹은 공개 주소를 표시한다.

스냅샷을 고정하므로 이후의 편집이 공개된 문서를 **즉시 바꾸지 않는다**. 지원서를 보낸 뒤에도 편집할 수 있어야 하고, 반대로 보낸 문서가 사용자도 모르게 달라지는 일은 없어야 한다. 공개 내용은 새로 배포할 때에만 갱신된다.

14) 문서 내보내기 기능

D9 · T9.2.3 · 유형 **파일 내보내기** · `POST /v1/portfolios/:id/exports`

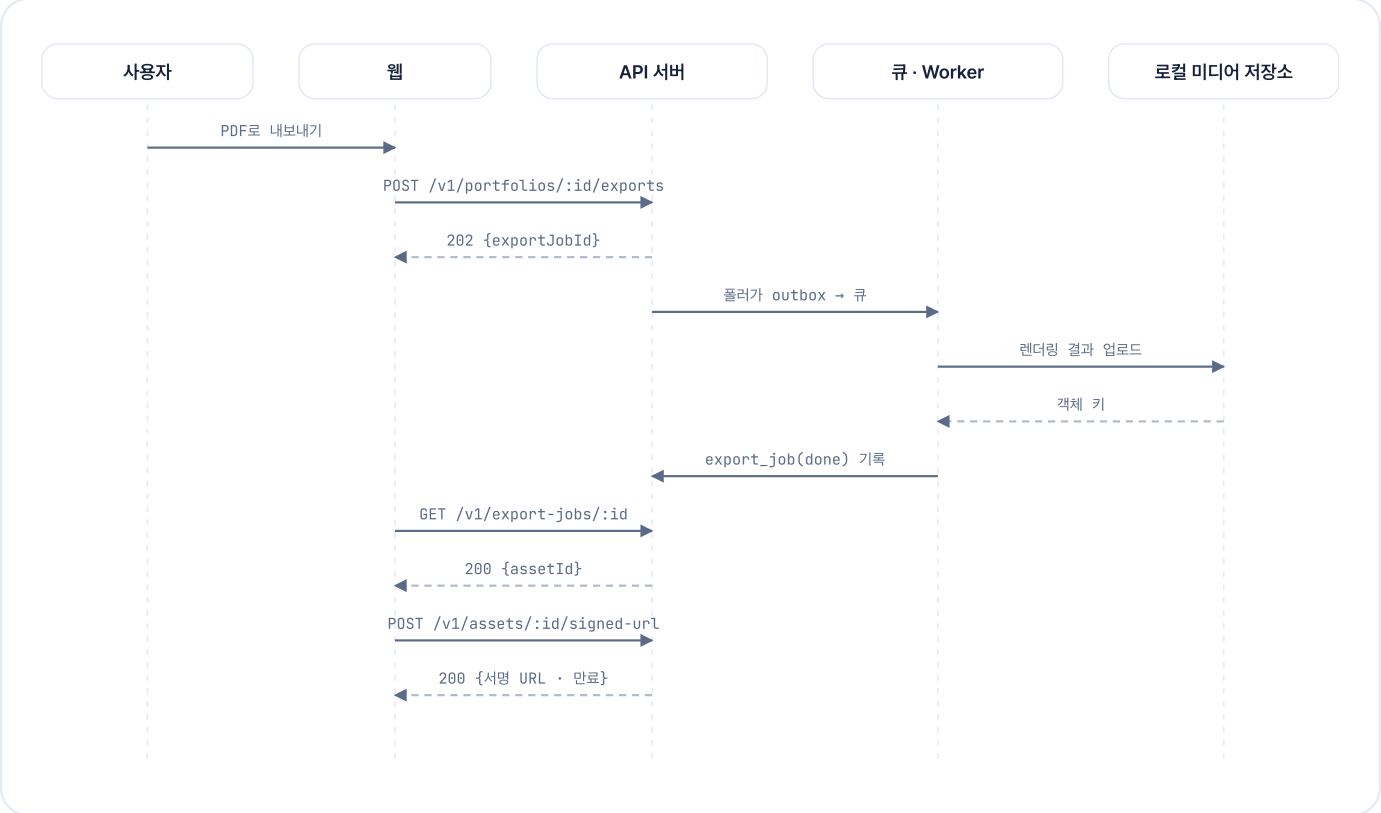


그림 6.2.14 문서 내보내기 기능의 시퀀스 다이어그램

내보내기 요청은 **202**로 접수되고 Worker가 문서를 렌더링하여 로컬 미디어 저장소에 올린다. 작업이 끝나면 자산 식별자를 조회할 수 있고, 별도의 요청으로 만료가 있는 **서명 URL**을 발급받아 내려받는다.

파일은 API 서버가 직접 전송하지 않는다. 렌더링에 시간이 오래 걸리며, 응답을 유지하면 서버의 처리량이 저하된다. 서명 URL에 **만료**를 두어 주소가 유출되어도 일정 시간이 지나면 쓸 수 없게 한다. 계정 삭제를 요청하면 이미 발급된 자산도 함께 **무효화**된다.

15) 방문 지표 수집 기능

D10 · T10.1.1 · 유형 **공개 사이트** · `GET /v1/public/portfolios/:slug · POST /v1/analytics/events`

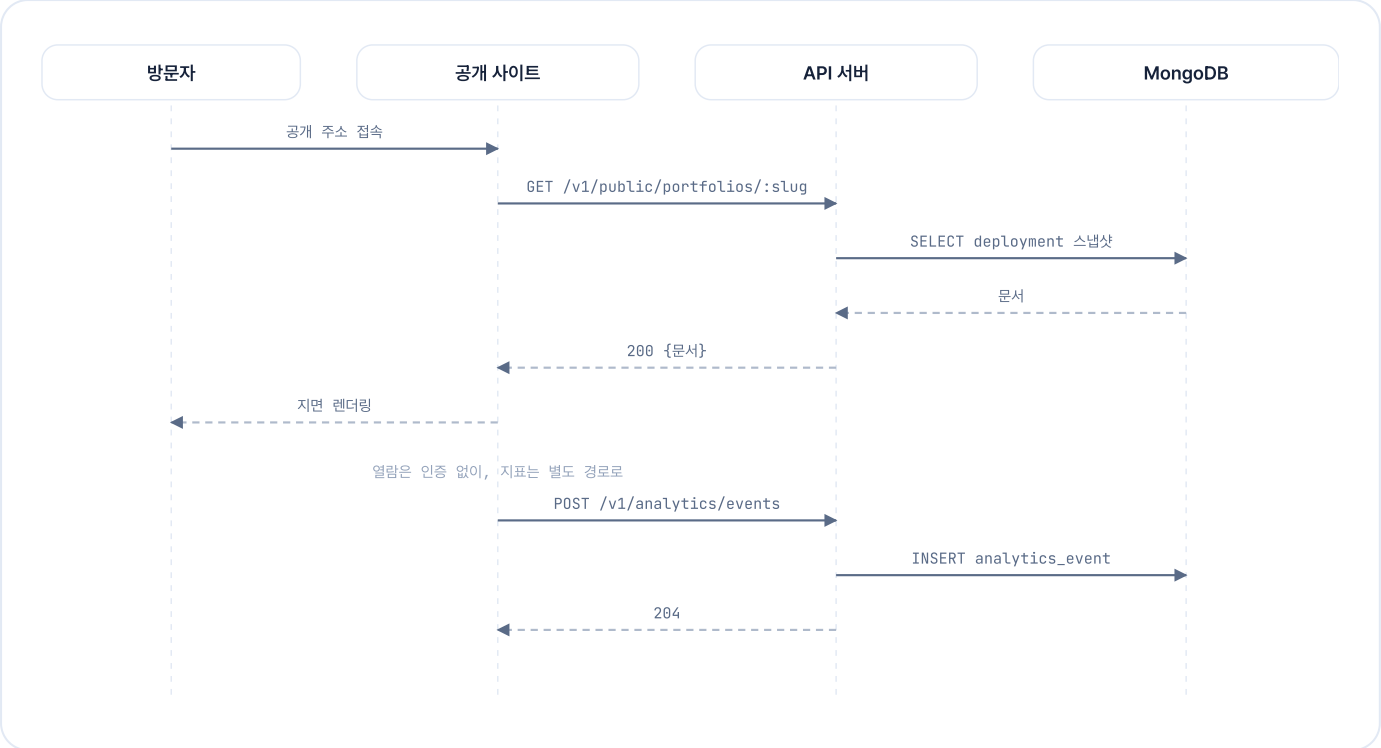


그림 6.2.15 방문 지표 수집 기능의 시퀀스 다이어그램

방문자가 공개 주소로 들어오면 **인증 없이** 배포 스냅샷을 읽어 지면을 그린다. 지면이 그려진 뒤 별도의 경로로 방문 이벤트를 보내고, 서버는 본문 없이 **204**로 답한다.

열람과 지표 수집은 서로 다른 경로로 분리한다. 지표 수집이 실패하여도 열람은 영향을 받지 않아야 한다. 수집하는 값은 **개인을 식별하지 않는** 범위로 한정한다.

16) 읽기 패턴 해설 기능

D10 · T10.3.1 · 유형 **AI 동기 호출** · `POST /v1/portfolios/:id/analytics/insight`

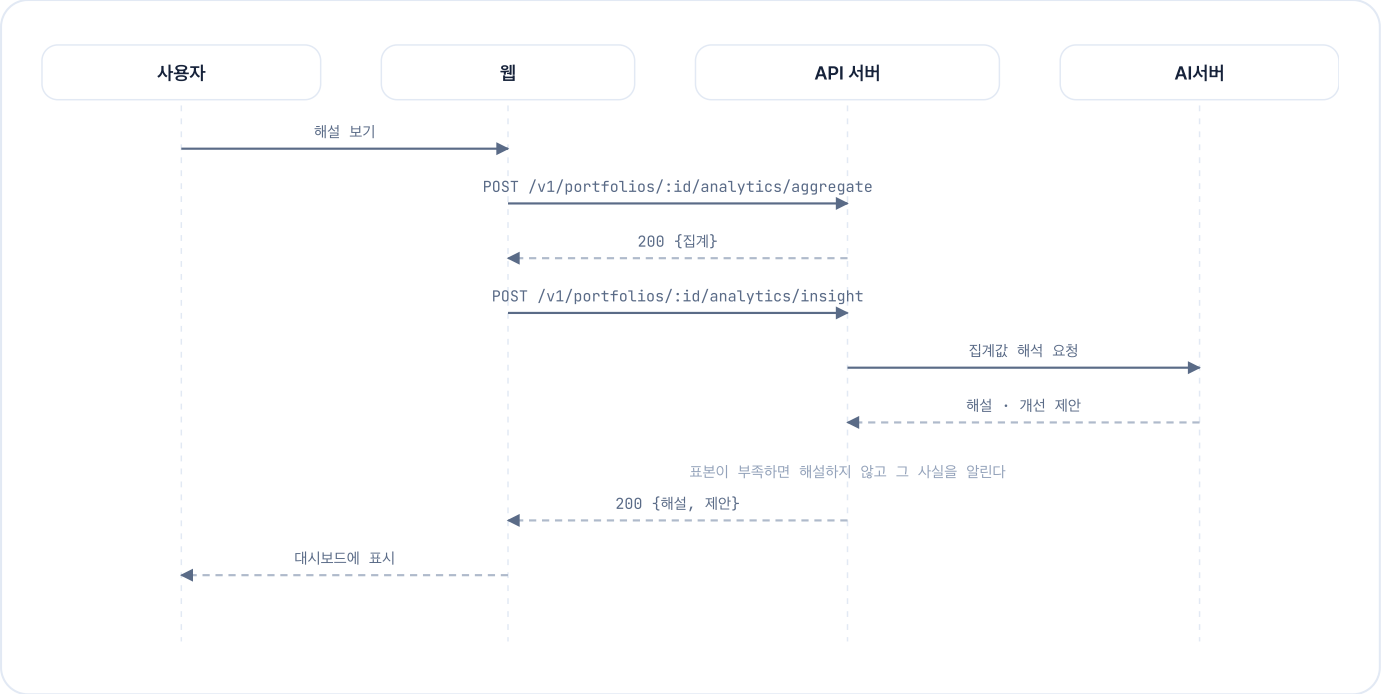


그림 6.2.16 읽기 패턴 해설 기능의 시퀀스 다이어그램

대시보드는 먼저 **집계 경로**로 수치를 받고, 그 수치를 **해설 경로**에 넘겨 AI서버의 해석과 개선 제안을 받는다. 표본이 부족하면 해설을 만들지 않고 그 사실을 알린다.

집계와 해설을 나눈 이유는 표본이 적을 때 해설을 멈추기 위함이다. 방문이 몇 건뿐인데 경향을 말하면 근거 없는 조언이 된다. 또한 해설은 **집계값만**을 입력으로 받으므로, 해설에 나오는 수치는 화면에 표시된 수치와 **반드시 일치**한다.

17) 홈 추천 공고 기능

D11 · T11.1.1 · 유형 **학습 모델 서빙** · `GET /v1/home`

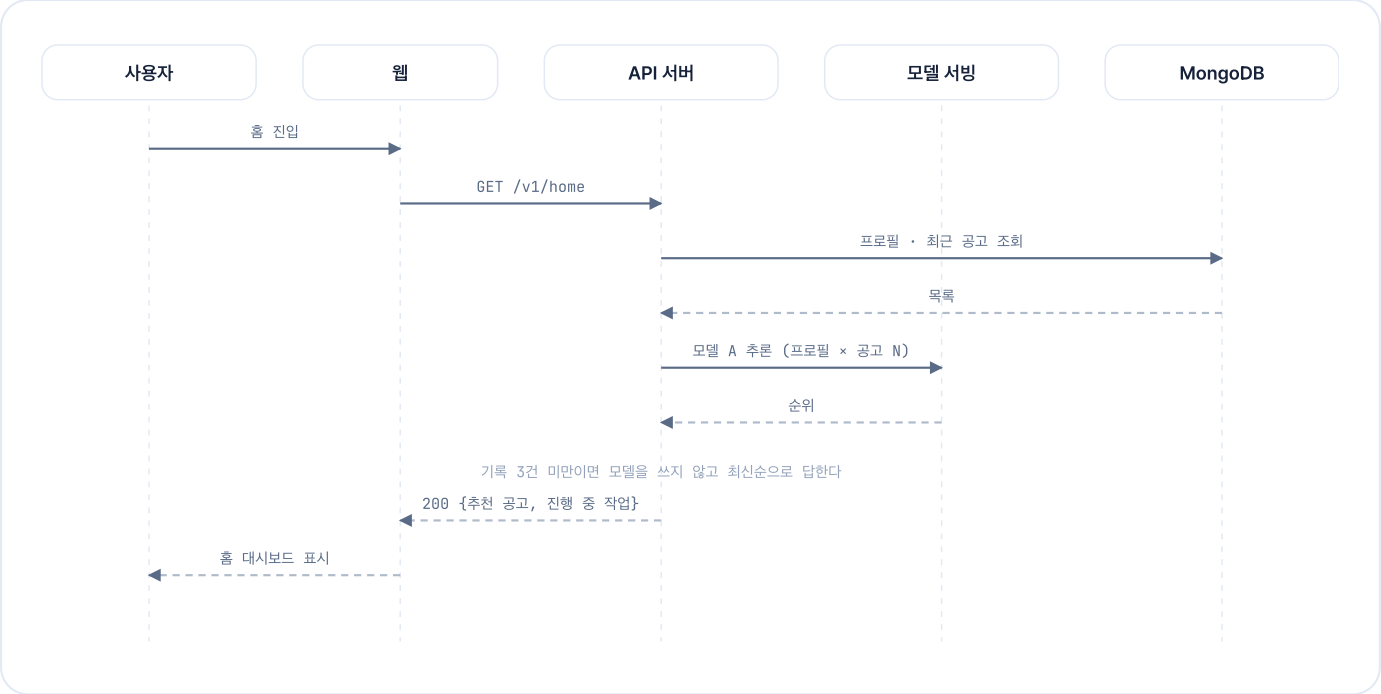


그림 6.2.17 홈 추천 공고 기능의 시퀀스 다이어그램

홈에 들어오면 서버가 프로필과 최근 공고를 읽어 모델 서빙에 넘기고, 받은 순위로 **추천 공고**와 **진행 중인 작업**을 함께 돌려준다.

홈은 사용자가 가장 자주 여는 화면이므로 모델 A의 지연이 그대로 체감된다. 그래서 이 경로는 목록을 세우는 데 필요한 만큼만 추론하고 공고별 **상세 판정은 하지 않는다**. 기록이 3건 미만인 프로필은 학습 분포 밖이므로 모델을 쓰지 않고 최신순으로 답한다.

6.3 관리자 기능설계 (Sequence Diagram)

2.6에서 정의한 **24개**의 관리자 기능을 시퀀스 다이어그램으로 설계한다. 지침은 6.2와 같으며, 흐름이 서로 다른 10개를 대표로 설계하고 나머지는 아래 유형에 귀속시킨다. 관리자 기능에는 사용자 기능에 없는 유형이 둘 있다. AI학습서버를 제어하는 **학습서버 비동기**와 **학습서버 동기**이다.

■ 상호작용 유형

유형	흐름의 특징	대표 그림
동기 조회	관리자 웹이 API를 한 번 호출하고 그 응답으로 화면을 그린다.	그림 6.3.5
동기 저장	스키마와 사전 조건을 확인한 뒤 데이터베이스에 기록한다.	그림 6.3.1 · 6.3.2
낙관적 잠금 저장	`If-Match`로 버전을 확인하고 파생 값을 같은 트랜잭션에서 다시 계산한다.	그림 6.3.6
비동기 작업	202를 먼저 돌려주고 outbox와 Worker가 외부 API를 호출한다.	그림 6.3.3

유형	흐름의 특징	대표 그림
AI 동기 호출	API 서버가 AI서버를 직접 호출하고 응답을 기다린다.	그림 6.3.4
학습서버 비동기	API 서버가 AI학습서버에 작업을 넘기고 식별자로 진행을 조회한다.	그림 6.3.7 · 6.3.9
학습서버 동기	AI학습서버에 조회하거나 제어를 보내고 즉시 응답을 받는다.	그림 6.3.8 · 6.3.10

일곱 유형이 24개 기능의 흐름을 모두 포함한다. 아래 다섯은 6.2와 같은 유형이고, 학습서버 두 유형이 관리자 기능에만 나타난다.

■ 주요기능별 귀속

주요기능	상세기능 수	나타나는 유형	대표 그림
관리자 인증	2	동기 저장	그림 6.3.1
공고 소스 관리	4	동기 조회 · 동기 저장 · 비동기 작업	그림 6.3.2 · 6.3.3
공고 관리	3	동기 저장 · AI 동기 호출	그림 6.3.4
수집 모니터링	2	동기 조회	그림 6.3.5
스케줄 관리	1	낙관적 잠금 저장	그림 6.3.6
학습 데이터 관리	2	학습서버 비동기 · 학습서버 동기	그림 6.3.7 · 6.3.8
모델 운영	5	학습서버 비동기 · 학습서버 동기	그림 6.3.9 · 6.3.10
장애 관리	1	학습서버 동기	그림 6.3.10
사용자 관리	2	동기 조회 · 동기 저장	그림 6.3.5 · 6.3.2
시스템 상태	2	동기 조회	그림 6.3.5
합계	**24**	7개 유형	그림 6.3.1 – 6.3.10

1) 관리자 인증과 역할 기반 접근 제어 기능

2.6의 관리자 인증 · 1 – 2 · 유형 **동기 저장** · `POST /v1/auth/login · 역할 검사` · 미구현

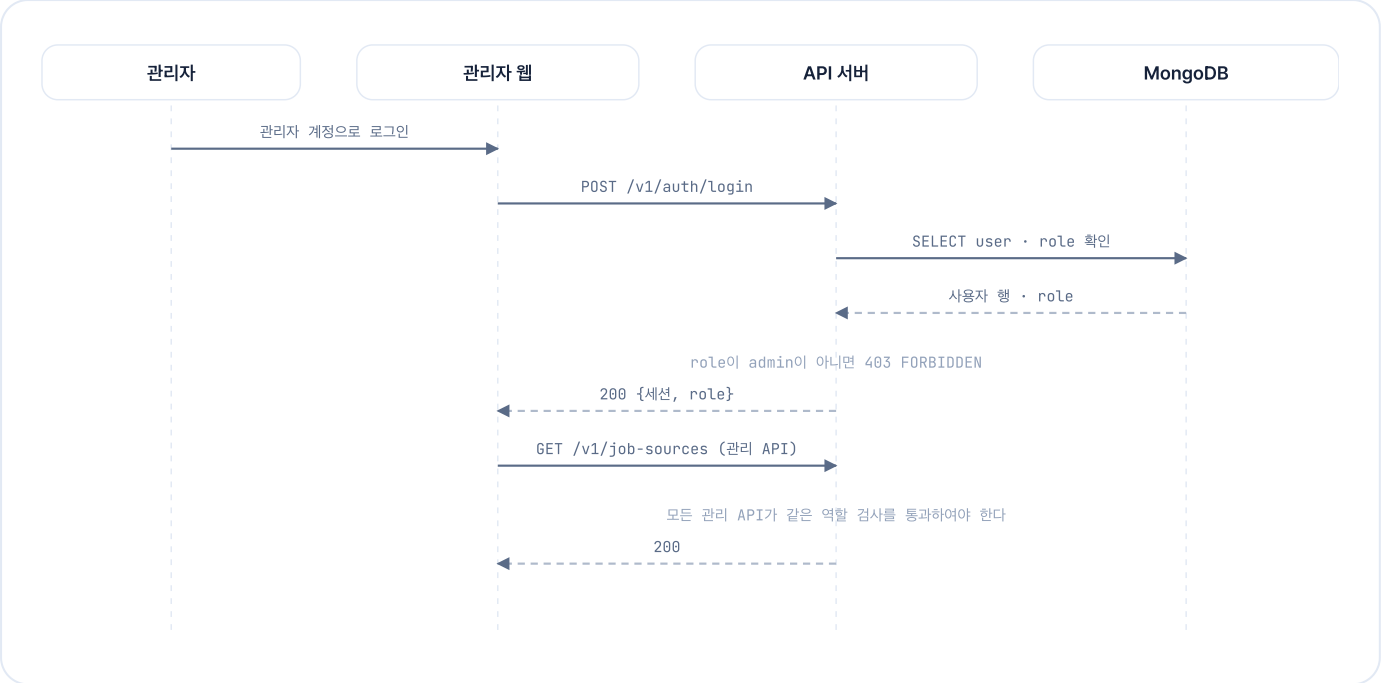


그림 6.3.1 관리자 인증과 역할 기반 접근 제어 기능의 시퀀스 다이어그램

관리자가 로그인하면 서버가 계정 행과 함께 역할을 읽어 세션에 담는다. 이후 관리 경로 호출은 **인증 훅**에서 역할을 확인하며, 관리자가 아니면 **403**으로 막는다.

역할 검사를 경로마다 두면 새 경로를 추가할 때 빠뜨리므로, 인증 훅 한 곳에 두어 모든 관리 경로가 같은 검사를 지나게 한다.

2) 수집 소스 등록 기능

2.6의 공고 소스 관리 · 2 · 유형 **동기 저장** · `POST /v1/job-sources` · 구현됨

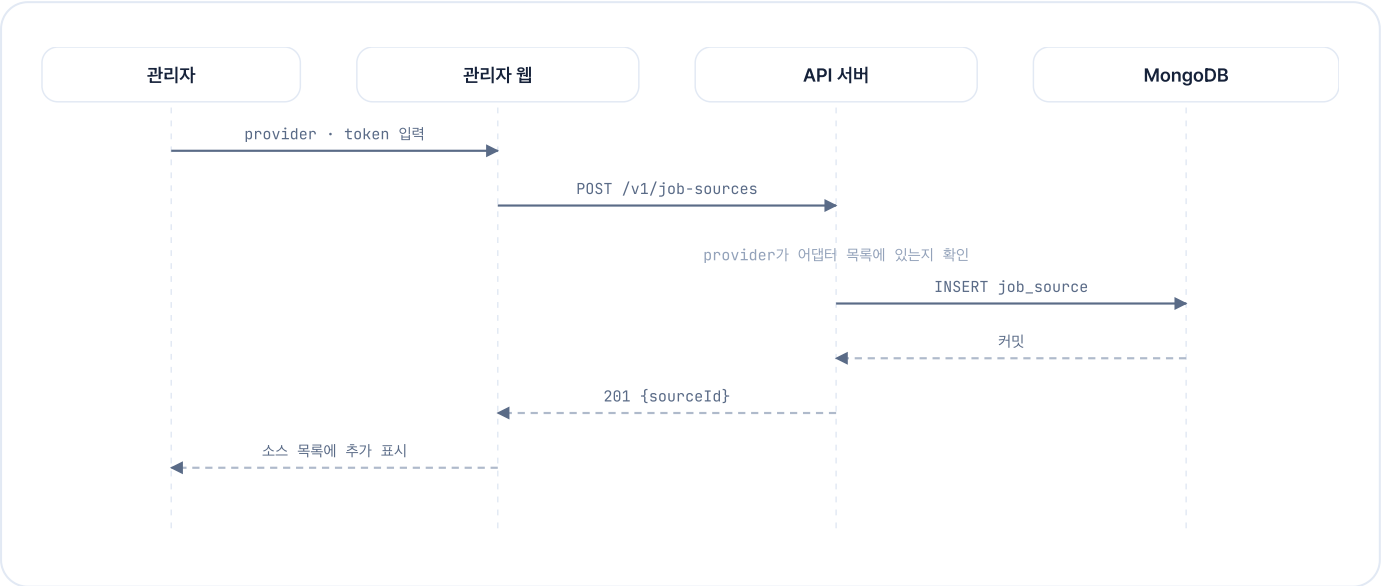


그림 6.3.2 수집 소스 등록 기능의 시퀀스 다이어그램

관리자가 채널과 접근 토큰을 입력하면 서버가 그 채널의 **어댑터**가 있는지 확인한 뒤 `job_sources` 행을 넣고 식별자를 돌려준다.

어댑터가 없는 채널을 등록하면 수집이 **매일 0건**을 모으고 **성공으로 기록**한다. 수집 실패가 성공으로 기록되므로 운영자가 인지할 수 없다. 그래서 등록 시점에 어댑터 목록과 대조하여 그런 소스가 만들어지는 것부터 막는다.

3) 소스 수집 실행 기능

2.6의 공고 소스 관리 · 3 – 4 · 유형 **비동기 작업** · `POST /v1/job-sources/runs · /:id/runs` · 구현됨

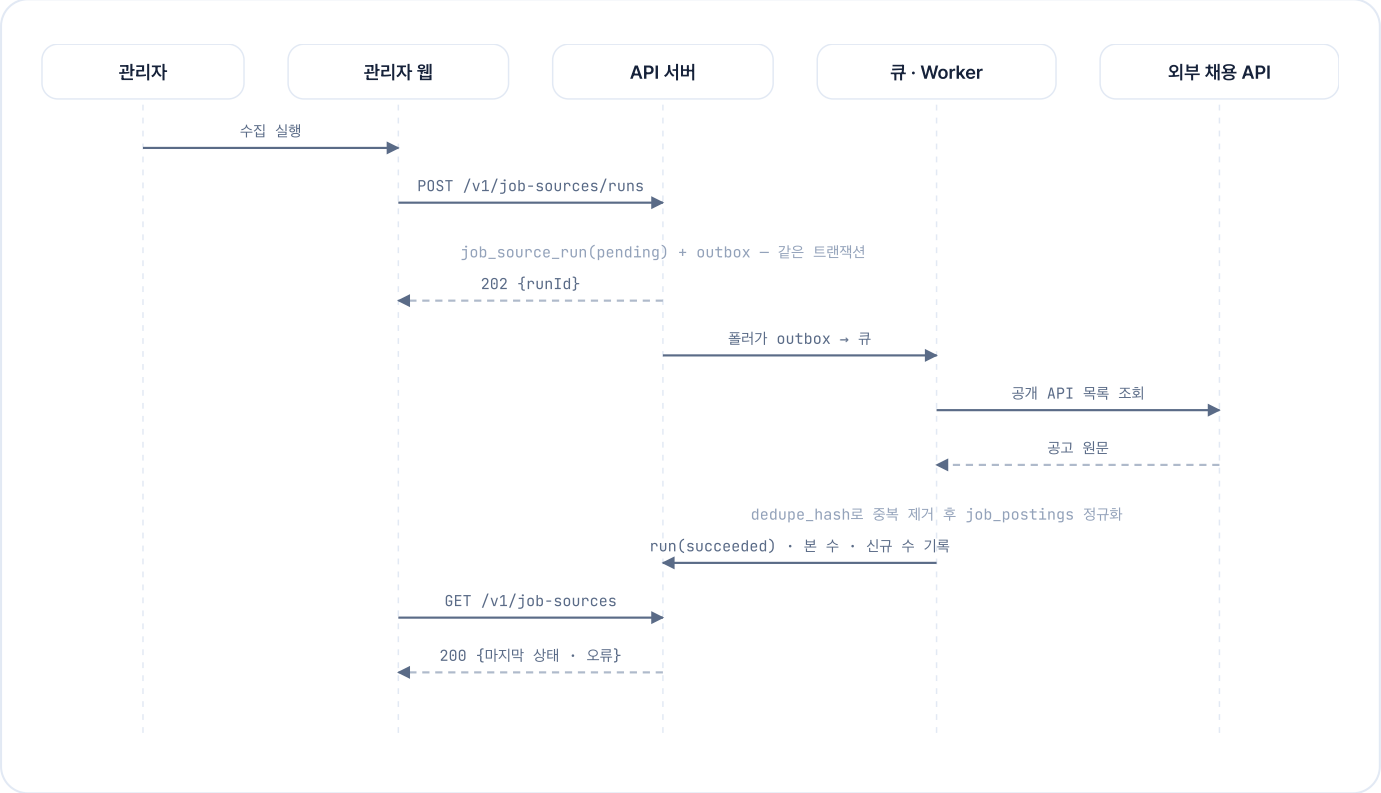


그림 6.3.3 소스 수집 실행 기능의 시퀀스 다이어그램

실행 요청은 `job_source_run` 행과 outbox를 **같은 트랜잭션**에 남기고 202로 접수된다. Worker가 외부 채용 API의 공개 목록을 조회하여 **중복 해시**로 걸러 낸 뒤 `job_postings`으로 정규화하고, 본 공고 수와 새로 들어온 수를 실행 결과에 기록한다.

수집은 외부 API에 의존하므로 실패가 정상적인 경우이다. 실행마다 본 수 · 신규 수 · 실패 사유를 남기는 것은 화면이 「왜 안 모였는가」에 답할 수 있게 하기 위함이다. 중복 제거를 **정규화 이전**에 두어 같은 공고가 채널을 달리하여 두 번 들어오는 것을 막는다.

4) URL로 공고 가져오기 기능

2.6의 공고 관리 · 6 · 유형 **AI 동기 호출** · `POST /v1/jobs/url-imports` · 구현됨

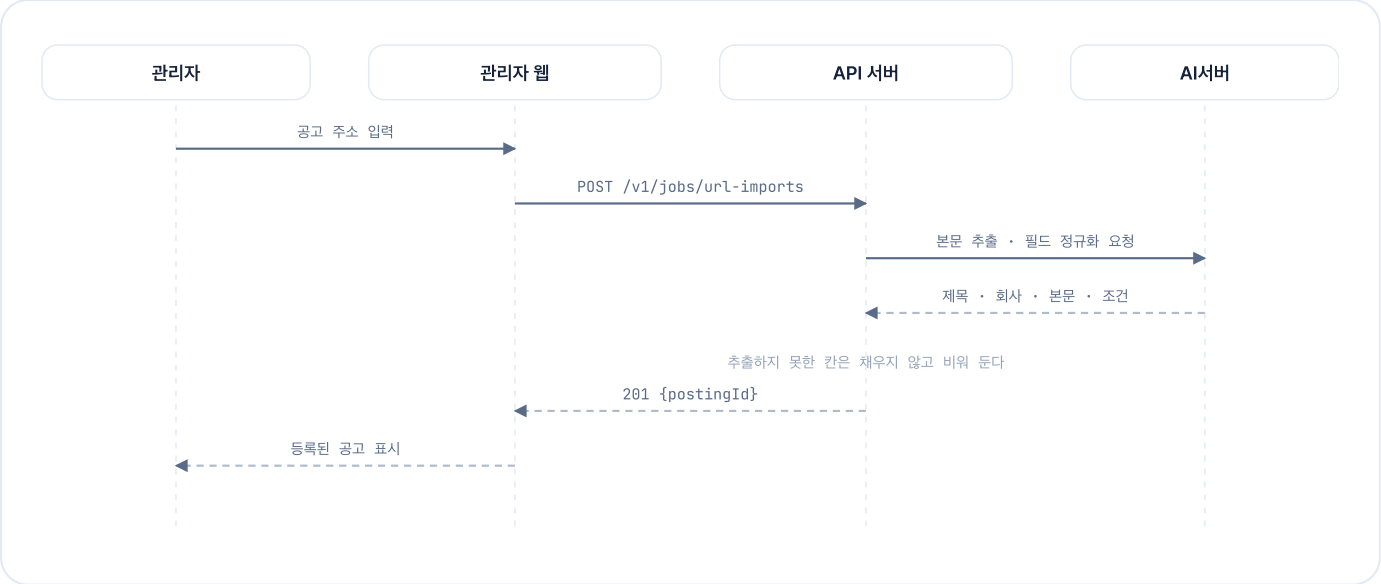


그림 6.3.4 URL로 공고 가져오기 기능의 시퀀스 다이어그램

공고 주소를 입력하면 서버가 AI서버에 **본문 추출**과 **항목 정규화**를 요청하고, 그 결과로 공고 한 건을 등록한다.

어댑터가 없는 사이트의 공고를 한 건씩 들여오는 경로이다. 추출하지 못한 항목을 짐작하여 채우면 화면이 거짓을 말하므로 모르는 값은 비워 둔다. 비어 있는 항목은 목록에서 드러나므로 관리자가 **직접 채울** 수 있다.

5) 수집 실행 이력 조회와 실패 알림 기능

2.6의 수집 모니터링 · 12 – 13 · 유형 **동기 조회** · `GET /v1/job-sources · 알림` · 컬렉션에 기록

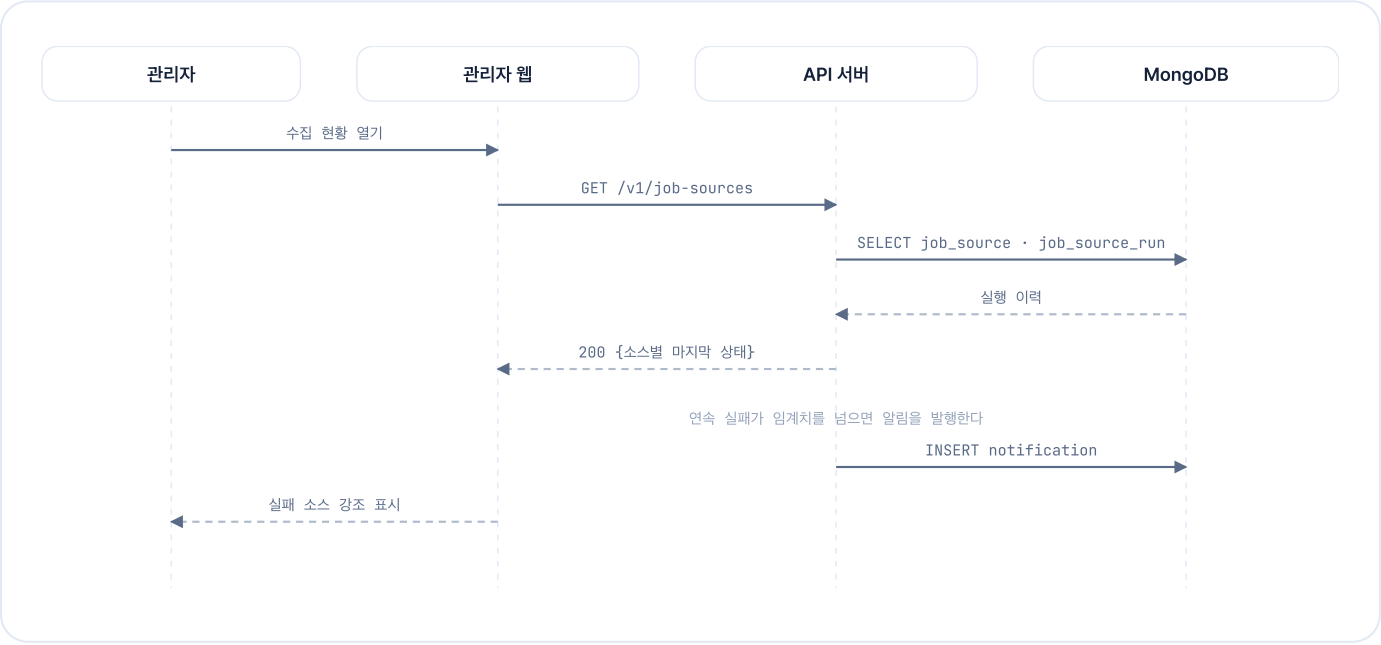


그림 6.3.5 수집 실행 이력 조회와 실패 알림 기능의 시퀀스 다이어그램

수집 현황 화면은 소스와 실행 이력을 함께 읽어 소스별 **마지막 상태**를 보인다. 연속 실패 횟수가 임계치를 넘으면 서버가 알림을 발행하고, 화면은 그 소스를 강조한다.

수집은 **조용히 실패**하는 것이 가장 위험하다. 며칠 뒤에야 공고가 들어오지 않은 것을 알아채면 그 사이의 추천이 모두 낡은 자료로 만들어진다. 그래서 실패를 세어 임계치에서 사람을 부른다.

6) 스케줄 작업 정의 관리 기능

2.6의 스케줄 관리 · 14 · 유형 **낙관적 잠금 저장** · `scheduled_job_definition` · 컬렉션에 기록

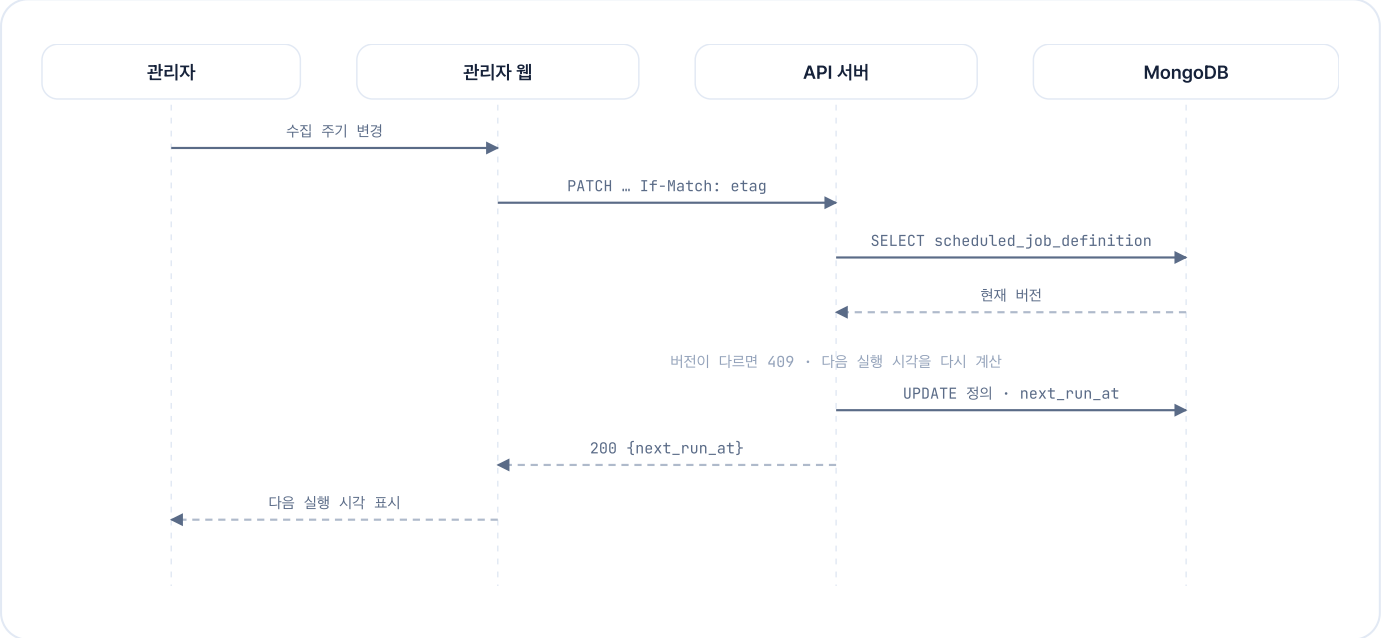


그림 6.3.6 스케줄 작업 정의 관리 기능의 시퀀스 다이어그램

주기를 바꾸는 요청은 `If-Match`로 버전을 확인한다. 버전이 같으면 서버가 정의와 다음 실행 시각을 한 트랜잭션에서 함께 고치고 새 실행 시각을 돌려준다. 버전이 다르면 **409**로 거부한다.

주기와 다음 실행 시각은 **서로 묶인 값**이다. 두 값을 따로 고치면 「주기는 여섯 시간인데 다음 실행은 내일」과 같은 상태가 생긴다. 그래서 다음 실행 시각은 관리자가 입력하지 않고 **서버가 계산**한다.

7) 학습 데이터세트 생성 기능

2.6의 학습 데이터 관리 · 15 · 유형 **학습서버 비동기** · `POST /training/datasets` · 미구현

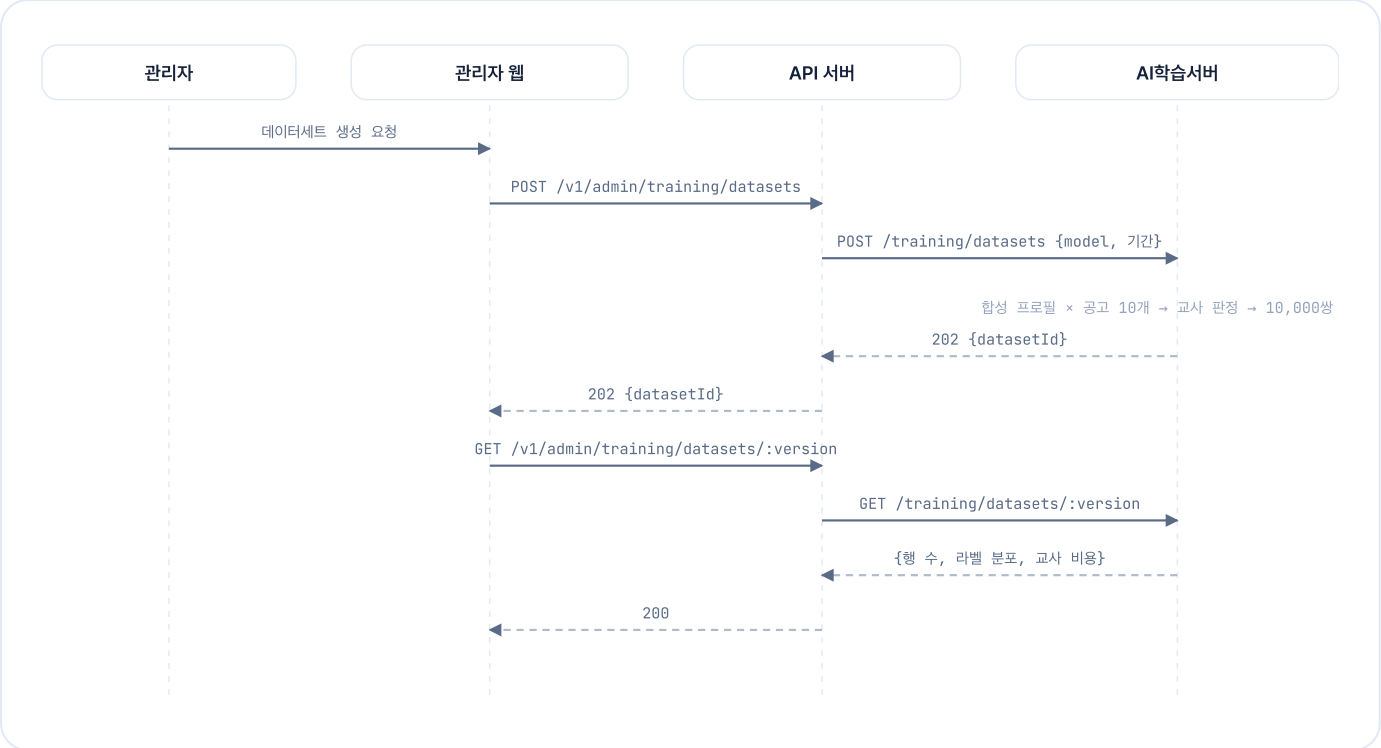


그림 6.3.7 학습 데이터세트 생성 기능의 시퀀스 다이어그램

생성 요청은 API 서버를 지나 AI학습서버로 전달된다. 학습서버는 합성 프로필마다 **후보 공고 10개**를 묶어 **교사 판정**을 요청하고, 루브릭으로 점수를 계산하여 데이터세트를 적재한다. 완료 후 행 수와 라벨 분포와 교사 호출 비용을 함께 조회할 수 있게 한다.

교사 호출에는 비용이 들고 되돌릴 수 없으므로, 학습을 시작하기 전에 데이터가 쓸 만한지 판단할 재료를 함께 돌려준다. **라벨 분포**가 한쪽 구간으로 치우친 데이터세트로 학습하면 지표는 높게 나오면서도 쓸 수 없는 모델이 된다.

8) 라벨 검수와 사람 평가 일치도 산출 기능

2.6의 학습 데이터 관리 · 16 · 유형 **학습서버 동기** · `PUT /training/datasets/:version/reviews` · 미구현

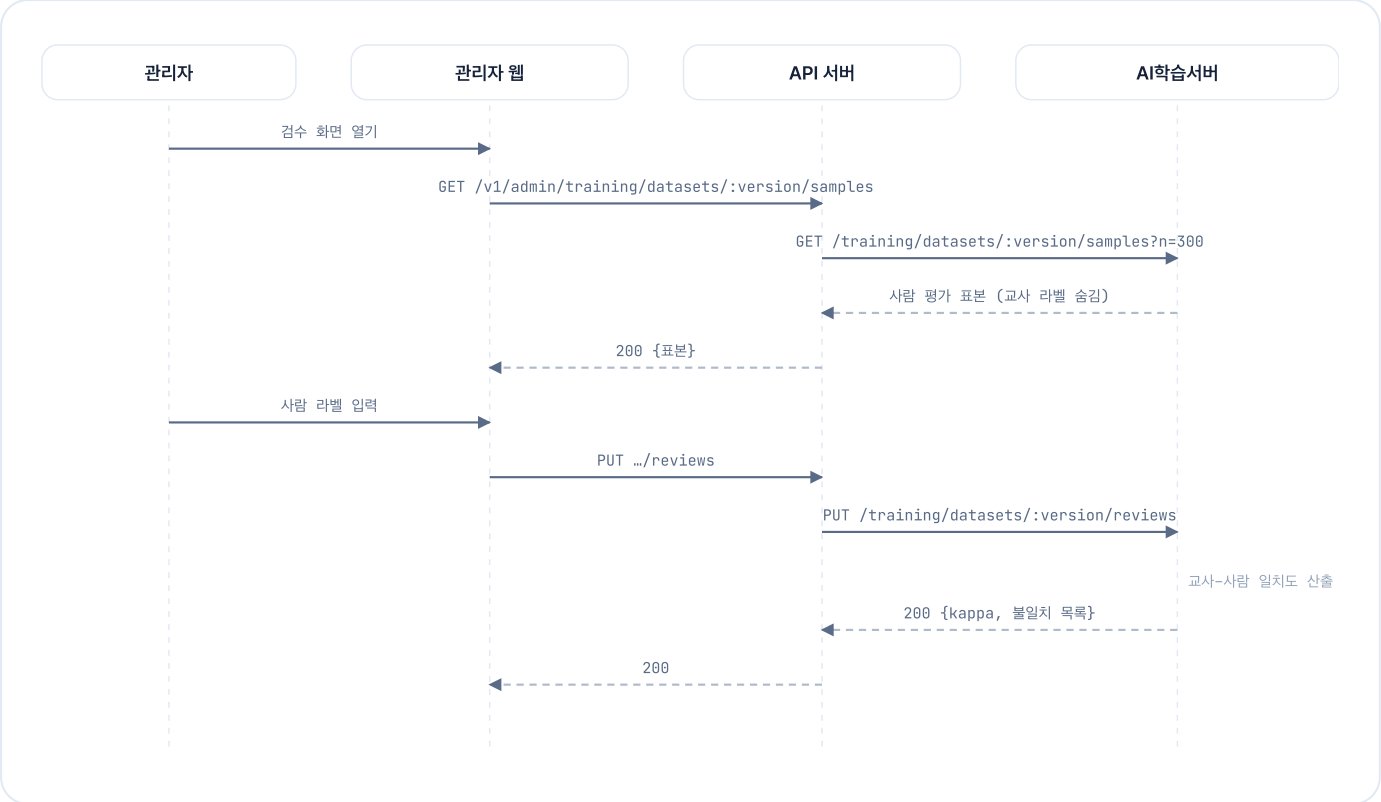


그림 6.3.8 라벨 검수와 사람 평가 일치도 산출 기능의 시퀀스 다이어그램

검수 화면은 사람 평가 표본을 **교사 라벨을 숨긴 상태**로 받는다. 사람이 라벨을 매겨 되돌려주면 학습서버가 교사와 사람의 **일치도**를 산출하고 불일치 목록을 함께 돌려준다.

교사 라벨을 먼저 보여 주면 사람이 그 판단에 끌려가고, 그렇게 얻은 일치도는 검증 값으로 사용할 수 없다. 그래서 표본을 내려줄 때 교사 라벨을 가린다. 이 일치도와 루브릭 가중치의 타당성은 라벨 구축 후의 **후속 검증**으로 확인한다.

9) 모델 학습 실행 · 배포 기능

2.6의 모델 운영 · 17 – 20 · 유형 **학습서버 비동기** · `POST /training/runs` · `/training/models/:version/deployments` · 미구현

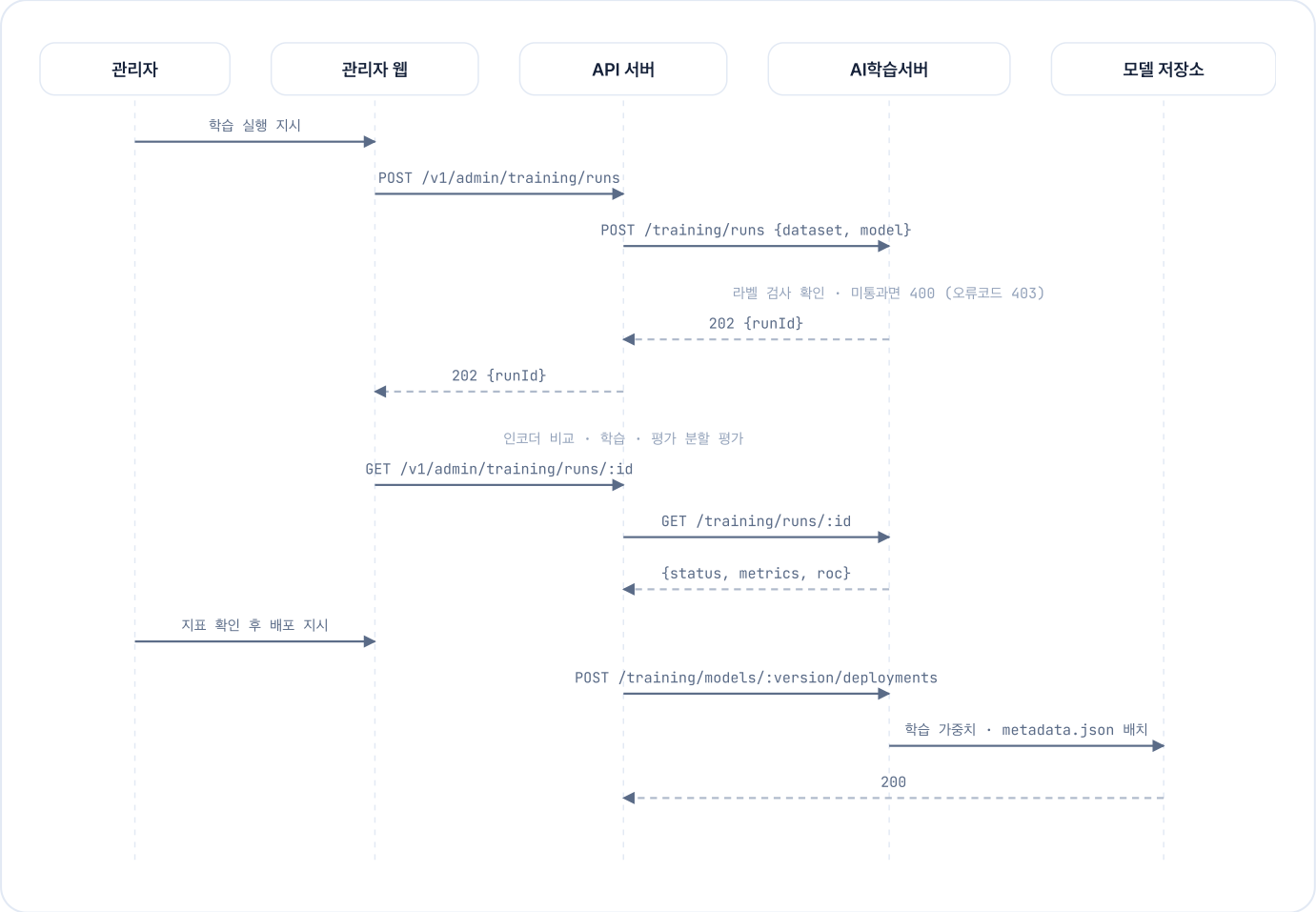


그림 6.3.9 모델 학습 실행 · 배포 기능의 시퀀스 다이어그램

학습 실행 지시는 먼저 데이터세트의 **라벨 검사**(교사 점수와 루브릭 계산의 일치)를 확인하고 통과하지 못하면 **전용 오류 코드**로 거부한다. 통과하면 202로 접수하고 학습서버가 인코더 비교와 학습과 평가 분할 평가를 수행하며, 관리자가 지표를 확인한 뒤 배포를 지시하면 학습 가중치와 메타데이터가 모델 저장소에 놓인다.

학습이 끝났다고 자동으로 배포하지 않는다. 지표가 목표치를 넘더라도 **오분류의 종류**가 나쁘면 배포하지 않아야 하기 때문이다. 적합한 공고나 기록을 하위로 떨어뜨리는 오류는 사용자의 기회를 버리는 결과가 되므로 **평균 지표보다 무겁게** 본다.

10) 모델 버전 조회 · 롤백 기능

2.6의 모델 운영 · 21 · 장애 관리 · 22 · 유형 **학습서버 동기** · `GET /training/models` · `POST /training/models/:version/deployments` · 미구현

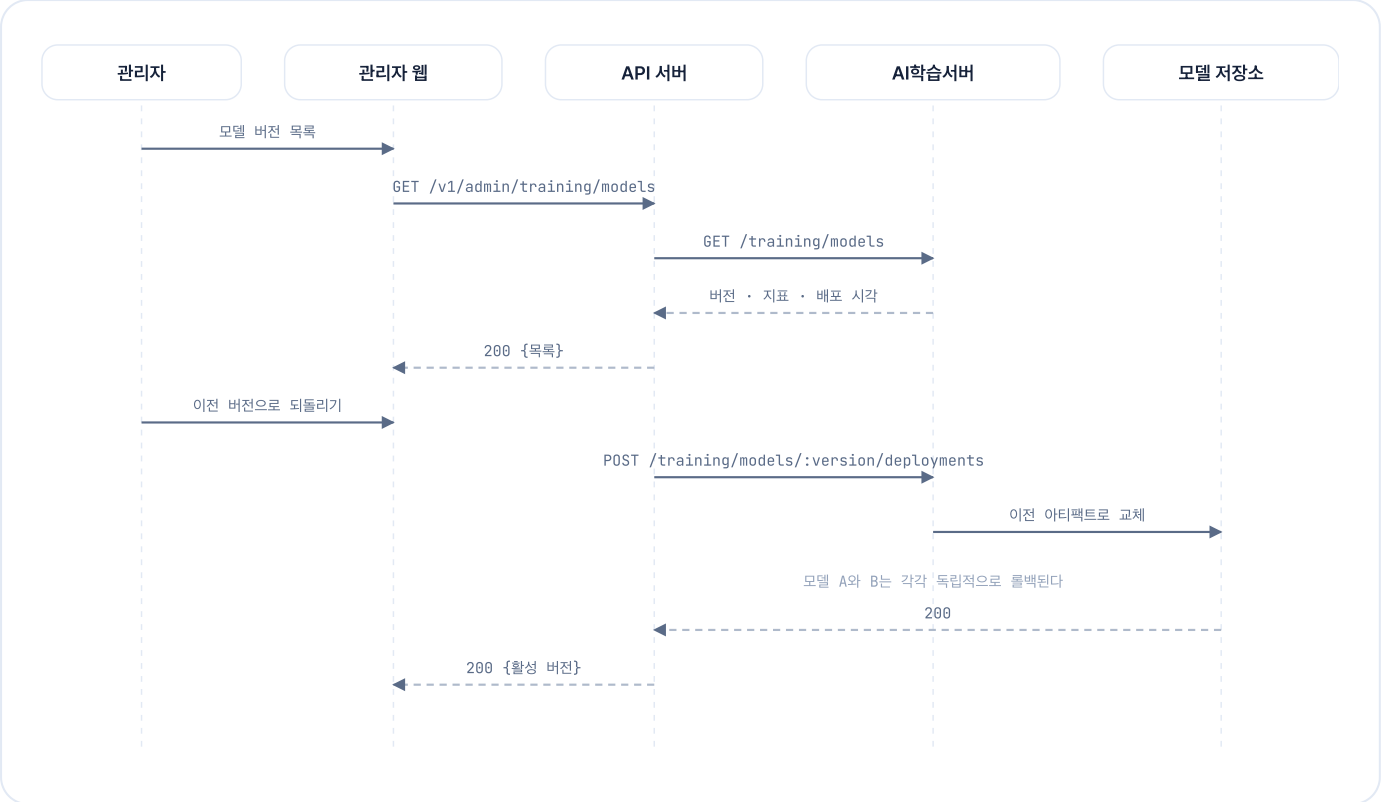


그림 6.3.10 모델 버전 조회 · 롤백 기능의 시퀀스 다이어그램

버전 목록은 지표와 배포 시각을 함께 보인다. 되돌리기는 **새 배포와 같은 경로**를 이전 버전으로 호출하는 것이며, 학습서버가 모델 저장소의 산출물을 교체하고 **활성 버전**을 돌려준다.

되돌리기 전용 경로는 두지 않는다. 경로를 분리하면 배포와 롤백의 동작이 서로 어긋날 수 있다. 모델 A와 B는 각각 **독립적으로 배포**되고 독립적으로 되돌려진다.

6.4 AI학습서버 기능설계

학습서버의 추론 기능은 **공고 추천**과 **커리어 기록 추천**으로 나눈다.



그림 6.4.1 두 추천 모델의 추론 흐름

경로	입력	출력	현재 기준
`POST /inference/match`	프로필과 공고 목록	공고별 점수 · 순위 · 근거 기록	`match-score.ts`
`POST /inference/record-rank`	공고와 기록 목록	기록별 점수와 추천 선택 조합	`materials/ranking.ts`

요건별 충족 값은 추천 순위와 별도로 관리한다.

다이어그램 귀속 회계

대상	상세기능	대표 그림	나머지의 처리
웹 사용자 (2.5)	151	17	그림 6.2.1 – 6.2.17 · 8개 상호작용 유형과 도메인별 귀속 표에 전부 대응시켰다
웹 관리자 (2.6)	24	10	그림 6.3.1 – 6.3.10 · 7개 유형과 주요기능별 귀속 표에 전부 대응시켰다
AI학습서버 (2.3)	35	1	관리자가 촉발하는 학습 · 검수 · 배포는 그림 6.3.7 – 6.3.10, 서비스가 촉발하는 추론은 그림 6.4.1
합계	**210**	**28**	귀속되지 않은 기능은 없다

상세기능 210개 가운데 주고받는 흐름이 서로 다른 28개를 대표로 그렸다. 같은 유형은 경로와 대상 컬렉션만 달라지므로 그림을 다시 그리지 않고 유형에 귀속시켰다.

07 API 설계

HTTP 경로 정의 163개

7.1 서버/클라이언트 구조

본 설계는 사용자의 웹 클라이언트와 그 클라이언트가 이용할 서버의 데이터베이스를 연동하여 개발하기 위해, 각 기능별로 API 계층 · 검증 계층 · 인증 계층의 구축을 설계한다.

클라이언트와 서버는 요청과 응답의 스키마를 같은 코드로 공유한다. `packages/contracts`에 zod 스키마를 두고 서버는 검증에, 클라이언트는 타입에 사용한다. 스키마가 바뀌면 두 쪽이 함께 바뀌므로 계약이 어긋나지 않는다.

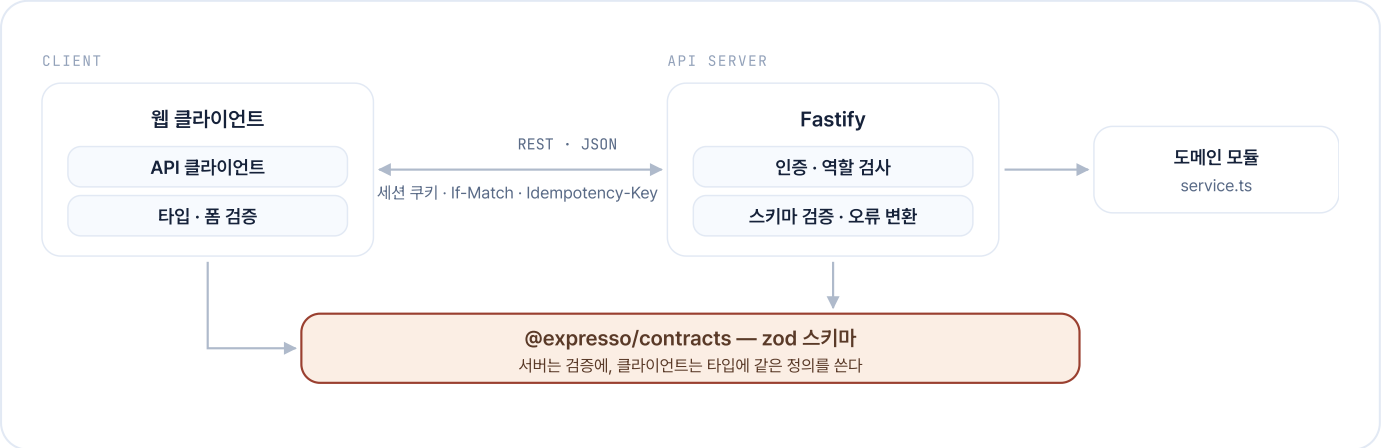


그림 7.1 서버 인터페이스 구조

그림 7.1을 보면 클라이언트와 API 서버가 `@expresso/contracts`를 함께 참조한다. 스키마를 한 곳에만 두었으므로 요청 형태가 바뀌면 서버의 검증과 클라이언트의 타입이 함께 바뀌며, 한쪽만 고쳐 계약이 어긋나는 일이 **컴파일 단계**에서 걸린다. API 서버는 **인증과 역할**을 먼저 확인하고 스키마로 요청을 검증한 뒤에야 도메인 모듈을 호출한다. 도메인 모듈은 HTTP를 알지 못하므로 같은 규칙을 HTTP 없이 시험할 수 있다.

항목	값
기본 경로	`/v1`
형식	`application/json`
인증	`ex_session` httpOnly 쿠키
요청 ID	응답 헤더 `x-request-id`
낙관적 잠금	`If-Match` 요청 헤더 · `ETag` 응답 헤더
멱등성	`Idempotency-Key` 헤더 (생성 계열)

항목	값
타임아웃	30초

■ 오류 응답 형식

모든 오류가 같은 형태로 응답한다

```
{
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed",
    "requestId": "req_0f8a2c91-...",
    "details": {}
  }
}
```

`message`는 상태 코드마다 고정된 문구이다. 내부 사정을 문장으로 흘리지 않기 위해서이다. 자세한 원인은 **서버 로그에만** 남으며, 클라이언트는 `requestId`로 그 로그를 찾는다.

7.2 서비스 REST API 정의

2026-09-28 시점의 Fastify HTTP 경로 정의는 **163개**다. 상태 확인 2개를 제외한 `/v1` 경로는 **161개**다. 이 중 커리어 편집기 경로 10개는 기능 플래그가 켜질 때 등록된다. 기본 설정에서 등록되는 경로는 **153개**다. 아래 표는 각 모듈의 경로 등록 코드에서 추출했고, 반복 등록되는 인터뷰 상태 경로 2개와 AI 제안 상태 경로 3개를 각각 센다. 커리어 문서 편집 경로는 `CAREER_EDITOR_V2_ENABLED=true`에서 등록된다. WebSocket 세션 경로는 REST 목록과 별도로 다룬다.

■ GET (62개)

METHOD	URI	모듈
GET	`/health/live`	system
GET	`/health/ready`	system
GET	`/v1/account/export`	account-lifecycle
GET	`/v1/analytics/metrics`	analytics
GET	`/v1/brew-jobs/:id`	brew-jobs
GET	`/v1/brews/:brewId/company-research`	company-research

METHOD	URI	모듈
GET	`/v1/brews/:id`	materials
GET	`/v1/brews/:id/materials`	materials
GET	`/v1/career/categories`	career
GET	`/v1/career/categories/:categoryId/view-configurations`	career
GET	`/v1/career/categories/:categoryId/views`	career
GET	`/v1/career/profile`	career
GET	`/v1/career/records`	career
GET	`/v1/career/records/:recordId`	career
GET	`/v1/career/records/:recordId/ai-proposals/:proposalId`	career-editor
GET	`/v1/career/records/:recordId/delete-impact`	career
GET	`/v1/career/records/:recordId/document`	career-editor
GET	`/v1/career/records/:recordId/document/revisions`	career-editor
GET	`/v1/career/records/:recordId/links`	career
GET	`/v1/career/records/:recordId/relations`	career
GET	`/v1/career/skills`	career
GET	`/v1/career/skills/:skillId/evidence`	career
GET	`/v1/career/view-configurations/:viewId/query`	career
GET	`/v1/companies/:id/logo`	jobs
GET	`/v1/consents`	consent

METHOD	URI	모듈
GET	`/v1/deployments/:id/analytics/derived`	analytics
GET	`/v1/deployments/:id/analytics/insight`	analytics
GET	`/v1/design-system-revisions/:id`	design-systems
GET	`/v1/design-systems`	design-systems
GET	`/v1/design-systems/:id`	design-systems
GET	`/v1/entitlements/:capability`	entitlements
GET	`/v1/export-jobs/:id`	publishing
GET	`/v1/generation-jobs/:id`	generation
GET	`/v1/generation-jobs/:id/page-stream`	generation
GET	`/v1/home`	engagement
GET	`/v1/interview-sessions/:id`	interview
GET	`/v1/job-analyses/:id`	job-analysis
GET	`/v1/job-sources`	jobs
GET	`/v1/jobs/postings`	jobs
GET	`/v1/jobs/postings/:id`	jobs
GET	`/v1/jobs/recent-searches`	jobs
GET	`/v1/jobs/saved-searches`	jobs
GET	`/v1/me`	identity
GET	`/v1/media`	media

METHOD	URI	모듈
GET	`/v1/media/:id`	media
GET	`/v1/notification-preferences`	engagement
GET	`/v1/notifications`	engagement
GET	`/v1/portfolios`	portfolios
GET	`/v1/portfolios/:id`	portfolios
GET	`/v1/portfolios/:id/analytics/dashboard`	analytics
GET	`/v1/portfolios/:id/deployments`	publishing
GET	`/v1/portfolios/:id/layouts`	layout
GET	`/v1/portfolios/:id/page`	page
GET	`/v1/portfolios/:id/page/document`	page
GET	`/v1/portfolios/:id/page/history`	page
GET	`/v1/portfolios/:id/page/stream`	page
GET	`/v1/portfolios/:id/revisions`	portfolios
GET	`/v1/public/assets/:id`	publishing
GET	`/v1/public/portfolios/:slug`	publishing
GET	`/v1/recipes/:id`	recipe
GET	`/v1/recipes/:id/template-previews`	templates
GET	`/v1/search`	engagement

POST (75개)

METHOD	URI	모듈
POST	`/v1/account/deletion`	account-lifecycle
POST	`/v1/account/deletion/cancel`	account-lifecycle
POST	`/v1/analytics/events`	analytics
POST	`/v1/assets/:id/signed-url`	publishing
POST	`/v1/auth/google`	identity
POST	`/v1/auth/google/link`	identity
POST	`/v1/auth/login`	identity
POST	`/v1/auth/logout`	identity
POST	`/v1/auth/signup`	identity
POST	`/v1/brews`	materials
POST	`/v1/brews/:id/design-selection`	design-systems
POST	`/v1/brews/:id/interview-sessions`	interview
POST	`/v1/brews/:id/recipes`	recipe
POST	`/v1/brews/free`	materials
POST	`/v1/career/categories`	career
POST	`/v1/career/categories/:categoryId/property-schema/:propertyId/restore`	career
POST	`/v1/career/categories/:categoryId/property-schema/apply`	career
POST	`/v1/career/categories/:categoryId/property-schema/preview`	career
POST	`/v1/career/categories/:categoryId/view-configurations`	career

METHOD	URI	모듈
POST	`/v1/career/categories/:categoryId/view-configurations/reorder`	career
POST	`/v1/career/categories/:categoryId/views`	career
POST	`/v1/career/formulas/preview`	career
POST	`/v1/career/records`	career
POST	`/v1/career/records/:recordId/ai-proposals`	career-editor
POST	`/v1/career/records/:recordId/ai-proposals/:proposalId/apply`	career-editor
POST	`/v1/career/records/:recordId/ai-proposals/:proposalId/cancel`	career-editor
POST	`/v1/career/records/:recordId/ai-proposals/:proposalId/reject`	career-editor
POST	`/v1/career/records/:recordId/ai-proposals/:proposalId/undo`	career-editor
POST	`/v1/career/records/:recordId/document/revisions/:revisionId/restore`	career-editor
POST	`/v1/career/records/:recordId/document/updates`	career-editor
POST	`/v1/career/records/:recordId/links`	career
POST	`/v1/career/records/:recordId/move`	career
POST	`/v1/career/records/:recordId/move/preview`	career
POST	`/v1/career/records/:recordId/restore`	career
POST	`/v1/career/rollups/preview`	career
POST	`/v1/career/skills/recompute`	career
POST	`/v1/career/view-configurations/:viewId/duplicate`	career
POST	`/v1/consents`	consent

METHOD	URI	모듈
POST	`/v1/deployments/:id/analytics/aggregate`	analytics
POST	`/v1/generation-jobs`	generation
POST	`/v1/interview-sessions/:id/pause`	interview
POST	`/v1/interview-sessions/:id/questions/:questionId/replace`	interview
POST	`/v1/interview-sessions/:id/questions/:questionId/skip`	interview
POST	`/v1/interview-sessions/:id/resume`	interview
POST	`/v1/job-analyses/:id/reanalyze`	job-analysis
POST	`/v1/job-sources`	jobs
POST	`/v1/job-sources/:id/runs`	jobs
POST	`/v1/job-sources/runs`	jobs
POST	`/v1/jobs/demand-summary`	jobs
POST	`/v1/jobs/postings/:id/analyses`	jobs
POST	`/v1/jobs/postings/:id/match`	jobs
POST	`/v1/jobs/saved-searches`	jobs
POST	`/v1/jobs/search/interpret`	jobs
POST	`/v1/jobs/submissions`	jobs
POST	`/v1/jobs/url-imports`	jobs
POST	`/v1/media`	media
POST	`/v1/portfolio-edit-proposals/:id/apply`	portfolio-editing

METHOD	URI	모듈
POST	`/v1/portfolio-revisions/:id/revert`	portfolio-editing
POST	`/v1/portfolios/:id/analytics/aggregate`	analytics
POST	`/v1/portfolios/:id/analytics/insight`	analytics
POST	`/v1/portfolios/:id/blocks/:blockId/duplicate`	portfolio-editing
POST	`/v1/portfolios/:id/blocks/:blockId/edit-preview`	portfolio-editing
POST	`/v1/portfolios/:id/dashboard-layout`	analytics
POST	`/v1/portfolios/:id/dashboard-views`	analytics
POST	`/v1/portfolios/:id/deployments`	publishing
POST	`/v1/portfolios/:id/deployments/:deploymentId/rollback`	publishing
POST	`/v1/portfolios/:id/exports`	publishing
POST	`/v1/portfolios/:id/layouts/:layoutId/select`	layout
POST	`/v1/portfolios/:id/layouts/remix`	layout
POST	`/v1/portfolios/:id/page`	page
POST	`/v1/portfolios/:id/restore`	portfolio-editing
POST	`/v1/portfolios/:id/resume-assets`	publishing
POST	`/v1/portfolios/:id/sections/:sectionId/media-blocks`	media
POST	`/v1/portfolios/:id/widgets`	analytics
POST	`/v1/recipes/:id/revisions/:revisionId/restore-item`	recipe

PUT (7개)

METHOD	URI	모듈
PUT	`/v1/brews/:brewId/company-research`	company-research
PUT	`/v1/brews/:id/materials`	materials
PUT	`/v1/career/profile`	career
PUT	`/v1/career/records/:recordId/relations`	career
PUT	`/v1/interview-sessions/:id/answers/:questionId`	interview
PUT	`/v1/jobs/postings/:id/interest`	jobs
PUT	`/v1/notification-preferences/:kind`	engagement

PATCH (10개)

METHOD	URI	모듈
PATCH	`/v1/brews/:id`	materials
PATCH	`/v1/career/categories/:categoryId/property-schema`	career
PATCH	`/v1/career/records/:recordId`	career
PATCH	`/v1/career/view-configurations/:viewId`	career
PATCH	`/v1/portfolios/:id/sections/:sectionId`	portfolio-editing
PATCH	`/v1/portfolios/:id/sections/:sectionId/blocks/order`	portfolio-editing
PATCH	`/v1/portfolios/:id/sections/order`	portfolio-editing
PATCH	`/v1/portfolios/:id/widgets/order`	analytics
PATCH	`/v1/recipes/:id`	recipe

METHOD	URI	모듈
PATCH	`/v1/widgets/:id`	analytics

DELETE (9개)

METHOD	URI	모듈
DELETE	`/v1/career/records/:recordId`	career
DELETE	`/v1/career/view-configurations/:viewId`	career
DELETE	`/v1/consents/:scope`	consent
DELETE	`/v1/identity/sessions/:sessionId`	identity
DELETE	`/v1/jobs/recent-searches/:id`	jobs
DELETE	`/v1/portfolios/:id/blocks/:blockId`	portfolio-editing
DELETE	`/v1/portfolios/:id/dashboard-layout`	analytics
DELETE	`/v1/portfolios/:id/publication`	publishing
DELETE	`/v1/widgets/:id`	analytics

URI 명사형 규칙의 위반

현재 형상에서 동사를 경로에 사용한 대표 경로와, 그에 대응하는 명사형 경로를 아래 표에 적는다.

현재 URI	수정안
`POST /v1/career/skills/recompute`	`POST /v1/career/skill-recomputations`
`POST /v1/jobs/search/interpret`	`POST /v1/jobs/search-interpretations`
`POST /v1/job-analyses/:id/reanalyze`	`POST /v1/job-analyses/:id/reruns`
`POST /v1/portfolios/:id/layouts/remix`	`POST /v1/portfolios/:id/layout-remixes`

현재 URI	수정안
`POST /v1/…/layouts/:layoutId/select`	`PUT /v1/portfolios/:id/selected-layout`
`POST /v1/portfolio-edit-proposals/:id/apply`	`POST /v1/portfolio-edit-proposals/:id/applications`
`POST /v1/portfolio-revisions/:id/revert`	`POST /v1/portfolio-revisions/:id/reversions`
`POST /v1/portfolios/:id/restore`	`POST /v1/portfolios/:id/restorations`
`POST /v1/…/deployments/:id/rollback`	`POST /v1/…/deployments/:id/rollbacks`
`POST /v1/…/questions/:id/replace`	`PUT /v1/interview-sessions/:id/questions/:id`
`POST /v1/…/questions/:id/skip`	`PUT /v1/…/questions/:id/skipped`
`POST /v1/interview-sessions/:id/pause | resume`	`PUT /v1/interview-sessions/:id/state`
`POST /v1/recipes/:id/revisions/:id/restore-item`	`POST /v1/recipes/:id/item-restorations`
`POST /v1/jobs/postings/:id/match`	`POST /v1/jobs/postings/:id/match-scores`
`POST /v1/jobs/demand-summary`	`GET /v1/jobs/demand-summaries`
`POST /v1/…/analytics/aggregate`	`POST /v1/…/analytics/aggregations`
`POST /v1/blocks/:id/duplicate`	`POST /v1/…/blocks/:id/duplications`
`POST /v1/blocks/:blockId/edit-preview`	`POST /v1/…/blocks/:id/edit-previews`

7.3 서비스 REST API 설계

7.2에서 정의한 모든 API를 상세하게 설계한다. 각 API마다 요청과 응답의 방향, 파라메터 목록, 실제 전송 내용을 적는다. 다만 163개 가운데 상당수는 요청과 응답의 **형태가 서로 같다**. 목록 조회는 어느 경로든 쿼리로 거르고 커서로 이어 받으며, 생성은 어느 경로든 `Idempotency-Key`를 요구한다.

따라서 형태가 서로 다른 9개(1 – 9)를 먼저 설계하고, 2.5 · 2.6의 주요기능 가운데 대표 API가 없는 기능마다 1개씩 17개(10 – 26)를 더해 **26개를 설계**한다. 나머지는 아래 유형에 귀속시킨다. 유형이 같으면 경로와 필드 이름만 달라지며, 그 값은 `packages/contracts/src/`의 zod 스키마에 있다. 스키마가 계약의 원본이므로 본 절과 어긋나면 스키마를 따른다.

■ 요청 · 응답 유형

유형	요청과 응답의 특징	대표	해당 범위
목록 조회	쿼리로 거르고 커서로 이어 받는다. 200	7.3.1 · 7.3.13 · 7.3.14 · 7.3.17	GET 62건 중 목록
단건 조회	경로 파라메터로 하나를 읽는다. 200	7.3.1 · 7.3.16 · 7.3.24 · 7.3.25 · 7.3.26	GET 62건 중 단건
생성	`Idempotency-Key` 로 중복 생성을 막는다. 201	7.3.2 · 7.3.10 · 7.3.11 · 7.3.15 · 7.3.18 · 7.3.21 · 7.3.22 · 7.3.23	POST 75건 중 생성
조건부 수정	`If-Match` 로 버전을 확인한다. 200 또는 409	7.3.3 · 7.3.12	PATCH 10 · PUT 7
삭제	영향 범위를 먼저 알린 뒤 지운다. 204	7.3.4	DELETE 9
비동기 접수	작업 식별자를 담아 202를 돌려준다	7.3.5 · 7.3.19 · 7.3.20	POST 75건 중 비동기
일치도 산출	공고 요건과 사용자 기록의 축별 충족 내역과 점수를 돌려준다. 200	7.3.6	일치도 경로
스트림	`text/event-stream` 으로 진행을 밀어 준다	7.3.7	1건
공개 조회	인증이 없다. 배포 스냅샷만 노출한다	7.3.8	4건
서명 URL	만료가 있는 임시 주소를 발급한다	7.3.9	1건

열 유형이 163개 HTTP 경로의 형태를 모두 포함한다. 메서드별 분포는 GET 62 · POST 75 · PATCH 10 · DELETE 9 · PUT 7이다.

1) 기록 목록 조회

유형 **목록 조회** · `GET /v1/career/records`

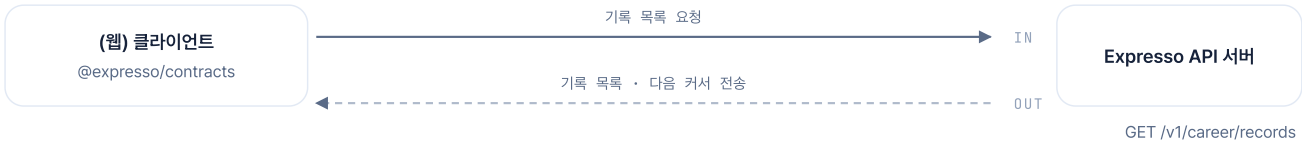


그림 7.3.1 기록 목록 조회의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`categoryId`	0	—	uuid	카테고리로 거른다. 없으면 전체
`status`	0	—	String	`draft` · `organized` · `verified`
`origin`	0	—	String	`manual` · `ai` · `interview` · `import`
`q`	0	—	String	제목에서 찾는다. 1 – 200자
`sort`	0	—	String	`updated_desc` · `updated_asc` · `title_asc` · `period_desc` · `period_asc` , 기본값 `updated_desc`
`cursor`	0	—	String	이어 받을 위치
`limit`	0	—	int	1 – 100, 기본값 50
`data`	—	0	Array	기록 목록
`page`	—	0	Object	`{nextCursor, hasNextPage}` . 다음 쪽이 없으면 `nextCursor` 는 null
`summary`	—	0	Object	`{total, draft, organized, verified, empty}` . 조건에 맞는 기록의 상태별 개수

표 7.3.1 기록 목록 조회의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/career/records?categoryId=7a1c9e42-2b3d-4f5e-8a6b-
1c2d3e4f5a6b&status=organized&sort=updated_desc&limit=50
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": [
    {
      "id": "0b6f2a3e-5c1d-4e8a-9f27-3d4c5b6a7e81",
      "categoryId": "7a1c9e42-2b3d-4f5e-8a6b-1c2d3e4f5a6b",
      "categoryKey": "project",
      "title": "사내 검색 개편",
      "status": "organized",
      "origin": "manual",
      "properties": { "role": "백엔드 리드" },
      "bodyMd": "검색 지연을 320ms에서 90ms로 줄였다.",
      "isEmpty": false,
      "periodFrom": "2025-03-01",
      "periodTo": "2025-09-30",
      "linkCount": 2,
      "usedInCount": 3,
      "version": 7,
      "createdAt": "2026-08-01T01:20:00.000Z",
      "updatedAt": "2026-08-11T04:12:33.000Z"
    }
  ],
  "page": { "nextCursor": null, "hasNextPage": false },
  "summary": { "total": 1, "draft": 0, "organized": 1, "verified": 0, "empty": 0 }
}
```

목록은 커서 방식으로 이어 받으며 쪽 번호는 사용하지 않는다. 기록이 추가되거나 지워져도 이미 본 항목이 다시 나오거나 건너뛰지 않기 때문이다. `usedInCount`는 그 기록을 인용한 포트폴리오 수이며, 화면의 「사용처 N곳」에 그대로 쓰인다.

2) 기록 생성

유형 **생성** · `POST /v1/career/records`



그림 7.3.2 기록 생성의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`Idempotency-Key`	O	—	Header	필수. 같은 키로 다시 보내면 이미 만든 기록을 돌려준다
`categoryId`	O	—	uuid	필수

속성	IN	OUT	TYPE	DESCRIPTION
`title`	O	–	String	최대 300자, 기본값 빈 문자열
`properties`	O	–	Object	카테고리 속성 스키마를 따른다
`bodyMd`	O	–	String	최대 200,000자
`ETag`	–	O	Header	생성된 기록의 버전. 형식은 `"v1"`
`data`	–	O	Object	생성된 기록. 처음 상태는 `draft`
`resource`	–	O	Object	`{version, updatedAt}`

표 7.3.2 기록 생성의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
Idempotency-Key: 4b1f9c2e-7a30-4d55-9c11-8e2f0a6d34bb
{
  "categoryId": "7a1c9e42-2b3d-4f5e-8a6b-1c2d3e4f5a6b",
  "title": "사내 검색 개편",
  "properties": { "period": { "type": "date", "value": { "start": "2025-03-01", "end": "2025-09-30",
"timezone": null } } },
  "bodyMd": "검색 지연을 320ms에서 90ms로 줄였다."
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
HTTP 201 Created
ETag: "v1"
{
  "data": {
    "id": "0b6f2a3e-5c1d-4e8a-9f27-3d4c5b6a7e81",
    "categoryId": "7a1c9e42-2b3d-4f5e-8a6b-1c2d3e4f5a6b",
    "title": "사내 검색 개편",
    "status": "draft",
    "origin": "manual",
    "properties": { "period": { "type": "date", "value": { "start": "2025-03-01", "end": "2025-09-30",
"timezone": null } } },
    "bodyMd": "검색 지연을 320ms에서 90ms로 줄였다.",
    "version": 1,
    "createdAt": "2026-08-14T02:10:11.000Z",
    "updatedAt": "2026-08-14T02:10:11.000Z"
  },
```

```
  "resource": { "version": 1, "updatedAt": "2026-08-14T02:10:11.000Z" }
}
```

`Idempotency-Key`는 **16자 이상 128자 이하**여야 하고 안전한 문자만 허용한다. 네트워크가 끊겨 클라이언트가 재시도하더라도 기록이 두 벌 생기지 않게 하는 장치이며, 서버는 새로 만들지 않고 첫 요청이 만든 기록을 **200**으로 돌려준다. 같은 키에 다른 본문을 보내면 409이다.

3) 기록 수정

유형 **조건부 수정** · `PATCH /v1/career/records/:recordId`

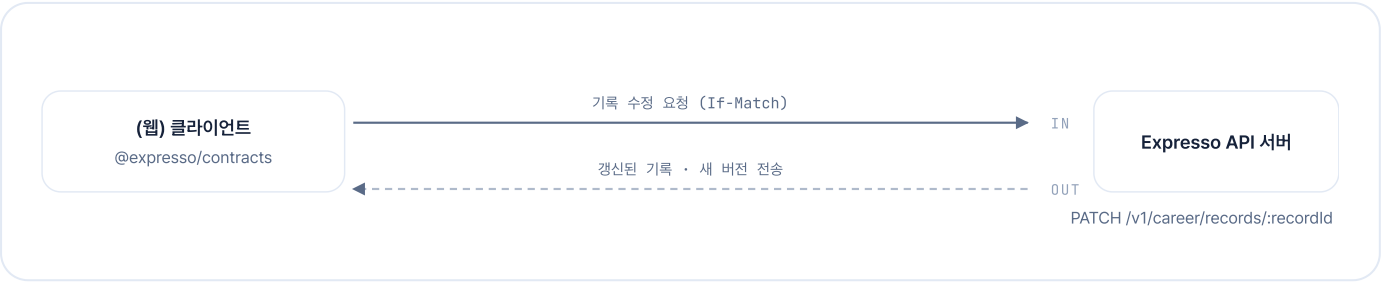


그림 7.3.3 기록 수정의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`recordId`	O	—	uuid	경로 파라메터
`If-Match`	O	—	Header	현재 버전의 ETag(`"v7"` 형식). 없으면 400, 어긋나면 412
`title`	O	—	String	넷 중 하나 이상은 있어야 한다
`status`	O	—	String	`draft` · `organized` · `verified`
`properties`	O	—	Object	—
`bodyMd`	O	—	String	—
`ETag`	—	O	Header	올라간 버전. 다음 수정의 `If-Match`에 쓴다
`data`	—	O	Object	갱신된 기록
`resource`	—	O	Object	올라간 버전

표 7.3.3 기록 수정의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
If-Match: "v7"
{
  "bodyMd": "검색 지연을 320ms에서 90ms로 줄였다. 색인 크기는 1.4배가 되었다."
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
ETag: "v8"
{
  "data": {
    "id": "0b6f2a3e-5c1d-4e8a-9f27-3d4c5b6a7e81",
    "categoryId": "7a1c9e42-2b3d-4f5e-8a6b-1c2d3e4f5a6b",
    "title": "사내 검색 개편",
    "status": "organized",
    "origin": "manual",
    "properties": { "role": "백엔드 리드" },
    "bodyMd": "검색 지연을 320ms에서 90ms로 줄였다. 색인 크기는 1.4배가 되었다.",
    "version": 8,
    "createdAt": "2026-08-01T01:20:00.000Z",
    "updatedAt": "2026-08-14T02:31:40.000Z"
  },
  "resource": { "version": 8, "updatedAt": "2026-08-14T02:31:40.000Z" }
}

버전이 어긋난 경우
HTTP 412
{ "error": { "code": "PRECONDITION_FAILED", "message": "Resource version is stale",
             "requestId": "req_0f8a2c91-3b4d-4e5f-a6b7-c8d9e0f1a2b3" } }
```

네 필드 가운데 하나도 없으면 **400**이다. 빈 수정 요청이 버전만 올리는 것을 막기 위해서이다. 412를 받은 클라이언트는 최신 상태를 다시 읽어 사용자에게 무엇이 달라졌는지 보여 준 뒤 재시도한다.

4) 기록 삭제와 영향 범위

유형 **삭제** · `GET /v1/career/records/:recordId/delete-impact · DELETE`



그림 7.3.4 기록 삭제와 영향 범위의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`recordId`	O	O	uuid	경로 파라메터. 응답에도 담긴다
`data`	—	O	Object	영향 범위. 아래 필드를 담는다. `DELETE` 도 같은 모양을 돌려준다
`portfolioCount`	—	O	int	이 기록을 인용한 포트폴리오 수
`blockCount`	—	O	int	영향을 받는 블록 수
`deletedAt`	—	O	Timestamp	삭제 시각. 영향 범위 조회에서는 조회 시각
`purgeAfter`	—	O	Timestamp	삭제 시각으로부터 30일. 이후에는 복구할 수 없다

표 7.3.4 기록 삭제와 영향 범위의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/career/records/0b6f2a3e-5c1d-4e8a-9f27-3d4c5b6a7e81/delete-impact
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "recordId": "0b6f2a3e-5c1d-4e8a-9f27-3d4c5b6a7e81",
    "portfolioCount": 3,
    "blockCount": 11,
    "deletedAt": "2026-08-14T02:40:00.000Z",
    "purgeAfter": "2026-09-13T02:40:00.000Z"
  }
}

삭제 실행
DELETE /v1/career/records/0b6f2a3e-5c1d-4e8a-9f27-3d4c5b6a7e81  →  HTTP 200
{ "data": { "recordId": "0b6f2a3e-5c1d-4e8a-9f27-3d4c5b6a7e81", "portfolioCount": 3, "blockCount": 11,
            "deletedAt": "2026-08-14T02:41:05.000Z", "purgeAfter": "2026-09-13T02:41:05.000Z" } }
```

삭제를 바로 실행하지 않고 영향 범위를 먼저 알려 준다. 이미 배포된 포트폴리오가 그 기록을 인용하고 있을 수 있기 때문이다. 삭제 후에도 `purgeAfter` 까지는 `POST …/restore` 로 **되돌릴 수 있다**.

5) 공고 분석 접수

유형 **비동기 접수** · `POST /v1/jobs/postings/:id/analyses`

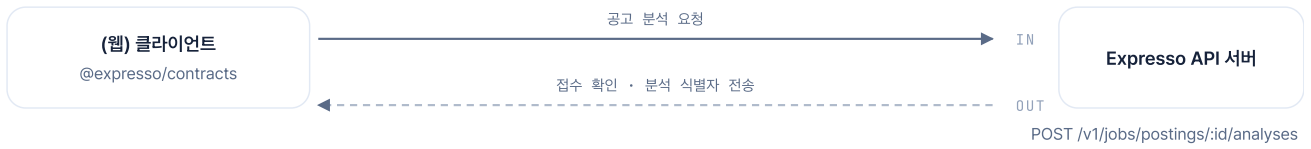


그림 7.3.5 공고 분석 접수의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`id`	O	—	uuid	공고 식별자, 경로 파라메터
`data`	—	O	Object	접수 결과. 아래 필드와 `jobPostingId` 를 담는다
`jobAnalysisId`	—	O	uuid	진행 조회에 사용한다
`status`	—	O	String	`queued` · `running` · `done` · `failed` . 새로 접수하면 `queued`
`reused`	—	O	Boolean	이 공고의 분석이 이미 있어 그것을 돌려주면 true

표 7.3.5 공고 분석 접수의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
POST /v1/jobs/postings/3e9d1f7a-6b2c-4d8e-a1f3-5b7c9d0e2f41/analyses
```

실제 전송내용 (OUT)

```
전송방향 : OUT
HTTP 201 Created
{
  "data": {
    "jobPostingId": "3e9d1f7a-6b2c-4d8e-a1f3-5b7c9d0e2f41",
    "jobAnalysisId": "8c4b2a19-7d3e-4f6a-b5c8-9e0d1f2a3b4c",
    "status": "queued",
    "reused": false
  }
}
```

분석에는 수십 초가 걸리므로 결과를 기다리지 않고 접수 응답을 반환한다. 새로 접수하면 201, 이 공고의 분석이 이미 있으면 그 분석을 200으로 돌려준다. 클라이언트는 `jobAnalysisId` 로 `GET /v1/job-analyses/:id` 를 **주기적으로 조회**하여 완료를 확인한다.

6) 공고 일치도 산출

유형 **일치도 산출** · `POST /v1/jobs/postings/:id/match`

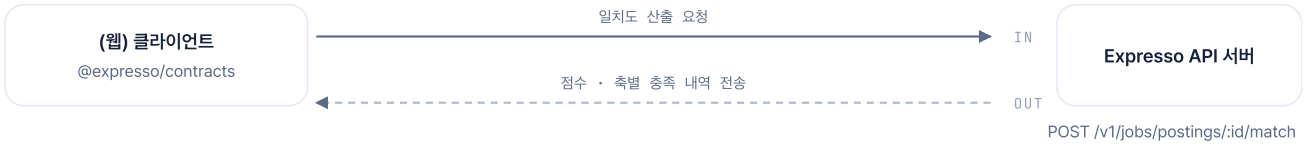


그림 7.3.6 공고 일치도 산출의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`id`	0	–	uuid	공고 식별자, 경로 파라메터
`data`	–	0	Object	산출 결과. 아래 필드와 `jobPostingId` 를 담는다
`total`	–	0	int	0 – 100. `round(100 × covered ÷ required)`
`required`	–	0	int	공고가 명시한 요건 수. 네 축의 합이며 1 이상
`covered`	–	0	int	기록에서 근거를 찾은 요건 수. 네 축의 합
`axes`	–	0	Object	`technology` · `impact` · `role` · `conditions` . 축마다 `{required, covered, matched, missing}`
`reason`	–	0	String	화면에 그대로 보이는 한 문장
`nextAction`	–	0	String	다음에 할 일을 적은 한 문장
`computedAt`	–	0	Timestamp	산출 시각

표 7.3.6 공고 일치도 산출의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
POST /v1/jobs/postings/3e9d1f7a-6b2c-4d8e-a1f3-5b7c9d0e2f41/match
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "jobPostingId": "3e9d1f7a-6b2c-4d8e-a1f3-5b7c9d0e2f41",
    "total": 80,
    "covered": 8,
    "required": 10,
    "axes": {
      "technology": {
        "required": 5, "covered": 4,
        "matched": ["React", "TypeScript", "Node.js", "PostgreSQL"],
        "missing": ["Kubernetes"]
      },
      "impact": {
        "required": 2, "covered": 2,
        "matched": ["지연 개선", "트래픽 규모"], "missing": []
      },
      "role": {
        "required": 2, "covered": 1,
        "matched": ["백엔드"], "missing": ["리드 경험"]
      },
      "conditions": {
        "required": 1, "covered": 1,
        "matched": ["서울"], "missing": []
      }
    },
    "reason": "역할·연차에서 리드 경험 근거가 비어 있습니다.",
    "nextAction": "리드 경험을 실제로 사용한 기록을 추가하거나 기존 기록의 원문 근거를 보강하세요.",
    "computedAt": "2026-08-14T03:05:12.000Z"
  }
}
```

`total`은 공고가 명시한 요건 가운데 기록에서 근거를 찾은 요건의 비율이며, 축은 요건을 묶어 보여 주는 분류이고 **배점이 아니다**. 요건을 하나도 읽지 못한 공고는 점수를 내지 않고 409로 답한다. 삭제되지 않은 기록이 **3개보다 적을 때**도 409이다. 0점과 판정 불가는 다른 상태이다.

7) 생성 진행 스트림

유형 **스트림** · `GET /v1/generation-jobs/:id/page-stream`

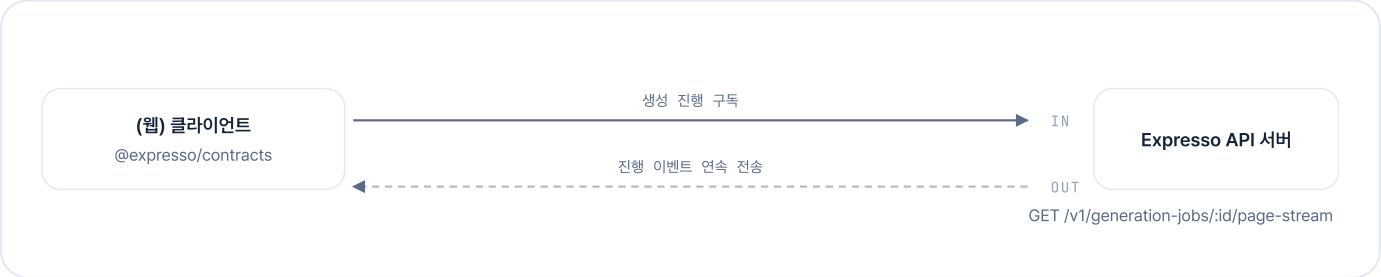


그림 7.3.7 생성 진행 스트림의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`id`	O	–	uuid	생성 작업 식별자
`Accept`	O	–	Header	`text/event-stream`
`event`	–	O	String	`begin` · `thinking` · `delta` · `done` · `failed`
`data`	–	O	Object	이벤트별 본문. `begin` 은 `{style}` , `thinking` 은 `{tokens}` , `delta` 는 `{text}` , `done` 은 `{pageId}` , `failed` 는 `{code}`

표 7.3.7 생성 진행 스트림의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/generation-jobs/5f3e8d21-9a4b-4c7d-8e6f-0a1b2c3d4e5f/page-stream
Accept: text/event-stream
```

실제 전송내용 (OUT)

```
전송방향 : OUT
Content-Type: text/event-stream; charset=utf-8

id: 1786676400000-0
event: begin
data: {"style":"null"}

id: 1786676412000-0
event: thinking
data: {"tokens":512}

id: 1786676530000-0
event: delta
data: {"text":"{\"html\":\"<section><h1>사내 검색 개편"}

id: 1786676590000-0
event: done
data: {"pageId":"2d7c6b5a-4e3f-4a1b-9c8d-7e6f5a4b3c2d"}
```

생성은 몇 분이 걸리고 그 사이 화면이 비어 있으면 사용자가 실패로 오해한다. 모델이 쓰는 지면 JSON을 조각(`delta`)이 나오는 대로 밀어 주어 진행을 보여 준다. 연결이 끊겼다 다시 붙으면 서버가 처음부터 다시 보내므로 클라이언트는 받은 내용을 비우고 처음부터 다시 받는다.

8) 공개 포트폴리오 열람

유형 **공개 조회** · `GET /v1/public/portfolios/:slug`

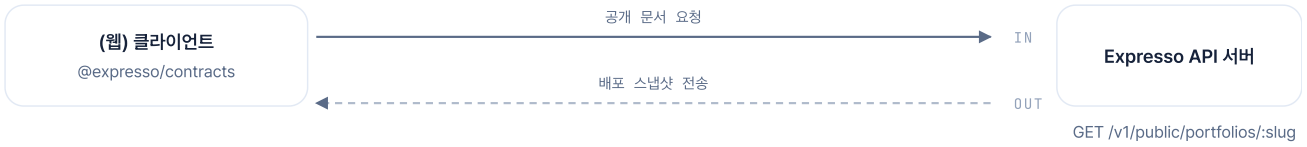


그림 7.3.8 공개 포트폴리오 열람의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`slug`	O	–	String	공개 주소, 경로 파라메터. 3 – 63자, 영문 소문자 · 숫자 · `-`
`data`	–	O	Object	`kind` 에 따라 `deployment` 또는 새 주소를 담는다
`kind`	–	O	String	`portfolio` · `redirect` . 바뀐 주소로 들어오면 `redirect` 와 `{from, to, expiresAt}`
`deployment`	–	O	Object	배포 시점에 고정된 문서. `snapshot` · 공유 카드용 `seo` · `publishedAt` 를 담는다

표 7.3.8 공개 포트폴리오 열람의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/public/portfolios/minjae-frontend-2026
(인증 헤더 없음)
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "kind": "portfolio",
    "deployment": {
      "id": "9a8b7c6d-5e4f-4321-8abc-def012345678",
      "portfolioId": "4c3b2a1d-0e9f-4a8b-9c7d-6e5f4a3b2c1d",
      "version": 3,
      "slug": "minjae-frontend-2026",
      "snapshot": {
        "portfolioId": "4c3b2a1d-0e9f-4a8b-9c7d-6e5f4a3b2c1d",
        "title": "박민재 · 프론트엔드",
        "templateId": "6e5d4c3b-2a1f-4e0d-8c9b-a8f7e6d5c4b3",
        "sections": [ { "id": "1a2b3c4d-5e6f-4a7b-8c9d-0e1f2a3b4c5d", "order": 0, "visible": true,
"blocks": [ ... ] } ],
        "generatedPage": { "id": "2d7c6b5a-4e3f-4a1b-9c8d-7e6f5a4b3c2d", "revision": 2, "document": "
<!doctype html>…" }
```

```
      },
      "seo": { "title": "박민재 · 프론트엔드", "description": "…", "indexable": true },
      "contactVisibility": "on_request",
      "publishedAt": "2026-08-12T09:00:00.000Z"
    }
  }
}
```


인증 없이 열리는 경로이다. 배포 시점의 스냅샷만 노출하므로 이후의 편집이 공개된 문서를 바꾸지 않는다. 사용자 식별자와 원본 기록은 응답에 **담기지 않는다**.

9) 자산 서명 URL 발급

유형 **서명 URL** · `POST /v1/assets/:id/signed-url`

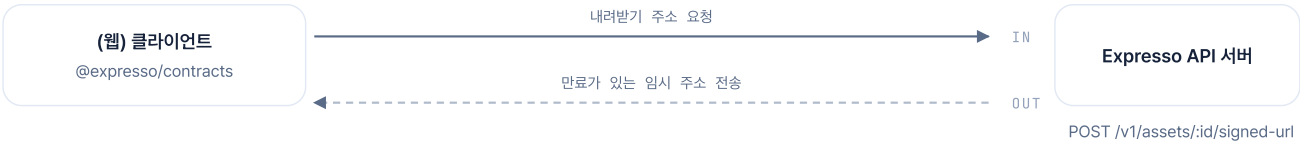


그림 7.3.9 자산 서명 URL 발급의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`id`	O	—	uuid	내보내기 자산 식별자, 경로 파라메터
`ttlSeconds`	O	—	int	1 – 86,400초, 기본값 300
`data.assetId`	—	O	uuid	자산 식별자
`data.url`	—	O	String	`/v1/public/assets/:id` 에 `version` · `expires` · `signature` 를 붙인 주소
`data.expiresAt`	—	O	datetime	이 시각 이후에는 403을 돌려준다

표 7.3.9 자산 서명 URL 발급의 파라메터

실제 전송내용 (IN)


```
전송방향 : IN
POST /v1/assets/4f1c2a9e-7b3d-4e8a-9c21-5d6e7f8a9b0c/signed-url
{
```

```
  "ttlSeconds": 600
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "assetId": "4f1c2a9e-7b3d-4e8a-9c21-5d6e7f8a9b0c",
    "url": "/v1/public/assets/4f1c2a9e-7b3d-4e8a-9c21-5d6e7f8a9b0c?
version=2&expires=1791201000&signature=Jb3x…",
    "expiresAt": "2026-10-05T11:50:00.000Z"
  }
}
```

파일을 API 서버가 직접 내려보내지 않고, **만료 시각과 서명**을 붙인 공개 자산 주소만 발급한다. 요금제가 `export.document` 를 허용하지 않으면 403을, 자산이 없거나 회수되었으면 404를 돌려준다. 발급된 주소는 만료 시각이 지나거나 서명이 맞지 않으면 403으로 거절되며, 내려받을 때마다 자산의 내려받기 수를 1 늘린다.

10) 로그인

유형 **생성** · `POST /v1/auth/login`



그림 7.3.10 로그인의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`email`	O	—	String	필수, 이메일 형식, 최대 320자
`password`	O	—	String	필수, 1–200자
`data.user.id`	—	O	uuid	사용자 식별자
`data.user.email`	—	O	String	사용자 이메일
`data.user.displayName`	—	O	String	1–200자
`data.user.planCode`	—	O	enum	`free` · `pro` · `team`

속성	IN	OUT	TYPE	DESCRIPTION
`data.session.sessionId`	–	0	uuid	발급된 세션 식별자
`data.session.accessToken`	–	0	String	`exps_` 접두사 뒤 43자
`data.session.expiresAt`	–	0	datetime	세션 만료 시각(ISO 문자열)

표 7.3.10 로그인의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
{
  "email": "jiwoo.kim@example.com",
  "password": "correct-horse-battery"
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "user": {
      "id": "dda90c64-d124-45ea-b5e8-3355341b2c05",
      "email": "jiwoo.kim@example.com",
      "displayName": "김지우",
      "planCode": "free"
    },
    "session": {
      "sessionId": "dfd6c113-d5ef-4626-9b9d-b9ed4140f89e",
      "accessToken": "exps_Q2xZ8kV4mT7pR1sW9nB3cD6fH0jK5lL2qA8eU4yI7oX",
      "expiresAt": "2026-11-04T02:10:11.000Z"
    }
  }
}
```

인증 없이 호출하는 경로이며, 본문이 스키마를 벗어나면 400을, 이메일이나 비밀번호가 맞지 않거나 삭제를 요청한 계정이면 401을 돌려준다. 성공하면 200과 함께 새 세션을 하나 발급하고, 세션 만료 시각은 발급 시점으로부터 **30일 뒤**로 정한다.

11) 카테고리 생성

유형 **생성** · `POST /v1/career/categories`

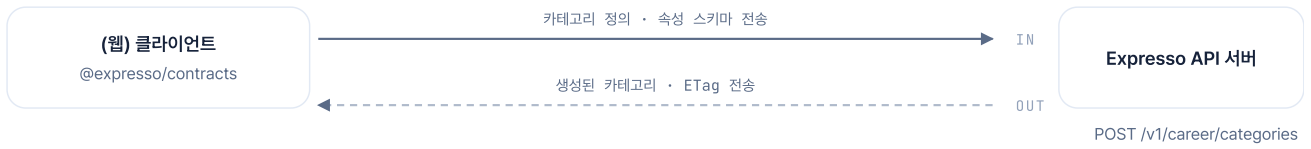


그림 7.3.11 카테고리 생성의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`key`	0	0	String	필수, 영문 소문자로 시작하는 2–64자(`a-z` · `0-9` · `_`)
`name`	0	0	String	필수, 1–120자
`icon`	0	0	String	필수, 1–80자
`defaultView`	0	0	enum	`table` · `gallery` · `timeline` · `board` · `list`
`propertySchema`	0	0	Object	속성 키 → `{label, type, required, system}` , 기본값 빈 객체. 응답에는 각 속성의 `id` 가 채워진다
`ETag`	–	0	Header	`"v{version}"` 형식
`data.id`	–	0	uuid	카테고리 식별자
`data.isSystem`	–	0	bool	사용자 카테고리는 `false`
`data.sortOrder`	–	0	int	7 + 기존 사용자 카테고리 수
`data.recordCount`	–	0	int	소속 기록 수
`data.version`	–	0	int	생성 직후 1

표 7.3.11 카테고리 생성의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
{
  "key": "side_project",
  "name": "사이드 프로젝트",
  "icon": "rocket",
```

```
  "defaultView": "gallery",
  "propertySchema": {
    "stack": { "label": "기술 스택", "type": "tags", "required": false, "system": false }
  }
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
ETag: "v1"
{
  "data": {
    "id": "2c4ada0e-7b25-42da-b360-86dfa0b7fa54",
    "key": "side_project",
    "name": "사이드 프로젝트",
    "icon": "rocket",
    "defaultView": "gallery",
    "isSystem": false,
    "propertySchema": {
      "stack": {
        "id": "4d52b09f-554f-4908-ae11-95f1cc1ba3c6",
        "label": "기술 스택", "type": "tags", "required": false, "system": false
      }
    },
    "sortOrder": 7,
    "recordCount": 0,
    "version": 1
  }
}
```

성공하면 **201**을 돌려주고 `ETag` 헤더에 카테고리 버전을 싣는다. 본문이 스키마를 벗어나면 400을, 그 사용자에게 같은 `key`의 카테고리가 이미 있거나 사용자 카테고리가 100개에 이르면 409를 돌려준다. 속성 정의에 `id`가 없으면 서버가 UUID를 발급해 채운다.

12) 기록 문서 편집 반영

유형 **조건부 수정** · `POST /v1/career/records/:recordId/document/updates`



그림 7.3.12 기록 문서 편집 반영의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`recordId`	O	O	uuid	경로 파라메터, 편집 대상 기록

속성	IN	OUT	TYPE	DESCRIPTION
clientId	O	O	uuid	필수, 편집 클라이언트 식별자
clientSequence	O	O	int	필수, 1 이상. 클라이언트가 매기는 업데이트 순번
expectedSequence	O	–	int	선택, 0 이상. 클라이언트가 알고 있는 문서 버전
updateBase64	O	–	String	필수, Base64 인코딩된 문서 업데이트. 디코딩 후 1MB 이하
checksum	O	–	String	필수, 디코딩한 바이트의 SHA-256 16진수 64자
serverSequence	–	O	int	서버가 부여한 업데이트 순번, 1 이상
documentVersion	–	O	int	반영 후 문서 버전

표 7.3.12 기록 문서 편집 반영의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
POST /v1/career/records/8aaade04-0aec-4175-927a-d8a5f4564253/document/updates
{
  "clientId": "c303fa51-ddf1-4457-a4d4-a33280028378",
  "clientSequence": 12,
  "expectedSequence": 41,
  "updateBase64": "AQHC5ZK7BAAHAQRib2R5AQA=",
  "checksum": "6a91343114156e74f65cef83a5a2476af562a241e8d96a32c63bf053acb07cde"
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "recordId": "8aaade04-0aec-4175-927a-d8a5f4564253",
  "clientId": "c303fa51-ddf1-4457-a4d4-a33280028378",
  "clientSequence": 12,
  "serverSequence": 42,
  "documentVersion": 42
}
```

디코딩한 업데이트가 1MB를 넘으면 413을, 체크섬이 맞지 않으면 422를, 기록이 없으면 404를, `expectedSequence`가 현재 문서 버전과 다르면 409를 돌려준다. 같은 `clientId` · `clientSequence` 조합이 다시 오면 업데이트를 새로 저장하지 않고 **처음 부여한 순번**을 그대로 돌려준다. 압축되지 않은 업데이트가 **100건 이상**이거나 합계 **512KB 이상**이 되면 문서 압축 작업을 큐에 넣는다.

13) 저장된 뷰 조회

유형 **목록 조회** · `GET /v1/career/view-configurations/:viewId/query`

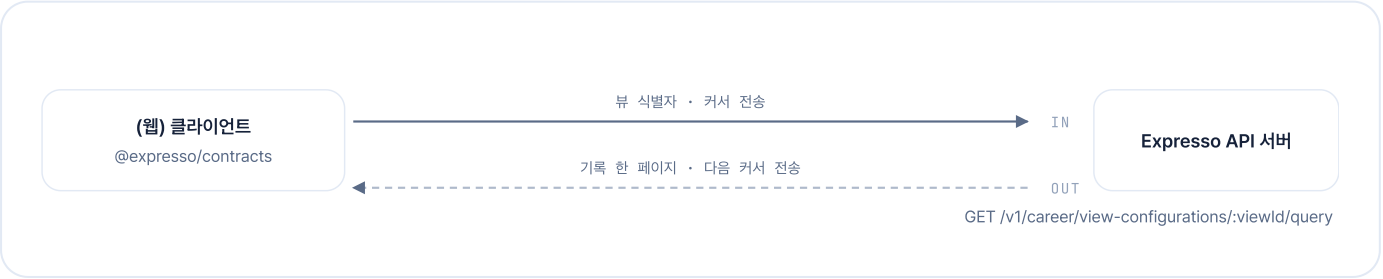


그림 7.3.13 저장된 뷰 조회의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`viewId`	O	–	uuid	경로 파라메터, 저장된 뷰 식별자
`cursor`	O	–	String	쿼리, 선택, 1–4096자. 이전 응답의 `nextCursor`
`limit`	O	–	int	쿼리, 선택, 1–100, 기본값 50
`data`	–	O	Array	뷰의 필터 · 정렬을 적용한 기록 목록
`data[].id`	–	O	uuid	기록 식별자
`data[].title`	–	O	String	최대 300자
`data[].status`	–	O	enum	`draft` · `organized` · `verified`
`data[].origin`	–	O	enum	`manual` · `ai` · `interview` · `import`
`data[].properties`	–	O	Object	카테고리 속성 값
`data[].version`	–	O	int	기록 버전
`page.hasNextPage`	–	O	bool	다음 페이지 존재 여부
`page.nextCursor`	–	O	String	다음 페이지 커서, 마지막 페이지면 `null`

표 7.3.13 저장된 뷰 조회의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/career/view-configurations/cd3ca7f4-a403-48c4-a8a8-00fe7b0637d7/query?limit=20
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": [
    {
      "id": "8aaade04-0aec-4175-927a-d8a5f4564253",
      "categoryId": "2c4ada0e-7b25-42da-b360-86dfa0b7fa54",
      "title": "사내 검색 개편",
      "status": "organized",
      "origin": "manual",
      "properties": { "stack": ["TypeScript", "Elasticsearch"] },
      "computedProperties": {},
      "bodyMd": "검색 지연을 320ms에서 90ms로 줄였다.",
      "version": 3,
      "createdAt": "2026-08-14T02:10:11.000Z",
      "updatedAt": "2026-09-02T07:41:30.000Z"
    }
  ],
  "page": {
    "hasNextPage": true,
    "nextCursor": "eyJ1c2VySWQiOiJkZGE5MGM2NCJ9.Hk3vQ9pZ2mN8rT1x"
  }
}
```

뷰나 그 뷰의 카테고리가 없으면 404를, 쿼리 문자열이 스키마를 벗어나면 400을 돌려준다. 커서는 서버가 서명하고 사용자 · 뷰 · 뷰 버전에 묶여 15분 동안 유효하며, 서명이 맞지 않거나 만료되었거나 그사이 뷰 설정이 바뀌었으면 400을 돌려준다. 서버는 `limit` 보다 **한 건 더** 읽어 `hasNextPage` 를 정한다.

14) 스킬 목록 조회

유형 **목록 조회** · `GET /v1/career/skills`

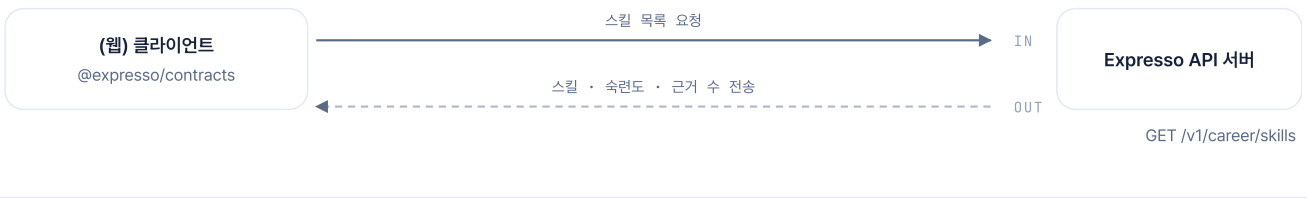


그림 7.3.14 스킬 목록 조회의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`data`	–	0	Array	근거가 1건 이상인 스킬 목록
`data[].id`	–	0	uuid	스킬 식별자
`data[].name`	–	0	String	1–120자
`data[].level`	–	0	int	숙련도 1–5
`data[].evidenceCount`	–	0	int	근거 수, 1 이상
`data[].strength`	–	0	enum	`weak` · `supported` · `strong`
`data[].lastUsedAt`	–	0	datetime	마지막 사용 시각(ISO 문자열)
`data[].computedAt`	–	0	datetime	스킬을 다시 계산한 시각(ISO 문자열)

표 7.3.14 스킬 목록 조회의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/career/skills
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": [
    {
      "id": "c2678c48-6d61-4830-87b6-3c9ad9349b59",
      "name": "TypeScript",
      "level": 4,
      "evidenceCount": 6,
      "strength": "strong",
      "lastUsedAt": "2026-09-02T07:41:30.000Z",
      "computedAt": "2026-09-02T07:42:05.000Z"
    }
  ]
}
```

요청 파라메터 없이 로그인한 사용자의 스킬만 돌려준다. **근거 수가 0인** 스킬은 빼고, `level` 내림차순 · `evidenceCount` 내림차순 · 이름 오름차순으로 정렬해 최대 **1,000건**까지 담는다. 조회만 하며 스킬을 다시 계산하지 않는다.

15) 자연어 검색 조건 변환

유형 **생성** · `POST /v1/jobs/search/interpret`



그림 7.3.15 자연어 검색 조건 변환의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`query`	O	–	String	필수, 앞뒤 공백 제거 후 2–1,000자
`resultCount`	O	–	int	선택, 0 이상, 기본값 0. 최근 검색에 함께 저장한다
`originalQuery`	–	O	String	해석한 검색어
`conditions`	–	O	Array	추출한 검색 조건 목록
`conditions[].field`	–	O	enum	`role` · `experience` · `work_type` · `location` · `salary` · `company_size` · `technology`
`conditions[].value`	–	O	String · number	1–200자 문자열 또는 0 이상의 수. `experience` · `salary` 는 수
`conditions[].enabled`	–	O	bool	조건 적용 여부
`conditions[].confidence`	–	O	number	0–1
`needsClarification`	–	O	bool	추출한 조건이 없으면 `true`
`example`	–	O	String	선택, 조건이 없을 때만 담는 예시 검색어
`recentSearchId`	–	O	uuid	저장된 최근 검색 식별자

표 7.3.15 자연어 검색 조건 변환의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
{
  "query": "서울 3년 이상 TypeScript 백엔드 원격",
  "resultCount": 0
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "originalQuery": "서울 3년 이상 TypeScript 백엔드 원격",
  "conditions": [
    { "field": "role", "value": "백엔드", "enabled": true, "confidence": 0.95 },
    { "field": "experience", "value": 3, "enabled": true, "confidence": 0.95 },
    { "field": "work_type", "value": "remote", "enabled": true, "confidence": 0.95 },
    { "field": "location", "value": "서울", "enabled": true, "confidence": 0.9 },
    { "field": "technology", "value": "typescript", "enabled": true, "confidence": 0.98 }
  ],
  "needsClarification": false,
  "recentSearchId": "dda90c64-d124-45ea-b5e8-3355341b2c05"
}
```

서버는 역할 · 경력 연수 · 근무 형태 · 지역 · 기술 · 연봉을 사전과 정규식으로 추출하며, 연봉 조건은 `enabled: false` 로 만든다. 본문이 스키마를 벗어나면 400을 돌려준다. 해석할 때마다 최근 검색에 기록하는데, 직전 최근 검색과 검색어가 같으면 그 항목을 갱신하고 사용자당 최근 **20건**만 남긴다.

16) 공고 상세 조회

유형 **단건 조회** · `GET /v1/jobs/postings/:id`

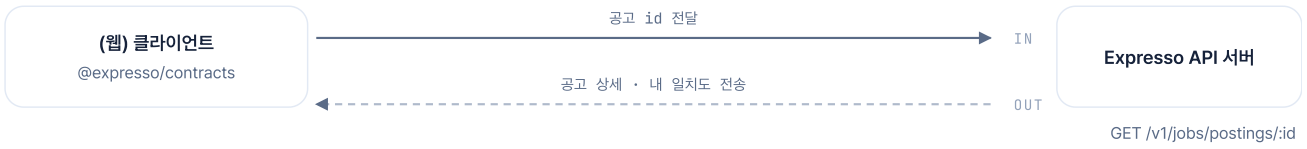


그림 7.3.16 공고 상세 조회의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`id`	O	—	uuid	경로 파라메터. 공고 id
`data.title`	—	O	String	1–300자
`data.company`	—	O	Object	회사 정보. 상세에서는 `brandColors` · `tonePalette` · `toneImpression` 이 더 붙는다

속성	IN	OUT	TYPE	DESCRIPTION
`data.descriptionRaw`	—	O	String	공고 원문. 요건의 `sourceSpan`이 이 문자열의 글자 위치를 가리킨다
`data.duties`	—	O	Array	`{group, items}` 목록. 주요 업무
`data.criteria`	—	O	Array	자격 요건. `kind`는 `must` · `nice` · `tone`, `coverage`는 `covered` · `partial` · `missing` 또는 null
`data.daysLeft`	—	O	int	마감까지 남은 날. 상시 모집이면 null, 지났으면 음수
`data.match`	—	O	Object	일치도. `total`은 0–100. 계산하지 않았으면 null
`data.interest`	—	O	Object	관심 공고 상태. `stage`는 `saved` · `applied` · `closed`. 없으면 null
`data.analysis`	—	O	Object	이 사용자의 공고 해석 작업. `status`는 `queued` · `running` · `done` · `failed`. 없으면 null
`data.rank`	—	O	Object	`{position, total}`. 점수가 매겨진 공고 사이의 순위, 없으면 null
`data.topRecords`	—	O	Array	`{id, title}` 최대 3건. 요건을 덮는 횟수가 많은 기록 순

표 7.3.16 공고 상세 조회의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/jobs/postings/7c0e2a51-3f4b-4d8e-9a61-2b5d8f0c1e47
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "id": "7c0e2a51-3f4b-4d8e-9a61-2b5d8f0c1e47",
    "title": "백엔드 엔지니어 (검색 플랫폼)",
    "company": {
      "id": "1a9d4e70-5b2c-4f3a-8e16-0c7b9d2f4a85",
      "name": "예시테크",
      "toneImpression": "짧고 단정한 문장"
    },
    "source": "api",
```

```
    "daysLeft": 12,
    "descriptionRaw": "[주요 업무]\n검색 API 설계와 운영...",
    "duties": [{ "group": "주요 업무", "items": ["검색 API 설계와 운영"] }],
    "criteria": [
      {
        "id": "3e6f1b2a-9c40-4d7e-8b15-6a2c0f9d1e33",
        "orderNo": 0,
        "label": "Elasticsearch 운영 경험",
        "kind": "must",
        "coverage": "covered",
        "coveredBy": [{ "id": "b4d2c8e1-6f30-4a97-9e52-1c8a7f0b3d64", "title": "사내 검색 개편" }],
        "sourceSpan": { "start": 42, "end": 61, "quote": "Elasticsearch 운영 경험" }
      }
    ],
    "match": { "total": 78, "covered": 7, "required": 9, "reason": "검색 지연 개선 기록이 핵심 요건을 덮는다.",
"nextAction": "대용량 색인 경험을 보강한다.", "computedAt": "2026-10-01T08:12:40.000Z" },
    "interest": null,
    "analysis": { "id": "5f8a3c19-2d6e-4b07-a1c4-9e0d7b2f6a18", "status": "done", "progressStage":
"done", "analyzedAt": "2026-10-01T08:12:38.000Z" },
    "rank": { "position": 2, "total": 14 },
    "topRecords": [{ "id": "b4d2c8e1-6f30-4a97-9e52-1c8a7f0b3d64", "title": "사내 검색 개편" }]
  }
}
```


경로의 `id` 가 UUID 형식이 아니면 400을, 해당 공고가 없으면 404를 돌려준다. 공고 본문과 요건은 모든 사용자에게 같고, `match` · `interest` · `analysis` · `rank` 와 요건의 `coverage` 는 요청한 사용자 기준으로 채워진다. 이 공고를 **아직 대조하지 않았으면** 요건의 `coverage` 는 null로 온다.

17) 재료 후보 조회

유형 **목록 조회** · `GET /v1/brews/:id/materials`

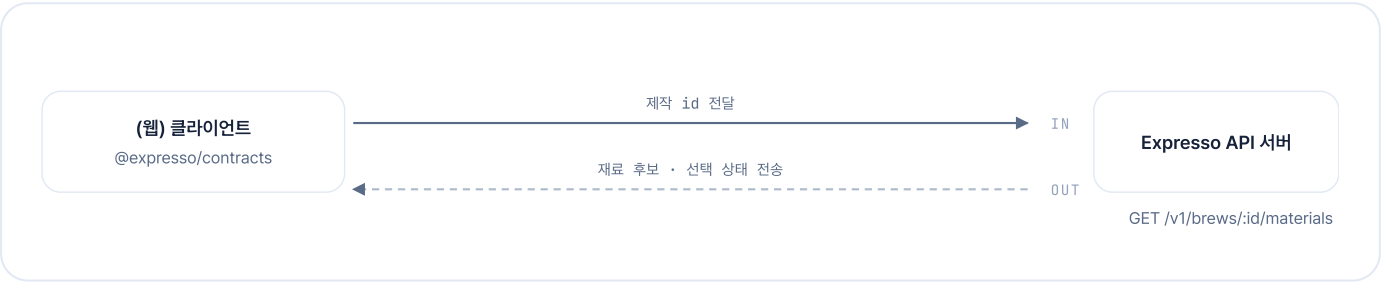


그림 7.3.17 재료 후보 조회의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`id`	0	—	uuid	경로 파라메터. 제작(brew) id
`data.selectionLimit`	—	0	int	한 번에 고를 수 있는 재료 수. 현재 10
`data.mode`	—	0	enum	`solo` · `collab`

속성	IN	OUT	TYPE	DESCRIPTION
`data.lengthPreset`	—	0	enum	`single` · `double` · `triple`
`data.materials`	—	0	Array	재료 후보. 최대 50건, `rank` 오름차순
`data.materials.recordId`	—	0	uuid	커리어 기록 id
`data.materials.status`	—	0	enum	`organized` · `verified`
`data.materials.score`	—	0	int	0 이상의 매칭 점수
`data.materials.rank`	—	0	int	매칭 순위. 선택을 바꿔도 바뀌지 않는다
`data.materials.selected`	—	0	bool	현재 선택 여부
`data.materials.selectedBy`	—	0	enum	`auto` · `user`
`data.materials.reason`	—	0	String	순위 근거 문장. 1–300자

표 7.3.17 재료 후보 조회의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/brews/2d7f4a10-8c3e-4b59-b6a1-0e9c5d3f7b22/materials
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "brewId": "2d7f4a10-8c3e-4b59-b6a1-0e9c5d3f7b22",
    "jobAnalysisId": "5f8a3c19-2d6e-4b07-a1c4-9e0d7b2f6a18",
    "updatedAt": "2026-10-01T08:20:05.000Z",
    "selectionLimit": 10,
    "mode": "solo",
    "lengthPreset": "single",
    "materials": [
      {
        "recordId": "b4d2c8e1-6f30-4a97-9e52-1c8a7f0b3d64",
        "title": "사내 검색 개편",
        "status": "verified",
        "score": 86,
        "rank": 0,
        "selected": true,
```

```
        "selectedBy": "auto",
        "excludedReason": null,
        "categoryName": "경험",
        "categoryIcon": "briefcase",
        "periodFrom": "2025-03-01",
        "periodTo": "2025-09-30",
        "origin": "manual",
        "reason": "공고의 검색 성능 요건과 기술이 겹친다."
      }
    ]
  }
}
```


경로의 `id` 가 UUID 형식이 아니면 400을, 요청한 사용자의 제작이 아니거나 없으면 404를 돌려준다. 후보는 제작을 만들 때 최대 50건까지 매겨지고 순위 0–9의 10건이 자동으로 선택된다. 이 API는 **읽기 전용**이며, 선택을 바꿀 때는 같은 경로의 `PUT` 에 `recordIds` (중복 없는 uuid **최대 10개**)를 보내고 같은 형태의 응답을 받는다.

18) 인터뷰 답변 저장

유형 **생성** · `PUT /v1/interview-sessions/:id/answers/:questionId`

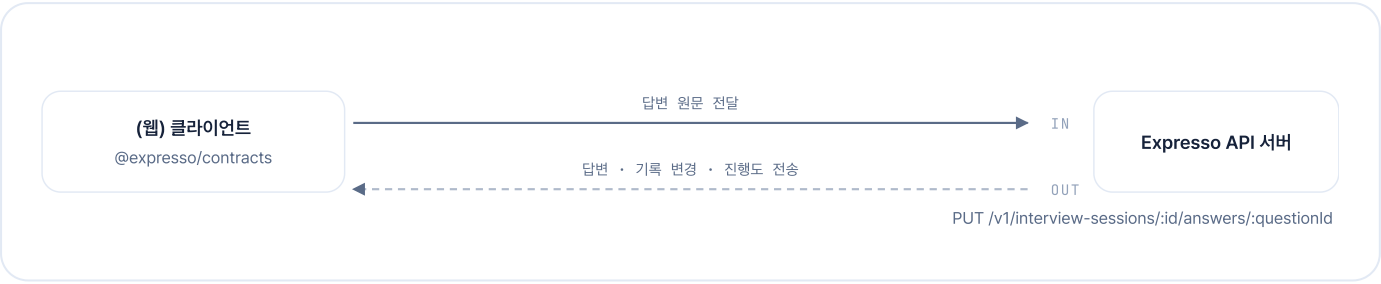


그림 7.3.18 인터뷰 답변 저장의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`Idempotency-Key`	0	–	Header	16–128자, 안전한 문자만 허용
`id`	0	–	uuid	경로 파라메터. 인터뷰 세션 id
`questionId`	0	–	uuid	경로 파라메터. 질문 id
`inputType`	0	–	enum	`text` · `voice`
`transcript`	0	–	String	앞뒤 공백을 지운 뒤 1–100,000자
`data.answer`	–	0	Object	`{id, questionId, inputType, transcript, createdRecordId, version, updatedAt}`

속성	IN	OUT	TYPE	DESCRIPTION
`data.answer.createdRecordId`	—	0	uuid	답변으로 만들어진 커리어 기록 id
`data.recordChange.type`	—	0	enum	`created` · `strengthened`
`data.recordChange.changedFields`	—	0	Array	바뀐 기록 필드 이름. 1개 이상
`data.recordChange.sourceQuote`	—	0	String	기록에 반영한 답변 원문
`data.progress`	—	0	Object	`{answered, total}` . `total` 은 3–6

표 7.3.18 인터뷰 답변 저장의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
PUT /v1/interview-sessions/9a3e7c21-4f5b-4d08-b2e6-1c7f0a8d5e39/answers/6b1d9f42-0e7a-4c35-8f21-
3d5a9c0e7b14
Idempotency-Key: 4b1f9c2e-7a30-4d55-9c11-8e2f0a6d34bb
{
  "inputType": "text",
  "transcript": "검색 색인을 샤드 단위로 나눠 재색인 시간을 6시간에서 40분으로 줄였다."
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "answer": {
      "id": "e2c4a7f1-3b9d-4e60-a815-7d0f2b6c9e43",
      "questionId": "6b1d9f42-0e7a-4c35-8f21-3d5a9c0e7b14",
      "inputType": "text",
      "transcript": "검색 색인을 샤드 단위로 나눠 재색인 시간을 6시간에서 40분으로 줄였다.",
      "createdRecordId": "c7f0e3b2-5a14-4d86-9b3e-2f8d1a6c0e57",
      "version": 1,
      "updatedAt": "2026-10-01T09:02:17.000Z"
    },
    "recordChange": {
      "type": "created",
      "recordId": "c7f0e3b2-5a14-4d86-9b3e-2f8d1a6c0e57",
      "changedFields": ["title", "body_md"],
      "sourceQuote": "검색 색인을 샤드 단위로 나눠 재색인 시간을 6시간에서 40분으로 줄였다."
    },
    "progress": { "answered": 2, "total": 5 }
  }
}
```

질문에 처음 답하면 경험 카테고리에 `origin`이 `interview`인 **초안 기록**을 새로 만들고, 같은 질문에 다시 답하면 그 기록의 제목과 본문을 덮어쓰며 `recordChange.type`을 `strengthened`로 돌려준다. 같은 `Idempotency-Key`로 같은 내용을 다시 보내면 첫 결과를 돌려주고, 다른 내용을 보내면 409를 돌려준다. 세션이나 질문이 없으면 404이며, 저장할 때마다 세션의 답변 수와 다음 질문 순서를 다시 계산하고 모든 질문이 답변되거나 건너뛰어지면 세션을 `done`으로 바꾼다.

19) 레시피 생성

유형 **비동기 접수** · `POST /v1/brews/:id/recipes`

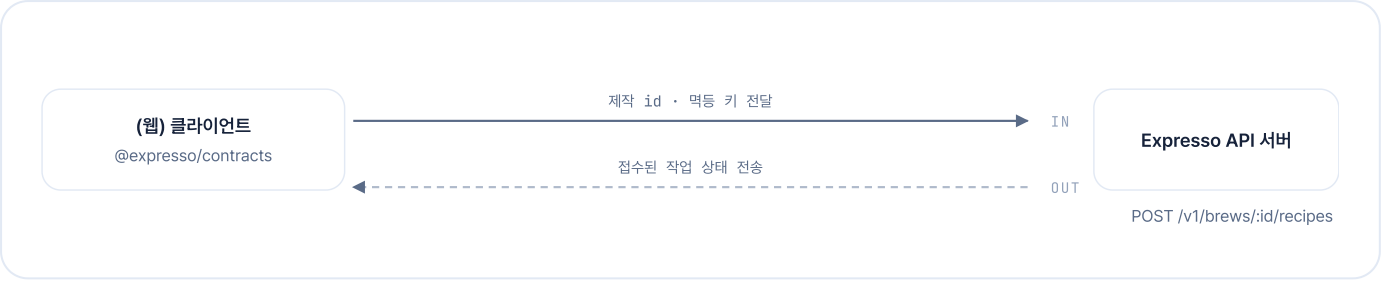


그림 7.3.19 레시피 생성의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`Idempotency-Key`	O	—	Header	16–128자, 안전한 문자만 허용
`id`	O	—	uuid	경로 파라메터. 제작(brew) id
`data.jobId`	—	O	uuid	접수된 작업 id
`data.type`	—	O	enum	`interview` · `recipe`. 이 API는 `recipe`
`data.status`	—	O	enum	`queued` · `running` · `succeeded` · `failed`
`data.stage`	—	O	String	진행 단계. 1–100자
`data.attempts`	—	O	int	실행 시도 횟수
`data.resultId`	—	O	uuid	완성된 레시피 id. 끝나기 전에는 null
`data.failure`	—	O	Object	`{code, retryable}`. 실패가 없으면 null

표 7.3.19 레시피 생성의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
POST /v1/brews/2d7f4a10-8c3e-4b59-b6a1-0e9c5d3f7b22/recipes
Idempotency-Key: 8e5d2a17-6c4f-4b90-a3d1-7f0b9e2c5a48
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "jobId": "f1a6c3e8-2b7d-4950-8e14-5c9a0d3f7b26",
    "type": "recipe",
    "status": "queued",
    "stage": "queued",
    "attempts": 0,
    "resultId": null,
    "failure": null
  }
}
```

요청 본문은 읽지 않으며, 작업 문서와 `recipe.draft` outbox 이벤트를 한 트랜잭션에 기록한 뒤 202로 작업 상태만 돌려준다. 같은 `Idempotency-Key`로 다시 보내면 새 작업을 만들지 않고 기존 작업을 돌려주며, 그 키가 다른 제작이나 다른 작업 유형에 쓰였으면 409를, 제작이 없으면 404를 돌려준다. 레시피는 **워커가 작업을 끝낸 뒤** 만들어지고, 클라이언트는 `GET /v1/brew-jobs/:id`로 상태를 확인해 `resultId`를 받는다.

20) 포트폴리오 생성 실행

유형 **비동기 접수** · `POST /v1/generation-jobs`

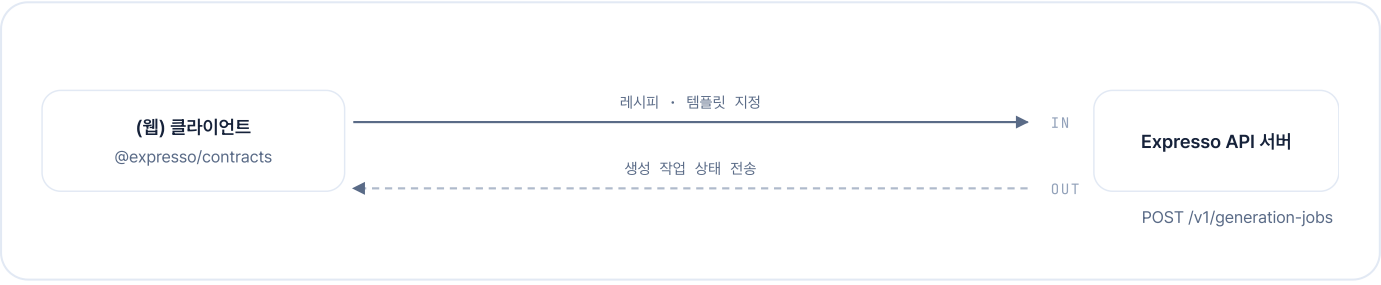


그림 7.3.20 포트폴리오 생성 실행의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`Idempotency-Key`	O	—	Header	16–128자, 안전한 문자만 허용
`recipeId`	O	—	uuid	필수. 생성에 쓸 레시피
`templateId`	O	—	uuid	필수. 활성 템플릿이어야 한다

속성	IN	OUT	TYPE	DESCRIPTION
`styleOverrides`	0	—	Object	선택. `background` · `text` · `accent` (`#RRGGBB`), `font` (`sans` · `serif` · `mono`), `density` (`compact` · `comfortable` · `spacious`), `structure` (`single-column` · `dense-grid` · `wide-margin`) 중 덮어쓸 축만 담는다
`data.generationJobId`	—	0	uuid	생성 작업 id
`data.status`	—	0	enum	`queued` · `running` · `done` · `failed`
`data.stage`	—	0	enum	`queued` · `validating` · `materializing` · `charging` · `done` · `failed`
`data.attempts`	—	0	int	실행 시도 횟수
`data.usageCharged`	—	0	bool	사용량 차감 여부
`data.portfolioId`	—	0	uuid	만들어진 포트폴리오 id. 만들어지기 전에는 null
`data.failure`	—	0	Object	`{code, retryable}` . 실패가 없으면 null

표 7.3.20 포트폴리오 생성 실행의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
Idempotency-Key: 0c9b4e72-1d5a-4f38-b6e0-9a2f7c3d1e85
{
  "recipeId": "a3e8d1c6-7b20-4f59-9d43-1e6c0b8f2a17",
  "templateId": "4f2b9e60-3c18-4a7d-b5e1-8d0a6c3f9b52",
  "styleOverrides": { "accent": "#3B5BDB", "structure": "wide-margin" }
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "generationJobId": "d8b3f0a5-6e21-4c97-8a4d-2f7e1c9b0a36",
    "status": "queued",
    "stage": "queued",
    "attempts": 0,
    "usageCharged": false,
    "portfolioId": null,
```

```
    "failure": null
  }
}
```

본문 검증에 실패하거나 `Idempotency-Key`가 없으면 400을, 레시피가 없거나 템플릿이 활성 상태가 아니면 404를 돌려준다. 같은 키와 같은 본문으로 다시 보내면 기존 작업을 돌려주고, 같은 키에 다른 본문을 보내면 409를 돌려준다. 작업 문서와 `portfolio.generate` outbox 이벤트를 한 트랜잭션에 기록한 뒤 **202**로 응답하며, 진행 상태는 `GET /v1/generation-jobs/:id`로, 만들어지는 지면은 `GET /v1/generation-jobs/:id/page-stream`으로 받는다.

21) 블록 수정 미리보기

유형 **생성** · `POST /v1/portfolios/:id/blocks/:blockId/edit-preview`

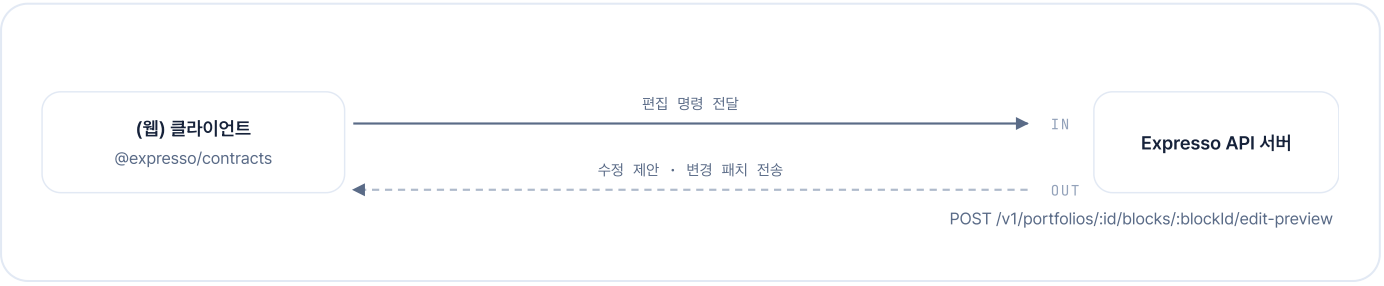


그림 7.3.21 블록 수정 미리보기의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`id`	0	–	uuid	경로 파라메터. 포트폴리오 id
`blockId`	0	–	uuid	경로 파라메터. 블록 id
`operation`	0	–	enum	`update_text` · `set_style` · `insert_record` · `instruct`
`path`	0	–	String	`update_text`에서 선택. 1–60자, 생략하면 `content.text`
`text`	0	–	String	`update_text`에서 필수. 1–100,000자
`style`	0	–	Object	`set_style`에서 필수. `size` · `weight` · `italic` · `strike` · `color` · `highlight` · `alignment` · `spacing` 중 하나 이상
`recordId`	0	–	uuid	`insert_record`에서 필수. 블록에 다시 이을 기록
`instruction`	0	–	String	`instruct`에서 필수. 앞뒤 공백을 지운 뒤 1–500자
`data.id`	–	0	uuid	수정 제안 id

속성	IN	OUT	TYPE	DESCRIPTION
`data.before / data.after`	—	O	Object	블록의 `content` · `style` · `sourceRecordId` · `syncState` · `locked` 수정 전후 값
`data.patches`	—	O	Array	`{path, before, after, label}`. 바뀐 자리 목록
`data.status`	—	O	enum	`pending` · `applied` · `rejected` · `expired`. 생성 직후 `pending`

표 7.3.21 블록 수정 미리보기의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
POST /v1/portfolios/6e0a3d58-9f21-4b7c-a4e3-0d8b5c2f1a69/blocks/1b7e4c90-5d32-4a8f-9e06-3c1a8f2d7b45/edit-preview
{
  "operation": "update_text",
  "path": "content.text",
  "text": "재색인 시간을 6시간에서 40분으로 줄였다."
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "id": "c5a2e9f3-0b48-4d17-86c2-7e1f4a0d3b98",
    "portfolioId": "6e0a3d58-9f21-4b7c-a4e3-0d8b5c2f1a69",
    "blockId": "1b7e4c90-5d32-4a8f-9e06-3c1a8f2d7b45",
    "targetPath": "portfolio:6e0a3d58-9f21-4b7c-a4e3-0d8b5c2f1a69/section:9d4f1b07-2e6a-4c38-b5d0-8a3e7c1f2b64/block:1b7e4c90-5d32-4a8f-9e06-3c1a8f2d7b45",
    "operation": "update_text",
    "before": { "content": { "text": "재색인 시간을 크게 줄였다." }, "style": {}, "sourceRecordId": null, "syncState": "synced", "locked": false },
    "after": { "content": { "text": "재색인 시간을 6시간에서 40분으로 줄였다." }, "style": {}, "sourceRecordId": null, "syncState": "detached", "locked": true },
    "sourceRecordId": null,
    "instruction": null,
    "patches": [
      { "path": "content.text", "before": "재색인 시간을 크게 줄였다.", "after": "재색인 시간을 6시간에서 40분으로 줄였다.", "label": "직접 고쳤습니다" }
    ],
    "status": "pending"
  }
}
```

이 API는 블록을 바꾸지 않고 24시간 뒤 만료되는 `pending` 상태의 수정 제안만 저장하며 201을 돌려준다. 블록 종류가 허용하지 않는 `path` (문단 · 제목 · 목록은 `content.text`, 수치는 `content.value` · `content.label` 만 허용)는

422로, 잠긴 블록에 대한 `instruct`와 미리보기 중 블록이 바뀐 경우는 **409**로, 편집 모델이 설정되지 않은 상태의 `instruct`는 **503**으로 거절한다. 제안은 `POST /v1/portfolio-edit-proposals/:id/apply`를 호출해야 블록에 반영된다.

22) 포트폴리오 배포

유형 **생성** · `POST /v1/portfolios/:id/deployments`



그림 7.3.22 포트폴리오 배포의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`id`	O	–	uuid	경로 변수. 배포할 포트폴리오
`slug`	O	–	String	필수. 3~63자, 소문자·숫자·하이픈, 하이픈으로 시작하거나 끝나지 않는다
`seo.title`	O	–	String	선택. 1~70자
`seo.description`	O	–	String	선택. 1~160자
`seo.indexable`	O	–	bool	기본값 `false`
`contactVisibility`	O	O	String	`public` · `on_request` · `hidden`, 기본값 `hidden`
`data.id`	–	O	uuid	새 배포의 식별자
`data.portfolioId`	–	O	uuid	배포한 포트폴리오
`data.version`	–	O	int	직전 배포 버전에 1을 더한 값, 첫 배포는 1
`data.slug`	–	O	String	배포된 주소
`data.snapshot`	–	O	Object	배포 시점의 섹션 · 블록 · 생성 페이지 사본

속성	IN	OUT	TYPE	DESCRIPTION
`data.publishedAt`	–	O	datetime	배포 시각

표 7.3.22 포트폴리오 배포의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
{
  "slug": "minjae-backend",
  "seo": {
    "title": "박민재 · 백엔드 엔지니어",
    "description": "검색 지연을 320ms에서 90ms로 줄인 기록",
    "indexable": true
  },
  "contactVisibility": "on_request"
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "id": "d3a0f6c8-2b41-4e9a-9c57-60e1b2a4f8d9",
    "portfolioId": "7c1e4a52-93d0-4f6b-8a2e-1b5d9f0c3e71",
    "version": 3,
    "slug": "minjae-backend",
    "snapshot": {
      "portfolioId": "7c1e4a52-93d0-4f6b-8a2e-1b5d9f0c3e71",
      "title": "백엔드 포트폴리오",
      "templateId": "0b6f2d3e-5a19-4c84-b7e2-9d1a4c6e8f30",
      "sections": [ { "id": "5e8c1a07-4d2b-4f93-a6e1-2c7b9d0f4a18", "order": 0, "visible": true,
"blocks": [] } ],
      "generatedPage": { "id": "a41c7e90-6b3d-4f25-8e0a-3d9b1c5f7e62", "revision": 2, "document": "
<!doctype html>..." }
    },
    "seo": { "title": "박민재 · 백엔드 엔지니어", "description": "검색 지연을 320ms에서 90ms로 줄인 기록",
"indexable": true },
    "contactVisibility": "on_request",
    "publishedAt": "2026-08-14T03:20:45.000Z"
  }
}
```

성공하면 201을 돌려주고 포트폴리오 상태를 `published` 로 바꾼다. 배포 한 건마다 `slug` 가 고유해야 하므로 (`deployments.subdomain` 고유 인덱스) 이전 배포에 쓴 `slug` 로 다시 요청하면 같은 포트폴리오라도 409로 거절하고, 품질 검사를 통과한(`qualityStatus: ready`) 생성 페이지가 없을 때도 409를 낸다. 직전 배포와 `slug` 가 다르면 옛 주소에서 새 주소로 가는 **30일짜리 리다이렉트**를 함께 기록한다.

23) 방문 이벤트 수집

유형 **생성** · `POST /v1/analytics/events`

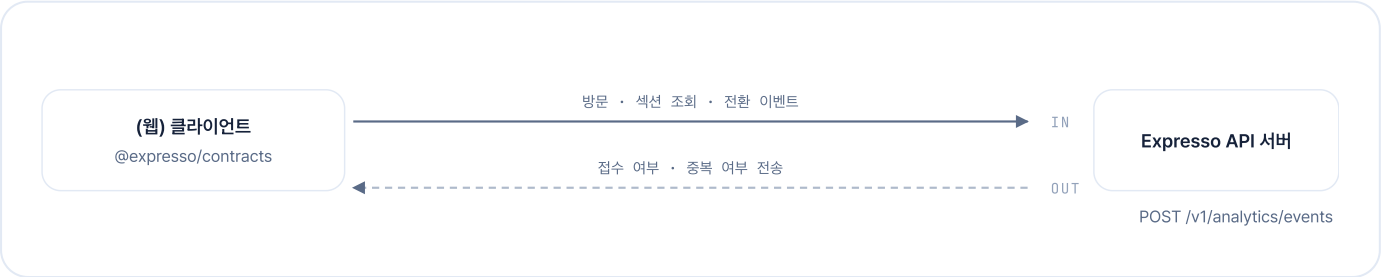


그림 7.3.23 방문 이벤트 수집의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`eventId`	0	0	uuid	필수. 같은 값의 재전송은 중복으로 처리한다
`slug`	0	–	String	필수. 배포된 포트폴리오 주소
`sessionId`	0	–	String	필수. 16~128자, 영문·숫자와 `._~-`
`type`	0	–	String	`visit` · `complete` · `section_view` · `contact_click` · `file_download` · `link_click`
`occurredAt`	0	–	datetime	필수. 시간대 오프셋 포함
`referrer`	0	–	String	선택. URL, 최대 2,000자
`sectionId`	0	–	uuid	`section_view` 일 때 필수
`dwellMs` · `scrollDepth`	0	–	int · number	`section_view` 일 때 필수. `dwellMs` 0~86,400,000, `scrollDepth` 0~1
`durationMs`	0	–	int	선택. 0~86,400,000
`target`	0	–	String	1~500자. 전환 이벤트(`contact_click` · `file_download` · `link_click`)일 때 필수
`data.accepted`	–	0	bool	접수 여부
`data.duplicate`	–	0	bool	이미 접수된 `eventId` 였는지

표 7.3.23 방문 이벤트 수집의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
{
  "eventId": "2f9d4c61-8a07-4b3e-9e15-7c0a6b2d8f43",
  "slug": "minjae-backend",
  "sessionId": "v1.3k9QmZp2Lx7Rt8Wd",
  "type": "section_view",
  "occurredAt": "2026-08-14T05:12:08.000Z",
  "sectionId": "5e8c1a07-4d2b-4f93-a6e1-2c7b9d0f4a18",
  "dwellMs": 14200,
  "scrollDepth": 0.8
}
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "accepted": true,
    "duplicate": false,
    "eventId": "2f9d4c61-8a07-4b3e-9e15-7c0a6b2d8f43"
  }
}
```

로그인 없이 호출할 수 있고 **202**를 돌려준다. `Authorization: Bearer` 헤더가 있으면 토큰을 검증해, 포트폴리오 주인의 방문은 **집계에서 빠지도록** 표시한다. 같은 `eventId` 를 같은 내용으로 다시 보내면 `duplicate: true` 로 응답하고 내용이 다르면 409를 낸다. 본문이 8KB를 넘으면 413, 같은 방문자·배포에서 1분에 120건을 넘으면 429, 공개 중인 배포가 없는 `slug` 이면 404, `visit` 이벤트보다 다른 이벤트가 먼저 오면 409로 거절한다.

24) 분석 대시보드 조회

유형 **단건 조회** · `GET /v1/portfolios/:id/analytics/dashboard`

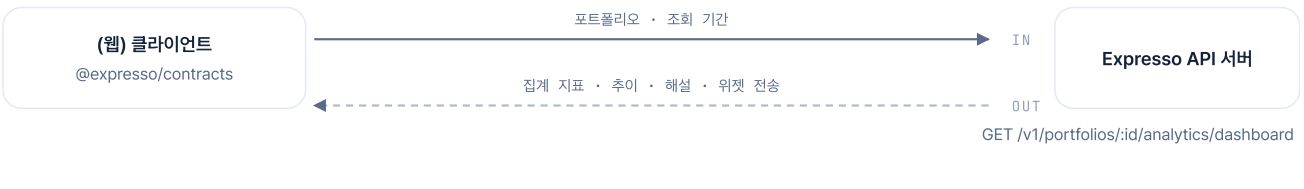


그림 7.3.24 분석 대시보드 조회의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`id`	O	—	uuid	경로 변수. 조회할 포트폴리오
`period`	O	—	String	쿼리. `7d` · `30d` · `all`, 기본값 `30d`
`data.deployment`	—	O	Object	`{id, subdomain, customDomain, publishedAt}`. 현재 배포
`data.period`	—	O	Object	`{preset, start, end}`. 날짜는 `YYYY-MM-DD`
`data.coverage`	—	O	Object	`{days, aggregatedDays, pendingDates}`. 방문이 있는 날 중 집계가 끝난 날과 남은 날
`data.metrics`	—	O	Object	`visits` · `completes` · `contactClicks` · `fileDownloads` · `linkClicks` · `sectionViews` · `sectionDwellMs`
`data.previous`	—	O	Object	같은 길이의 직전 기간 지표. `all`이거나 집계가 없으면 `null`
`data.trend`	—	O	Array	`{date, visits, completes}`, 최대 400개
`data.sections` · `data.referrers`	—	O	Array	섹션별 조회 수·체류 시간(최대 50개), 유입 출처 origin별 방문 수(최대 8개)
`data.organizations`	—	O	Object	`entitled`가 `false`면 목록 없이 자격이 없다는 값만 온다
`data.insight`	—	O	Object	`state` : `insufficient_sample` · `none` · `ready`
`data.widgets`	—	O	Array	`{id, metricKey, visualization, span, compareTo, order}`, 최대 40개. `data.customized`가 `false`면 기본 배치이고 `id`는 `null`

표 7.3.24 분석 대시보드 조회의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/portfolios/7c1e4a52-93d0-4f6b-8a2e-1b5d9f0c3e71/analytics/dashboard?period=7d
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "portfolio": { "id": "7c1e4a52-93d0-4f6b-8a2e-1b5d9f0c3e71", "title": "백엔드 포트폴리오" },
    "deployment": { "id": "d3a0f6c8-2b41-4e9a-9c57-60e1b2a4f8d9", "subdomain": "minjae-backend",
"customDomain": null, "publishedAt": "2026-08-14T03:20:45.000Z" },
    "period": { "preset": "7d", "start": "2026-08-14", "end": "2026-08-20" },
    "coverage": { "days": 6, "aggregatedDays": 5, "pendingDates": ["2026-08-20"] },
    "metrics": { "visits": 24, "completes": 9, "contactClicks": 2, "fileDownloads": 1, "linkClicks": 5,
"sectionViews": 61, "sectionDwellMs": 734000 },
    "previous": null,
    "trend": [ { "date": "2026-08-15", "visits": 6, "completes": 2 } ],
    "sections": [ { "sectionId": "5e8c1a07-4d2b-4f93-a6e1-2c7b9d0f4a18", "title": "", "views": 18,
"dwellMs": 251000 } ],
    "referrers": [ { "origin": "https://www.linkedin.com", "visits": 11 }, { "origin": null, "visits":
7 } ],
    "organizations": { "entitled": false },
    "insight": { "state": "insufficient_sample", "sampleSize": 24, "minimumSample": 30 },
    "views": [],
    "customized": false,
    "widgets": [ { "id": null, "metricKey": "visits", "visualization": "number", "span": 3,
"compareTo": "prev_period", "order": 0 } ]
  }
}
```


이 API는 저장된 집계(`metricsDaily`)를 **읽기만 하고** 아무것도 쓰지 않으며, 집계되지 않은 날의 방문은 `metrics` 와 `trend` 에 들어가지 않고 `coverage.pendingDates` 에만 나타난다. 다른 사용자의 포트폴리오이거나 없으면 404, 한 번도 배포하지 않은 포트폴리오이면 409를 돌려준다. 정의되지 않은 쿼리 파라미터가 섞이면 **400**으로 거절한다.

25) 홈 화면 조회

유형 **단건 조회** · `GET /v1/home`



그림 7.3.25 홈 화면 조회의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`data.activeBrews`	—	0	Array	`{id, status, updatedAt}` . 완료되지 않은 작업, 최근 갱신 순 최대 5개
`data.portfolios`	—	0	Array	`{id, title, status}` . 제목 순 최대 10개

속성	IN	OUT	TYPE	DESCRIPTION
`data.recommendedJobs`	—	0	Array	적합도 점수가 높은 공고 최대 5개
`data.recommendedJobs.score`	—	0	number	적합도 점수
`data.recommendedJobs.companyLogoUrl`	—	0	String	최대 300자. 저장된 로고가 없으면 `null`
`data.keyMetrics`	—	0	Array	`{key, value}`. 지표 키별 합계, 키 이름 순
`data.empty`	—	0	Object	`{brews, portfolios, recommendations, metrics}`. 각 목록이 비었는지

표 7.3.25 홈 화면 조회의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/home
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "activeBrews": [ { "id": "9b2e7d14-3c58-4a61-8f0d-5e1a7c3b9d26", "status": "generating",
"updatedAt": "2026-08-14T02:41:09.000Z" } ],
    "portfolios": [ { "id": "7c1e4a52-93d0-4f6b-8a2e-1b5d9f0c3e71", "title": "백엔드 포트폴리오", "status":
"published" } ],
    "recommendedJobs": [
      {
        "id": "c6f1a830-2d9e-4b75-a4c3-8e0b6d2f1a97",
        "title": "백엔드 엔지니어 (검색)",
        "company": "예시컴퍼니",
        "score": 0.82,
        "companyLogoUrl": "/v1/companies/4a7d2e91-6c03-4f58-b1e9-0d3c5a8f7b24/logo?v=3f9a1c7e5b2d8e60"
      }
    ],
    "keyMetrics": [ { "key": "completes", "value": 9 }, { "key": "visits", "value": 24 } ],
    "empty": { "brews": false, "portfolios": false, "recommendations": false, "metrics": false }
  }
}
```

로그인한 사용자 **본인의 데이터만** 읽고 요청 파라미터는 없다. 아무것도 쓰지 않는 조회이며, `keyMetrics` 는 기간을 나누지 않고 지표 키별로 모든 일별 집계를 더한 값이다. 목록이 비어 있으면 빈 배열과 함께 `empty` 의 해당 값이 `true` 로 온다.

26) 기능 사용 권한 조회

유형 **단건 조회** · `GET /v1/entitlements/:capability`



그림 7.3.26 기능 사용 권한 조회의 요청 · 응답 방향

속성	IN	OUT	TYPE	DESCRIPTION
`capability`	O	O	String	경로 변수. `portfolio.generate` · `recipe.auto` · `brew.cowork` · `template.pro` · `analytics.full` · `analytics.organization_domains` · `analytics.returning_visitors` · `export.document` · `publishing.custom_domain` · `publishing.remove_badge` · `analysis.advanced` · `collaboration.team`
`data.planCode`	–	O	String	`free` · `pro` · `team`
`data.allowed`	–	O	bool	지금 이 기능을 쓸 수 있는지
`data.reason`	–	O	String	`ENTITLED` · `PLAN_REQUIRED` · `QUOTA_EXHAUSTED`
`data.usage`	–	O	Object	선택. `portfolio.generate`를 요금제가 허용할 때만 온다
`data.usage.periodStart`	–	O	String	한국 시간 기준 이번 달 1일, `YYYY-MM-DD`
`data.usage.resetsAt`	–	O	datetime	다음 달 1일 0시(한국 시간)
`data.usage.used`	–	O	int	이번 달 사용 횟수
`data.usage.limit` · `data.usage.remaining`	–	O	int	월 한도와 남은 횟수. 무제한이면 둘 다 `null`

표 7.3.26 기능 사용 권한 조회의 파라메터

실제 전송내용 (IN)

```
전송방향 : IN
GET /v1/entitlements/portfolio.generate
```

실제 전송내용 (OUT)

```
전송방향 : OUT
{
  "data": {
    "capability": "portfolio.generate",
    "planCode": "free",
    "allowed": true,
    "reason": "ENTITLED",
    "usage": {
      "periodStart": "2026-08-01",
      "resetsAt": "2026-08-31T15:00:00.000Z",
      "used": 2,
      "limit": 3,
      "remaining": 1
    }
  }
}
```

목록에 없는 `capability` 이면 **400**을 돌려준다. 요금제가 기능을 허용하지 않으면 `allowed: false` , `reason: PLAN_REQUIRED` 로 응답하고, `portfolio.generate` 는 이번 달 사용 횟수가 한도에 닿으면 `QUOTA_EXHAUSTED` 가 된다. 조회만 하며 **사용량을 늘리지 않는다**.

7.4 AI학습서버 REST API 설계

서비스 API와 프로세스를 분리하고, 내부 네트워크와 서비스 토큰으로 접근을 제한한다. **모델 A**는 공고를, **모델 B**는 해당 공고에 쓸 커리어 기록을 정렬한다. 학습 단위는 Bi-Encoder+MLP(`bi`)와 Cross-Encoder(`cross`) 둘이며, 모델 B는 활성 `cross` 버전을 사용한다.

METHOD	URI	설계 기능
GET	`/training/datasets`	데이터세트 목록 조회
POST	`/training/datasets`	데이터세트 생성 — 프로필–공고 묶음 구성 · 교사 라벨링 · 루브릭 점수 계산
GET	`/training/datasets/:version`	데이터세트 상세(행 수 · 라벨 분포 · 교사 모델 · 루브릭 버전 · 라벨 검사 결과)
DELETE	`/training/datasets/:version`	데이터세트 삭제

METHOD	URI	설계 기능
GET	`/training/datasets/:version/review-samples`	사람 평가 표본 조회 (교사 라벨 가림)
PUT	`/training/datasets/:version/review-samples`	사람 평가 결과 제출 · 교사–사람 일치도 산출
GET	`/training/runs`	학습 실행 목록 조회
POST	`/training/runs`	학습 실행 (인코더 비교 · Bi-Encoder+MLP · Cross-Encoder)
GET	`/training/runs/:id`	학습 진행 · 결과 조회
DELETE	`/training/runs/:id`	학습 중단
GET	`/training/runs/:id/metrics`	성능 지표 조회 (NDCG@10 · Recall@K · MAE · 순서 일치율 · ROC 좌표 · baseline 비교)
GET	`/training/models`	모델 버전 목록 조회 (`?kind=bi|cross`)
GET	`/training/models/:version`	모델 상세 조회
POST	`/training/models/:version/deployments`	모델 배포
DELETE	`/training/models/:version`	모델 삭제
GET	`/training/active-models`	활성 모델 조회 (Bi-Encoder+MLP · Cross-Encoder 각각)
PUT	`/training/active-models/:kind`	활성 모델 변경(롤백 포함)
GET	`/training/parameters`	학습 파라미터 조회
PUT	`/training/parameters`	학습 파라미터 수정
GET	`/training/teacher-prompts`	교사 프롬프트 · 루브릭 버전 조회
PUT	`/training/teacher-prompts/:kind`	교사 프롬프트 수정
GET	`/training/failures`	학습 장애기록 조회

METHOD	URI	설계 기능
POST	`/inference/match`	**모델 A** — 공고 선별 · 재정렬
POST	`/inference/record-rank`	**모델 B** — 공고에 맞는 기록 채점 · 선택 조합
GET	`/health/live`	생존 확인
GET	`/health/ready`	준비 확인(인코더 · 모델 로드 여부)

`/inference/match`는 저장된 공고 벡터로 후보를 선별한 뒤 상위 후보를 Cross-Encoder로 재정렬하여 공고별 점수 · 순위 · 근거 기록과 모델 버전을, `/inference/record-rank`는 기록별 점수와 추천 선택 조합 · 제외 사유와 모델 버전을 반환하도록 설계한다.

08 데이터 설계

MongoDB 컬렉션

8.1 데이터 모델

Expresso는 **MongoDB 8.0** 단일 노드 replica set의 `expresso` 데이터베이스에 도메인 문서를 저장한다. `packages/database/src/collections.ts` 의 접근자에 **컬렉션 82개**가 선언돼 있다. 문서 타입은 `packages/database/src/documents/` , validator와 인덱스의 기준은 `packages/database/src/collection-specs.ts` 다.

문서 식별자는 **UUID 문자열** `_id` 이고 컬렉션 사이의 참조는 `userId` , `recordId` , `jobPostingId` 같은 필드로 보존한다. 사용자 소유 문서는 `userId` 로 조회 범위를 제한한다. 독립적으로 증가하는 이력·업데이트·이벤트는 **별도 컬렉션**에 둔다.

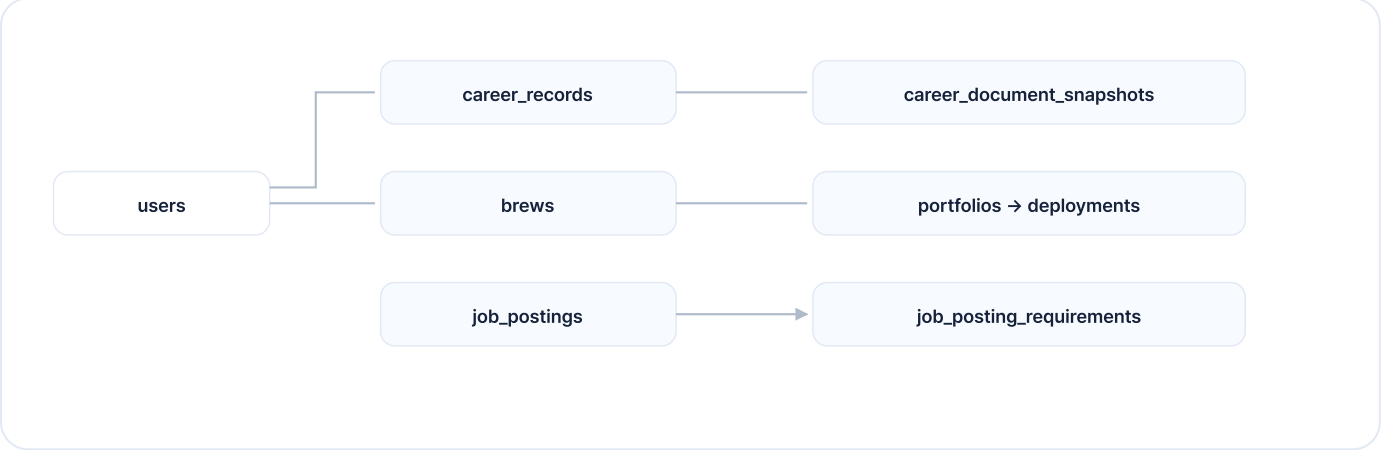


그림 8.1 핵심 컬렉션의 참조 관계. 화살표는 문서의 식별자 참조를 뜻한다.

공개 포트폴리오의 문장 근거는 `generation_sentence_evidence` 가 `recipe_evidence_paths` 를 거쳐 `career_records` 의 **원천 기록**을 가리키도록 보존한다.

8.2 논리적 설계(컬렉션 명세서)

커리어 본문은 스냅샷과 순서가 있는 업데이트로 관리한다. 저장된 뷰는 `career_views.configuration` 에 둔다. 관계·수식·롤업 결과는 커리어 기록과 관계 컬렉션에 연결한다. AI 편집 제안의 상태와 기준 버전은 `career_ai_proposals` 에 남긴다.

■ 컬렉션 목록

영역	컬렉션	개수
계정·인증	`plans` , `users` , `consents` , `usage_counters` , `identity_oauth_accounts` , `identity_sessions`	6

영역	컬렉션	개수
커리어 기록	`career_categories`, `career_records`, `career_views`, `record_links`, `record_usages`, `revisions`, `skills`, `skill_evidence`	8
커리어 편집	`career_document_snapshots`, `career_document_updates`, `career_record_revisions`, `career_record_relations`, `career_ai_proposals`	5
채용 공고	`companies`, `job_postings`, `job_posting_requirements`, `job_sources`, `recent_searches`	5
제작·매칭	`job_analyses`, `brews`, `brew_jobs`, `brew_sources`, `company_research_items`, `interests`, `match_scores`, `requirement_coverages`	8
AI 인터뷰	`interview_sessions`, `questions`, `answers`, `answer_record_changes`	4
레시피	`templates`, `recipes`, `recipe_sections`, `recipe_items`, `recipe_evidence_paths`, `recipe_revisions`, `recipe_unused_sources`	7
생성·편집	`portfolios`, `portfolio_sections`, `blocks`, `generation_jobs`, `generation_sentence_evidence`, `generation_usage_ledger`	6
지면·레이아웃	`generated_pages`, `layout_specs`, `portfolio_edit_proposals`, `portfolio_snapshots`	4
배포·미디어	`deployments`, `deployment_slug_redirects`, `export_assets`, `export_jobs`, `media_assets`, `media_variants`	6
분석	`analytics_event_receipts`, `annotations`, `visit_events`, `conversion_events`, `dashboard_views`, `derived_metrics`, `insights`, `metrics_daily`, `notifications`, `saved_searches`, `section_views`, `widgets`	12
운영·플랫폼	`account_deletion_requests`, `account_deletion_events`, `scheduled_job_definitions`, `scheduled_job_runs`, `schema_migrations`, `migration_locks`, `snapshot_chunks`, `analytics_rate_limits`, `import_runs`, `import_checkpoints`	10
비동기 이벤트	`outbox_events`	1

각 컬렉션의 필드와 인덱스는 아래 표와 같다. 표는 마이그레이션을 모두 적용한 스키마의 validator와 인덱스에서 생성한다.

계정·인증

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`code`	문자열	O	`free` · `pro` · `team` · 최대 255자
`generationQuota`	정수	O	≥ 0
`features`	객체	O	
`isPublicListed`	참거짓	O	

표 8.2.1 `plans` · 인덱스 `code ↑` (유일)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`email`	문자열	O	최대 320자
`displayName`	문자열	O	
`planId`	UUID 문자열	O	
`deletionRequestedAt`	날짜	–	null 허용
`createdAt`	날짜	O	
`passwordHash`	문자열	–	null 허용
`profile`	객체	–	null 허용
`profile.targetRoles`	임의 값	O	null 허용
`profile.experienceYears`	정수	O	≥ 0 · ≤ 12
`profile.primaryGoal`	문자열	O	`explore` · `build` · `organize`
`profile.updatedAt`	날짜	O	
`notificationPreferences`	객체	–	null 허용
`notificationPreferences.deadline`	참거짓	–	
`notificationPreferences.saved_search`	참거짓	–	
`notificationPreferences.generation`	참거짓	–	
`notificationPreferences.traffic`	참거짓	–	
`writeVersion`	정수	–	≥ 0

표 8.2.2 `users` · 인덱스 `email ↑` (유일) · `planId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`scope`	문자열	O	`job_posting_analysis` · `career_records` · 최대 255자
`policyVersion`	정수	O	≥ 0
`grantedAt`	날짜	O	
`revokedAt`	날짜	–	null 허용

표 8.2.3 `consents` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, scope ↑` (유일 · 부분) · `userId ↑, scope ↑, revokedAt ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`periodStart`	문자열	O	형식 `^\d{4}-\d{2}-\d{2}$`
`used`	정수	O	≥ 0
`resetsAt`	날짜	O	

표 8.2.4 `usage_counters` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, periodStart ↑` (유일)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`provider`	문자열	O	최대 255자
`providerAccountId`	문자열	O	최대 255자
`email`	문자열	O	최대 320자
`linkedAt`	날짜	O	
`lastLoginAt`	날짜	–	null 허용

표 8.2.5 `identity_oauth_accounts` · 인덱스 `provider ↑, providerAccountId ↑` (유일) · `userId ↑, provider ↑` (유일)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`tokenHash`	문자열	O	최대 255자
`expiresAt`	날짜	O	
`revokedAt`	날짜	–	null 허용
`lastSeenAt`	날짜	–	null 허용
`createdAt`	날짜	O	

표 8.2.6 `identity_sessions` · 인덱스 `tokenHash ↑` (유일) · `userId ↑, _id ↑` (유일) · `tokenHash ↑, expiresAt ↑` · `userId ↑, createdAt ↓` · `expiresAt ↑` (만료 0초)

■ 커리어 기록

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	–	null 허용
`key`	문자열	O	최대 255자
`isSystem`	참거짓	O	
`propertySchema`	객체	O	
`sortOrder`	정수	O	
`name`	문자열	O	

필드	타입	필수	제약
`icon`	문자열	O	
`defaultView`	문자열	O	`table` · `gallery` · `timeline` · `board` · `list`
`version`	정수	O	≥ 0
`updatedAt`	날짜	O	

표 8.2.7 `career_categories` · 인덱스 `userId ↑, key ↑` (유일 · 부분) · `key ↑` (유일 · 부분)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`categoryId`	UUID 문자열	O	
`title`	문자열	O	
`status`	문자열	O	`draft` · `organized` · `verified`
`origin`	문자열	O	`manual` · `ai` · `interview` · `import`
`properties`	객체	O	
`bodyMd`	문자열	O	
`periodStart`	문자열	–	형식 `^\d{4}-\d{2}-\d{2}$` · null 허용
`periodEnd`	문자열	–	형식 `^\d{4}-\d{2}-\d{2}$` · null 허용
`version`	정수	O	≥ 0
`updatedAt`	날짜	O	
`deletedAt`	날짜	–	null 허용
`purgeAfter`	날짜	–	null 허용
`createIdempotencyKey`	문자열	–	최대 255자 · null 허용
`createRequestHash`	문자열	–	null 허용
`referenceVersion`	정수	–	≥ 0
`documentSchemaVersion`	정수	–	≥ 0 · null 허용
`documentVersion`	정수	–	≥ 0 · null 허용
`latestSnapshotId`	문자열	–	null 허용
`computedProperties`	객체	–	null 허용
`unmappedProperties`	객체	–	null 허용
`propertyValueTombstones`	객체	–	null 허용
`editorMigratedAt`	날짜	–	null 허용

표 8.2.8 `career_records` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, createIdempotencyKey ↑` (유일 · 부분) · `userId ↑, categoryId ↑` · `categoryId ↑` · `userId ↑, deletedAt ↑, categoryId ↑, updatedAt ↓, _id ↓`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`categoryId`	UUID 문자열	O	
`name`	문자열	O	최대 255자
`viewType`	문자열	O	`table` · `gallery` · `timeline` · `board` · `list`

필드	타입	필수	제약
filters	배열	O	
sorts	배열	O	
visibleProperties	임의 값	O	null 허용
sortOrder	정수	O	≥ 0
createdAt	날짜	O	
configuration	객체	–	
version	정수	–	≥ 1
updatedAt	날짜	–	

표 8.2.9 career_views · 인덱스 userId ↑, categoryId ↑, name ↑ (유일) · userId ↑, _id ↑ (유일) · userId ↑, categoryId ↑, sortOrder ↑ · categoryId ↑ · userId ↑, categoryId ↑, configuration.order ↑, _id ↑ (부분) · userId ↑, _id ↑, configuration.version ↑ (부분)

필드	타입	필수	제약
_id	UUID 문자열	O	
userId	UUID 문자열	O	
fromRecordId	UUID 문자열	O	
toRecordId	UUID 문자열	O	
relation	문자열	O	related · parent · duplicate_of · 최대 255자
createdBy	문자열	O	user · ai

표 8.2.10 record_links · 인덱스 userId ↑, fromRecordId ↑, toRecordId ↑, relation ↑ (유일) · userId ↑, _id ↑ (유일) · userId ↑, toRecordId ↑

필드	타입	필수	제약
_id	UUID 문자열	O	
userId	UUID 문자열	O	
recordId	UUID 문자열	O	
blockId	UUID 문자열	O	
quotedText	문자열	O	
firstUsedAt	날짜	O	

표 8.2.11 record_usages · 인덱스 userId ↑, _id ↑ (유일) · userId ↑, recordId ↑, blockId ↑ (유일) · userId ↑, blockId ↑

필드	타입	필수	제약
_id	UUID 문자열	O	
userId	UUID 문자열	O	
portfolioId	UUID 문자열	O	
blockId	UUID 문자열	–	null 허용
actor	문자열	O	user · ai
before	임의 값	–	null 허용
after	임의 값	–	null 허용

필드	타입	필수	제약
`restoreLabel`	문자열	–	null 허용
`proposalId`	UUID 문자열	–	null 허용
`revertedRevisionId`	UUID 문자열	–	null 허용
`changeKind`	문자열	O	`edit` · `revert` · `restore`
`summary`	문자열	O	
`createdAt`	날짜	O	

표 8.2.12 `revisions` · 인덱스 `userId ↑, _id ↑` (유일) · `blockId ↑` · `proposalId ↑` · `revertedRevisionId ↑` · `userId ↑, portfolioId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`name`	문자열	O	최대 255자
`level`	정수	O	≥ 1 · ≤ 5
`computedAt`	날짜	O	
`demandScore`	실수	–	null 허용
`evidenceCount`	정수	O	≥ 0
`lastUsedAt`	날짜	–	null 허용
`strength`	문자열	O	`weak` · `supported` · `strong`

표 8.2.13 `skills` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, name ↑` (유일)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`skillId`	UUID 문자열	O	
`recordId`	UUID 문자열	O	
`weight`	실수	O	
`extractedSpan`	객체	O	

표 8.2.14 `skill_evidence` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, skillId ↑, recordId ↑` (유일) · `userId ↑, recordId ↑`

■ 커리어 편집

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`recordId`	UUID 문자열	O	
`documentVersion`	정수	O	≥ 0
`version`	정수	–	≥ 0
`schemaVersion`	정수	O	≥ 0

필드	타입	필수	제약
`content`	임의 값	O	
`stateVector`	이진	O	
`serverSequence`	정수	O	≥ 0
`checksum`	문자열	O	형식 `^[a-f0-9]{64}$`
`actor`	문자열	O	`user` · `ai` · `migration`
`createdAt`	날짜	O	

표 8.2.15 `career_document_snapshots` · 인덱스 `recordId ↑, version ↓`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`recordId`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`clientId`	문자열	O	최대 128자 · 최소 1자
`clientSequence`	정수	O	≥ 0
`serverSequence`	정수	O	≥ 0
`update`	이진	O	
`byteLength`	정수	O	≥ 0 · ≤ 1048576
`updateHash`	문자열	O	형식 `^[a-f0-9]{64}$`
`actor`	문자열	O	`user` · `ai` · `migration`
`receivedAt`	날짜	O	
`compactedAt`	날짜	O	null 허용

표 8.2.16 `career_document_updates` · 인덱스 `recordId ↑, clientId ↑, clientSequence ↑` (유일) · `recordId ↑, serverSequence ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`recordId`	UUID 문자열	O	
`actor`	문자열	O	`user` · `ai` · `migration`
`summary`	문자열	O	최대 2000자
`beforeVersion`	정수	O	≥ 0
`afterVersion`	정수	O	≥ 0
`snapshotId`	문자열	–	null 허용
`proposalId`	문자열	–	null 허용
`expiresAt`	날짜	–	null 허용
`createdAt`	날짜	O	

표 8.2.17 `career_record_revisions` · 인덱스 `recordId ↑, createdAt ↓`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`sourceRecordId`	UUID 문자열	O	
`sourcePropertyId`	UUID 문자열	O	
`targetRecordId`	UUID 문자열	O	
`inversePropertyId`	문자열	—	null 허용
`cardinality`	문자열	O	`single` · `multiple`
`deletePolicy`	문자열	O	`restrict` · `nullify`
`createdBy`	문자열	O	`user` · `ai`
`createdAt`	날짜	O	
`updatedAt`	날짜	O	

표 8.2.18 `career_record_relations` · 인덱스 `userId ↑, sourceRecordId ↑, sourcePropertyId ↑, targetRecordId ↑` (유일) · `userId ↑, targetRecordId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`recordId`	UUID 문자열	O	
`status`	문자열	O	`draft` · `streaming` · `ready` · `applied` · `rejected` · `cancelled` · `expired` · `conflicted`
`baseDocumentVersion`	정수	O	≥ 0
`selection`	객체	O	
`prompt`	문자열	O	최대 4000자
`summary`	문자열	O	최대 2000자 · null 허용
`commands`	배열	O	최대 100개
`propertyChanges`	배열	O	최대 50개
`progress`	객체	O	null 허용
`beforeSnapshotId`	문자열	O	null 허용
`afterSnapshotId`	문자열	O	null 허용
`beforeChecksum`	문자열	O	null 허용
`afterChecksum`	문자열	O	null 허용
`revisionId`	문자열	O	null 허용
`appliedDocumentVersion`	정수	O	≥ 0 · null 허용
`expiresAt`	날짜	O	
`createdAt`	날짜	O	
`updatedAt`	날짜	O	

표 8.2.19 `career_ai_proposals` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, recordId ↑, status ↑` · `expiresAt ↑` (만료 0초)

채용 공고

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`name`	문자열	O	
`domain`	문자열	–	null 허용
`industry`	문자열	–	null 허용
`toneSummary`	문자열	–	null 허용
`dedupeKey`	문자열	–	최대 255자 · null 허용
`tonePalette`	객체	–	null 허용
`avatarBackground`	문자열	–	null 허용
`avatarColor`	문자열	–	null 허용
`brandColors`	임의 값	O	null 허용
`toneImpression`	문자열	–	null 허용
`initial`	문자열	–	null 허용
`logoData`	이진	–	null 허용
`logoMediaType`	문자열	–	null 허용
`logoSourceUrl`	문자열	–	null 허용
`logoChecksum`	문자열	–	null 허용
`logoReadAt`	날짜	–	null 허용
`importRunId`	문자열	–	최대 64자
`sourceHash`	문자열	–	형식 `^[a-f0-9]{64}$` · 최대 64자 · 최소 64자

표 8.2.20 `companies` · 인덱스 `dedupeKey ↑` (유일 · 부분) · `importRunId ↑, _id ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`companyId`	UUID 문자열	O	
`source`	문자열	O	`api` · `partner` · `user_input` · 최대 255자
`externalId`	문자열	–	최대 255자 · null 허용
`title`	문자열	O	
`descriptionRaw`	문자열	O	
`requirements`	임의 값	O	null 허용
`expiresAt`	날짜	–	null 허용
`dedupeHash`	문자열	O	최대 255자
`sourceUrl`	문자열	–	null 허용
`createdAt`	날짜	O	
`normalizedAt`	날짜	–	null 허용
`location`	문자열	–	null 허용
`workType`	문자열	–	null 허용
`experienceLabel`	문자열	–	null 허용
`employmentType`	문자열	–	null 허용
`salaryNote`	문자열	–	null 허용

필드	타입	필수	제약
`jobFamily`	문자열	–	최대 255자 · null 허용
`duties`	배열	O	
`preferred`	배열	O	
`hiringProcess`	배열	O	
`processNote`	문자열	–	null 허용
`notice`	문자열	–	null 허용
`team`	문자열	–	null 허용
`deadlineNote`	문자열	–	null 허용
`sourceBoard`	문자열	–	null 허용
`locationRegion`	문자열	–	최대 255자 · null 허용
`experienceNote`	문자열	–	null 허용
`factsReadAt`	날짜	–	null 허용
`experienceMinYears`	정수	–	null 허용
`importRunId`	문자열	–	최대 64자
`sourceHash`	문자열	–	형식 `^[a-f0-9]{64}$` · 최대 64자 · 최소 64자

표 8.2.21 `job_postings` · 인덱스 `dedupeHash ↑` (유일) · `source ↑, externalId ↑` (유일 · 부분) · `experienceMinYears ↑` · `expiresAt ↑` · `createdAt ↑` · `jobFamily ↑, locationRegion ↑` · `jobFamily ↑` · `locationRegion ↑` · `companyId ↑` · `importRunId ↑, _id ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`jobPostingId`	UUID 문자열	O	
`orderNo`	정수	O	≥ 0
`label`	문자열	O	
`kind`	문자열	O	`must` · `nice` · `tone`
`sourceSpan`	객체	O	
`extractorVersion`	정수	O	≥ 0
`extractedAt`	날짜	O	
`axis`	문자열	–	`technology` · `impact` · `role` · `conditions` · `null` · 최대 255자 · null 허용
`importRunId`	문자열	–	최대 64자
`sourceHash`	문자열	–	형식 `^[a-f0-9]{64}$` · 최대 64자 · 최소 64자

표 8.2.22 `job_posting_requirements` · 인덱스 `jobPostingId ↑, axis ↑` · `jobPostingId ↑, orderNo ↑` · `importRunId ↑, _id ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`provider`	문자열	O	`greenhouse` · `lever` · `ashby` · `workable` · `greeting` · `work24` · `work24web` · 최대 255자
`token`	문자열	O	최대 255자
`displayName`	문자열	O	

필드	타입	필수	제약
`isActive`	참거짓	O	
`lastRunAt`	날짜	–	null 허용
`lastStatus`	문자열	–	`succeeded` · `failed` · `null` · null 허용
`lastError`	문자열	–	null 허용
`lastSeenCount`	정수	O	≥ 0
`lastAddedCount`	정수	O	≥ 0
`createdAt`	날짜	O	
`siteUrl`	문자열	–	null 허용
`importRunId`	문자열	–	최대 64자
`sourceHash`	문자열	–	형식 `^[a-f0-9]{64}$` · 최대 64자 · 최소 64자

표 8.2.23 `job_sources` · 인덱스 `provider ↑, token ↑` (유일) · `provider ↑, token ↑` · `importRunId ↑, _id ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`queryText`	문자열	O	
`conditions`	배열	O	
`resultCount`	정수	O	≥ 0
`createdAt`	날짜	O	

표 8.2.24 `recent_searches` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, createdAt ↓`

■ 제작·매칭

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`jobPostingId`	UUID 문자열	–	null 허용
`inputType`	문자열	O	`url` · `paste` · `file` · `board` · `free`
`status`	문자열	O	`queued` · `running` · `done` · `failed` · 최대 255자
`attachments`	배열	O	
`analyzedAt`	날짜	–	null 허용
`inputIdempotencyKey`	문자열	–	최대 255자 · null 허용
`inputRequestHash`	문자열	–	null 허용
`progressStage`	문자열	O	`queued` · `extracting` · `validating` · `covering` · `done` · `failed`
`attempts`	정수	O	≥ 0
`resultVersion`	정수	O	≥ 0
`targetVersion`	정수	O	≥ 0
`failureCode`	문자열	–	null 허용
`failureRetryable`	참거짓	–	null 허용

필드	타입	필수	제약
`history`	객체	–	null 허용
`history.userId`	UUID 문자열	O	
`history.previousVersion`	정수	O	≥ 0
`history.requirements`	배열	O	
`history.archivedAt`	날짜	O	

표 8.2.25 `job_analyses` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, inputIdempotencyKey ↑` (유일 · 부분) · `userId ↑, status ↑` · `jobPostingId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`jobAnalysisId`	UUID 문자열	O	
`freeTitle`	문자열	–	최대 300자 · null 허용
`freeBrief`	문자열	–	null 허용
`mode`	문자열	O	`solo` · `collab`
`lengthPreset`	문자열	O	`single` · `double` · `triple`
`status`	문자열	O	`draft` · `interviewing` · `recipe` · `generating` · `done` · 최대 255자
`deadlineAt`	날짜	–	null 허용
`resumedAt`	날짜	–	null 허용
`createdAt`	날짜	O	
`updatedAt`	날짜	O	

표 8.2.26 `brews` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, status ↑` · `userId ↑, jobAnalysisId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`type`	문자열	O	`interview` · `recipe` · 최대 255자
`status`	문자열	O	`queued` · `running` · `succeeded` · `failed`
`stage`	문자열	O	
`attempts`	정수	O	≥ 0
`input`	객체	O	
`resultId`	UUID 문자열	–	null 허용
`errorCode`	문자열	–	null 허용
`failureRetryable`	참거짓	–	null 허용
`inputIdempotencyKey`	문자열	–	최대 255자 · null 허용
`createdAt`	날짜	O	
`updatedAt`	날짜	O	

표 8.2.27 `brew_jobs` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, inputIdempotencyKey ↑` (유일 · 부분) · `userId ↑, type ↑, createdAt ↓`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`brewId`	UUID 문자열	O	
`recordId`	UUID 문자열	O	
`rank`	정수	O	≥ 0
`selectedBy`	문자열	O	`auto` · `user`
`excludedReason`	문자열	–	null 허용
`score`	정수	O	≥ 0
`reasonText`	문자열	O	
`isSelected`	참거짓	O	
`updatedAt`	날짜	O	

표 8.2.28 `brew_sources` · 인덱스 `userId ↑, brewId ↑, recordId ↑` (유일) · `userId ↑, _id ↑` (유일) · `userId ↑, brewId ↑, rank ↑` (유일 · 부분) · `userId ↑, recordId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`companyId`	UUID 문자열	O	
`kind`	문자열	O	`fact` · `signal` · 최대 255자
`topic`	문자열	O	
`statement`	문자열	O	
`sourceUrl`	문자열	–	null 허용
`publishedAt`	날짜	–	null 허용
`capturedAt`	날짜	O	
`confidence`	문자열	O	`low` · `medium` · `high`
`basisFactIds`	임의 값	O	null 허용

표 8.2.29 `company_research_items` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, companyId ↑, kind ↑, capturedAt ↓` · `companyId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`jobPostingId`	UUID 문자열	O	
`stage`	문자열	O	`saved` · `applied` · `closed`
`deadlineAt`	날짜	–	null 허용
`memo`	문자열	–	null 허용
`updatedAt`	날짜	O	

표 8.2.30 `interests` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, jobPostingId ↑` (유일) · `jobPostingId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`jobPostingId`	UUID 문자열	O	
`total`	실수	O	
`axes`	객체	O	
`reasonText`	문자열	O	
`computedAt`	날짜	O	
`nextAction`	문자열	O	

표 8.2.31 `match_scores` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, jobPostingId ↑` (유일) · `jobPostingId ↑`

필드	타입	필수	제약
`userId`	UUID 문자열	O	
`requirementId`	UUID 문자열	O	
`coverage`	문자열	O	`covered` · `partial` · `missing`
`coveredBy`	임의 값	O	null 허용
`computedAt`	날짜	O	
`_id`	문자열	O	

표 8.2.32 `requirement_coverages` · 인덱스 `userId ↑` · `requirementId ↑`

AI 인터뷰

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`brewId`	UUID 문자열	O	
`status`	문자열	O	`open` · `paused` · `done`
`questionCount`	정수	O	≥ 0
`transcriptUrl`	문자열	–	null 허용
`inputIdempotencyKey`	문자열	–	최대 255자 · null 허용
`currentOrder`	정수	O	≥ 0
`answeredCount`	정수	O	≥ 0
`pausedAt`	날짜	–	null 허용
`createdAt`	날짜	O	
`updatedAt`	날짜	O	

표 8.2.33 `interview_sessions` · 인덱스 `userId ↑, brewId ↑` (유일) · `userId ↑, _id ↑` (유일) · `userId ↑, inputIdempotencyKey ↑` (유일 · 부분)

필드	타입	필수	제약
`_id`	UUID 문자열	O	

필드	타입	필수	제약
`userId`	UUID 문자열	O	
`interviewSessionId`	UUID 문자열	O	
`requirementId`	UUID 문자열	—	null 허용
`replacedFromId`	UUID 문자열	—	null 허용
`orderNo`	정수	O	≥ 0
`text`	문자열	O	
`skipped`	참거짓	O	
`basis`	객체	O	
`active`	참거짓	O	
`variant`	정수	O	≥ 0
`createdAt`	날짜	O	
`rationale`	문자열	—	null 허용

표 8.2.34 `questions` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, interviewSessionId ↑, orderNo ↑` (유일 · 부분) · `replacedFromId ↑` · `requirementId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`questionId`	UUID 문자열	O	
`inputType`	문자열	O	`text` · `voice`
`transcript`	문자열	O	
`createdRecordId`	UUID 문자열	—	null 허용
`inputIdempotencyKey`	문자열	—	최대 255자 · null 허용
`requestHash`	문자열	—	null 허용
`version`	정수	O	≥ 0
`updatedAt`	날짜	O	

표 8.2.35 `answers` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, questionId ↑` (유일) · `userId ↑, inputIdempotencyKey ↑` (유일 · 부분) · `createdRecordId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`answerId`	UUID 문자열	O	
`recordId`	UUID 문자열	O	
`changeType`	문자열	O	`created` · `strengthened`
`changedFields`	배열	O	
`sourceQuote`	문자열	O	
`createdAt`	날짜	O	

표 8.2.36 `answer_record_changes` · 인덱스 `userId ↑, answerId ↑` (유일) · `userId ↑, _id ↑` (유일) · `userId ↑, recordId ↑`

레시피

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`code`	문자열	O	최대 255자
`name`	문자열	O	
`toneTags`	임의 값	O	null 허용
`supportedSections`	임의 값	O	null 허용
`planRequired`	문자열	O	`free` · `pro`
`description`	문자열	O	
`rendererVersion`	정수	O	≥ 0
`style`	객체	O	
`industries`	임의 값	O	null 허용
`isActive`	참거짓	O	

표 8.2.37 `templates` · 인덱스 `code ↑` (유일)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`brewId`	UUID 문자열	O	
`version`	정수	O	≥ 0
`status`	문자열	O	`draft` · `confirmed`
`completeness`	실수	O	
`generatedAt`	날짜	O	
`inputIdempotencyKey`	문자열	–	최대 255자 · null 허용
`updatedAt`	날짜	O	
`promptVersion`	정수	O	≥ 0
`portfolioPlan`	객체	–	null 허용
`planningManifest`	객체	–	null 허용

표 8.2.38 `recipes` · 인덱스 `userId ↑, brewId ↑, version ↑` (유일) · `userId ↑, _id ↑` (유일) · `userId ↑, inputIdempotencyKey ↑` (유일 · 부분)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`recipeId`	UUID 문자열	O	
`orderNo`	정수	O	≥ 0
`title`	문자열	O	
`purpose`	문자열	O	
`targetLength`	정수	O	≥ 0
`context`	객체	O	

필드	타입	필수	제약
`locked`	참거짓	O	
`editedBy`	문자열	O	`ai` · `user`
`updatedAt`	날짜	O	

표 8.2.39 `recipe_sections` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, recipeId ↑, orderNo ↑` (유일)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`recipeSectionId`	UUID 문자열	O	
`orderNo`	정수	O	≥ 0
`pointText`	문자열	O	
`evidence`	배열	O	
`locked`	참거짓	O	
`editedBy`	문자열	O	`ai` · `user`
`updatedAt`	날짜	O	

표 8.2.40 `recipe_items` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, recipeSectionId ↑, orderNo ↑` (유일)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`recipeId`	UUID 문자열	O	
`recipeItemId`	UUID 문자열	O	
`sourceType`	문자열	O	`requirement` · `record` · `answer` · 최대 255자
`sourceId`	UUID 문자열	O	
`sourceLabel`	문자열	O	
`targetPath`	문자열	O	
`createdAt`	날짜	O	

표 8.2.41 `recipe_evidence_paths` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, recipeId ↑, sourceType ↑, sourceId ↑, recipeItemId ↑` (유일) · `userId ↑, recipeItemId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`recipeId`	UUID 문자열	O	
`actor`	문자열	O	`ai` · `user`
`action`	문자열	O	
`snapshot`	객체	O	
`diff`	배열	O	
`createdAt`	날짜	O	

표 8.2.42 `recipe_revisions` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, recipeId ↑, createdAt ↓, _id ↓`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`recipeId`	UUID 문자열	O	
`recordId`	UUID 문자열	O	
`reason`	문자열	O	
`createdAt`	날짜	O	

표 8.2.43 `recipe_unused_sources` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, recipeId ↑, recordId ↑` (유일) · `userId ↑, recordId ↑`

■ 생성·편집

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`brewId`	UUID 문자열	O	
`templateId`	UUID 문자열	O	
`currentDeploymentId`	UUID 문자열	—	null 허용
`title`	문자열	O	
`status`	문자열	O	`draft` · `published` · `unlisted` · 최대 255자
`createdAt`	날짜	O	
`updatedAt`	날짜	O	
`styleOverrides`	객체	O	

표 8.2.44 `portfolios` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, status ↑` · `userId ↑, updatedAt ↓, _id ↓` · `currentDeploymentId ↑` · `templateId ↑` · `userId ↑, brewId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`recipeSectionId`	UUID 문자열	—	null 허용
`orderNo`	정수	O	≥ 0
`visible`	참거짓	O	
`hiddenReason`	문자열	—	null 허용

표 8.2.45 `portfolio_sections` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, portfolioId ↑, orderNo ↑` (유일) · `userId ↑, portfolioId ↑, recipeSectionId ↑` (유일 · 부분) · `recipeSectionId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioSectionId`	UUID 문자열	O	
`kind`	문자열	O	`heading` · `paragraph` · `list` · `metric` · `chart` · `media`
`content`	객체	O	
`style`	객체	O	
`sourceRecordId`	UUID 문자열	–	null 허용
`syncState`	문자열	O	`synced` · `stale` · `detached`
`locked`	참거짓	O	
`orderNo`	정수	O	≥ 0

표 8.2.46 `blocks` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, portfolioSectionId ↑, orderNo ↑, _id ↑` · `userId ↑, sourceRecordId ↑` · `sourceRecordId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`brewId`	UUID 문자열	O	
`recipeId`	UUID 문자열	O	
`templateId`	UUID 문자열	O	
`status`	문자열	O	`queued` · `running` · `done` · `failed` · 최대 255자
`usageCharged`	참거짓	O	
`errorCode`	문자열	–	null 허용
`inputIdempotencyKey`	문자열	–	최대 255자 · null 허용
`requestHash`	문자열	–	null 허용
`stage`	문자열	O	`queued` · `validating` · `materializing` · `charging` · `done` · `failed`
`attempts`	정수	O	≥ 0
`failureRetryable`	참거짓	–	null 허용
`portfolioId`	UUID 문자열	–	null 허용
`createdAt`	날짜	O	
`updatedAt`	날짜	O	
`styleOverrides`	객체	O	

표 8.2.47 `generation_jobs` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, inputIdempotencyKey ↑` (유일 · 부분) · `userId ↑, status ↑` · `portfolioId ↑` · `templateId ↑` · `userId ↑, brewId ↑` · `userId ↑, recipeId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`generationJobId`	UUID 문자열	O	

필드	타입	필수	제약
`blockId`	UUID 문자열	O	
`recipeEvidencePathId`	UUID 문자열	O	
`sourceQuote`	문자열	O	
`createdAt`	날짜	O	

표 8.2.48 `generation_sentence_evidence` · 인덱스 `userId ↑, generationJobId ↑, blockId ↑, recipeEvidencePathId ↑` (유일) · `userId ↑, _id ↑` (유일) · `recipeEvidencePathId ↑` · `userId ↑, blockId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`generationJobId`	UUID 문자열	O	
`usageCounterId`	UUID 문자열	–	null 허용
`amount`	정수	O	`-1` · `1`
`reason`	문자열	O	최대 255자
`createdAt`	날짜	O	

표 8.2.49 `generation_usage_ledger` · 인덱스 `userId ↑, generationJobId ↑, reason ↑` (유일) · `userId ↑, _id ↑` (유일) · `userId ↑, usageCounterId ↑`

■ 지면·레이아웃

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`generationJobId`	UUID 문자열	–	null 허용
`html`	문자열	O	
`css`	문자열	O	
`rationale`	문자열	O	
`revision`	정수	O	≥ 0
`instruction`	문자열	–	null 허용
`promptVersion`	정수	O	
`ungroundedNumbers`	임의 값	O	null 허용
`removed`	임의 값	O	null 허용
`createdAt`	날짜	O	
`qualityStatus`	문자열	O	`ready` · `failed_qa`
`qaReport`	객체	O	
`generationManifest`	객체	O	
`portfolioPlanSnapshot`	객체	–	null 허용
`styleSpecSnapshot`	객체	–	null 허용
`designPrinciplesVersion`	정수	O	≥ 0

표 8.2.50 `generated_pages` · 인덱스 `portfolioId ↑, revision ↑` (유일) · `userId ↑, _id ↑` (유일) · `generationJobId ↑` (유일 · 부분) · `portfolioId ↑, revision ↓` · `userId ↑, portfolioId ↑` · `userId ↑, generationJobId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`batchId`	UUID 문자열	O	
`generationJobId`	UUID 문자열	—	null 허용
`seedTemplateId`	UUID 문자열	—	null 허용
`spec`	객체	O	
`promptVersion`	정수	O	≥ 0
`editedBy`	문자열	O	`ai` · `user`
`orderNo`	정수	O	≥ 0
`selected`	참거짓	O	
`createdAt`	날짜	O	
`instruction`	문자열	—	null 허용

표 8.2.51 `layout_specs` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, portfolioId ↑` (유일 · 부분) · `userId ↑, portfolioId ↑, createdAt ↓, batchId ↑, orderNo ↑` · `generationJobId ↑` · `seedTemplateId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`targetPath`	문자열	O	
`blockId`	UUID 문자열	O	
`operation`	문자열	O	`update_text` · `set_style` · `insert_record` · `instruct`
`beforeState`	객체	O	
`afterState`	객체	O	
`sourceRecordId`	UUID 문자열	—	null 허용
`status`	문자열	O	`pending` · `applied` · `rejected` · `expired` · 최대 255자
`createdAt`	날짜	O	
`expiresAt`	날짜	O	
`appliedAt`	날짜	—	null 허용
`patches`	배열	O	
`instruction`	문자열	—	null 허용

표 8.2.52 `portfolio_edit_proposals` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, portfolioId ↑, status ↑, expiresAt ↑` · `userId ↑, blockId ↑` · `userId ↑, sourceRecordId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`kind`	문자열	O	`initial_generation` · `edit` · `manual`
`snapshot`	객체	O	
`createdAt`	날짜	O	

표 8.2.53 `portfolio_snapshots` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, portfolioId ↑, createdAt ↓, _id ↓`

■ 배포·미디어

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`version`	정수	O	≥ 0
`subdomain`	문자열	O	최대 255자
`customDomain`	문자열	–	최대 255자 · null 허용
`seoIndexable`	참거짓	O	
`contactVisibility`	문자열	O	`public` · `on_request` · `hidden`
`publishedAt`	날짜	–	null 허용
`hasUnpublishedChanges`	참거짓	O	
`snapshot`	객체	O	
`seo`	객체	O	

표 8.2.54 `deployments` · 인덱스 `subdomain ↑` (유일) · `userId ↑, _id ↑` (유일) · `userId ↑, portfolioId ↑, version ↑` (유일) · `customDomain ↑` (유일 · 부분) · `userId ↑, portfolioId ↑, version ↓`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`oldSlug`	문자열	O	최대 255자
`newSlug`	문자열	O	
`createdAt`	날짜	O	
`expiresAt`	날짜	O	

표 8.2.55 `deployment_slug_redirects` · 인덱스 `oldSlug ↑` (유일) · `oldSlug ↑, expiresAt ↑` · `userId ↑, portfolioId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	

필드	타입	필수	제약
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`kind`	문자열	O	`pdf` · `deck` · `resume_file` · 최대 255자
`fileUrl`	문자열	O	
`pageFormat`	문자열	—	`letter` · `a4` · `null` · null 허용
`downloadCount`	정수	O	≥ 0
`version`	정수	O	≥ 0
`accessNonce`	UUID 문자열	O	
`revokedAt`	날짜	—	null 허용
`createdAt`	날짜	O	

표 8.2.56 `export_assets` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, portfolioId ↑` (유일 · 부분) · `userId ↑, portfolioId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`deploymentId`	UUID 문자열	—	null 허용
`kind`	문자열	O	`pdf` · `deck`
`pageFormat`	문자열	—	`letter` · `a4` · `null` · null 허용
`status`	문자열	O	`queued` · `running` · `done` · `failed`
`attempts`	정수	O	≥ 0
`idempotencyKey`	문자열	O	최대 255자
`requestHash`	문자열	O	
`assetId`	UUID 문자열	—	null 허용
`errorCode`	문자열	—	null 허용
`createdAt`	날짜	O	
`updatedAt`	날짜	O	

표 8.2.57 `export_jobs` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, idempotencyKey ↑` (유일) · `assetId ↑` · `deploymentId ↑` · `userId ↑, portfolioId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`storageKey`	문자열	O	
`mimeType`	문자열	O	`image/png` · `image/jpeg` · `image/webp` · `image/gif`
`width`	정수	O	≥ 0
`height`	정수	O	≥ 0
`byteSize`	정수	O	≥ 0
`checksum`	문자열	O	최대 255자

필드	타입	필수	제약
`createdAt`	날짜	O	

표 8.2.58 `media_assets` · 인덱스 `userId ↑, checksum ↑` (유일) · `userId ↑, _id ↑` (유일) · `userId ↑, createdAt ↓`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`mediaAssetId`	UUID 문자열	O	
`storageKey`	문자열	O	
`mimeType`	문자열	O	
`width`	정수	O	≥ 0
`height`	정수	O	≥ 0
`byteSize`	정수	O	≥ 0
`createdAt`	날짜	O	

표 8.2.59 `media_variants` · 인덱스 `mediaAssetId ↑, width ↑` (유일) · `userId ↑, _id ↑` (유일) · `mediaAssetId ↑, width ↑` · `userId ↑, mediaAssetId ↑`

분석

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`deploymentId`	UUID 문자열	O	
`eventType`	문자열	O	`visit` · `complete` · `section_view` · `contact_click` · `file_download` · `link_click`
`visitorHash`	문자열	O	최대 64자 · 최소 64자
`payloadHash`	문자열	O	최대 64자 · 최소 64자
`payloadBytes`	정수	O	≥ 1 · ≤ 8192
`occurredAt`	날짜	O	
`receivedAt`	날짜	O	

표 8.2.60 `analytics_event_receipts` · 인덱스 `userId ↑, _id ↑` (유일) · `visitorHash ↑, receivedAt ↓` · `userId ↑, deploymentId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`date`	문자열	O	형식 `^\d{4}-\d{2}-\d{2}$`
`label`	문자열	O	
`note`	문자열	O	

표 8.2.61 `annotations` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, portfolioId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`deploymentId`	UUID 문자열	O	
`sessionId`	문자열	O	
`referrer`	문자열	–	null 허용
`orgDomain`	문자열	–	null 허용
`isOwner`	참거짓	O	
`startedAt`	날짜	O	
`eventId`	UUID 문자열	–	null 허용
`completed`	참거짓	O	
`durationMs`	정수	–	≥ 0 · null 허용

표 8.2.62 `visit_events` · 인덱스 `userId ↑, _id ↑` (유일) · `eventId ↑` (유일 · 부분) · `userId ↑, deploymentId ↑, startedAt ↓`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`visitEventId`	UUID 문자열	O	
`kind`	문자열	O	`contact_click` · `file_download` · `link_click`
`target`	문자열	O	
`eventId`	UUID 문자열	–	null 허용
`occurredAt`	날짜	O	

표 8.2.63 `conversion_events` · 인덱스 `userId ↑, _id ↑` (유일) · `eventId ↑` (유일 · 부분) · `userId ↑, visitEventId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`portfolioId`	UUID 문자열	O	
`name`	문자열	O	최대 255자
`period`	문자열	O	`7d` · `30d` · `all`
`isDefault`	참거짓	O	

표 8.2.64 `dashboard_views` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, portfolioId ↑, name ↑` (유일) · `userId ↑, portfolioId ↑` (유일 · 부분)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`name`	문자열	O	최대 255자
`numeratorKey`	문자열	O	
`denominatorKey`	문자열	O	

표 8.2.65 `derived_metrics` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, name ↑` (유일)

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`deploymentId`	UUID 문자열	O	
`periodStart`	문자열	–	형식 `^\d{4}-\d{2}-\d{2}$` · null 허용
`periodEnd`	문자열	–	형식 `^\d{4}-\d{2}-\d{2}$` · null 허용
`narrative`	문자열	O	
`evidenceMetrics`	임의 값	O	null 허용
`suggestions`	배열	O	
`generatedAt`	날짜	O	

표 8.2.66 `insights` · 인덱스 `userId ↑, _id ↑` (유일) · `userId ↑, deploymentId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`deploymentId`	UUID 문자열	O	
`date`	문자열	O	형식 `^\d{4}-\d{2}-\d{2}$`
`metricKey`	문자열	O	최대 255자
`value`	실수	O	
`sampleSize`	정수	O	≥ 0

표 8.2.67 `metrics_daily` · 인덱스 `userId ↑, deploymentId ↑, date ↑, metricKey ↑` (유일) · `userId ↑, _id ↑` (유일) · `userId ↑, deploymentId ↑, metricKey ↑, date ↓`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	O	
`kind`	문자열	O	`deadline` · `saved_search` · `generation` · `traffic`
`targetUrl`	문자열	O	
`dedupeKey`	문자열	O	최대 255자
`readAt`	날짜	–	null 허용
`dedupeDate`	문자열	O	형식 `^\d{4}-\d{2}-\d{2}$`

필드	타입	필수	제약
deliveryStatus	문자열	O	queued · sending · sent · failed · suppressed · 최대 255자
attempts	정수	O	≥ 0
nextAttemptAt	날짜	O	
lastError	문자열	–	null 허용
createdAt	날짜	O	
deliveredAt	날짜	–	null 허용

표 8.2.68 notifications · 인덱스 userId ↑, _id ↑ (유일) · userId ↑, dedupeKey ↑, dedupeDate ↑ (유일) · deliveryStatus ↑, nextAttemptAt ↑

필드	타입	필수	제약
_id	UUID 문자열	O	
userId	UUID 문자열	O	
queryText	문자열	O	
filters	객체	O	
notify	참거짓	O	
lastRunAt	날짜	–	null 허용
name	문자열	O	최대 255자
createdAt	날짜	O	
updatedAt	날짜	O	

표 8.2.69 saved_searches · 인덱스 userId ↑, _id ↑ (유일) · userId ↑, name ↑ (유일)

필드	타입	필수	제약
_id	UUID 문자열	O	
userId	UUID 문자열	O	
visitEventId	UUID 문자열	O	
portfolioSectionId	UUID 문자열	O	
dwellMs	정수	O	≥ 0
scrollDepth	실수	O	≥ 0 · ≤ 1
exited	참거짓	O	
eventId	UUID 문자열	–	null 허용
occurredAt	날짜	O	

표 8.2.70 section_views · 인덱스 userId ↑, _id ↑ (유일) · eventId ↑ (유일 · 부분) · userId ↑, portfolioSectionId ↑ · userId ↑, visitEventId ↑

필드	타입	필수	제약
_id	UUID 문자열	O	
userId	UUID 문자열	O	
dashboardViewId	UUID 문자열	O	

필드	타입	필수	제약
`metricKey`	문자열	–	null 허용
`derivedMetricId`	UUID 문자열	–	null 허용
`visualization`	문자열	O	`number` · `spark` · `line` · `bar` · `donut` · `list` · `note`
`compareTo`	문자열	–	`prev_period` · `other_portfolio` · `industry` · `null` · null 허용
`position`	객체	O	

표 8.2.71 `widgets` · 인덱스 `userId ↑, _id ↑` (유일) · `derivedMetricId ↑` · `userId ↑, dashboardViewId ↑`

■ 운영·플랫폼

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`userId`	UUID 문자열	–	null 허용
`subjectId`	UUID 문자열	O	
`status`	문자열	O	`pending` · `cancelled` · `purged` · 최대 255자
`requestedAt`	날짜	O	
`purgeAfter`	날짜	O	
`cancelledAt`	날짜	–	null 허용
`purgedAt`	날짜	–	null 허용
`cancellationTokenHash`	문자열	O	최대 64자 · 최소 64자
`restoration`	객체	O	
`phase`	문자열	O	

표 8.2.72 `account_deletion_requests` · 인덱스 `subjectId ↑` (유일 · 부분) · `userId ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`requestId`	UUID 문자열	O	
`phase`	문자열	O	최대 255자
`affectedRows`	정수	O	≥ 0
`occurredAt`	날짜	O	

표 8.2.73 `account_deletion_events` · 인덱스 `requestId ↑, phase ↑` (유일)

필드	타입	필수	제약
`_id`	문자열	O	`saved_searches` · `expire_postings` · `notification_batch` · `analytics_daily` · `deletion_grace` · `retention` · `job_ingest` · `posting_facts` · 최대 255자
`intervalSeconds`	정수	O	≥ 60 · ≤ 604800
`nextRunAt`	날짜	O	
`lastStartedAt`	날짜	–	null 허용

필드	타입	필수	제약
`lastFinishedAt`	날짜	–	null 허용
`lastStatus`	문자열	–	`succeeded` · `failed` · `null` · null 허용
`failureCount`	정수	0	≥ 0

표 8.2.74 `scheduled_job_definitions` · 인덱스 `nextRunAt ↑`

필드	타입	필수	제약
`_id`	UUID 문자열	0	
`jobKey`	문자열	0	최대 255자
`scheduledFor`	날짜	0	
`status`	문자열	0	`queued` · `running` · `succeeded` · `failed` · 최대 255자
`attempts`	정수	0	≥ 0
`startedAt`	날짜	–	null 허용
`finishedAt`	날짜	–	null 허용
`lastError`	문자열	–	null 허용
`result`	객체	–	null 허용
`createdAt`	날짜	0	

표 8.2.75 `scheduled_job_runs` · 인덱스 `jobKey ↑, scheduledFor ↑` (유일) · `status ↑, scheduledFor ↑`

필드	타입	필수	제약
`_id`	문자열	0	
`name`	문자열	0	
`checksum`	문자열	0	
`state`	문자열	0	`applying` · `applied`
`completedSteps`	배열	0	
`appliedAt`	날짜	–	

표 8.2.76 `schema_migrations`

필드	타입	필수	제약
`_id`	문자열	0	
`owner`	문자열	0	
`token`	문자열	0	
`expiresAt`	날짜	0	

표 8.2.77 `migration_locks`

필드	타입	필수	제약
`_id`	문자열	0	
`payloadId`	문자열	0	

필드	타입	필수	제약
`userId`	문자열	O	
`part`	정수 · 실수	O	
`bytes`	이진	O	
`sha256`	문자열	O	

표 8.2.78 `snapshot_chunks` · 인덱스 `payloadId ↑, part ↑` (유일)

필드	타입	필수	제약
`_id`	문자열	O	
`visitorHash`	문자열	O	
`targetId`	문자열	O	
`period`	문자열	O	
`count`	정수 · 실수	O	
`expiresAt`	날짜	O	

표 8.2.79 `analytics_rate_limits` · 인덱스 `expiresAt ↑` (만료 0초) · `visitorHash ↑, targetId ↑, period ↑` (유일)

필드	타입	필수	제약
`_id`	문자열	O	
`schemaHash`	문자열	O	
`tables`	객체	O	
`validation`	객체	O	
`completed`	참거짓	O	

표 8.2.80 `import_runs`

필드	타입	필수	제약
`_id`	문자열	O	
`runId`	문자열	O	
`tableName`	문자열	O	
`lastKey`	문자열	–	
`processedCount`	정수 · 실수	O	

표 8.2.81 `import_checkpoints` · 인덱스 `runId ↑, tableName ↑` (유일)

■ 비동기 이벤트

필드	타입	필수	제약
`_id`	UUID 문자열	O	
`topic`	문자열	O	
`payload`	객체	O	
`idempotencyKey`	문자열	O	최대 255자

필드	타입	필수	제약
`state`	문자열	O	`pending` · `publishing` · `published` · `dead_letter` · 최대 255자
`attempts`	정수	O	≥ 0
`availableAt`	날짜	O	
`lockedAt`	날짜	–	null 허용
`publishedAt`	날짜	–	null 허용
`lastError`	문자열	–	null 허용
`createdAt`	날짜	O	
`updatedAt`	날짜	O	
`leaseToken`	문자열	–	null 허용
`leaseUntil`	날짜	–	null 허용
`userId`	문자열	–	null 허용

표 8.2.82 `outbox_events` · 인덱스 `idempotencyKey ↑` (유일) · `availableAt ↑, createdAt ↑` · `state ↑, availableAt ↑, _id ↑` · `state ↑, leaseUntil ↑`

■ 검증과 일관성

- 요청·응답은 `packages/contracts`의 **Zod 스키마**로 검증한다. 저장 문서는 컬렉션 validator와 인덱스를 적용한다.
- 여러 문서에 걸친 상태 변경은 MongoDB 트랜잭션으로 묶는다. 작업 발행은 `outbox_events`를 **같은 트랜잭션**에 기록하고 디스패처가 Redis 큐로 전달한다.
- 참조 대상의 존재와 소유권은 **서비스가 검증**한다.

8.3 로그파일 설계

모든 기능과 서비스가 정상 동작하는지 확인하기 위해 **실행 로그와 운영 로그**를 파일로 저장한다. 로그는 JSON 한 줄에 한 사건으로 기록하며, 사람이 읽는 형식이 아닌 기계가 필터링하는 형식으로 작성한다. 장애 발생 시 `requestId`로 한 요청의 전 구간을 조회할 수 있어야 한다.

프로세스	파일명	회전
API 서버	`expresso-api.log`	일 단위 · 14일 보관
Worker	`expresso-worker.log`	일 단위 · 14일 보관
AI학습서버	`expresso-training.log`	일 단위 · 30일 보관
오류만 모은 파일	`expresso-error.log`	일 단위 · 90일 보관

학습 로그와 오류 로그를 더 오래 둡니다. 모델 성능 문제와 드문 오류는 재발 주기가 2주보다 깁니다.

세션 토큰 · 접속 문자열 · 비밀번호 · 이메일은 **로그에 남기지 않으며**, 쿼리스트링은 **통째로 잘라 낸다**.

Log 예시

```
{"level":30,"time":1786665012345,"requestId":"req_0f8a2c91",
 "req":{"method":"POST","url":"/v1/generation-jobs"},"msg":"incoming request"}

{"level":30,"time":1786665071204,"requestId":"req_0f8a2c91","jobId":"d5e7a2b9",
 "durationMs":58816,"tokensIn":18442,"tokensOut":6103,"msg":"generation completed"}

{"level":40,"time":1786665130017,"requestId":"req_3b71de4a",
 "msg":"training server unavailable"}

{"level":50,"time":1786665188902,"requestId":"req_91cf07b2",
 "error":{"name":"GenerationValidationError","statusCode":422},"msg":"request failed"}
```

8.4 오류코드 및 오류파일 설계

발생할 수 있는 모든 오류를 식별하여 아래와 같이 코드로 정의한다. 오류가 발생할 때마다 기록하는 오류 파일의 포맷과 파일명은 8.3의 로그파일 설계를 따른다.

코드	이름	설명
200	—	정상 처리(성공)
400	`VALIDATION_ERROR`	요청 스키마 검증 실패
401	`AUTH_REQUIRED`	인증이 필요하거나 세션이 만료됨
403	`FORBIDDEN`	인증은 되었으나 권한이 없음
404	`NOT_FOUND`	리소스가 없음
409	`CONFLICT`	현재 상태와 충돌하는 요청
412	`PRECONDITION_FAILED`	`If-Match` 버전이 낡음
422	`UNPROCESSABLE`	요청은 읽었으나 그대로는 처리할 수 없음
429	`RATE_LIMITED`	사용량 한도 초과
503	`DEPENDENCY_UNAVAILABLE`	MongoDB · Redis · LLM 등 의존 서비스 장애
500	`INTERNAL_ERROR`	그 밖의 서버 오류

401과 403은 구분하여 반환한다. 두 경우에 클라이언트가 수행할 다음 행동이 다르다. 401은 다시 로그인하면 되지만, 403은 다시 로그인하여도 해결되지 않는다.

코드	AI학습서버 오류	코드	AI학습서버 오류
000	정의되지 않은 에러코드	421	성능 검증 오류
401	데이터세트 생성 오류	422	배포 승인 기준 미달
402	원천 데이터 부족 오류	431	모델 배포 오류
403	라벨 계산 불일치 오류	432	모델 버전 관리 오류
404	인코딩 오류	433	모델 롤백 오류
405	임베딩 모델 로드 오류	434	모델 삭제 오류
411	모델 학습 오류	441	기록 순위 산출 오류
412	하이퍼파라미터 탐색 오류	442	**인코더 불일치 오류**
413	학습 중단 오류	443	활성 모델 없음 (규칙 폴백)
–	—	451	장애기록 반환 오류

442는 오류를 내지 않고 조용히 틀린 답을 내놓는 유일한 경우이므로, 서빙 시작 시 `metadata.json`의 인코더 이름 · 버전을 로드한 인코더와 대조하여 미리 잡는다. 403은 교사가 낸 점수와 루브릭 계산이 다른 라벨이 남아 있을 때 학습을 막는다.

09 성능시험지표

서비스 로컬 측정 · 모델 미측정

9.1 AI서비스 성능평가지표

개발할 AI서비스의 성능을 측정하기 위한 평가지표를 아래와 같이 발굴하여 정의한다. 평가항목마다 시험내용 · 시험환경 · 시험절차를 함께 작성한다.

평가항목	단위	개발목표치	평가방법
공고 추천 순위 품질 (모델 A · NDCG@10)	–	확정 전	팀 자체시험
기록 추천 순위 품질 (모델 B · NDCG@10)	–	확정 전	팀 자체시험
포트폴리오 생성 성공률	%	≥ 90	팀 자체시험
대표 read 경로 p95	ms	≤ 300	자동화 시험
대표 write 경로 p95	ms	≤ 300	자동화 시험
이벤트 수집 p95	ms	≤ 150	자동화 시험
큐 등록 p95	ms	≤ 200	자동화 시험

평가항목	단위	개발목표치	평가방법
공고 추천 순위 품질 (모델 A)	–	확정 전	팀 자체시험

시험내용

모델 A가 매긴 공고 순위가 교사 적합도 라벨의 순위와 얼마나 일치하는지 측정한다. 평가 분할의 프로필마다 연결된 공고 10개를 모델 A의 점수로 정렬한 뒤 NDCG@10을 산출하고 프로필 전체의 평균을 낸다. 예측 점수와 교사 점수의 MAE와 공고쌍 순서 일치율을 함께 산출하며, 같은 시험을 규칙 baseline에도 수행하여 개선 폭을 확인한다.

시험환경

1. 평가 분할 700쌍 (프로필 70개, 프로필 단위로 분리된 것)
2. 학습이 완료된 `bi` · `cross` 모델과 `metadata.json`
3. 규칙 baseline — `match-score.ts`의 `calculateExplainableMatch()`
4. 지표 산출 스크립트와 공고 벡터 캐시
5. 결과를 확인할 PC 환경

시험절차

1. 평가 분할에서 프로필 1건을 선택한다.
2. 그 프로필에 연결된 공고 10개를 모델 A로 채점한다(선별 후 재정렬).
3. 점수 내림차순으로 정렬한다.
4. 교사 적합도 라벨을 정답으로 하여 NDCG@10을 계산한다.
5. 같은 프로필 안에서 정답 점수 차가 5점 이상인 공고쌍의 순서 일치율과, 예측 점수의 MAE를 계산한다.
6. 모든 프로필에 대하여 1~5를 반복하고 평균을 낸다.
7. 같은 절차를 규칙 baseline으로 수행하여 두 값을 비교한다.

평가항목	단위	개발목표치	평가방법
기록 추천 순위 품질 (모델 B · NDCG@10)	–	확정 전	팀 자체시험

시험내용

모델 B가 (채용 공고, 커리어 기록 1건) 쌍에 매긴 점수 순서가 교사의 기록 단위 판정과 일치하는지 측정한다. 기록의 정답 관련도는 교사가 그 공고의 요구사항에 연결한 충족 점수 가운데 최댓값으로 하고, 어느 요구사항에도 연결되지 않은 기록은 0으로 둔다. Cross-Encoder를 학습 때와 다른 입력 단위로 쓰므로, 이 시험이 그 분포 차이의 영향을 확인하는 자리이다.

시험환경

1. 평가 분할 700쌍의 교사 판정 — 요구사항별 근거 기록 ID와 충족 수준
2. 활성 `cross` 모델
3. 규칙 baseline — `materials/ranking.ts` 의 `rankMaterials()`
4. NDCG@10을 산출하는 스크립트

시험절차

1. 평가 분할에서 (공고, 프로필) 쌍 1건을 선택한다.
2. 그 프로필의 기록마다 모델 B로 (공고, 기록) 점수를 매긴다.
3. 점수 내림차순으로 정렬한다.
4. 교사의 기록 단위 판정을 정답으로 하여 NDCG@10을 계산한다.
5. 모든 쌍에 대하여 1~4를 반복하고 평균을 낸다.
6. 같은 절차를 규칙 baseline으로 수행하여 두 값을 비교한다.

■ 응답 시간 — 2026-09-28 MongoDB 로컬 시험

경로	예산	측정 P95	결과
대표 read `GET /v1/home`	300ms	7.59ms	PASS

경로	예산	측정 P95	결과
대표 write `POST /v1/career/records`	300ms	10.30ms	PASS
event collection	150ms	6.89ms	PASS
queue registration	200ms	7.39ms	PASS

2026-09-28 현재 코드, MongoDB 8.0 단일 노드 rs0의 독립된 임시 테스트 DB에서 측정했다.

추가 측정값은 읽기 100건 동시 요청 118.01ms, 공고 1,000건 목록 85.90ms, 9MiB 스냅샷 10조각 쓰기 132.94ms·읽기 78.12ms다. 같은 시험에서 3개 검증이 통과했다. 실행 명령은 `EXPRESSO_LOAD_TEST=1 node scripts/test-infra.mjs test/load/performance-budget.test.ts --disableConsoleIntercept` 다.

9.2 AI학습 성능 (ROC)

학습 모델의 성능은 추천 순위 · 후보 선별 · 점수 예측 · 처리 효율을 구분하여 평가한다. 주요 지표는 NDCG@10이며, 설계서 양식이 요구하는 ROC · AUC는 보조 지표로 함께 산출한다. ROC는 적합도 라벨 **60점 이상**을 양성으로 두고, 예측 점수의 임계값을 바꾸어 가며 그린다.

평가 지표	무엇을 평가하는가	해석 방향	적용 범위
NDCG@10	적합도가 높은 공고를 상위에 배치하는가	높을수록 좋음	주요 지표 · 모델 A · B
Recall@K	적합한 공고를 스크리닝 단계에서 충분히 남기는가	높을수록 좋음	스크리닝 (`bi`)
MAE	예측 점수와 교사 라벨 점수의 평균 절대 차이	낮을수록 좋음	점수 예측
공고쌍 순서 일치율	같은 프로필에서 더 적합한 공고에 높은 점수를 주는가	높을수록 좋음	순서 예측
p95 지연 · 처리량	요청의 95%가 완료되는 시간과 단위 시간당 처리량	지연은 낮게 · 처리량은 높게	운영 효율
ROC · AUC	60점 이상 쌍을 나머지와 구분하는가	높을수록 좋음	양식 보조 지표

■ 순위 지표의 이해

추천은 점수 하나를 맞히는 일이 아니라 공고의 순서를 정하는 일이므로, 주요 지표로 순위 지표를 쓴다. NDCG@k는 상위 k개의 **순서**가 이상적인 순서에 얼마나 가까운지를, Recall@k는 적합한 공고를 상위 k개 안에 **빠짐없이** 남겼는지를 본다. 아래 그림 9.1은 두 지표가 추천 단계에서 각각 무엇을 평가하는지를, 그림 9.2는 공고 몇 건으로 두 지표를 직접 계산한 과정을 나타낸다. 두 그림의 숫자는 설명을 위한 예시이다.



그림 9.1 순위 지표가 평가하는 것 (Recall@K · NDCG@10)



그림 9.2 순위 지표의 계산 예시 (NDCG@5 · Recall@k)

NDCG@k(Normalized Discounted Cumulative Gain)는 **0부터 1 사이의 값**이며, 1이면 모델의 순서가 교사 점수 내림차순과 같다. 아래 세 단계로 계산한다.

01 **gain**

공고마다 교사 적합도 점수를 그대로 gain으로 쓴다(선형 gain).

02 **DCG@k**

모델이 매긴 순서대로 gain을 $\log_2$(순위 + 1)로 나누어 더한다. DCG@k = $\Sigma_{i=1...k}$ $gain_i \div \log_2(i+1)$. 1위는 1로, 2위는 1.585로, 3위는 2로 나누므로 같은 공고라도 위에 놓일수록 합에 크게 기여한다.

03 **NDCG@k**

같은 공고를 교사 점수 내림차순으로 놓았을 때의 DCG(IDCG@k)로 나눈다. NDCG@k = DCG@k ÷ IDCG@k.

그림 9.2 위쪽에서 모델이 공고를 B(75) · A(90) · C(60) · E(10) · D(40) 순으로 놓으면 DCG@5는 181.56이고, 이상적인 순서 A · B · C · D · E의 IDCG@5는 188.42이므로 NDCG@5는 **0.964**이다. 모자란 6.86 가운데 1 · 2위가 바뀐 손실이 5.54, 4 · 5위가 바뀐 손실이 1.31이다. 이처럼 NDCG는 **위쪽 순위의 실수를 더 크게 깎는다**. 사용자는 목록의 위쪽부터 보므로 위쪽 순서가 더 중요하다는 점을 반영한 것이다.

선형 gain에서는 모든 공고의 점수가 0보다 크므로 순서를 완전히 뒤집어도 NDCG가 0이 되지 않는다. 예시의 다섯 공고를 E · D · C · B · A로 정반대로 놓아도 NDCG@5는 **0.702**이다. 따라서 NDCG 값 하나의 크기로 판단하지 않고, 같은 평가 분할에서 기준선(Char TF-IDF 등)과의 차이와 그 신뢰구간으로 읽는다. 아래 파일럿 결과에서 NDCG@10이 0.95 안팎으로 높게 나오는 것도 이 성질 때문이다. 본 설계서의 NDCG@10은 프로필마다 공고 10개를 모델 점수로 정렬하여 계산하고 프로필 전체의 평균을 낸다(9.1 시험절차).

Recall@k는 적합한 공고 전체 가운데 상위 k개 안에 들어온 비율이다. 적합한 공고는 교사 적합도 **60점 이상**인 공고로 보며, ROC의 양성과 MAP@10 · MRR@10의 기준과 같다. Recall@k = (상위 k개 안의 적합 공고 수) ÷ (후보 전체의 적합 공고 수). 그림 9.2 아래쪽에서 후보 20건 가운데 적합 공고 4건이 1 · 3 · 4 · 8위에 있으면 Recall@5는 3 ÷ 4 = 0.75, Recall@10은 4 ÷ 4 = 1.00이다.

Recall@k는 상위 k개 안의 순서를 보지 않는다. 적합 공고 4건이 1–4위에 있든 7–10위에 있든 Recall@10은 같다. 대신 **적합한 공고를 놓쳤는가**를 본다. 그래서 **스크리닝 단계의 지표**로 쓴다. 단계적 활용(그림 4.4)에서 Cross-Encoder는 Bi-Encoder+MLP가 남긴 상위 K개만 다시 평가하므로, 스크리닝에서 빠진 적합 공고는 재정렬 단계에서 되살릴 수 없다. 스크리닝 후보 수 K는 Recall@K가 충분히 높으면서 Cross-Encoder의 처리 시간이 허용되는 값으로 정한다. 스크리닝은 Recall@K로, 재정렬의 결과 순서는 NDCG@10으로 평가하는 것은 이 때문이다.

■ 평가 조건

- [ ] 점수와 순위는 **교사 라벨**을 기준으로 평가한다.
- [ ] 공고쌍은 같은 프로필 안에서 정답 점수 차이가 **5점 이상**인 쌍으로 구성한다.
- [ ] Recall@K는 **K보다 큰 후보 풀**과 관련성 기준을 정해 측정한다. 프로필당 공고 10개 묶음 평가와는 별도로 구성한다.
- [ ] 인코더 네 종은 **같은 데이터와 같은 평가 조건**에서 비교하고, 임베딩 생성 시간 · 처리량 · GPU 메모리 · 임베딩 저장 용량을 함께 기록한다.
- [ ] 모델은 검증 분할에서 임계값과 하이퍼파라미터를 정하고, 평가 분할에서 한 번만 평가한다.
- [ ] 사람 평가와의 일치와 루브릭 가중치의 타당성은 라벨 구축 후 **후속 검증**으로 확인한다.

■ 파일럿 학습 조건

2026-09-04에 수행한 파일럿 학습의 결과를 기록한다. **합성 프로필 1,000개**에 공고 10개씩을 붙인 **10,000쌍**을 사용하였고, 후보 공고는 프로필마다 같은 직무 5개 · 인접 직무 3개 · 무작위 2개이다. 공고는 JTH 공고 데이터(`jobs.csv`)의 구조화 필드를 사용하였다. 라벨은 루브릭 `job-profile-fit-v1` 로 산출하였으며 GPT-5.6 Luna가 9,260건, Claude Sonnet이 740건을 판정하였다. 프로필 ID와 공고 ID는 분할 사이에 겹치지 않는다. 두 학습 모델은 선별 · 재정렬로 연결하지 않고 각각 독립으로 평가하였다.

분할	프로필	공고	쌍	용도
학습	840	2,366	8,400	가중치 학습 · 기준선 점수 보정
검증	90	292	900	최적 epoch 선택
평가	70	228	700	최종 성능 비교

학습 설정	BI-ENCODER+MLP	CROSS-ENCODER
기반 모델	`multilingual-e5-base` · 동결	`bge-reranker-v2-m3`
목적 함수	점수 회귀 + 0.1 × 순서 학습 손실	점수 회귀
학습률	0.0005	0.00002
선택 epoch	47 epoch 중 검증 MAE 최저 39 (조기 종료)	3 epoch 중 3
학습한 가중치	MLP 헤드 · 3.0 MiB	마지막 4개 층 + 분류 헤드 · 약 5,144만 파라미터 · 196.23 MiB
실행 장비	RTX 5080 · CUDA 12.8	RTX 5080 · CUDA 12.8

■ 파일럿 평가 결과

모델	NDCG@10 ↑	순서 일치율 ↑	MAP@10 ↑	MRR@10 ↑
Word TF-IDF	0.8653	0.5870	0.4955	0.5581
BM25	0.8635	0.5827	0.4371	0.4647
Char TF-IDF	0.8799	0.6334	0.5280	0.6253
E5 cosine (학습 전)	0.8370	0.5151	0.4112	0.4965
BGE zero-shot (학습 전)	0.8535	0.5685	0.4302	0.4824
Bi-Encoder+MLP	**0.9555**	**0.8025**	0.6909	0.7225
Cross-Encoder	**0.9608**	0.7903	0.7464	0.7610

순위 지표. 평가 분할 70명 · 700쌍. NDCG@10은 선형 gain, 순서 일치율은 동점 라벨을 제외한 2,714개 공고쌍, MAP@10 · MRR@10은 60점 이상 공고가 있는 13명을 대상으로 계산하였다.

모델	MAE ↓	RMSE ↓	SPEARMAN ↑
Word TF-IDF	15.04	17.76	0.2522
BM25	14.95	17.71	0.2675
Char TF-IDF	14.91	17.57	0.3189
E5 cosine (학습 전)	15.29	17.98	0.1605
BGE zero-shot (학습 전)	15.41	18.06	0.2315
Bi-Encoder+MLP	**10.54**	13.44	0.6832
Cross-Encoder	**10.51**	12.93	0.6819

점수 지표. 평가 분할 700쌍.

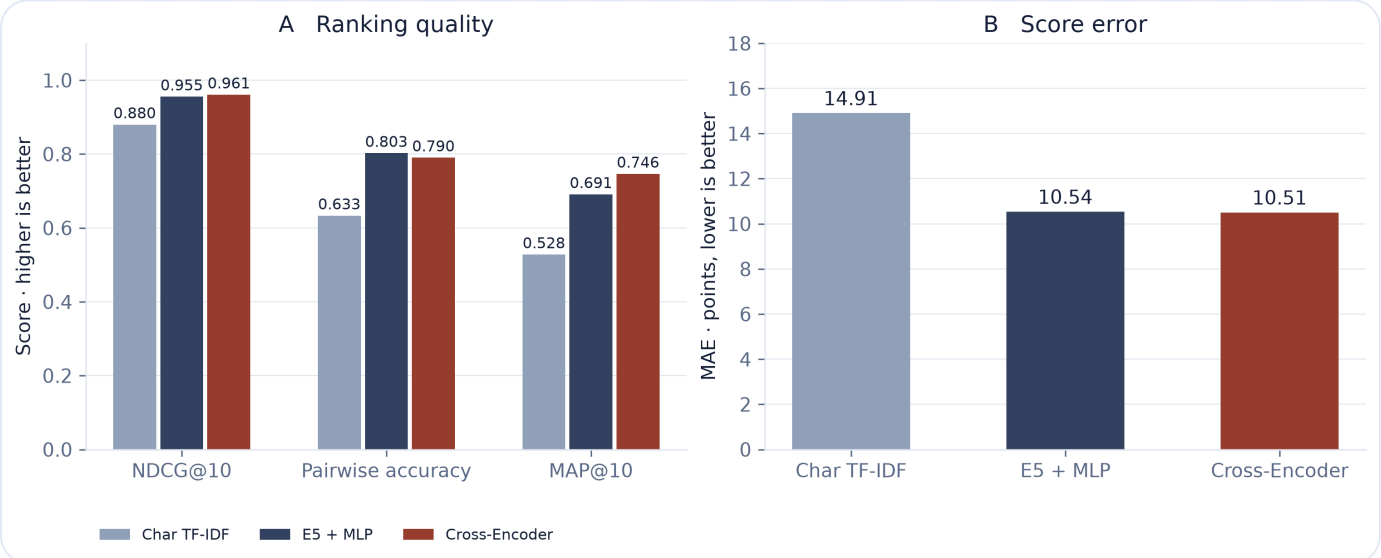


그림 9.3 모델별 순위 품질과 점수 오차 (평가 분할)

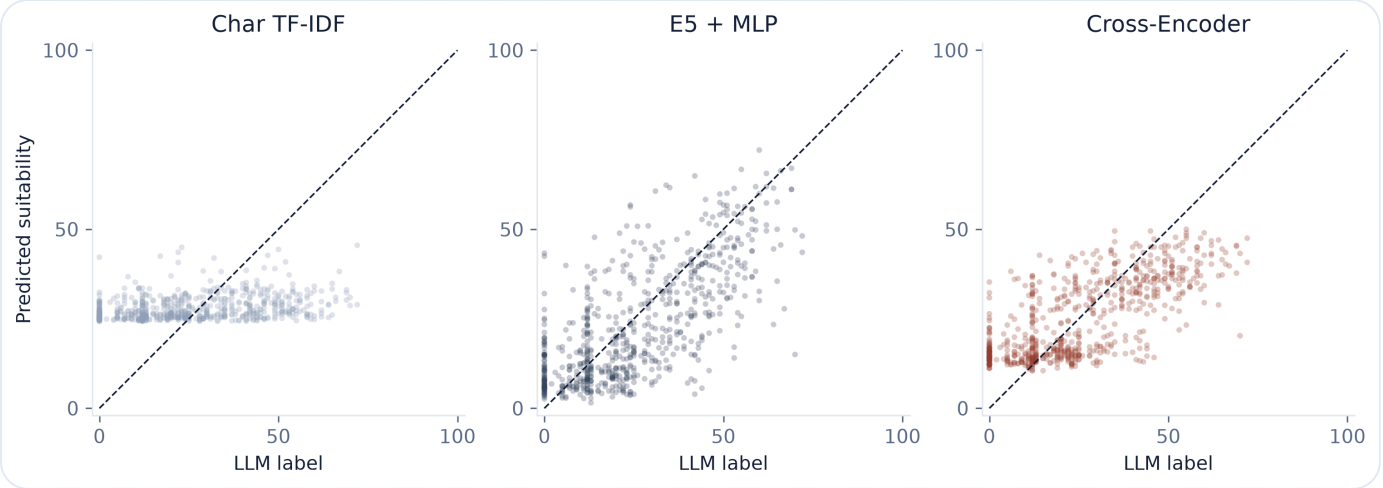


그림 9.4 LLM 라벨과 모델 예측값 (평가 분할 700쌍)

모델	같은 직무 (350쌍)	인접 직무 (210쌍)	무작위 (140쌍)
Char TF-IDF	15.31	14.58	14.37
Bi-Encoder+MLP	11.19	10.42	9.08
Cross-Encoder	11.45	10.49	8.18

후보 유형별 MAE (평가 분할).

비교	NDCG@10 차이 [95% 구간]	MAE 차이 [95% 구간]
Bi-Encoder+MLP – Char TF-IDF	+0.0756 [+0.0548, +0.0967]	-4.37 [-5.65, -3.02]
Cross-Encoder – Char TF-IDF	+0.0809 [+0.0597, +0.1036]	-4.40 [-5.25, -3.53]
Cross-Encoder – Bi-Encoder+MLP	+0.0054 [-0.0046, +0.0160]	-0.03 [-0.94, +0.85]

평가 프로필 70명 단위 paired bootstrap · 20,000회 · seed 42.

두 학습 모델은 기준선 가운데 가장 높은 Char TF-IDF보다 NDCG@10이 높고 MAE가 낮으며, 두 차이의 95% 구간이 모두 0을 포함하지 않는다. Cross-Encoder와 Bi-Encoder+MLP의 차이는 두 지표 모두 **구간이 0을 포함한다**. 검증 분할의 NDCG@10은 Bi-Encoder+MLP **0.9406**, Cross-Encoder **0.9383**이고 MAE는 각각 10.41, 10.35이다.

지표별 측정 현황

모델	지표	파일럿 측정값
A	NDCG@10	Bi-Encoder+MLP 0.9555 · Cross-Encoder 0.9608
	MAE	Bi-Encoder+MLP 10.54 · Cross-Encoder 10.51
	공고쌍 순서 일치율	Bi-Encoder+MLP 0.8025 · Cross-Encoder 0.7903
	Recall@K (스크리닝)	측정하지 않음 — 후보 10개 전체를 평가하여 Recall@10이 항상 1
	p95 지연 · 처리량	측정하지 않음
	AUC	산출하지 않음
B	NDCG@10 (교사 기록 단위 판정 기준)	측정하지 않음

모델 A의 ROC 곡선 — 평가 분할 점수로 산출 후 삽입

그림 9.5 ROC 곡선 자리

10 개발환경 구성

양식 10장

10.1 개발환경

본 프로젝트의 개발환경은 아래와 같다.

서버

구분	개발환경	규격
개발툴	편집툴	VS Code
	런타임	Node.js 24 이상
	패키지 관리자	pnpm 11.16.0 (워크스페이스 모노레포)
	언어 · 프레임워크	TypeScript 5 (ESM) · Fastify 5.11
	사용할 미들웨어	zod 4.4 · mongodb 7.5 · ioredis 6.0 · BullMQ 6.0 · sanitize-html 2.17 · sharp 0.35 · turndown 7.2
서버	TCP Port주소	`4000 (API). 학습서버 4100은 설계 포트`
	OS	Ubuntu 24.04 LTS (운영) / macOS · Linux (개발)
	DB	MongoDB 8.0 — 개발 포트 57017
	캐시 · 큐	Redis 8.2.1 — 개발 포트 56379
서버 하드웨어	CPU	운영 사양 확정 필요
	RAM	운영 사양 확정 필요
	DISK	운영 사양 확정 필요
	Network	운영 사양 확정 필요

클라이언트 · AI학습서버

구분	개발환경	규격
클라이언트	프레임워크	Next.js 16.3 (App Router)
	라이브러리	React 19.2 · Phosphor Icons 2.1 · react-markdown 10.1
	지원 브라우저	Chrome · Safari · Edge 최신 2개 버전
AI학습서버	언어	Python
	학습	PyTorch · Hugging Face Transformers
	인코더	`multilingual-e5-base` 외 비교 후보 3종 (동결) · `bge-reranker-v2-m3` (마지막 4개 층 학습)
	데이터 · 시각화	pandas · pyarrow · matplotlib
	서빙 · GPU	FastAPI · Cross-Encoder 미세조정에 GPU 사용 사양 확정 필요

LLM 프로바이더

`AI_PROVIDER`	동작	용도
`off`	AI 클라이언트를 만들지 않는다. 요건 추출과 생성 문장은 사용 불가 상태로 끝난다	기본값 · AI 없이 개발
`claude-code`	로그인된 Claude Code CLI를 호출한다	개발
`codex`	로그인된 Codex CLI를 호출한다	개발
`fixture`	녹화된 응답을 재생한다	CI · 회귀 시험
`anthropic`	API 키로 호출한다	운영 미구현

인프라 실행

로컬 개발 시작

```
pnpm install
pnpm infra:up
cp services/backend/.env.example services/backend/.env
pnpm db:migrate
pnpm dev:backend
```

`pnpm infra:up`은 기존 로컬 서비스와 충돌하지 않도록 **포트를 고정**하여 띄운다. 두 인프라가 없어도 API 프로세스는 시작되지만 `/health/ready`가 503을 반환한다.

10.2 소스디렉터리 구조



현재 저장소의 디렉터리 구조다. AI학습서버의 디렉터리와 실행 경로는 4장·7.4의 설계안이다.

모든 도메인 모듈이 **같은 형태**를 가진다. `routes.ts`가 HTTP를, `service.ts`가 비즈니스 규칙을 맡고, `*.test.ts`와 `*.integration.test.ts`가 그 옆에 놓인다. 파일 이름이 곧 역할이므로 새 모듈을 추가할 때에도 같은 구성을 따른다.

10.3 명칭표준안 (변수, 함수, 파일명)

영역	대상	규칙	예
파일	디렉터리 · TS 파일	kebab-case	`job-analysis/` · `build-app.ts`
	시험 파일	`<대상>.test.ts`	`coverage.test.ts`
	MongoDB 문서 타입	`<도메인>.ts`	`career-editor.ts`

영역	대상	규칙	예
TypeScript	변수 · 함수	camelCase	`calculateRequirementCoverage`
	타입 · 클래스	PascalCase	`GenerationEvidence`
	zod 스키마	PascalCase + `Schema`	`CreateCareerRecordSchema`
	상수 · 열거 값	UPPER_SNAKE_CASE	`API_PREFIX` · `VALIDATION_ERROR`
	오류 클래스	`<맥락>Error`	`GenerationValidationError`
데이터베이스	컬렉션	snake_case 복수	`job_posting_requirements`
	참조 필드	camelCase	`jobPostingId`
	시각 필드	`<동사>At`	`extractedAt`
	인덱스	collection spec에서 선언	`job_posting_requirements` 조회 인덱스
REST API	경로	kebab-case 복수 명사	`/v1/job-analyses`
	파라미터 · JSON 필드	camelCase	`:portfolioId` · `usageRemaining`
	오류 코드	UPPER_SNAKE_CASE	`PRECONDITION_FAILED`
Python	변수 · 함수	snake_case	`build_profile_text`
	데이터세트 버전	`fit-v<번호>`	`fit-v1`
	모델 버전	`bi-v<세트>-<일련>` · `cross-v<세트>-<일련>`	`cross-v1-002`

TypeScript 문서 타입과 MongoDB 문서 필드는 camelCase를 사용한다. 컬렉션 이름은 snake_case 복수형이다. 루브릭이나 인코더 입력의 텍스트 구성을 바꾸면 라벨과 벡터의 성격이 바뀌므로 데이터세트 버전을 올린다.

2026-10-13 v1.0 설계서 작성 완료 — Expresso 팀

2026-10-13 v0.9 사용자용 유스케이스 다이어그램(그림 2.3)의 연결선 교차 정리 — Expresso 팀

2026-10-06 v0.8 순위 지표가 평가하는 것을 나타낸 개념 그림(그림 9.1) 추가, 순위 지표 계산 예시를 그림 9.2로 이동 — Expresso 팀

2026-10-06 v0.7 서비스 개요 그림과 세 가지 장점, 기존 서비스 지원 범위 벤 다이어그램, 전체 사용자 플로우, 순위 지표(NDCG@k · Recall@k) 설명 추가, 핵심 문장 하이라이트 · 강조, 제공 효과 정리 — Expresso 팀

2026-10-06 v0.6 구조 그림 9개를 제안서 양식으로 반영(교사 지식 증류 · 두 모델의 단계적 활용 추가), 방문자 용어 통일, 모델 A 판단 기준에서 근무 조건 제외, 웹 화면을 PDF와 같은 구성으로 정리 — Expresso 팀

2026-10-05 v0.5 표지 · 문서 정보 · 머리말에 Expresso 브랜딩과 공식 명칭 적용, 개발필요성을 제안서 기획 배경 · 벤치마킹 차별점 기준으로 재작성, 폐기된 근거 검증 컨셉 정리 — Expresso 팀

2026-10-05 v0.4 파일럿 학습 결과(4.3 · 9.2)와 ROC 양성 기준 60점 반영, 7.3 API 설계 26개로 확장, 8.2 컬렉션 명세서 82개 자동 생성 — Expresso 팀

2026-10-05 v0.3 프로젝트 제안서 기준 AI 모델(Bi-Encoder+MLP · Cross-Encoder) · 합성 학습 데이터 · 평가 지표 · 15주 일정 · 개발조직 정합 — Expresso 팀

2026-09-28 v0.2 MongoDB 데이터·시스템 구조와 현행 규모·성능 수치 반영 — Expresso 팀

변경 이력 · 2026-08-14 v0.1 저장소 형상 기준 전 장 초안 작성 — Expresso 팀

2026-08-14 5.3 사용자 화면설계 29개를 화면별로 다시 작성 · 5.2 초기화면에 실제 캡처 적용 — Expresso 팀

2026-08-14 6.2 · 6.3 · 6.4 다이어그램 해설 28개를 흐름과 근거로 나누어 다시 작성 · 6.4의 작성 현황 표를 귀속 회계로 교체 — Expresso 팀

2026-08-14 AI 개발 방향을 2.1에 정의하고, 본문 전체의 대구 · 비유 · 판단 표현 55곳을 양식의 규정문으로 교정 — Expresso 팀